

Anne Baanen
Alexander Bentkamp
Jasmin Blanchette
Xavier Génèreux
Johannes Hölzl
Jannis Limperg

Oling Cat • 欧林猫
Lean-zh 项目组 译

逻辑验证漫游指南

2026 桌面版

(2026 年 6 月 18 日)



github.com/Lean-zh/LoVe-zh

目录

目录	iii
前言	vii
一 基础	1
1 类型与项	3
1.1 类型	3
1.2 项	4
1.3 类型检查与类型推断	6
1.4 类型居留	8
1.5 新引入的 Lean 结构总结	9
2 程序与定理	11
2.1 类型定义	11
2.2 函数定义	16
2.3 定理陈述	20
2.4 新引入的 Lean 结构总结	22
3 逆向证明	23
3.1 策略模式	24
3.2 基本策略	26
3.3 关于连结词和量词的推理	28
3.4 关于相等的推理	32
3.5 重写策略	33
3.6 数学归纳法证明	35
3.7 归纳策略	36
3.8 清理策略	37
3.9 新引入的 Lean 结构总结	37
	iii

4	正向证明	39
4.1	结构化证明	39
4.2	结构化构造	42
4.3	关于联结词和量词的正向推理	44
4.4	计算式证明	50
4.5	使用策略进行正向推理	51
4.6	依值类型	52
4.7	PAT 原理	54
4.8	通过模式匹配与递归进行归纳	57
4.9	新引入的 Lean 结构总结	58
二	函数式与逻辑编程	61
5	函数式编程	63
5.1	归纳类型	63
5.2	结构归纳	64
5.3	结构化递归	66
5.4	模式匹配表达式	67
5.5	结构体	69
5.6	类型类	70
5.7	列表	74
5.8	二叉树	80
5.9	Cases 策略	82
5.10	依值归纳类型	83
5.11	新引入的 Lean 结构总结	85
6	归纳谓词	87
6.1	入门示例	87
6.2	逻辑符号	94
6.3	规则归纳	95
6.4	线性算术策略	98
6.5	消去	98
6.6	更多示例	101
6.7	归纳中的陷阱	106
6.8	新引入的 Lean 结构总结	108
7	带作用的编程	109
7.1	入门示例	109

7.2	两个操作与三个定律	112
7.3	一种类型类	114
7.4	无作用	115
7.5	基本异常	116
7.6	可变状态	118
7.7	非确定性	121
7.8	通用证明策略	122
7.9	一个通用算法：在列表上迭代	122
7.10	新引入的 Lean 结构总结	124
8	元编程	125
8.1	策略组合子	126
8.2	宏	129
8.3	元编程单子	130
8.4	第一个示例：一个 Assumption 策略	132
8.5	表达式	133
8.6	第二个示例：一个合取式分解策略	135
8.7	第三个示例：一个直接证明查找器	138
8.8	杂项策略	141
8.9	新引入的 Lean 结构总结	141
三	程序语义	143
9	操作语义	145
9.1	形式语义	145
9.2	一个极简的命令式语言	146
9.3	大步语义	147
9.4	大步语义的性质	150
9.5	小步语义	152
9.6	小步语义的性质	155
10	霍尔逻辑	157
10.1	霍尔三元组	157
10.2	霍尔规则	158
10.3	霍尔逻辑的语义方法	160
10.4	第一个程序：交换两个变量	163
10.5	第二个程序：两个数相加	164
10.6	验证条件生成器	165

10.7	重新审视第二个程序：两个数相加	168
10.8	完全正确性的霍尔三元组	169
11	指称语义	171
11.1	组合性	171
11.2	关系式指称语义	172
11.3	不动点	173
11.4	单调函数	174
11.5	完备格	174
11.6	最小不动点	176
11.7	关系式指称语义（续）	176
11.8	在程序等价上的应用	176
11.9	基于归纳谓词的更简单方法	179
四	数学	181
12	数学的逻辑基础	183
12.1	宇宙	183
12.2	Prop 的特殊性	184
12.3	选择公理	187
12.4	子类型	188
12.5	商类型	192
12.6	新引入的 Lean 构造总结	198
13	基本数学结构	201
13.1	单个二元算子上的类型类	201
13.2	两个二元算子上的类型类	205
13.3	强制转换	209
13.4	正规化策略	210
13.5	列表、多重集和有限集	210
13.6	序类型类	211
13.7	新引入的 Lean 构造总结	213
14	有理数与实数	215
14.1	有理数	215
14.2	实数	219
14.3	最后的勉励	224
14.4	新引入的 Lean 构造总结	224

前言

形式证明助手是一些旨在帮助用户进行计算机验证证明的软件。我们通常称之为**证明助手**或**交互式定理证明器**，但一位沮丧的学生创造了「**阻止证明的野兽**」一词来形容它，而语音识别软件则偶尔会将「**定理证明器**」误识别为「**恐惧改进器**」，敬请留意。

严格证明与形式证明 交互式定理证明有其自己的术语，始于「证明」的概念。**形式证明**是用逻辑形式主义表达的逻辑论证。在本文中，「形式」表示「逻辑的」或「基于逻辑的」。逻辑学家，也就是研究逻辑学的数学家，在计算机出现之前的几十年里就在纸上进行形式证明。但如今，形式证明几乎总是使用证明助手来完成。

相比之下，**非形式证明**就是数学家通常所说的证明。这类证明通常用 $\text{\LaTeX}$ 或黑板完成，也被称为「纸笔证明」。其详细程度可能差异很大，诸如「显然」、「清楚地」和「不失一般性」之类的短语将部分证明负担转移给了读者。**严格证明**则是一种非常详细的非形式证明。

证明助手的主要优势在于，它们能够运用精确的逻辑，帮助学生构建高度可靠、无歧义的数学陈述证明。它们可以用来证明任意高级的结果，远远超越了玩具示例和逻辑谜题。形式证明还能帮助学生理解有效定义或有效证明的构成。引用 Scott Aaronson 的话：¹

我还记得自己曾经批改过数百份试卷，学生们一开始只是假设需要证明的内容，或者一页一页地写满胡言乱语，希望在一片混乱中，他们**可能**会偶然说出一些正确的话。

当我们发展一个新理论时，形式证明可以帮助我们探索它。当我们推广、重构或以其他方式修改现有证明时，形式证明非常有用，就像编译器帮助我们开发正确的程序一样。形式证明提供了高度的可信度，使其他人更容易对其进行审查。此外，形式证明可以构成经过验证的计算工具（例如，经过验证的计算机代数系统）的基础。

¹<https://www.scottaaronson.com/teaching.pdf>

成功案例 在数学和计算机科学领域，辅助证明已取得诸多成功。数学形式化领域的一些里程碑式成果包括：Gonthier 等人证明四色定理 [7]，Gonthier 等人证明奇数阶定理 [8]，Hales 等人证明开普勒猜想 [11]，以及 Buzzard 等人定义完美拟态空间 [5]。该领域最早的研究始于 20 世纪 60 年代，由 Nicolaas de Bruijn 及其同事在一个名为 AUTOMATH 的系统中开展。²

包括 AMD [26] 和英特尔 [12] 在内的一些公司一直在使用证明助手来验证其设计。在学术界，一些里程碑式的成果包括操作系统内核 seL4 [14] 和 CertiKOS [10] 的验证，以及经过验证的编译器 CompCert [17]、JinjaThreads [20] 和 CakeML [16] 的开发。

证明助手 全球范围内，有许多正在开发或使用的证明助手。以下列出了一些主要的助手，按其逻辑基础进行分类：

- Set Theory **集合论**：Isabelle/ZF、Metamath、Mizar；
- Simple Type Theory **简单类型论**：HOL4、HOL Light、Isabelle/HOL、PVS；
- Dependent Type Theory **依值类型论**：Agda、Lean、Matita、Rocq（以前叫做 Coq）；
- First-Order Logic **类 Lisp 的一阶逻辑**：ACL2。

有关证明助手和交互式定理证明的历史，我们参考了 Harrison、Urban 和 Wiedijk 撰写的资料丰富的章节 [13]。

Lean Lean 是一个证明助手，主要由 Leonardo de Moura（亚马逊网络服务，AWS）自 2012 年以来开发。其数学库 mathlib 最初由 Jeremy Avigad（卡内基梅隆大学，CMU）领导开发，现由 Lean 用户社区维护并进一步扩展 [21]。³

本指南使用 Lean 版本 v4.24.0，mathlib 修订版 f897ebcf72cd16f8，以及一些扩展，收集在名为 LoVeLib 的小型库中。⁴ 虽然这只是一个研究项目，还存在一些不完善的地方，但 Lean 非常适合用于交互式定理证明教学，原因如下：

- 它具有表达能力很强且非常有趣的，基于 Calculus of Inductive Construction **归纳构造演算**（一种 Dependent Type Theory **依值类型论**）的逻辑。
- 它扩展了经典公理和 Quotient Type **商类型**，使其可用于验证数学。
- 它包含一个便捷的元编程框架，可用于编写自定义证明的自动化程序。
- 它通过 Visual Studio Code 插件包含一个现代化的用户界面。
- 它具有非常易读且完备的文档。
- 它是开源的。

²<https://www.win.tue.nl/automath/>

³<https://github.com/leanprover-community/mathlib4>

⁴https://github.com/lean-forward/logical_verification_2026/raw/main/lean/LoVe/LoVelib.lean

Lean 的核心库仅包含基本定义和工具。mathlib 则提供了更多功能。尽管名为 mathlib，但它不仅仅是一个数学库，它还在 Lean 的核心库之上提供了大量的自动化功能，满足了人们对现代证明助手的需要。

关于本指南 本指南最初是作为阿姆斯特丹自由大学理学硕士课程《逻辑验证》(LoVe) 的配套课程而设计的。其主要目的是教授交互式定理证明。Lean 是手段，而非目的本身。因此，本指南并非旨在成为全面的 Lean 教程——为此，我们推荐 *Lean 4 定理证明* [?]。本指南也不能替代练习和作业。定理证明不是用来看的，它只能通过实践来学习。

具体来说，你的目标是：

- 学习交互式定理证明的基本理论和技术；
- 学习如何使用逻辑作为一种精确的语言来建模系统并描述它们的性质；
- 熟悉一些证明助手可以成功应用的领域，例如函数式编程、命令式编程语言的语义以及数学；
- 培养可应用于大型项目的实践技能（无论是个人项目、硕士、博士项目还是工业界）；
- 能够转换到其他证明助手并应用所学的知识；
- 对该领域有充分的了解，可以开始阅读在国际会议（例如《认证程序与证明》(CPP) 和《交互式定理证明》(ITP)）或期刊（例如《自动推理杂志》(JAR)）上发表的相关科学论文。

具备良好的 Lean 知识后，应该可以轻松迁移到其他基于依值类型论的证明助手，例如 Agda 或 Rocq，或者基于简单类型论的系统，例如 HOL4 或 Isabelle/HOL。

本指南的一个重要特点是，它与 Knuth 的 *T_EXbook* [15] 一样，并非总是说实话。为了简化阐述，本书提出了一些关于 Lean 的基本但错误的论断。大多数这些论断都会在后续章节中得到纠正。与 Knuth 一样，我们认为「这种故意撒谎的技巧实际上会让你更容易学习这些理念。一旦你理解了一个简单但错误的规则，用它的例外来补充它就不难了。」

本指南附带的 Lean 文件可在公共源码库中找到。⁵ 文件的命名方案遵循本指南的章节名，因此，LoVe07_EffectiveProgramming_Demo.lean 是与课堂上复习的第 7 章「副作用编程」对应的主文件，LoVe07_EffectiveProgramming_ExerciseSheet.lean 是习题表，而 LoVe07_EffectiveProgramming_HomeworkSheet.lean 是家庭作业表。

我们非常感谢《Lean 4 定理证明》[2] 和《具体语义: Isabelle/HOL 描述》[23] 的作者，他们教会了我们 Lean 和编程语言语义学。他们的许多想法都体现在了本指南中。

⁵<https://github.com/Lean-zh/LoVe2026-zh/>

我们感谢 Robert Lewis 和 Assia Mahboubi 对本指南的组织 and 重点提出的宝贵意见。我们感谢 Kiran Gopinathan 和 Ilya Sergey 分享了第 2 章脚注 3 中提到的轶事，并允许我们进一步分享。我们感谢 Daniel Fabian 设计了本指南的第一个平板电脑优化版本。我们感谢 Paul Chisholm 撰写了相关 Lean 文件中的一些评论。我们感谢 Pietro Monticone 随着 Lean 和 mathlib 的发展而帮助更新 Lean 文件。We thank Moritz Roos for completing a proof about the real numbers, which is now included in the main Lean file associated with Chapter 14. We thank Henrik Böving for his comments on the Lean files associated with the guide. 我们感谢 Moritz Roos 完成了关于实数的证明，该证明现在包含在第 14 章对应的主 Lean 文件中。我们感谢 Henrik Böving 对与本指南相关的 Lean 文件提出的意见。我们感谢 Mark Summerfield 提出的许多文本建议。最后，我们感谢 Fabian Axer Avila、Chris Bailey、Kevin Buzzard、Paul Chisholm、Dominique Danco、Raufs Dunamalijevs、Wan Fokkink、Lina Gerlach、Alexandra Graß、Robert Lewis、Alexandre Rademaker、Antonius Danny Reyes、Robert Schütz、Kristina Sojakova、Patrick Thomas、Balazs Toth、Richard Tschärke、Huub Vromen、Floris Westerman、Wijnand van Woerkom 和 Yiming Xu 报告了他们在本指南早期版本中发现的错别字和一些更严重的错误。如果您发现本文中有任何错误，请告知对应的作者。⁶

特殊符号 在本指南中，我们假设你使用 Visual Studio Code 及其「lean4」扩展来编辑 .lean 文件。Visual Studio Code 允许你通过输入反斜杠 \ 后跟 ASCII 标识符来输入 Unicode 符号。例如， $\rightarrow$ 、 $\forall$ 或 $\in$ 可以通过输入 \->、\fo 或 \in 并按下 Tab 键或空格键来输入。我们将自由使用这些符号。作为参考，我们提供了本指南中使用的主要非 ASCII 符号列表，并为每个符号提供了其对应的 ASCII 表示形式。在 Visual Studio Code 中，按住 Control 键或 Command 键的同时将鼠标悬停在某个符号上，就能看到该符号的不同输入方式。

⁶Jasmin Blanchette (jasmin.blanchette@ifi.lmu.de)

$\neg$	<code>\not</code>	$\wedge$	<code>\and</code>	$\vee$	<code>\or</code>
$\rightarrow$	<code>\-></code>	$\leftrightarrow$	<code>\<-></code>	$\forall$	<code>\fo</code>
$\exists$	<code>\ex</code>	$\leq$	<code>\<=</code>	$\geq$	<code>\>=</code>
$\neq$	<code>\neq</code>	$\approx$	<code>\sim</code>	$\times$	<code>\x</code>
$\circ$	<code>\circ</code>	$\emptyset$	<code>\empty</code>	$\cup$	<code>\union</code>
$\cap$	<code>\intersect</code>	$\in$	<code>\in</code>	$\Downarrow$	<code>\downlefttharpoon</code>
$\bigcirc$	<code>\bigcirc</code>	$\leftarrow$	<code>\<-</code>	$\mapsto$	<code>\mapsto</code>
$\Rightarrow$	<code>\=></code>	$\Rightarrow$	<code>\==></code>	$\llbracket$	<code>\llbracket</code>
$\rrbracket$	<code>\rrbracket</code>	$\cdot$	<code>\.</code>	α	<code>\a</code>
β	<code>\b</code>	γ	<code>\g</code>	ε	<code>\e</code>
σ	<code>\s</code>	θ	<code>\theta</code>	$\subscript{1}$	<code>\1</code>
$\subscript{2}$	<code>\2</code>	$\subscript{3}$	<code>\3</code>	$\subscript{4}$	<code>\4</code>
$\subscript{5}$	<code>\5</code>	$\subscript{6}$	<code>\6</code>	$\subscript{7}$	<code>\7</code>
$\subscript{8}$	<code>\8</code>	$\subscript{9}$	<code>\9</code>		

第一部分

基础

类型与项

我们从学习 Lean 的基础知识开始，首先了解^{Term}项（也称为^{Expression}表达式）及其^{Type}类型。

Lean 的逻辑基础是一种丰富的逻辑，叫做^{Calculus of Inductive Construction}归纳构造演算，它支持^{Dependent Types}值类型。Lean 的逻辑灵感源自 λ -演算，类似于 Haskell、OCaml 和 Standard ML 等类型化函数式编程语言。即使您从未使用过这些语言，也能从现代语言（例如 C++、Java、Python）中找到许多类似的概念。大致来说：

Lean = 函数式编程 + 逻辑

如果你有数学背景，那么你可能已经了解过函数式编程的大部分关键概念了，有时它们有不同的名称。例如，Haskell 程序：

```
fib 0 = 0
fib 1 = 1
fib n = fib (n - 2) + fib (n - 1)
```

与数学定义非常吻合：

$$fib(n) = \begin{cases} 0 & \text{若 } n = 0 \\ 1 & \text{若 } n = 1 \\ fib(n-2) + fib(n-1) & \text{若 } n \geq 2 \end{cases}$$

在本章中，我们将关注 Lean 语言中一个不包含依值类型的部分，称为^{Simple Type Theory}简单类型论（或^{Higher-Order Logic}高阶逻辑）。这大致相当于简单类型的 λ -演算 [4]，并添加了一个相等运算符 (=)。它是一种抽象的、极简版的编程语言，其函数调用机制预示了函数式编程的出现。

1.1 类型

^{Type}类型可以是基本类型，例如 $\mathbb{Z}$ 、 $\mathbb{Q}$ 和 `Bool`，也可以是^{Total Function}全函数 $\sigma \rightarrow \tau$ ，其中 σ 和 τ 本身就是类型。类型指明了表达式可以求出何种值，它们引入了一个在数学中隐

式遵循的规则。原则上来说，没有什么可以阻止数学家陈述 $1 \in 2$ ，但类型规则会将其标记为可能有误。

从语义上讲，类型可以被视为集合。我们通常会定义类型 $\mathbb{Z}$ 、 $\mathbb{Q}$ 和 Bool ，以便它们忠实地刻画数学家的 $\mathbb{Z}$ 和 $\mathbb{Q}$ ，以及计算机科学家的布尔值，函数箭头 ($\rightarrow$) 也是如此。然而，尽管它们有相似之处，Lean 和数学却是截然不同的语言。Lean 的类型可以被^{Interpreted}解释为集合，但它们本身并非集合。

^{Higher-Order Type}
高阶类型是箭头 $\rightarrow$ 左侧包含嵌套的 $\rightarrow$ 的类型，例如类型 $(\mathbb{Z} \rightarrow \mathbb{Z}) \rightarrow \mathbb{Q}$ 。此类类型的值是接受其他函数作为参数的函数。因此， $(\mathbb{Z} \rightarrow \mathbb{Z}) \rightarrow \mathbb{Q}$ 是一个一元函数类型，它接受一个 $\mathbb{Z} \rightarrow \mathbb{Z}$ 类型的函数作为参数，并返回一个 $\mathbb{Q}$ 类型的值。

1.2 项

^{Term}
简单类型论的**项**，也称为表达式，包括以下几种：

- ^{Constant}**常量** c ;
- ^{Variable}**变量** x ;
- ^{Application}**应用** $t\ u$;
- ^{Anonymous Function}**匿名函数** $\text{fun } x \mapsto t$ (也称为 λ -表达式)。

上面的 t 和 u 表示任意项。我们也可以写作 $t : \sigma$ 来表示 t 具有类型 σ 。

每一种项的具体解释如下：

- 常量 $c : \sigma$ 是一个类型为 σ 的符号，它的含义在当前的全局^{Context}**语境**下是固定的。例如，一个算术理论可能包含常量 $0 : \mathbb{Z}$ 、 $1 : \mathbb{Z}$ 、 $\text{abs} : \mathbb{Z} \rightarrow \mathbb{N}$ 、 $\text{square} : \mathbb{N} \rightarrow \mathbb{N}$ 和 $\text{prime} : \mathbb{N} \rightarrow \text{Bool}$ 。常量包括函数（例如 abs ）和谓词（例如 prime ）。
- 变量 $x : \sigma$ 要么是^{Bound}**绑定的**，要么是^{Free}**自由的**。一个绑定的变量会引用一个匿名函数 $\text{fun } x : \sigma \mapsto t$ 中的输入 x 。反之，一个自由变量是在局部^{Context}**语境**下声明的，这一概念将在后面解释。
- 应用 $t\ u$ ，其中 $t : \sigma \rightarrow \tau$ 和 $u : \sigma$ ，是一个类型为 τ 的项，它表示将函数 t 应用于参数 u 的结果。例如，如果 t 是 abs ， u 是 0 ，则应用是 $\text{abs } 0$ 。除非它是一个复杂的项，否则不需要在参数周围加上括号，例如 $\text{prime } (\text{abs } 0)$ 。
- 给定一个项 $t : \tau$ ，一个匿名函数 $\text{fun } x : \sigma \mapsto t$ 表示一个类型为 $\sigma \rightarrow \tau$ 的^{Total Function}**全函数**，它将每个类型为 σ 的输入值 x 映射到函数体 t ，其中 x 可能出现在 t 中。因此， $\text{fun } x : \mathbb{Z} \mapsto \text{square } (\text{abs } x)$ 表示一个函数，它将 0 映射到 $\text{square } (\text{abs } 0)$ ，将 1 映射到 $\text{square } (\text{abs } 1)$ ，依此类推。变量 x 称为被 fun 所^{Binder}**绑定**。因此， fun 被称为**绑定器**。对于数学家来说，一个更熟悉的语法可能是 $x \mapsto \text{square } (\text{abs } x)$ ，没有 fun 关键字。

常量和变量在语法上看起来很相似，但它们被视为不同的。常量是全局声明的，而变量则通过 `fun` 或其他绑定器在局部引入。

应用和匿名函数互为镜像：匿名函数「构造」一个函数；应用则「解构」一个函数。如果我们将两者结合起来会发生什么？如果我们将一个类型为 σ 的参数 u 应用到匿名函数 $\text{fun } x : \sigma \mapsto t[x]$ ，其中 $t[x]$ 表示某个可能包含 x 的项，那么我们就得到项 $t[u]$ 。（这是一个稍微简化的视角。实际上，如果 $t[x]$ 包含一些绑定器，则可能需要重命名绑定变量以避免捕获 u 的自由变量。在 Lean 中，这种重命名会自动进行。）以下是一些应用匿名函数的示例：

```
(fun n : ℕ ↦ n) 4 产生 4
(fun n : ℕ ↦ square (square n)) 5 产生 square (square 5)
(fun y : ℤ ↦ 1) 0 产生 1
(fun x : ℤ ↦ (fun y : ℤ ↦ x)) 1 产生 fun y : ℤ ↦ 1
```

虽然我们的函数是一元函数（即它们只接受一个参数），但我们可以通过嵌套 `fun` 来构建 n 元函数，这需要使用一种名为^{Currying}柯里化的巧妙技巧。例如， $\text{fun } x : \sigma \mapsto (\text{fun } y : \tau \mapsto x)$ 表示类型为 $\sigma \rightarrow (\tau \rightarrow \sigma)$ 的函数，它接受两个参数并返回第一个参数。严格来说， $\sigma \rightarrow (\tau \rightarrow \sigma)$ 接受一个参数并返回一个函数，而该函数又接受一个参数。

应用的方式与之类似：若 $K := (\text{fun } x : \mathbb{Z} \mapsto (\text{fun } y : \mathbb{Z} \mapsto x))$ ，则 $K\ 1 = (\text{fun } y : \mathbb{Z} \mapsto 1)$ 且 $(K\ 1)\ 0 = 1$ 。函数 K 在 $K\ 1$ 中，如果只应用于一个参数，则被称为^{Partially Applied}部分应用，因为它可以接受一个额外的参数。

柯里化是一个非常有用的概念，我们可以省略大多数的括号：

```
 $\sigma \rightarrow \tau \rightarrow u$  表示  $\sigma \rightarrow (\tau \rightarrow u)$ 
 $t\ u\ v$  表示  $(t\ u)\ v$ 
 $\text{fun } x : \sigma \mapsto \text{fun } y : \tau \mapsto t$  表示  $\text{fun } x : \sigma \mapsto (\text{fun } y : \tau \mapsto t)$ 
```

我们还将使用以下缩写：

```
 $\text{fun } (x : \sigma) (y : \tau) \mapsto t$  表示  $\text{fun } x : \sigma \mapsto \text{fun } y : \tau \mapsto t$ 
 $\text{fun } x\ y : \sigma \mapsto t$  表示  $\text{fun } (x : \sigma) (y : \sigma) \mapsto t$ 
```

此外，在匿名函数 $\text{fun } x : \sigma \mapsto t$ 中，我们通常可以省略类型注解 $:\sigma$ ：

```
 $\text{fun } x \mapsto t$  表示  $\text{fun } x : \sigma \mapsto t$ 
```

然后，Lean 会尝试根据函数体 t 和匿名函数周围的语境推断其类型。类型推断可以简化符号，并节省一些按键。

在数学中，通常用中缀语法来书写二元运算符（例如， $x + y$ ）。这样的记法在 Lean 中也可以，它其实是柯里化函数应用的语法糖（例如，`add x y`）。

使用 Lean 的一种方法是用 `opaque` 命令声明我们需要的类型和常量。考虑以下声明：

```

opaque a : ℤ
opaque b : ℤ
opaque f : ℤ → ℤ
opaque g : ℤ → ℤ → ℤ

#check fun x : ℤ ↦ g (f (g a x)) (g x b)
#check fun x ↦ g (f (g a x)) (g x b)

```

前四行声明了四个常量 (a、b、f、g)，它们可用于构成项。最后两行使用 #check 命令对项进行类型检查并显示其类型。使用传统的数学符号，最后一行的项应写作 $x \mapsto g(f(g(a, x)), g(x, b))$ 。# 前缀表示诊断命令：这些命令可用于调试，但我们通常不会将它们保存在 Lean 文件中。

1.3 类型检查与类型推断

当 Lean 在解析一个项时，它会检查该项的类型是否正确。在此过程中，如果省略了绑定变量的类型，它就会尝试推断这些变量的类型，例如 `fun x ↦ square (square x)` 中 `x` 的类型。

对于简单类型论来说，类型检查和类型推断是可判定的问题。诸如重载（多个常量可以重用相同的名称，例如 $0 : \mathbb{N}$ 和 $0 : \mathbb{R}$ ）等高级特性可能会导致不可判定性 [25]。Lean 采用务实的、面向计算机科学的方法，并假设如果存在多种可能的类型，则数字 0、1、2、... 属于 $\mathbb{N}$ 类型。

Lean 的类型系统可以表示为一个形式系统。一个 ^{Formal System Judgment} **形式系统** 由 ^{Derivation Rule} **判断** 和用于生成判断的 **推导规则** 组成。类型判断的形式为 $C \vdash t : \sigma$ ，表示项 `t` 在局部语境 `C` 中具有类型 σ 。例如，判断 $\vdash \text{abs} : \mathbb{Z} \rightarrow \mathbb{N}$ 表示常数 `abs` 在空的局部语境中具有类型 $\mathbb{Z} \rightarrow \mathbb{N}$ 。

局部语境给出了 `t` 中未受 `fun` 绑定的变量的类型。它用于跟踪由 `t` **外部** 的绑定器绑定的变量。如果同一个变量 `x` 被绑定多次，则最后一次出现的变量会覆盖其他变量。例如，判断 $x : \mathbb{Z}, y : \mathbb{N}, y : \mathbb{Z} \vdash y : \mathbb{Z}$ 表示变量 `y` 在局部语境 $x : \mathbb{Z}, y : \mathbb{N}, y : \mathbb{Z}$ 中具有类型 $\mathbb{Z}$ 。

对于简单类型论而言，类型判断由四条类型规则产生，每种项对应一条：

$$\begin{array}{c}
 \frac{}{C \vdash c : \sigma} \text{CST} \quad \text{当 } c \text{ 是全局声明的，类型为 } \sigma \text{ 的常量} \\
 \\
 \frac{}{C \vdash x : \sigma} \text{VAR} \quad \text{当 } x : \sigma \text{ 是出现在 } C \text{ 中最右边的 } x \\
 \\
 \frac{C \vdash t : \sigma \rightarrow \tau \quad C \vdash u : \sigma}{C \vdash t u : \tau} \text{APP}
 \end{array}$$

$$\frac{C, x : \sigma \vdash t : \tau}{C \vdash (\text{fun } x : \sigma \mapsto t) : \sigma \rightarrow \tau} \text{FUN}$$

每条规则都有零个或多个前提（水平线上方）、一个结论（水平线下方），以及可能的一个附加条件（水平线右侧）。前提是类型判断，而附加条件则是规则中出现的数学变量的任意数学条件。为了证明前提，需要继续向上推导，我们稍后就会看到。至于附加条件，我们可以运用所有数学知识来证明它们为真。

前两条规则分别标记为 CST 和 VAR，没有前提，但它们有附加条件，这些条件必须满足才能应用规则。后两条规则以一或两个判断作为前提，并产生一个新的判断。FUN 是唯一修改局部上下文的规则：当我们进入一个匿名函数的函数体 t 时，我们需要记录绑定变量 x 的存在，以及它的类型，以便在 t 中遇到 x 时做好准备。

我们可以使用此规则系统来证明给定项的类型正确，方法是：通过逆向（即向上）推导并应用规则，构建一个类型判断的^{Formal Derivation}**形式推导**。与现实中的树一样，推导树的根节点位于底部。推导的判断出现在根节点处，每个分支都以应用一个无前提的规则结束。规则的应用由水平线和标签表示。以下类型推导确定项 $\text{fun } x : \mathbb{Z} \mapsto \text{abs } x$ 在任意局部上下文 C 中具有类型 $\mathbb{Z} \rightarrow \mathbb{N}$ ：

$$\frac{\frac{}{C, x : \mathbb{Z} \vdash \text{abs} : \mathbb{Z} \rightarrow \mathbb{N}} \text{CST} \quad \frac{}{C, x : \mathbb{Z} \vdash x : \mathbb{Z}} \text{VAR}}{C, x : \mathbb{Z} \vdash \text{abs } x : \mathbb{N}} \text{APP}$$

$$\frac{}{C \vdash (\text{fun } x : \mathbb{Z} \mapsto \text{abs } x) : \mathbb{Z} \rightarrow \mathbb{N}} \text{FUN}$$

从根节点向上阅读证明，注意局部语境是如何贯穿始终的，以及它是如何被 FUN 规则扩展的。该规则将 fun 绑定的变量移动到局部语境，使得 VAR 的应用可以在树的上层进行。如果变量 x 已经在 C 中声明，则在进入 fun 表达式后，它将被 $x : \mathbb{Z}$ 覆盖。

总而言之，类型系统由推导规则组成，这些规则可以 (1) 用其数学变量的任意值进行实例化，以及 (2) 连接起来形成推导树。

下面是第二个示例，这次从一个空的局部语境开始：

$$\frac{\frac{\frac{}{x : \mathbb{Z}, y : \mathbb{Z} \vdash x : \mathbb{Z}} \text{VAR}}{x : \mathbb{Z} \vdash (\text{fun } y : \mathbb{Z} \mapsto x) : \mathbb{Z} \rightarrow \mathbb{Z}} \text{FUN}}{\vdash (\text{fun } x : \mathbb{Z} \mapsto \text{fun } y : \mathbb{Z} \mapsto x) : \mathbb{Z} \rightarrow \mathbb{Z} \rightarrow \mathbb{Z}} \text{FUN}$$

到目前为止，两个例子都是^{Well-Typed}**类型良好的**。如果我们从一个类型错误的项开始，或者在推导树根部的判断中指定了错误的类型或语境，我们就会发现无法完成推导。推导构成了对项而言类型良好的证明，并且在给定的语境中具有指定的类型。

上述类型系统仅检查项是否类型良好，并不检查类型是否^{Well-formed}形式良好。例如，给定一元类型构造子 `List`, `List ℤ`（整数列表的类型）是形式良好的，而 `ℤ List` 和 `List List` 是^{Ill-formed}形式不良的。对于简单类型理论，形式良好性检查很容易：只应使用声明的类型构造子，并且每个 n 元类型构造子都应传递恰好为 n 类型的参数。

1.4 类型居留

给定一个类型 σ , ^{Type Inhabitation}类型居留问题是指在空的局部语境中寻找该类型的一个^{Inhabitant}居留元，即一个类型为 σ 的项。这练习看似毫无意义，但正如我们将在第 4 章中看到的那样，这个问题与寻找命题的证明密切相关。类似「寻找一个类型为 σ 的项」这种看似简单的练习，其实是掌握定理证明的好方法。

虽然这个问题总体上是不可判定的，但只要策略得当，我们就能取得很大进展。要创建给定类型的项，请从占位符 `_` 开始，并递归地应用以下两个步骤的组合：

1. 如果类型的形式为 $\sigma \rightarrow \tau$ ，则可能的居留项是一个匿名函数，形式为 `fun x :  $\sigma \mapsto \_$` ，其中 `_` 是一个类型为 τ 的缺失项的占位符。Lean 会将 `_` 标记为错误；如果在 Visual Studio Code 中将鼠标悬停在该项上，提示将显示缺失项的类型以及在局部语境中声明的任何变量。
2. 给定一个类型 σ （可以是函数类型），你可以使用任何常量 c 或类型为 $\tau_1 \rightarrow \dots \rightarrow \tau_n \rightarrow \sigma$ 的变量 x 来构建一个类型为 σ 的项。对于每个参数，你需要放置一个占位符，从而得到 `$c \_ \dots \_$` 或 `$x \_ \dots \_$` 。

占位符可以递归地使用相同的策略来消除。

作为示例，我们将应用该策略来查找以下类型的项：

$$(\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow ((\beta \rightarrow \alpha) \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma$$

最初，只有步骤 1 适用，其中：

$$\sigma := \alpha \rightarrow \beta \rightarrow \gamma \quad \text{且} \quad \tau := ((\beta \rightarrow \alpha) \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma$$

（回想一下， $\rightarrow$ 是右结合的。）这得到了项 `fun f  $\mapsto \_$` ，它拥有正确的类型，但左边有一个占位符。由于参数 f 的类型为 σ ，它是一个函数类型，因此使用名称 f 来表示它是合理的。然后，我们继续递归地处理类型为 τ 的占位符。同样，只有步骤 1 可行，因此我们最终得到项 `fun f g  $\mapsto \_$` ，其中 g 的类型为 $(\beta \rightarrow \alpha) \rightarrow \beta$ ，而占位符的类型为 $\alpha \rightarrow \gamma$ 。步骤 1 的第三次应用得到 `fun f g a  $\mapsto \_$` ，其中 a 的类型为 α ，占位符的类型为 γ 。

此时，步骤 1 不再适用。让我们看看步骤 2 是否可行。占位符周围的语境包含以下变量：

$$f : \alpha \rightarrow \beta \rightarrow \gamma, \quad g : (\beta \rightarrow \alpha) \rightarrow \beta, \quad a : \alpha$$

回想一下，我们正在尝试构建一个 γ 类型的项。我们唯一能用来实现这一点的变量是 f ：它接受两个参数并返回一个 γ 类型的值。因此，我们用项 $f _ _$ 替换占位符，其中，两个新的占位符代表两个缺失的参数。将所有内容放在一起，我们现在得到了项 $\text{fun } f \text{ } g \text{ } a \mapsto f _ _$ 。

根据 f 的类型，占位符的类型分别为 α 和 β 。第一个占位符很容易填充，同样使用步骤 2，只需提供 α 类型的 a ，不带任何参数即可。对于第二个占位符，我们将步骤 2 与变量 g 结合使用，它是 β 的唯一来源。由于 g 接受一个参数，因此我们必须提供一个占位符。这意味着我们当前的项是 $\text{fun } f \text{ } g \text{ } a \mapsto f \text{ } a \text{ } (g _)$ 。

我们快完成了。剩下的唯一占位符的类型是 $\beta \rightarrow \alpha$ ，它是 g 的参数类型。应用步骤 1，我们将占位符替换为 $\text{fun } b \mapsto _$ ，其中 $_$ 的类型为 α 。在这里，我们可以简单地提供 a 。我们的最终项是 $\text{fun } f \text{ } g \text{ } a \mapsto f \text{ } a \text{ } (g \text{ } (\text{fun } b \mapsto a))$ 。

上述推导过程冗长乏味，但却具有确定性：在每个点，步骤 1 或步骤 2 中的其中一项适用，但并非同时适用。一般来说，情况并非总是如此。对于其他一些类型，我们可能会遇到死胡同，需要回溯。我们也可能会完全失败，无处可回溯。

关键在于，项应始终保持语法正确。我们在 Visual Studio Code 中唯一应该看到的红色下划线应该出现在占位符下方。一般来说，开发软件的一个好的原则是，从可以编译的程序开始，进行尽可能小的更改，以获得新的可以编译的程序，并重复此过程，直到程序完成。

1.5 新引入的 Lean 结构总结

在本章和大多数其他章节的末尾，我们都会简要概述本章引入的构造。某些语法具有多重含义，我们将逐步介绍。有关详细信息，请参阅 *Lean 4 参考手册* [1]、*Lean 4 定理证明* [?] 教程以及 *mathlib* 文档¹。

诊断命令

`#check` 检查并打印一个项的类型

声明

`opaque` 声明一个未指定的新常量或类型

¹https://leanprover-community.github.io/mathlib4_docs/

程序与定理

我们继续学习 Lean 的基础知识，重点关注程序和定理，但暂不进行任何证明。具体来说，我们将回顾如何定义新的类型和函数，以及如何将其预期的性质表述为定理。

2.1 类型定义

Lean 归纳构造演算的一个显著特点是它内置了对归纳类型的支持。^{Inductive Type}归纳类型是一种类型，其值是通过应用称为^{Constructor}构造子的特殊常量来构建的。归纳类型是在程序中表示非循环数据的一种简洁方式。你可能还知道一些其他的、大致相同的名称，包括^{Algebraic Data Type}代数数据类型、^{Inductive Data Type}归纳数据类型、^{Freely Generated Data Type}自由生成的数据类型、^{Recursive Data Type}递归数据类型和^{Data Type}数据类型。

2.1.1 自然数的定义

归纳类型的「Hello, World!」示例是自然数类型 Nat ($\mathbb{N}$)。在 Lean 中，它可以定义如下：

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

第一行向世界声明我们引入了一个名为 Nat 的新类型，用于表示自然数。第二行和第三行声明了两个新的构造子， $\text{Nat.zero} : \text{Nat}$ 以及 $\text{Nat.succ} : \text{Nat} \rightarrow \text{Nat}$ ，它们可用于构造类型为 Nat 的值。按照计算机科学和逻辑学中既定的惯例，计数从 0 开始。第二个构造子让这个归纳定义变得有趣——它需要一个类型为 Nat 的参数来产生一个类型为 Nat 的值。以下这些项

Nat.zero

```
Nat.succ Nat.zero
Nat.succ (Nat.succ Nat.zero)
⋮
```

表示类型为 `Nat` 的不同值——0、0 的后继数、0 的后继数的后继数，以此类推。这种记法被称为 ^{Unary}一进制记法，也叫 ^{Peano}皮亚诺记法，以逻辑学家 ^{Giuseppe Peano}朱塞佩·皮亚诺的名字命名。若想了解在 Lean 中对皮亚诺数的另一种解释（还有一些令人惊叹的电子游戏画面），请参阅凯文·巴扎德（Kevin Buzzard）的文章《计算机能证明定理吗？》。¹

类型声明的通用格式为

```
inductive 类型名 (参数1 : 类型1) ... (参数k : 类型k) : Type
where
  | 构造子名称1 : 构造子类型1
  ⋮
  | 构造子名称n : 构造子类型n
```

对于自然数，我们可以让 Lean 使用大家熟悉的名称 0、1、2 等等，实际上，预定义的 `ℕ` 类型就提供了这样的语法糖。不过，使用更详细的表示法能让我们更清楚地了解 Lean 的类型定义。

在本章配套的 Lean 文件中，`Nat` 的定义被放在一个命名空间内，通过 `namespace MyNat` 和 `end MyNat` 进行界定，以将其作用域限制在文件的某一部分。在 `end MyNat` 之后，出现的所有 `Nat`、`Nat.zero` 或 `Nat.succ` 都将指向 Lean 预定义的自然数类型。同样，整个文件被放在 `LoVe` 命名空间下，以避免与 Lean 现有库中的名称冲突。

我们可以在 Lean 的任何位置使用 `#print` 命令来查看之前的定义。例如，在 `MyNat` 命名空间内输入 `#print Nat`，会显示如下信息：

```
inductive LoVe.MyNat.Nat : Type
number of parameters: 0
constructors:
LoVe.MyNat.Nat.zero : Nat
LoVe.MyNat.Nat.succ : Nat → Nat
```

本书侧重于自然数，这也是本书许多内容偏向计算机科学的体现之一。对于数论学家来说，他们可能更关注整数 $\mathbb{Z}$ 和有理数 $\mathbb{Q}$ ；分析学家则更希望处理实数 $\mathbb{R}$ 和复数 $\mathbb{C}$ 。然而，自然数在计算机科学中无处不在，并且作为归纳类型有着非常简单的定义。正如我们将在第 12 章中看到的，自然数还可以用来构建其他类型。

¹<http://chalkdustmagazine.com/features/can-computers-prove-theorems/>

2.1.2 算数表达式的定义

下一个例子会用到整数。如果我们要实现一个计算器程序或一种编程语言，那么就需要定义一种类型，用来将算术表达式表示为抽象语法树。下面的例子展示了如何在 Lean 中实现这一点：

```
inductive AExp : Type where
  | num : ℤ → AExp
  | var : String → AExp
  | add : AExp → AExp → AExp
  | sub : AExp → AExp → AExp
  | mul : AExp → AExp → AExp
  | div : AExp → AExp → AExp
```

从数学上讲，该定义等价于用以下生成规则来归纳地定义类型 AExp：

1. 对于每个整数 i ，项 $\text{AExp.num } i$ 都是一个 AExp 值。（直观上，构造子 AExp.num 就是把一个整数「包装」为一个算术表达式。）
2. 对于每个字符串 x ，项 $\text{AExp.var } x$ 都是一个 AExp 值。
3. 若 e_1 和 e_2 都是 AExp 值，那么 $\text{AExp.add } e_1 \ e_2$ 、 $\text{AExp.sub } e_1 \ e_2$ 、 $\text{AExp.mul } e_1 \ e_2$ 和 $\text{AExp.div } e_1 \ e_2$ 也都是 AExp 值。

上述定义穷尽了所有情况。AExp 所有可能的取值都只能通过生成规则 1 到 3 构造得到。此外，使用不同生成规则构造出的 AExp 值彼此不同。这两条归纳类型的性质可以用 Joseph Goguen 提出的口号来概括：「无垃圾，无混淆」。

将上面 Lean 中简洁的 AExp 定义与实现相同功能的 Java 程序作对比也许会很有启发意义。该 Java 程序包含一个接口和六个实现该接口的类，分别对应于 AExp 类型及其六个构造子：

```
public interface AExp { }

public class Num implements AExp {
    public int num;

    public Num(int num) { this.num = num; }
}

public class Var implements AExp {
    public String var;

    public Var(String var) { this.var = var; }
}
```

```
public class Add implements AExp {
    public AExp left;
    public AExp right;

    public Add(AExp left, AExp right)
    { this.left = left; this.right = right; }
}

public class Sub implements AExp {
    public AExp left;
    public AExp right;

    public Sub(AExp left, AExp right)
    { this.left = left; this.right = right; }
}

public class Mul implements AExp {
    public AExp left;
    public AExp right;

    public Mul(AExp left, AExp right)
    { this.left = left; this.right = right; }
}

public class Div implements AExp {
    public AExp left;
    public AExp right;

    public Div(AExp left, AExp right)
    { this.left = left; this.right = right; }
}
```

在 Python 中，同样的类声明如下：

```
class AExp:
    pass

class Num(AExp):
```

```

def __init__(self, num):
    self.num = num

class Var(AExp):
    def __init__(self, var):
        self.var = var

class Add(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

class Sub(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

class Mul(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

class Div(AExp):
    def __init__(self, left, right):
        self.left = left
        self.right = right

```

2.1.3 列表的定义

我们考虑的下一个类型是有限列表：

```

inductive List ( $\alpha$  : Type) where
| nil   : List  $\alpha$ 
| cons  :  $\alpha \rightarrow \text{List } \alpha \rightarrow \text{List } \alpha$ 

```

Polymorphism

该类型是**多态的**：它由一个类型参数 α 参数化，我们可以用具体类型对其进行实例化。例如， $\text{List } \mathbb{Z}$ 表示整数列表的类型， $\text{List } (\text{List } \mathbb{R})$ 表示实数列表的列表类型。类型构造子 `List` 以一个类型作为参数并返回一个类型。类似于 Java 的泛型和 C++ 的模板，多态性是一种提供参数化类型的机制。

以下命令显示了构造子的类型：

```
#check List.nil
#check List.cons
```

Lean 的内置列表提供了语法糖来编写列表：

```
[] 表示 List.nil
x :: xs 表示 List.cons x xs
[x1, ..., xn] 表示 x1 :: ... :: xn :: []
```

和所有其他二元运算符一样，`::` 运算符的结合优先级低于函数应用。因此，`f x :: List.reverse ys` 会被解析为 `(f x) :: (List.reverse ys)`。避免不必要的括号是一种良好的编程习惯，因为括号会降低可读性。此外，在中缀运算符两侧加上空格，有助于表达正确的优先级。

函数式程序员通常会用 `xs`、`ys`、`zs` 这样的名字来表示列表，虽然在 Lean 中 `l` 也很常见。由于列表包含多个元素，使用复数形式是很自然的。例如，猫的列表可以命名为 `cats`，猫的列表的列表可以命名为 `catss`。当我们将一个非空列表表示为表头和表尾时，通常会写作 `x :: xs` 或 `cat :: cats`。

2.2 函数定义

如果我们只想声明一个函数，那么可以使用 `opaque` 命令（章节 1.2）。但通常来说，我们想要**定义**函数的行为，那么可以使用 `def` 命令。由于 Lean 的根基是函数式变成，函数会定义为可求值的数学表达式，而非修改某些状态的指令时程序。与此相应，Lean 并不使用 `while` 或 `for` 来迭代，而是使用递归作为遍历数据的主要机制。

2.2.1 在自然数上递归

我们来看一个简单的例子，看看递归是如何工作的。斐波那契数的 Lean 定义如下：

```
def fib : ℕ → ℕ
| 0      => 0
| 1      => 1
| n + 2 => fib (n + 1) + fib n
```

左侧的模式对应于函数的参数。在这里，`fib` 的声明中只有一个类型为 `ℕ` 的参数，因此我们只需对一个自然数进行**模式匹配**。当输入具有给定的形式时，相应的模式匹配就会被触发。例如，如果输入是 1，那么模式 1 就会匹配它，并计算对应的右

侧表达式。如果输入是 5，则会触发模式 $n + 2$ ，此时 $n := 3$ 。然后将 n 取值为 3 计算右侧表达式，得到 $\text{fib } 4 + \text{fib } 3$ 。

递归定义的一般形式为：

```
def 名称(参数1 : 类型1) ... (参数m : 类型m) : 类型
  | 模式1 => 结果1
    .....
  | 模式n => 结果n
```

参数 参数₁ 到 参数_m 不能用于模式匹配，只有在 类型 中声明的剩余参数才能进行模式匹配。如果一行中提供了多个模式，它们之间用逗号分隔。模式中 can 包含变量，这些变量在对应的右侧表达式中是可见的，也可以包含构造子。

基本的自然数算术运算，如加法、乘法和幂运算，都可以通过递归来定义。当然，这些运算在 Lean 中已经预定义了（分别为 $+$ 、 $*$ 和 $^$ ），但自己实现一遍也是很有意义的练习。我们先从加法开始：

```
def add : ℕ → ℕ → ℕ
  | m, Nat.zero    => m
  | m, Nat.succ n => Nat.succ (add m n)
```

我们对两个参数同时进行模式匹配，区分第二个参数为零和非零的情况。每一次对 `add` 的递归调用，都会从第二个参数中剥离出一个 `Nat.succ` 构造子。Lean 允许我们用 `0` 和 `n + 1` 这样的语法糖来代替 `Nat.zero` 和 `Nat.succ n`。

我们可以使用 `#eval` 或 `#reduce` 来计算 `add` 应用于数字的结果：

```
#eval add 2 7
#reduce add 2 7
```

两个命令都会在 Visual Studio Code 中打印出 9。`#eval` 会使用一个经过优化的解释器，而 `#reduce` 则会使用 Lean 的推理内核，它的效率更低。

当定一个函数时，最佳实践是每次提供一些测试来确保函数的行为符合预期。你甚至可以在 Lean 文件中保留 `#eval` 或 `#reduce` 作为文档。

乘法的定义与加法类似：

```
def mul : ℕ → ℕ → ℕ
  | _, Nat.zero    => Nat.zero
  | m, Nat.succ n => add m (mul m n)
```

下划线 (`_`) 表示未使用的变量。我们本可以使用一个名字（例如 `m`），但 `_` 能更好地记录我们的意图。

下面的 `#eval` 命令会打印 14，符合预期：

```
#eval mul 2 7
```

指数（「 m 的 n 次方」）有多种定义的方式。我们的第一个方案在结构上与乘法的定义完全相同：

```
def power : ℕ → ℕ → ℕ
| _, Nat.zero    => 1
| m, Nat.succ n => mul m (power m n)
```

我们可以将第一个参数 m 提取出来，并作为 参数 放在函数名旁边，位于引入函数类型的冒号之前（排除该参数 m ）：

```
def powerParam (m : ℕ) : ℕ → ℕ
| Nat.zero    => 1
| Nat.succ n => mul m (powerParam m n)
```

从外部看，这两个定义没有区别。事实上，我们已经在 List 构造子的类型参数 α 中见过这种语法了（第 2.1 节）。

还有另一种定义方式，即先引入一个通用的迭代器，然后使用正确的参数来调用它：

```
def iter (α : Type) (z : α) (f : α → α) : ℕ → α
| Nat.zero    => z
| Nat.succ n => f (iter α z f n)

def powerIter (m n : ℕ) : ℕ :=
  iter ℕ 1 (mul m) n
```

`iter` 函数接收一个类型 α ，一个「零」值 z ，一个一元函数 f ，以及一个自然数 n ，并计算出

$$\underbrace{f(f(\dots(f\ z)\dots))}_{n\text{次}}$$

它是高阶函数的一个例子：一种将另一个函数（此处为 f ）作为参数的函数。在函数式编程中，函数被视为与其他对象一样的普通对象，可以用作参数或函数的返回结果。在这里，我们使用 `iter` 来计算 m 的 n 次幂：

$$\underbrace{\text{mul } m (\text{mul } m (\dots (\text{mul } m\ 1) \dots))}_{n\text{次}}$$

最后，请注意 `powerIter` 的定义不是递归的。对于不使用模式匹配的非递归函数，其语法简单如下：

```
def name (参数1 : 类型1) ... (参数m : 类型m) : 类型 :=
  结果
```

2.2.2 算术表达式上的递归

回到算术表达式类型，如果我们想在 Java 中实现一个 `eval` 函数，我们可能会将其作为 `AExp` 接口的一部分，并在每个子类中实现它。对于 `Add`、`Sub`、`Mul` 和 `Div`，我们会递归地在 `left` 和 `right` 对象上调用 `eval`。

在 Lean 中，其语法非常简洁。我们定义一个函数，并使用模式匹配来区分这六种情况：

```
def eval (env : String → ℤ) : AExp → ℤ
| AExp.num i      => i
| AExp.var x      => env x
| AExp.add e1 e2 => eval env e1 + eval env e2
| AExp.sub e1 e2 => eval env e1 - eval env e2
| AExp.mul e1 e2 => eval env e1 * eval env e2
| AExp.div e1 e2 => eval env e1 / eval env e2
```

请注意，此函数是高阶函数：它接收一个函数 `env`，该函数表示一个将值分配给变量的「Environment环境」。在 `AExp.var` 情况下，环境用于求值变量，并在递归情况 (`AExp.add`、`AExp.sub`、`AExp.mul` 和 `AExp.div`) 中传递。

也许你会担心 `AExp.div` 情况下的除以零问题。让我们看看 `#eval` 对此有何评价，使用一个将 7 分配给所有变量的环境：

```
#eval eval (fun x ↦ 7) (AExp.div (AExp.var "y") (
  AExp.num 0))
```

输出结果是 0。在 Lean 中，除法被方便地定义为一个全函数，当分母为零时返回零。关于为什么这并不危险的清晰解释，请参阅 Buzzard 的博客。²

2.2.3 列表上的递归

列表上的递归函数可以用类似的方式定义：

```
def append (α : Type) : List α → List α → List α
| List.nil,      ys => ys
| List.cons x xs, ys => List.cons x (append α xs ys)
```

`append` 函数接受三个参数：一个类型 α 和两个类型为 `List  $\alpha$` 的列表。

```
#eval append ℕ [3, 1] [4, 1, 5]
#eval append _ [3, 1] [4, 1, 5]
```

通过传入占位符 `_`，我们让 Lean 从其他两个参数的类型中推断出类型 $\mathbb{N}$ 。

为了使类型参数 α 成为隐式参数，我们可以将其放在花括号 `{ }` 中：

²<https://xenaproject.wordpress.com/2020/07/05/division-by-zero-in-type-theory-a-faq/>

```
def appendImplicit {α : Type} : List α → List α → List
  α
| List.nil,      ys => ys
| List.cons x xs, ys => List.cons x (appendImplicit xs
  ys)

#eval appendImplicit [3, 1] [4, 1, 5]
```

@ 符号可用于使隐式参数显式化。有时这对于指导 Lean 的解析器是必要的。

```
#eval @appendImplicit N [3, 1] [4, 1, 5]
#eval @appendImplicit _ [3, 1] [4, 1, 5]
```

我们可以在定义中使用语法糖，无论是在 => 左侧的模式中，还是在右侧：

```
def appendPretty {α : Type} : List α → List α → List α
| [],      ys => ys
| x :: xs, ys => x :: appendPretty xs ys
```

在 Lean 的标准库中，追加函数是一个名为 ++ 的中缀运算符。我们可以用它来定义一个反转列表的函数：

```
def reverse {α : Type} : List α → List α
| []      => []
| x :: xs => reverse xs ++ [x]
```

2.3 定理陈述

Lean 既是证明助手又是编程语言的原因在于，我们可以对定义的类型和常量陈述定理，并证明它们成立。在交互式定理证明中，术语**定理**、**引理**、**推论**、**事实**、**性质**以及**真陈述**的使用或多或少是可以互换的。类似地，**命题**、**逻辑公式**和**陈述**的意思也都是一样的。

在 Lean 中，命题仅仅是 Prop 类型的项。这与一阶逻辑形成对比，在一阶逻辑中，传统上在语法中区分项和公式。可以证明的命题被称为定理（或引理、推论等）；否则，它就是一个非定理或假陈述。数学家有时将术语「命题」作为定理的同义词（例如，「命题 3.14」），但在形式逻辑中，命题也可以是假的。

以下是关于第 2.2 节中定义的增加、乘法和列表反转操作的真陈述示例：

```
theorem add_comm (m n : N) :
  add m n = add n m :=
  sorry
```

```

theorem add_assoc (l m n : ℕ) :
  add (add l m) n = add l (add m n) :=
  sorry

theorem mul_comm (m n : ℕ) :
  mul m n = mul n m :=
  sorry

theorem mul_assoc (l m n : ℕ) :
  mul (mul l m) n = mul l (mul m n) :=
  sorry

theorem mul_add (l m n : ℕ) :
  mul l (add m n) = add (mul l m) (mul l n) :=
  sorry

theorem reverse_reverse {α : Type} (xs : List α) :
  reverse (reverse xs) = xs :=
  sorry

```

其一般形式为

```

theorem name (参数1 : 类型1) ... (参数m : 类型m) :
  陈述 :=
  证明

```

`:=` 符号将定理的陈述与其证明隔开。`theorem` 的语法与没有模式匹配的 `def` 命令非常相似，用 `statement` 代替 `type`，用 `proof` 代替 `result`。在上面的例子中，我们放置了标记 `sorry` 作为实际证明的占位符。这个标记字面上是向未来的读者和 Lean 表示道歉，因为缺乏证明。这也是值得担心的一个理由，直到我们设法消除它。在第 3 章和第 4 章中，我们将看到如何实现这一点。

带有 `sorry` 证明的 `theorem` 命令的直观语义是：「这个命题应该是可以证明的，但我还没有进行证明——抱歉。」有时，我们想表达一个相关的想法，即：「让我们假设这个命题成立。」Lean 为此提供了 `axiom` 命令，通常与 `opaque` 结合使用。例如：

```

opaque a : ℤ
opaque b : ℤ

axiom a_less_b :
  a < b

```

在 `opaque` 命令之后，除了它们的类型之外，我们没有关于 `a` 和 `b` 的任何信息。公理指定了它们应该具有的性质。该命令的通用格式为：

```
axiom name (参数1 : 类型1) ... (参数m : 类型m) :  
    陈述
```

公理是危险的，因为它们可能导致逻辑不一致，从而使我们能够推导出 `False`。例如，如果我们添加第二个公理陈述 `a > b`，我们就可以很容易地推导出 `False`。交互式定理证明的历史充满了不一致的公理。这里有一个轶事：在 2020 年的「认证程序与证明」会议上，一篇提交的论文因为一个有缺陷的公理而被拒绝，其中一位同行评审员从中推导出了 `False`。³因此，我们通常会避免使用公理。

从 Lean 的角度来看，带有 `sorry` 证明的定理实际上就是一个公理，可能会破坏逻辑的一致性。为了防止误解，最好只在开发证明时将 `sorry` 作为临时措施，而不是将其作为更明确且诚实的 `axiom` 的替代品。

2.4 新引入的 Lean 结构总结

诊断命令

<code>#eval</code>	使用优化的解释器执行一个项
<code>#print</code>	打印一个常量的定义
<code>#reduce</code>	使用 Lean 的 Inference Kernel 推断内核执行一个项

声明

<code>axiom</code>	陈述一个公理
<code>def</code>	定义一个新的常量
<code>inductive</code>	引入一个类型及其构造子
<code>namespace ... end</code>	Named Scope 在命名作用域内收集声明
<code>theorem</code>	陈述一个定理及其证明

证明命令

<code>sorry</code>	代表缺失的证明或定义
--------------------	------------

³该论文的一位作者报告说，该公理现在已经经过修订并作为定理推导出来。整个形式化开发现在已不含公理。修订后的论文已在「计算机辅助验证 2020」会议上被接受 [9]。

逆向证明

Tactic **策略**是在^{Goal}**目标**(即要证明的^{Proposition}**命题**)上操作的过程, 要么完全证明它, 要么产生新的子目标, 或者失败。当我们陈述一个定理时, 定理的陈述就是初始目标。一旦使用合适的策略消除了所有(子)目标, 证明就完成了。此处介绍的大多数策略在^{Theorem Proving in Lean 4}**《Lean 4 定理证明》**第 5 章中有更详细的说明 [2]。

Tactic **策略**是一种^{Backward}**逆向**证明机制。它们从^{Goal}**目标**出发, 向前推进到已证明的定理。考虑定理 a 、 $a \rightarrow b$ 、 $b \rightarrow c$ 以及目标 $\vdash c$ 。与类型判断类似, 目标用符号 $\vdash$ 标识。一个非形式的逆向证明如下:

为了证明 c , 由 $b \rightarrow c$ 可知, 只需证明 b 即可。
为了证明 b , 由 $a \rightarrow b$ 可知, 只需证明 a 即可。
为了证明 a , 我们直接使用 a 。 □

^{Backward Proof}**逆向证明**的显著特征是使用了「^{it suffices to}**只需证明**」这一短语。请注意我们是如何从一个^{Goal}**目标**变换到另一个目标 ($\vdash c$ 、 $\vdash b$ 、 $\vdash a$), 直到没有目标需要证明为止的。相比之下, ^{Forward Proof}**正向证明**会从定理 a 开始, 每次处理一个定理, 最终推导出所需的定理 c :

由 a 与 $a \rightarrow b$ 可得 b 。
由 b 与 $b \rightarrow c$ 可得 c , 正如预期。 □

^{Forward Proof}**正向证明**只操作^{Theorem}**定理**, 不操作^{Goal}**目标**。我们将在第 4 章深入学习正向证明。
非形式化的证明有时会混合使用这两种风格。只要能通过「为了证明....., 只需证明.....」之类的措辞清晰地识别出^{Backward}**逆向**步骤, 这种混合就是可行的。在目标前面加上 $\vdash$ 符号有助于提醒这些断言尚未被证明。

你可能熟悉的另一种表达证明的格式是^{Natural Deduction}**自然演绎**。下面给出了对应于上述证明的自然演绎推导:

$$\begin{array}{c}
 \frac{\frac{\frac{}{\vdash b \rightarrow c}}{\vdash b \rightarrow c} \quad \frac{\frac{\vdash a \rightarrow b \quad \vdash a}{\vdash b} \rightarrow E}{\vdash c} \rightarrow E
 \end{array}$$

从上往下读时，该推导对应于^{Forward Proof}**正向证明**；从下往上读时，它对应于^{Backward Proof}**逆向证明**。

3.1 策略模式

在第 2 章中，每当需要证明时，我们只是简单地使用了 `sorry` 占位符。对于^{Tactical Proof}**策略证明**，我们现在将写下 `by` 来进入^{Tactic Mode}**策略模式**。在此模式下，我们可以调用一系列^{Tactic}**策略**。

策略在目标上操作，目标由我们想要证明的命题 Q 和^{Local Context}**局部语境** C 组成。局部语境由形如 $x : \sigma$ 的变量声明和形如 $h : P$ 的^{Hypothesis}**假设**组成。我们用 $C \vdash Q$ 表示一个目标，其中 C 是变量和假设的列表，而 Q 是^{Goal}**目标**的^{Target}**目的**。

为了让事情更具体，请考虑以下 Lean 示例：

```

theorem fst_of_two_props :
  ∀ a b : Prop, a → b → a :=
  by
    intro a b
    intro ha hb
    apply ha
  
```

请注意，蕴含箭头 $\rightarrow$ 是右结合的；这意味着 $a \rightarrow b \rightarrow a$ 与 $a \rightarrow (b \rightarrow a)$ 相同。直观地说，该陈述的意思是「 a 蕴含了 b 蕴含 a 」，或者等价地，「 a 和 b 蕴含 a 」。

我们将逐行审阅这个证明。

`by`

^{Keyword}**关键字** `by` 表示我们进入了^{Tactic Mode}**策略模式**，在该模式下我们可以通过^{Tactic}**策略**来指定证明。随后是策略，每行一个，并且所有策略的缩进相同。

如果我们将光标置于 `by` 关键字上，Visual Studio Code 会报告当前的目标仅仅是原始定理的陈述：

$\vdash \forall a b : \text{Prop}, a \rightarrow b \rightarrow a$

接下来的策略

`intro a b`

告诉 Lean 固定两个^{Free Variable}自由变量 a 和 b ，它们对应于同名的两个^{Bound Variable}绑定变量。该策略模仿了数学家在纸上的工作方式：为了证明一个 $\forall$ 量化的命题，只需证明该命题对于绑定变量的某些任意但固定的值成立即可。目标变为：

$$a\ b : \text{Prop} \vdash a \rightarrow b \rightarrow a$$

其中命题 a 和 b 在目标的^{Local Context}局部语境(即 $\vdash$ 符号左侧)中声明。我们通常为自由变量使用与绑定变量相同的名称，正如这里所做的，但这并不是强制性的。通常，使用唯一的名称并避免遮蔽现有变量是良好的做法。

除了使用两个变量名调用一次 `intro` 之外，我们也可以调用两次 `intro`，即 `intro a` 后面跟着 `intro b`。通常，每个带有 n 个参数的 `intro` 调用都会提取接下来的 n 个量化的变量。

`intro` 策略不仅限于量化变量。在我们的证明脚本中，

```
intro ha hb
```

告诉 Lean 将 (来自 $a \rightarrow b \rightarrow$ 的) 前提 a 和 b 移至局部语境，并称这些假设为 ha 和 hb 。

事实上，为了证明一个蕴含式，只需将其左侧作为假设，并证明其右侧即可。于是目标变为：

$$a\ b : \text{Prop},\ ha : a,\ hb : b \vdash a$$

习惯上会在假设名称前加前缀 h 。

最后，策略

```
apply ha
```

告诉 Lean 将名为 ha 的假设 a 与目标 $\vdash a$ 进行匹配。由于 a 在语法上等价于 a ，匹配成功。这就完成了证明。

非形式化地，我们可以用类似纸笔数学的风格，将证明写成如下形式：

令 a 和 b 为命题。

假设 $(ha)\ a$ 和 $(hb)\ b$ 为真。

为了证明 a ，我们需要使用假设 ha 。 □

(数学家可能会使用数字标签，如 (1) 和 (2)，而非具有描述性的假设名称。)

回到 Lean 证明，我们可以通过将变量和假设声明为定理的参数，来避免调用 `intro`，如下所示：

```
theorem fst_of_two_props_params (a b : Prop) (ha : a) (
  hb : b) :
  a :=
by apply ha
```

目标是：

$a, b : \text{Prop}, ha : a, hb : b \vdash a$

定理的所有参数在语境中立即生效。就像前面的例子一样，该目标通过 `apply ha` 即可证明。

下面是一个连续使用多个 `apply` 的例子：

```
theorem prop_comp (a b c : Prop) (hab : a → b) (hbc : b
  → c) :
  a → c :=
by
  intro ha
  apply hbc
  apply hab
  apply ha
```

站在数学家的角度，我们可以将最后的证明表述如下：

假设 $(ha) a$ 为真。

为了证明 c ，根据假设 hbc ，只需证明 b 即可。

为了证明 b ，根据假设 hab ，只需证明 a 即可。

为了证明 a ，我们使用假设 ha 。

□

3.2 基本策略

`intro` 和 `apply` 策略是策略证明的基础。其他基本策略包括 `exact`、`assumption`、`rfl` 和 `ac_rfl`。如果我们有足够的耐心使用这些策略进行推理，而不依赖更强大的 Proof Automation 的**证明自动化**，它们可以发挥很大作用。它们也可以用来解决各种逻辑谜题。

下面，大而细的方括号 `[]` 表示可选的语法。

`intro`

```
intro [名字1 ... 名字n]
```

`intro` 策略将最前面的 $\forall$ 量化变量或最前面的^{Assumption}前提 $a \rightarrow$ 从目标的^{Target}目的移动到局部语境中。该策略接受一个可选参数，用于指定语境中变量或前提的名称，从而覆盖默认名称。如果提供了多个名称（即 $n > 1$ ），则会移动多个变量或前提。

给定一个可证明的目标，`intro` 总是会产生一个可证明的目标。因此，它被称为是^{Safe}**安全的**。

apply

`apply` 定理或假设

`apply` 策略将目标的^{Target}目的与指定定理或假设的结论进行匹配，并将该定理或假设的前提作为新的目标加入。匹配是在^{Up to Computation}计算等价的意义下进行的。

我们必须谨慎调用 `apply`，因为它可能将一个可证明的目标转换为不可证明的子目标。例如，如果目标是 $\vdash \text{True}$ ，而我们 `apply` 了定理 $\text{False} \rightarrow \text{True}$ ，则结论 True 会与目标的目的 True 匹配，最后我们得到了不可证明的子目标 $\vdash \text{False}$ 。我们称 `apply` 是^{Unsafe}不安全的。

exact

`exact` 定理或假设

`exact` 策略将目标的目的与指定的定理或假设进行匹配，从而证明该目标。在这种情况下，我们通常也可以使用 `apply`，但 `exact` 能更好地传达我们的意图。

assumption

`assumption` 策略从局部语境中寻找一个与目标的目的匹配的假设，并将其应用以证明目标。

rfl

`rfl` 策略证明形如 $\vdash l = r$ 的目标，其中左右两边 l 和 r 在^{Up to Computation}计算等价的意义下在语法上是相等的。计算首先是指定义的^{Expansion}展开，但也包括将匿名函数应用于参数的^{Reduction}归约等等。这些^{Conversion}转换有传统的名称。下面列出了主要转换及其示例，其中全局语境包含 `def double (n : ℕ) : ℕ := n + n`：

α -转换 $(\text{fun } x \mapsto f\ x) = (\text{fun } y \mapsto f\ y)$
 β -转换 $(\text{fun } x \mapsto f\ x)\ a = f\ a$
 δ -转换 `double 5 = 5 + 5`
 ζ -转换 $(\text{let } n : \mathbb{N} := 2; n + n) = 4$
 η -转换 $(\text{fun } x \mapsto f\ x) = f$
 ι -转换 `Prod.fst (a, b) = a`

将这些转换作为从左向右的重写规则来重复应用的过程称为^{Reduction}归约；将转换反向应用一次称为^{Expansion}展开。

简而言之，为了证明 $\vdash l = r$ ，`rfl` 策略会展开 l 和 r 中的定义，并执行 β -归约及其他归约。如果在归约过程中， l 和 r 在某一点变得语法上完全相同，则证明成功。通常，`rfl` 在数学家会说「根据定义」的地方起作用。

ac_rfl

`ac_rfl` 与 `rfl` 类似，但它可以用于在^{Associativity}**结合律**（例如 $(a + b) + c = a + (b + c)$ ）和^{Commutativity}**交换律**（例如 $a + b = b + a$ ）的意义下进行推理。这适用于已注册为具有结合律和交换律的二元运算，例如算术类型上的 $+$ 和 $*$ ，以及集合上的 $\cup$ 和 $\cap$ 。我们将在第 3.6 节中看到例子。

sorry

我们在第 2 章中遇到的 `sorry` 证明命令可以作为策略在策略证明中的任何位置使用。它「证明」了当前目标，但实际上并未进行证明。请谨慎使用。

3.3 关于连结词和量词的推理

在学习对自然数、列表或其他数据类型进行推理之前，我们必须首先学习如何对 Lean 的逻辑连结词和量词进行推理。让我们从一个简单的例子开始：^{Conjunction}**合取** ($\wedge$) 的^{Commutativity}**交换律**。

```
theorem And_swap (a b : Prop) :
  a ∧ b → b ∧ a :=
by
  intro hab
  apply And.intro
  apply And.right
  exact hab
  apply And.left
  exact hab
```

此时，我们建议您将光标置于 Visual Studio Code 中的示例之上，以查看证明状态的序列。通过将光标放在每个命令之上或紧随其后，您可以查看该行命令的效果。对于最后一行，Lean 只会报告「No goals」。

该证明是典型的 `intro-apply-exact` 组合。它使用了以下定理：

```
And.intro : ?a → ?b → ?a ∧ ?b
And.left  : ?a ∧ ?b → ?a
And.right : ?a ∧ ?b → ?b
```

其中问号 (?) 表示可以被**实例化**的变量 —— 例如, 通过将目标的**目的**与定理的**结论**进行匹配。这些变量被称为**元变量**。

上述三个定理是与**合取**相关的**引入规则**和两个**消去规则**。逻辑符号 (例如 $\wedge$) 的引入规则是结论以该符号作为最外层符号的定理。对偶地, 消去规则在其前提中包含该符号。对于每个逻辑符号, 引入规则告诉我们**如何证明**一个以该符号为最外层位置的命题。相比之下, 消去规则告诉我们该命题**必然是如何被证明的**。

在上述证明中, 我们应用 $\wedge$ 的引入规则来证明目标 $\vdash b \wedge a$, 并应用两个消去规则从假设 $a \wedge b$ 中提取 b 和 a 。我们在 $\vdash b \wedge a$ 中通过使用所谓的引入规则来「消去」 $\wedge$, 这听起来可能很奇怪。该术语之所以是「逆向」的, 是因为我们的证明是**逆向的**。

问号也可能出现在目标中。它们表示可以任意**实例化**的变量。在上面的证明过程中, 在 `apply And.right` 策略之后, 我们得到了目标:

`a b : Prop, hab : a  $\wedge$  b  $\vdash$  ?left.a  $\wedge$  b`

其中 `?left.a` 是一个**元变量**。策略 `exact hab` 将 (目的中的) `?left.a` 与 (hab 中的) `a` 进行匹配。这种通过实例化变量使两个项在语法上等价的过程称为**合一**。**匹配**是合一的一种特殊情况, 即其中一个项不包含变量, 如本例所示。

顺便提一下, 每当元变量出现在目标中时, 还会出现额外的子目标, 每个元变量的类型也会作为一个子目标 (例如 $\vdash \text{Prop}$)。这些令人困惑的子目标仅仅是一个提醒, 说明我们需要用正确类型的项来实例化这些元变量。我们通常可以忽略这些子目标。一旦元变量被实例化 (通常是在我们解决另一个子目标时), 其相关的子目标也会随之消失。

在 Lean 中, **合一**是在**计算等价**的意义下进行的。例如, 项 $(\text{fun } x \mapsto ?m) a$ 和 b 可以通过取 `?m := b` 来合一, 因为 $(\text{fun } x \mapsto b) a$ 和 b 在 β -转换的意义下是语法相等的。

下面是定理 `And_swap` 的另一种证明:

```
theorem And_swap_braces :
  ∀ a b : Prop, a  $\wedge$  b  $\rightarrow$  b  $\wedge$  a :=
by
  intro a b hab
  apply And.intro
  · exact And.right hab
  · exact And.left hab
```

该定理的陈述方式有所不同, 将 a 和 b 作为 $\forall$ 量化变量, 而不是定理的参数。从逻辑上讲, 这是等价的, 但在证明中我们必须在 `hab` 之外额外**引入** a 和 b 。

另一个区别是在子证明前面使用了项目符号 ($\cdot$)。当我们面临两个或多个目标需要证明时, 通常良好的风格是在每个子证明前面加上一个点。**策略组合子** $\cdot$

专注于第一个子目标；其后的策略必须证明该子目标。在我们的例子中，`apply And.intro` 策略创建了两个子目标： $\vdash b$ 和 $\vdash a$ 。

第三个区别是，我们现在通过 ^{Juxtaposition}并置，直接将 `And.right` 和 `And.left` 应用于 ^{Hypothesis}假设 $a \wedge b$ ，分别获得 b 和 a ，而不是等待定理的前作为新的子目标出现。这是在 ^{Backward Proof}逆向证明中迈出的微小 ^{Forward}正向步骤。同样的语法既可以用于 ^{Discharge}解除（即证明）一个假设，也可以用于 ^{Instantiate}实例化一个 $\forall$ 量词。这种方法的一个好处是，我们避免了可能令人困惑的 `?left.a` 元变量。

在下一个例子中，我们使用并置来实例化 $\forall$ 量词：

```
theorem f5_if (h :  $\forall n : \mathbb{N}, f\ n = n$ ) :  
  f 5 = 5 :=  
  by exact h 5
```

如果 h 是定理 $\forall n, f\ n = n$ ，那么 $h\ 5$ 就是定理 $f\ 5 = 5$ 。

^{Disjunction}析取 ($\vee$) 的引入规则和消去规则如下：

```
Or.inl :  $\forall b : \text{Prop}, ?a \rightarrow ?a \vee b$   
Or.inr :  $\forall b : \text{Prop}, ?a \rightarrow b \vee ?a$   
Or.elim :  $?a \vee ?b \rightarrow (?a \rightarrow ?c) \rightarrow (?b \rightarrow ?c) \rightarrow ?c$ 
```

^{Juxtaposition}`Or.inl`（「左侧引入」）和 `Or.inr`（「右侧引入」）中的 $\forall$ 量词可以通过简单的并置，将定理名称应用于我们要实例化的值来直接实例化。因此，`Or.inl False` 对应于定理 $?a \rightarrow ?a \vee \text{False}$ 。这就是 ^{Forward}正向风格。

或者，我们可以在形如 $\dots \vdash c \vee d$ 的目标上调用 `apply Or.inl`。这会设置定理中的 $?a := c$ ，以及 $b := d$ 。新的子目标是 $\dots \vdash c$ 。这就是 ^{Backward}逆向风格。

^{Unsafe}`Or.inl` 和 `Or.inr` 都是不安全的：如果您应用了两者中错误的一个，或者在证明中过早地应用了其中任何一个，您可能会得到一个不可证明的子目标。如果您考虑可证明的目标 $\vdash \text{True} \vee \text{False}$ ，就可以很容易地看到这一点：`apply Or.inr` 会产生不可证明的子目标 $\vdash \text{False}$ 。

`Or.elim` 规则乍一看可能违反直觉。本质上，它指出如果我们有 $a \vee b$ ，那么为了证明任意的 c ，只需证明在 a 成立时 c 成立，以及在 b 成立时 c 成立即可。您可以将 $(?a \rightarrow ?c) \rightarrow (?b \rightarrow ?c) \rightarrow ?c$ 看作是一个巧妙的技巧，仅使用蕴含来表达析取的含义。

^{Equivalence}等价 ($\leftrightarrow$) 的引入规则和消去规则如下：

```
Iff.intro :  $(?a \rightarrow ?b) \rightarrow (?b \rightarrow ?a) \rightarrow (?a \leftrightarrow ?b)$   
Iff.mp :  $(?a \leftrightarrow ?b) \rightarrow ?a \rightarrow ?b$   
Iff.mpr :  $(?a \leftrightarrow ?b) \rightarrow ?b \rightarrow ?a$ 
```

^{Existential Quantification}存在量化 ($\exists$) 的引入规则和消去规则如下：

存在量化 ($\exists$) 的引入规则和消去规则如下：

```
Exists.intro : ∀w, (?P w → (∃x, ?P x))
Exists.elim : (∃x, ?P x) → (∀a, ?P a → ?c) → ?c
```

$\exists$ 的引入规则可以用于为一个存在量词提供一个^{Witness}证据进行^{Instantiate}实例化——证据是使量词的主体为真的值。例如：

```
theorem Exists_double_iden :
  ∃n : ℕ, double n = n :=
  by
    apply Exists.intro 0
    rfl
```

同样地，我们以^{Forward}正向方式实例化一个 $\forall$ 量词：Exists.intro 0 是定理 $?P 0 \rightarrow (\exists x, ?P x)$ 。该规则是^{Unsafe}不安全的：为 x 选择错误的证体会导致不可证明的目标。例如，如果目标是 $\vdash \exists n, n > 5$ ，而我们选择 3 作为证体，最后会得到不可证明的子目标 $\vdash 3 > 5$ 。

$\exists$ 的消去规则让人联想到 $\vee$ 的消去规则。事实上，一种富有成效的思考量化 $\exists n, ?P n$ 的方式是，将其看作是一个可能^{Infinitary}无穷的析取 $?P 0 \vee ?P 1 \vee \dots$ 。类似地， $\forall n, ?P n$ 可以被看作是 $?P 0 \wedge ?P 1 \wedge \dots$ 。

对于真 (True)，只有一个引入规则：

```
True.intro : True
```

「真」不包含任何信息。如果它作为^{Hypothesis}假设出现，它是完全没用的，并且没有消去规则可以从中提取任何信息。下面第 3.8 节中描述的 clear 策略可以用来移除此类无用的假设。

对偶地，对于假 (False)，只有一个消去规则：

```
False.elim : False → ?a
```

无法证明「假」，但如果我们以某种方式从某处（例如从假设中）得到了它，那么我们可以推导出任何东西 (?a)。

事实上，^{Negation}否定 (Not) 是根据蕴含和「假」来定义的： $\neg a$ 是 $a \rightarrow \text{False}$ 的缩写。直观上，它们的含义相同。我们可以把「非 a 」看作是「 a 会导致荒谬的事情（因此非 a ）」。

Lean 的逻辑是^{Classical}经典的，支持 ^{Law of Excluded Middle}排中律 和 ^{Proof by Contradiction}反证法：

```
Classical.em : ∀a : Prop, a ∨ ¬ a
Classical.byContradiction : (¬ ?a → False) → ?a
```

蕴含 ($\rightarrow$) 和 ^{Universal Quantification}全称量化 ($\forall$) 就像是谚语中那种「^{dogs that did not bark}不叫的狗」。它们没有引入规则或消去规则。相反, 对于这两者, ^{Application}intro 策略就是引入原理, 而应用 (如 λ -演算中那样) 就是消去原理。例如, 给定定理 $hab : a \rightarrow b$ 和 $ha : a$, 应用 $hab\ ha$ 就是一个陈述 b 的定理。

为了证明涉及连结词和量词的逻辑谜题, 我们提倡一种「盲目」的、「电子游戏」式的推理风格, 它主要依赖于 intro 和 apply 等基本策略。以下是一些通常奏效的策略:

- 如果目标的目的是一个蕴含式 $P \rightarrow Q$, 调用 `intro hP` 将 P 移动到你的假设中: $\dots, hP : P \vdash Q$ 。
- 如果目标的目的是一个全称量化 $\forall x : \sigma, Q$, 调用 `intro x` 将 x 移动到局部语境中: $\dots, x : \sigma \vdash Q$ 。
- 寻找一个结论与目标的形状相同 (可能包含可以匹配的变量) 的定理或假设, 并对其调用 `apply`。例如, 如果目标的目的是 Q , 而你有一个形如 $hPQ : P \rightarrow Q$ 的定理或假设, 请尝试 `apply hPQ`。
- 一个被否定的目标 $\vdash \neg P$ 在计算上等价于 $\vdash P \rightarrow \text{False}$, 因此你可以调用 `intro hP` 来产生子目标 $hP : P \vdash \text{False}$ 。通过调用 `rw [Not]` (在第 3.5 节中描述) 来展开否定的定义通常是一个好策略。
- 有时你可以通过输入 `apply False.elim` 将目标替换为 `False` 来取得进展。作为下一步, 你通常会 `apply` 一个形如 $P \rightarrow \text{False}$ 或 $\neg P$ 的定理或假设。
- 当你面临多个选择时 (例如在 `Or.inl` 和 `Or.inr` 之间选择), 请记住你所做的选择, 并在遇到 ^{Dead End}死胡同或感觉没有进展时进行 ^{Backtrack}回溯。
- 如果你怀疑自己可能遇到了死胡同, 请检查在给定的前提下目标是否确实是可证明的。即使你最初陈述的是一个可证明的定理, 当前的目标也可能是不可证明的 (例如, 如果你应用了不安全的规则)。

3.4 关于相等的推理

^{Equality}

相等 ($=$) 也是一个基本的逻辑常量。它的特征由以下引入规则和消去规则给出:

```
Eq.refl : ∀a, a = a
Eq.symm : ?a = ?b → ?b = ?a
Eq.trans : ?a = ?b → ?b = ?c → ?a = ?c
Eq.subst : ?a = ?b → ?P ?a → ?P ?b
```

^{Equivalence Relation}

前三个定理是指定 $=$ 为 ^{Equivalence Relation}等价关系的引入规则。第四个定理是一个消去规则, 它允许我们在任意语境 (由元变量 $?P$ 表示) 中进行等量替换。

下面的例子展示了其中一些规则的应用。我们应用 `Eq.trans` 和 `Eq.symm`，利用等式 $a = b$ 和 $c = b$ 来证明 $a = c$ ：

```
theorem Eq_trans_symm {α : Type} (a b c : α)
  (hab : a = b) (hcb : c = b) :
  a = c :=
by
  apply Eq.trans
  • exact hab
  • apply Eq.symm
    exact hcb
```

由于这种重写操作非常常见，Lean 提供了一个 `rw` 策略来实现相同的结果。如果适用，该策略还会自动应用 `rfl`：

```
theorem Eq_trans_symm_rw {α : Type} (a b c : α)
  (hab : a = b) (hcb : c = b) :
  a = c :=
by
  rw [hab]
  rw [hcb]
```

关于解析的说明：相等的结合优先级高于逻辑连结词。因此， $a = b \wedge c = d$ 被解读为 $(a = b) \wedge (c = d)$ 。

3.5 重写策略

重写策略 `rw` 及其相关的 `simp` 进行等量替换。它们将等式作为自左向右的重写规则，将出现的左侧项替换为右侧项。

默认情况下，它们在目标的目的上操作，但也可以通过 `at` 关键字来重写指定的假设：

```
at h1 ... hn  重写指定的假设
at *           重写所有假设和目标
```

rw

```
rw [定理或常量1, ..., 定理或常量n] [at 位置]
```

`rw` 策略使用一个或多个等式作为自左向右的重写规则来重写目标。它会搜索与第一个等式左侧匹配的第一个子项；一旦找到，该子项的所有出现都会被替换为右侧项。如果等式包含变量，则会根据需要进行实例化。要反向使用定理（即作为自右

向左的重写规则)，请在定理名称前加上一个短左箭头 ($\leftarrow$)。如果指定了多个等式，它们将依次被应用。

因此，给定定理 $hg : \forall x, g\ x = f\ x$ 和目标 $\vdash h\ (f\ a)\ (g\ b)\ (g\ c)$ ，策略 $rw\ [hg]$ 会产生子目标 $\vdash h\ (f\ a)\ (f\ b)\ (g\ c)$ ，而 $rw\ [\leftarrow hg]$ 则产生子目标 $\vdash h\ (g\ a)\ (g\ b)\ (g\ c)$ 。

除了定理之外，我们还可以指定一个常数的名称。这将尝试使用该常数的定义等式之一作为重写规则。

simp

`simp [at 位置]`

`simp` 策略穷尽地使用一组标准的重写规则（称为 ^{Simp Set}**simp 集合**）来重写目标。`simp` 集合中的每个等式都用作自左向右的重写规则。`simp` 集合包含关于预定义符号（例如算术和列表运算符）的各种规则，并且可以通过在适当的定理上添加 `@[simp]` 属性来扩展。

`simp [定理或常量1, ..., 定理或常量n] [at 位置]`

对于上述 `simp` 变体，指定的定理会被临时添加到 `simp` 集合中。在定理列表中，星号 (*) 可以用来代表所有假设。定理名称前的减号 (-) 会临时将该定理从 `simp` 集合中移除。一个既能简化假设又能利用结果简化目标的目的的强大命令是 `simp [*] at *`。

给定定理 $hg : \forall x, g\ x = f\ x$ 和目标 $\vdash h\ (f\ a)\ (g\ b)\ (g\ c)$ ，策略 `simp [hg]` 产生子目标 $\vdash h\ (f\ a)\ (f\ b)\ (f\ c)$ ，其中 $g\ b$ 和 $g\ c$ 都已被重写。除了定理之外，我们还可以指定常数的名称。这会临时将常数的定义等式添加到 `simp` 集合中。

说到这里，你可能会想，「那么 `simp` 到底做了什么呢？」当然，你可以去研究源代码或查阅科学文献。但这可能不是对你时间最有效的利用。事实上，即使是证明助手的专家用户也并不完全理解他们每天使用的策略的行为。最成功的用户会采取一种轻松、随性的态度，依次尝试各种策略，并研究出现的子目标（如果有的话），看看自己是否走在正确的道路上。

随着你不断地使用 `simp` 和其他策略，你会对它们擅长处理什么样的目标产生一些直觉。这是交互式定理证明只能通过实践来学习的众多原因之一。通常，你不会确切地理解 Lean 做了什么——为什么一个策略成功了，或者失败了。定理证明有时会非常令人沮丧。^{The Hitchhiker's Guide to the Galaxy}《银河系漫游指南》封面上的那些醒目而友好的大字所给出的建议在这里同样适用：**不要恐慌**。

3.6 数学归纳法证明

^{Structural induction}
induction 归纳策略对一个归纳类型的值执行结构归纳。**结构归纳**意味着归纳遵循归纳类型的结构。对于由 `Nat.zero` 和 `Nat.succ` 构造的自然数，结构归纳对应于标准的数学归纳法：要证明 $p\ n$ ，只需证明 $p\ 0$ 和 $\forall k, p\ k \rightarrow p\ (k + 1)$ 。借助 `induction`，我们可以推理 2.2 一节中通过递归定义的和乘运算。

加法是通过对其第二个参数进行递归定义的。我们将证明两个定理，`add_zero` 和 `add_succ`，它们为我们提供了在**第一个**参数上递归的替代等式。我们从 `add_zero` 开始：

```
theorem add_zero (n : ℕ) :
  add 0 n = n :=
by
  induction n with
  | zero      => rfl
  | succ n' ih => simp [add, ih]
```

^{Induction}
归纳策略后面跟着两个由它们对应的构造子标识的子证明。此外，任何特定于子证明的变量或假设都可以明确地在构造子名称之后命名。

第一个情况，标记为 `zero`，对应于基本情况 $\vdash \text{add } 0\ 0 = 0$ 。第二个情况，标记为 `succ`，对应于归纳步骤

```
n' : ℕ, ih : add 0 n' = n' ⊢ add 0 (Nat.succ n') = Nat.succ n'
```

`succ` 后面指定的名称 `n'` 和 `ih` 分别是我们给 `Nat.succ` 的参数和归纳假设所起的名称。用其他名称也可以，但通常将归纳假设称为 `ih`。

我们可以继续通过结构归纳证明定理：

```
theorem add_succ (m n : ℕ) :
  add (Nat.succ m) n = Nat.succ (add m n) :=
by
  induction n with
  | zero      => rfl
  | succ n' ih => simp [add, ih]
```

```
theorem add_comm (m n : ℕ) :
  add m n = add n m :=
by
  induction n with
  | zero      => simp [add, add_zero]
```

```

| succ n' ih => simp [add, add_succ, ih]

theorem add_assoc (l m n : ℕ) :
  add (add l m) n = add l (add m n) :=
  by
    induction n with
    | zero      => rfl
    | succ n' ih => simp [add, ih]

```

一旦我们证明了一个二元运算满足交换律和结合律，就应该让 Lean 的自动化机制，尤其是 `ac_rfl` 知道这一点。下面的命令即为 `add` 实现了这一点：

```

instance Associative_add : Std.Associative add :=
  { assoc := add_assoc }

instance Commutative_add : Std.Commutative add :=
  { comm := add_comm }

```

(instance 机制将在第 5 章讲解。) 下面的例子使用 `ac_rfl` 策略，在 `add` 的结合律与交换律意义下进行推理：

```

theorem mul_add (l m n : ℕ) :
  mul l (add m n) = add (mul l m) (mul l n) :=
  by
    induction n with
    | zero      => rfl
    | succ n' ih =>
      simp [add, mul, ih]
      ac_rfl

```

下面给出一些关于如何使用数学归纳法进行证明的提示：

- 按照目标中出现的某个函数的定义结构，来进行归纳证明，通常是有益的。特别地，如果一个函数是通过对其第 n 个参数进行递归来定义的，那么对该参数进行归纳通常是合理的。
- 如果归纳的基本情况难以证明，这通常表明选错了变量，或者需要先证明一些辅助引理。

3.7 归纳策略

我们已经在上面看到了 `induction` 策略的实际用法。为方便查阅，下面给出它的完整语法。一般而言，本指南正文中介绍的每一种策略，都会在名为「X 策略」的小节中附带此类语法描述。

induction

```
induction 项 [with
| 构造子1 名字1 => 策略1
  :
| 构造子n 名字n => 策略n]
```

`induction` 策略对指定项执行结构归纳。这会生成与该项类型定义中构造子数量相同的子目标。在对应递归构造子（如 `Nat.succ` 或 `List.cons`）的子目标中，归纳假设将作为可用假设出现。可选的名字 `名字1`、...和 `名字n` 用于命名生成的变量或假设。

3.8 清理策略

下面这些策略有助于简化证明目标。目前我们尚未用到它们，但在探索证明思路的过程中，它们会很有帮助。

clear

```
clear 变量或假设1 ... 变量或假设n
```

只要指定的变量和假设没有在目标的其他任何地方使用，`clear` 就会移除它们。

rename

```
rename 变量的类型或假设的命题 => 新名字
```

`rename` 策略会修改变量或假设的名字。

3.9 新引入的 Lean 结构总结**属性**

`@[simp]` 将定理添加到 `simp` 集

证明命令

`by` 开始一个策略式的证明

策略

<code>ac_refl</code>	证明 $l = r$ 满足交换律和结合律
<code>apply</code>	将目标的目的与定理的结论进行匹配
<code>assumption</code>	使用假设证明目标
<code>clear</code>	从目标中移除变量或假设
<code>exact</code>	使用指定的定理证明目标
<code>induction</code>	对归纳类型的变量进行结构归纳
<code>intro</code>	将 $\forall$ -量化的变量移入到目标的假设中
<code>rename</code>	重命名变量或假设
<code>rfl</code>	证明 $l = r$ 计算等价
<code>rw</code>	使用给定的定理作为重写规则重写一次
<code>simp</code>	使用一组预先注册的重写规则穷举重写
<code>sorry</code>	表示缺少证明

策略组合子

- 专注于第一个子目标；需要证明该子目标。

正向证明

策略证明是逆向进行的。它们从目标开始，将其归约到已有的定理和假设。通常，以正向方式进行证明也是合理的：从已有的定理和假设出发，逐步向我们的目标推进。

Structured Proofs
结构化证明就是一种支持这种推理方式的风格。策略证明往往易于编写但难于阅读。大多数用户会结合这两种风格，根据具体情况使用最合适的一种。结构化证明较高的可读性使其深受某些用户的喜爱。

Proof Terms
结构化证明是撒在 Lean **证明项**之上的一层语法糖。它们使用 `assume`、`have`、`let` 和 `show` 等关键字来构建，模仿了纸笔形式的证明。所有 Lean 的证明，无论是策略证明还是结构化证明，在系统内部都会被归约成证明项。在第 3 章中，我们已经看到了一些示例：给定假设 $ha : a$ 和 $hab : a \rightarrow b$ ，项 $hab\ ha$ 即是命题 b 的证明项，因此我们可以写成 $hab\ ha : b$ 。定理和假设的名称（例如 ha 和 hab ）也是证明项。沿着这一思路继续，给定 $hbc : b \rightarrow c$ ，我们可以为命题 c 构造证明项 $hbc\ (hab\ ha)$ 。我们可以将 hab 视为一个函数，它可将 a 的证明转换为 b 的证明，对于 hbc 也同理。

Tactic Mode
结构化证明是 Lean 中的默认方式。它们可以在**策略模式**之外使用。要进入策略模式，我们必须使用 `by` 命令。

Theorem Proving in Lean 4
此处涵盖的概念在 **Lean 4 定理证明**[2] 的第 2 至 4 章中有更详细的描述。Nederpelt 和 Geuvers 的教科书 [22] 是另一个有用的参考资料。

4.1 结构化证明

作为第一个例子，考虑以下结构化证明：

```
theorem fst_of_two_props :  
  ∀ a b : Prop, a → b → a :=
```

```

fix a b : Prop
assume ha : a
assume hb : b
show a from
  ha

```

每个由 $\forall$ ^{Quantifier} 量词绑定的变量以及 ^{Implication} 蕴涵的每个假设, 都在证明中通过使用 `fix` 和 `assume` 命令显式地引入。可以同时引入多个变量。我们经常会省略变量的类型, 特别是当它们可以从名称中猜出时; 然而, 我们总是会写出完整的 ^{Proposition} 命题, 并且每行放置一个, 以提高可读性 —— 毕竟, 这是 ^{Structured Style} 结构化风格的主要潜在优势之一。结尾处的 `show ... from` 命令为了可读性重复了要证明的命题, 并在关键字 `from` 之后给出证明。此时的 ^{Goal} 目标是 $a b : \text{Prop}, ha : a, hb : b \vdash a$ 。

非正式地, 我们可以将证明写为如下形式:

固定一些命题 a 和 b 。

假设 $(ha) a$ 且 $(hb) b$ 为真。

我们必须证明 a 。这根据 ha 平凡地得出。

□

一些作者会在「命题」之前插入诸如「^{Arbitrary but Fixed}任意但固定」之类的限定词, 或者他们会写「令 a 和 b 是某些命题」。所有这些变体都是等价的。此外, 我们也可以用「QED」来代替「□」以结束证明。

上述 Lean 证明是非典型的, 因为目标的目的出现在 ^{Hypothesis} 假设之中。通常, 我们必须执行一些中间推理步骤, 其形式基本上是「由某某得到某某」。在 Lean 中, 每个中间步骤都采用 `have` 命令的形式, 如下例所示:

```

theorem prop_comp (a b c : Prop) (hab : a → b) (hbc : b
  → c) :
  a → c :=
  assume ha : a
  have hb : b :=
    hab ha
  have hc : c :=
    hbc hb
  show c from
    hc

```

非正式地:

假设 $(ha) a$ 为真。

由 ha 和 hab , 我们得到 $(hb) b$ 。

由 hb 和 hbc , 我们得到 $(hc) c$ 。

我们必须证明 c 。这根据 hc 平凡地得出。

□

注意这是一个^{Forward Proof}**正向证明**：它从假设 a 开始，逐个定理地向目标定理 c 推进。
^{Modus Ponens}**肯定前件**($\lceil$ 从 a 和 $a \rightarrow b$ 推导出 b $\rceil$) 通过简单的并置来表达 (例如 $hab\ ha$)。

一般而言, `fix-assume-show` 骨架重复了定理的陈述。我们经常会根据目标中的绑定变量来命名固定变量, 正如我们在这里所做的那样, 但这并不是强制性的。
^{Shadowing}此外, 避免遮蔽已有变量是一个良好的实践。在最后一个 `fix` 或 `assume` 与 `show` 之间, 我们可以根据需要的论证详细程度, 放入任意数量的 `have` 命令。细节可以增加可读性, 但提供过多细节可能会令读者难以忍受。

`have` 命令的语法与 `theorem` 相似, 但它出现在结构化证明之中。我们也可以将 `have` 视为一个定义。在 `have hb : b := hab ha` 中, 右侧的 $hab\ ha$ 是 b 的证明项, 而左侧的 hb 被定义为该证明项的同义词。从此以后, hb 和 $hab\ ha$ 即可互换使用。由于 hb 和 hc 只被使用了一次, 且它们的证明非常简短, 专家们往往倾向于将它们^{Inline}**内联**, 把 hc 替换为 $hbc\ hb$, 然后把 hb 替换为 $hab\ ha$, 从而得到:

```
theorem prop_comp_inline (a b c : Prop) (hab : a → b)
  (hbc : b → c) :
  a → c :=
  assume ha : a
  show c from
    hbc (hab ha)
```

一个典型的结构化证明具有如下的 `fix-assume-have-show` 格式:

```
theorem hr :
  ∀(c₁ : σ₁) ... (c₁ : σ₁), P₁ → ... → Pₘ → R :=
  fix (c₁ : σ₁) ... (c₁ : σ₁)
  assume h₁ : P₁
  ⋮
  assume hₘ : Pₘ
  have k₁ : Q₁ := ...
  ⋮
  have kₙ : Qₙ := ...
  show R from ...
```

4.2 结构化构造

上一节介绍了用于编写结构化证明的主要命令：fix、assume、have 和 show。现在我们更系统地回顾结构化证明的各组成部分。

定理或假设

除了 sorry 之外，最简单的结构化证明是定理或假设的名称。如果我们有：

```
theorem two_add_two_Eq_four :
  2 + 2 = 4 :=
  by ...
```

那么在此之后，定理名称 two_add_two_Eq_four 即可被用作 $2 + 2 = 4$ 的证明。例如：

```
theorem this_time_with_feeling :
  2 + 2 = 4 :=
  two_add_two_Eq_four
```

我们可以将参数传递给定理，以^{Instantiate}实例化 $\forall$ 量词并^{Discharge}解除假设。假设定理 add_comm ^{Instance}($m : \mathbb{N}$) : $\text{add } m \ n = \text{add } n \ m$ 可用，并且我们想要证明它的^{Instance}实例 $\text{add } 0 \ n = \text{add } n \ 0$ 。这可以通过使用定理名称和两个参数来巧妙地完成：

```
theorem add_comm_zero_left (n : ℕ) :
  add 0 n = add n 0 :=
  add_comm 0 n
```

这等价于策略证明 by exact add_comm 0 n，但更为简洁。exact 策略可以被看作是 by 的逆操作。为何要进入策略模式又立刻离开它呢？

与 exact 和 apply 一样，定理或假设的陈述会在计算等价的意义上与当前目标进行匹配。这提供了一些灵活性。

fix

```
fix names : type
```

fix 命令将由 $\forall$ 绑定的量化变量从目标的目的移动到^{Local Context}局部语境。它是 intro 策略的结构化版本。

请注意，Lean 中用于固定变量的标准策略是 fun。我们倾向于使用由 LoVelib 提供的 fix，作为一个更具可读性的替代方案。我们将把 fun 保留用于指定匿名函数。

assume

```
assume name : proposition
```

assume 命令将最前面的假设从目标的目的移动到局部语境中。它可以被看作是 intro 策略的结构化版本。

请注意，Lean 中用于陈述假设的标准策略是 fun。我们倾向于使用由 LoVelib 提供的 assume，作为一个更具可读性的替代方案。我们将把 fun 保留用于指定匿名函数。

have

```
have 名称 : 命题 :=
  证明
```

have 命令允许我们陈述并证明一个中间定理，该定理可以引用之前由 fix、assume 和 have 引入的名称。证明过程可以是策略式的^{Tactical}，也可以是结构化的。通常，我们倾向于使用结构化证明来勾勒主要论证过程，而在证明子目标或无关紧要的中间步骤时则求助于策略式证明。

当我们将参数传递给定理名称时，会出现另一种混合情况。例如，给定 $hab : a \rightarrow b$ 和 $ha : a$ ，策略 `exact hab ha` 将证明目标 $\vdash b$ 。在这里，`hab ha` 是一个嵌套在策略内部的证明项。

let

```
let 名称 [: 类型] := 项
```

let 命令引入一个新的局部定义^{Local Definition}。它可以用于为证明后续部分多次出现的复杂对象命名。它与 have 类似，但它是为可计算数据^{Computable Data}而不是证明设计的。展开或引入一个 let 对应于 λ -变换 ^{λ -conversion} (第 3.2 节)。

构造「`let 名称 [: 类型] := 项`」之后必须跟随换行符或分号 (;)。

show

```
show 命题 from
  证明
```

show 命令允许我们重复要证明的目标，这可以起到文档的作用。它还允许我们在计算等价的意义上改写目标。如果我们不想重复目标且不需要改写它，可以直接写 证明，而不必使用「`show 命题 from 证明`」的语法。证明可以是策略式的，也可以是结构化的。

4.3 关于联结词和量词的正向推理

以正向方式对^{Logical Connectives Quantifier}逻辑联结词和量词进行推理，使用的是与策略模式（第 3.3 节）相同的^{Introduction and Elimination Rules}引入和消去规则。通过几个例子可以初步体会其风格。让我们从^{Conjunction}合取开始：

```
theorem And_swap (a b : Prop) :
  a ∧ b → b ∧ a :=
  assume hab : a ∧ b
  have ha : a :=
    And.left hab
  have hb : b :=
    And.right hab
  show b ∧ a from
    And.intro hb ha
```

即便是不了解 And.left 等含义的读者，也能理解我们是从 $a \wedge b$ 中提取出 a 和 b ，然后将它们重新组合为 $b \wedge a$ 。数学家们理解这一证明可能比理解其对应的策略证明要容易得多：

```
theorem And_swap_tactical (a b : Prop) :
  a ∧ b → b ∧ a :=
  by
    intro hab
    apply And.intro
    apply And.right
    exact hab
    apply And.left
    exact hab
```

一般来说，逆向证明更容易「不假思索地」推导出来，而且证明助手提供的大多数自动化功能也是逆向工作的。这很有道理：假设你是在调查谋杀案的马普尔小姐或赫尔克里·波洛。逆向调查会从犯罪现场开始，尝试提取线索，将少数嫌疑人与犯罪联系起来。相比之下，正向调查可能会从询问多达 80 亿人开始，以确定他们是否有不在场证明。哪种方法更有可能成功？

接下来的例子涉及到^{One-Point Rules}单点规则。在绑定变量实际上只能取一个值时，这些定理可以用来消去量词。例如，命题 $\forall n, n = 666 \rightarrow \text{beast} \geq n$ 可以简化为 $\text{beast} \geq 666$ 。下面的定理证明了这种简化的合理性：

```
theorem Forall.one_point {α : Type} (t : α) (P : α →
  Prop) :
  (∀ x, x = t → P x) ↔ P t :=
```

```

Iff.intro
  (assume hall :  $\forall x, x = t \rightarrow P\ x$ 
    show  $P\ t$  from
      by
        apply hall t
        rfl)
  (assume hp :  $P\ t$ 
    fix x :  $\alpha$ 
    assume heq :  $x = t$ 
    show  $P\ x$  from
      by
        rw [heq]
        exact hp)

```

证明过程看起来可能有些令人生畏，但其实并不难。关键在于一步步来。起初，我们观察到目标是一个^{Equivalence}等价式，于是我们写下：

```
Iff.intro ( _ ) ( _ )
```

其中 `Iff.intro` 是 $\leftrightarrow$ 的引入规则（第 3.3 节）。

两个^{Placeholders}占位符对于使证明格式良好非常重要。我们使用圆括号是因为我们强烈怀疑证明是非平凡的。由于证明基本上就是项，也就是程序，我们在第 1.4 节给出的建议在此同样适用，只需作相应调整（*mutatis mutandis*）：

关键在于，proof 应始终保持语法正确。我们在 Visual Studio Code 中唯一应该看到的红色下划线应该出现在占位符下方。一般来说，开发 proof 的一个好的原则是，从可以编译的 proof 开始，进行尽可能小的更改，以获得新的可以编译的 proof，并重复此过程，直到 proof 完成。

将鼠标悬停在第一个占位符上，会显示对应的^{Subgoal}子目标。我们可以看到 Lean 期望得到 $\vdash (\forall x, x = t \rightarrow P\ x) \rightarrow P\ t$ 的证明，于是我们提供了一个合适的^{Skeleton}骨架。蕴涵的结构化证明由 `assume` 后跟 `show` 组成：

```

Iff.intro
  (assume hall :  $\forall x, x = t \rightarrow P\ x$ 
    show  $P\ t$  from
      _ )
  ( _ )

```

剩余的每一个占位符都可以用结构化证明或策略证明来替代。为了消除第一个占位符，我们进入策略模式，并将假设 $\forall x, x = t \rightarrow P\ x$ 的^{Instance}实例 $t = t \rightarrow P\ t$ 应用于目标 $\vdash P\ t$ 。这留下了^{Proof Obligation}证明义务 $t = t$ ，我们可以使用 `rfl` 策略来完成它：

```
Iff.intro
  (assume hall :  $\forall x, x = t \rightarrow P x$ 
   show P t from
     by
       apply hall t
       rfl))
  (_)
```

第二个占位符代表 $\vdash P t \rightarrow \forall x, x = t \rightarrow P x$ 。为了消除它，我们根据第一个蕴涵的假设编写 `assume`，为 $\forall$ 编写 `fix`，为第二个蕴涵的假设编写 `assume`，并为结论编写 `show`：

```
(assume hp : P t
 fix x :  $\alpha$ 
 assume heq :  $x = t$ 
 show P x from
  _)
```

`show` 的证明是策略式的：我们将 $\vdash P x$ 中的 x 重写为 t ，得到 $\vdash P t$ ，这与假设 `hp` 相符：

```
(assume hp : P t
 fix x :  $\alpha$ 
 assume heq :  $x = t$ 
 show P x from
  by
    rw [heq]
    exact hp)
```

让我们检查一下单点规则在我们的引导示例上是否真的有效：

```
theorem beast_666 (beast :  $\mathbb{N}$ ) :
  ( $\forall n, n = 666 \rightarrow \text{beast} \geq n$ )  $\leftrightarrow$   $\text{beast} \geq 666$  :=
  Forall.one_point _ _
```

它确实有效。除了 `Forall.one_point _ _`，我们也可以写成 `Forall.one_point 666 (fun m  $\mapsto$   $\text{beast} \geq m$ )`。

最后， $\exists$ 的单点规则演示了如何在结构化证明中使用 $\exists$ 的引入和消去规则：

```
theorem Exists.one_point { $\alpha$  : Type} (t :  $\alpha$ ) (P :  $\alpha \rightarrow$ 
  Prop) :
  ( $\exists x : \alpha, x = t \wedge P x$ )  $\leftrightarrow$   $P t$  :=
  Iff.intro
```

```

(assume hex :  $\exists x, x = t \wedge P x$ 
 show P t from
   Exists.elim hex
   (fix x :  $\alpha$ 
    assume hand :  $x = t \wedge P x$ 
    have hxt :  $x = t$  :=
      And.left hand
    have hpx :  $P x$  :=
      And.right hand
    show P t from
      by
        rw [←hxt]
        exact hpx))
(assume hp : P t
 show  $\exists x : \alpha, x = t \wedge P x$  from
   Exists.intro t
   (have tt :  $t = t$  :=
    by rfl
    show  $t = t \wedge P t$  from
      And.intro tt hp))

```

虽然证明看起来可能有些复杂，但它的开发过程基本上是不假思索的，使用了与 $\forall$ 单点规则证明相同的步骤。

首先，我们使用 $\leftrightarrow$ 的引入规则：

```
Iff.intro (_) ( _ )
```

对于第一个占位符，我们需要在假设 $\exists x : \alpha, x = t \wedge P x$ 下提供 $P t$ 的证明。因此，我们将证明骨架展开为：

```

Iff.intro
  (assume hex :  $\exists x : \alpha, x = t \wedge P x$ 
   show P t from
     _ )
  ( _ )

```

此时，我们想要从假设 hex 中提取 x 的^{Witness}见证。为了达到这个目的，我们使用 $\exists$ 的消去规则：

```

Iff.intro
  (assume hex :  $\exists x : \alpha, x = t \wedge P x$ 

```

```

show P t from
  Exists.elim hex ( )
( )

```

第一个占位符需要子目标 $\vdash \forall x, x = t \wedge P x \rightarrow P t$ 的证明。我们先把它勾勒出来：

```

Iff.intro
(assume hex :  $\exists x : \alpha, x = t \wedge P x$ 
 show P t from
  Exists.elim hex
    (fix x :  $\alpha$ 
      assume hand :  $x = t \wedge P x$ 
      show P t from
        _))
( )

```

请注意子证明是如何镜像子目标的：fix 对应 $\forall$ ，assume 对应 $\rightarrow$ 的假设，而 show 对应 $\rightarrow$ 的结论。现在我们可以访问见证 x 及其特征属性 $x = t \wedge P x$ 。

接下来，我们使用 have 命令和 $\wedge$ 的消去规则 (And.left 和 And.right) 提取 $x = t \wedge P x$ 的两个 ^{Conjuncts}合取项：

```

Iff.intro
(assume hex :  $\exists x : \alpha, x = t \wedge P x$ 
 show P t from
  Exists.elim hex
    (fix x :  $\alpha$ 
      assume hand :  $x = t \wedge P x$ 
      have hxt :  $x = t$  :=
        And.left hand
      have hpx :  $P x$  :=
        And.right hand
      show P t from
        _))
( )

```

现在我们可以尝试证明 $P t$ 。我们将 t 重写为 x ，并应用假设 $P x$ ：

```

Iff.intro
(assume hex :  $\exists x : \alpha, x = t \wedge P x$ 
 show P t from
  Exists.elim hex

```

```

(fix x :  $\alpha$ 
  assume hand :  $x = t \wedge P\ x$ 
  have hxt :  $x = t$  :=
    And.left hand
  have hpx :  $P\ x$  :=
    And.right hand
  show  $P\ t$  from
    by
      rw [←hxt]
      exact hpx))
( )

```

对于剩余的占位符，我们首先根据子目标 $\vdash P\ t \rightarrow \exists x : \alpha, x = t \wedge P\ x$ 提供一个证明骨架：

```

(assume hp :  $P\ t$ 
  show  $\exists x : \alpha, x = t \wedge P\ x$  from
    _ )

```

然后我们使用 $\exists$ 的引入规则提供见证 t ：

```

(assume hp :  $P\ t$ 
  show  $\exists x : \alpha, x = t \wedge P\ x$  from
    Exists.intro t ( ))

```

接下来是更多的样板代码：

```

(assume hp :  $P\ t$ 
  show  $\exists x : \alpha, x = t \wedge P\ x$  from
    Exists.intro t
      (show  $t = t \wedge P\ t$  from
        _))

```

最后的合取项 $t = t \wedge P\ t$ 很容易证明：其左侧通过^{Reflexivity}自反性得出，而右侧仅仅是假设 hp 。最后，我们得到：

```

(assume hp :  $P\ t$ 
  show  $\exists x : \alpha, x = t \wedge P\ x$  from
    Exists.intro t
      (have tt :  $t = t$  :=
        by rfl
      show  $t = t \wedge P\ t$  from
        And.intro tt hp))

```

为了开发上述结构化证明，我们使用了与展示类型的^{Inhabitant}居留元(第 1.4 节)时大致相同的步骤，将函数箭头理解为蕴涵。这两个步骤之间的相似性并非巧合，正如我们将在第 4.7 节中看到的那样。

4.4 计算式证明

在非正式数学中，我们经常将证明表示为等式、不等式或等价式的^{Transitive Chain}传递链(例如 $a = b = c$ 、 $a \geq b \geq c$ 或 $a \leftrightarrow b \leftrightarrow c$)。在 Lean 中，此类^{Calculational Proofs}计算式证明由 `calc` 命令提供支持，该命令提供了一种轻量级语法，并负责为等式及算术比较运算符等^{Preregistered Relations}预注册关系应用^{Transitivity Theorems}传递性定理。

其通用语法如下：

```
calc
  项0 运算符1 项1 :=
    证明1
  _ 运算符2 项2 :=
    证明2
    ⋮
  _ 运算符n 项n :=
    证明n
```

下划线 (`_`) 是语法的一部分。每个 ^{Compatible}证明_{*i*} 证明了陈述 项_{*i-1*} 运算符_{*i*} 项_{*i*}。运算符_{*i*} 不必完全相同，但它们必须彼此兼容。例如，`=`、`<` 和 `≤` 是兼容的，而 `>` 和 `<` 则不兼容。

下面是一个简单的例子：

```
theorem two_mul_example (m n : ℕ) :
  2 * m + n = m + n + m :=
calc
  2 * m + n = m + m + n :=
    by rw [Nat.two_mul]
  _ = m + n + m :=
    by ac_rfl
```

数学家们（假设他们愿意为这样一个平凡的结论提供证明）可能会将上述证明大致写成如下形式：

$$\begin{aligned} 2 * m + n &= (m + m) + n && \text{(因为 } 2 * m = m + m \text{)} \\ &= m + n + m && \text{(通过 } + \text{ 的} \text{ } \overset{\text{Associativity Commutativity}}{\text{结合律和交换律}} \text{)} \end{aligned}$$

□

在 Lean 证明中，下划线代表项 $(m + m) + n$ ，如果我们不使用 `calc` 来编写证明，我们就不得不重复写出该项：

```
theorem two_mul_example_have (m n : ℕ) :
  2 * m + n = m + n + m :=
  have hmul : 2 * m + n = m + m + n :=
    by rw [Nat.two_mul]
  have hcomm : m + m + n = m + n + m :=
    by ac_rfl
  show _ from
    Eq.trans hmul hcomm
```

请注意，使用 `have` 还需要我们显式地调用 `Eq.trans`，并为这两个中间步骤命名。

4.5 使用策略进行正向推理

许多用户更喜欢^{Tactic Mode}**策略模式**而非结构化证明。但即使在策略模式下，以正向方式进行推理、混合正向和逆向推理步骤也是非常有用的。结构化证明命令 `have`、`let` 和 `calc` 也可以作为策略使用，这使得上述操作成为可能。

下面的例子在一个我们已经见过好多次的定理上演示了 `have` 和 `let` 策略：

```
theorem prop_comp_tactical (a b c : Prop) (hab : a → b)
  (hbc : b → c) :
  a → c :=
  by
    intro ha
    have hb : b :=
      hab ha
    let c' := c
    have hc : c' :=
      hbc hb
    exact hc
```

have

```
have 名称 : 命题 :=
  证明
```

`have` 策略允许我们在策略模式下陈述并证明一个中间定理。之后，该定理在^{Goal State}**目标状态**中可用作一个假设。

let

`let 名称 [: 类型] := 项`

`let` 策略允许我们在策略模式下引入一个局部定义。之后，所定义的符号及其定义在目标状态中可用。

calc

`calc`

```
项0 运算符1 项1 :=
  证明1
_ 运算符2 项2 :=
  证明2
  ⋮
_ 运算符n 项n :=
  证明n
```

`calc` 策略允许我们在策略模式下进入计算式证明（第 4.4 节）。该策略与同名的结构化证明命令具有相同的语法。我们可以将作为策略的 `calc ...` 看作是作为上述结构化证明命令的 `apply (exact calc ...)` 的别名。

4.6 依值类型

^{Dependent Type} **依值类型** 是 ^{Dependent Type Theory} **依值类型论** 这一系列逻辑系统的定义性特征。虽然你可能不熟悉这个术语，但你很可能以某种形式熟悉这个概念。

考虑一个函数 `pick`，它接受一个 ^{Natural Number} **自然数** n （即来自 $\mathbb{N} = \{0, 1, 2, \dots\}$ 的一个值），并返回一个介于 0 和 n 之间的自然数。直观上，`pick n` 的类型应该是 $\{0, 1, \dots, n\}$ （即由所有满足 $i \leq n$ 的自然数组成的类型）。在 Lean 中，这个类型写为 $\{i : \mathbb{N} // i \leq n\}$ 。这就是 `pick n` 的类型。具有数学背景的读者可能会倾向于将 `pick` 看作一个由 $\mathbb{N}$ ^{Index} **索引** 的 ^{Indexed Family of Terms} **索引项族**：

$$(\text{pick } n : \{i : \mathbb{N} // i \leq n\})_{n:\mathbb{N}}$$

其中每个项的类型都依赖于索引——例如，`pick 5 : {i :  $\mathbb{N}$  //  $i \leq 5$ }`。但是 `pick` 本身的类型是什么呢？我们希望表达 `pick` 是一个函数，它接受一个参数 $n : \mathbb{N}$ ，并返回一个类型为 $\{i : \mathbb{N} // i \leq n\}$ 的值。为了体现这一点，我们可以写作：

$$\text{pick} : (n : \mathbb{N}) \rightarrow \{i : \mathbb{N} // i \leq n\}$$

这是一个依值类型：结果的类型依赖于参数 n 的值。（变量名 n 本身是无关紧要的；我们也可以写成 m 或 x_0 。）

除非另有说明，否则「依值类型」指的是依赖于一个（非类型）项的类型，如上所示，其中 $n : \mathbb{N}$ 是项，而 $\{i : \mathbb{N} // i \leq n\}$ 是依赖于它的类型。但是：

- 类型也可以依赖于另一个类型——例如，类型构造子 `List`，其 η -展开的变体 $\text{fun } \alpha : \text{Type} \mapsto \text{List } \alpha$ ，或者具有相同定义域和陪域的函数的多态类型 $\text{fun } \alpha : \text{Type} \mapsto \alpha \rightarrow \alpha$ 。
- 项可以依赖于类型——例如，多态恒等函数 $\text{fun } \alpha : \text{Type} \mapsto \text{fun } x : \alpha \mapsto x$ 。
- 当然，项也可以依赖于项——例如， $\text{fun } n : \mathbb{N} \mapsto n + 2$ 。

综上所述， $\text{fun } x \mapsto t$ 共有四种情况：

函数体 (t)	参数 (x)	描述
项	依赖于 项	简单类型 λ -表达式
类型	依赖于 项	狭义的依值类型
项	依赖于 类型	多态项
类型	依赖于 类型	多态类型构造子

最后三行对应于 Henk Barendregt 的 λ -立方的三条轴。¹

第 1.3 节中介绍的 APP 和 FUN 规则必须经过泛化，才能适用于依值类型：

$$\begin{array}{c}
 \frac{C \vdash t : (x : \sigma) \rightarrow \tau[x] \quad C \vdash u : \sigma}{C \vdash t u : \tau[u]} \text{APP}' \\
 \\
 \frac{C, x : \sigma \vdash t : \tau[x]}{C \vdash (\text{fun } x : \sigma \mapsto t) : (x : \sigma) \rightarrow \tau[x]} \text{FUN}'
 \end{array}$$

记号 $\tau[x]$ 代表一个可能包含 x 的类型，而 $\tau[u]$ 代表将其中所有出现的 x 替换为 u 后得到的相同类型。

当 x 不出现在 $\tau[x]$ 中时，就出现了简单类型的情况。此时，我们可以简单地将 $(x : \sigma) \rightarrow$ 写为 $\sigma \rightarrow$ 。我们所熟悉的记号 $\sigma \rightarrow \tau$ 等价于 $(_ : \sigma) \rightarrow \tau$ 。很容易验证，当 x 不出现在 $\tau[x]$ 中时，APP' 和 FUN' 与 APP 和 FUN 是重合的。

下面的例子演示了 APP'：

$$\frac{\vdash \text{pick} : (n : \mathbb{N}) \rightarrow \{i : \mathbb{N} // i \leq n\} \quad \vdash 5 : \mathbb{N}}{\vdash \text{pick } 5 : \{i : \mathbb{N} // i \leq 5\}} \text{APP}'$$

¹https://en.wikipedia.org/wiki/Lambda_cube

下一个例子演示了 FUN'：

$$\frac{\frac{\alpha : \text{Type}, x : \alpha \vdash x : \alpha}{\alpha : \text{Type} \vdash (\text{fun } x : \alpha \mapsto x) : \alpha \rightarrow \alpha} \text{FUN or FUN'}}{\vdash (\text{fun } \alpha : \text{Type} \mapsto \text{fun } x : \alpha \mapsto x) : (\alpha : \text{Type}) \rightarrow \alpha \rightarrow \alpha} \text{FUN'}$$

这一图景并不完整，因为我们只检查了项——即冒号（:）左侧的实体——是否类型正确。冒号右侧的实体——即类型——也应该使用相同的类型系统进行检查。例如，`Nat.succ` 的类型是 $\mathbb{N} \rightarrow \mathbb{N}$ ，其类型是 `Type`。类型的类型（例如 `Type` 和 `Prop`）被称为^{Universe}**宇宙**。我们将在第 12 章中更深入地研究它们。

值得注意的是，^{Universal Quantification}**全称量化**仅仅是^{Dependent Type}**依值类型**的一个别名： $\forall x : \sigma, \tau$ 是 $(x : \sigma) \rightarrow \tau$ 的缩写。这一点在下文中会变得更加清晰。

4.7 PAT 原理

你可能已经注意到，同样的符号 $\rightarrow$ 既用于^{Implication}**蕴含**（例如 `False → True`），也作为^{Type Constructor}**类型构造子**（例如 $\mathbb{Z} \rightarrow \mathbb{N}$ ）。如果没有^{Context}**语境**，我们无法分辨 $a \rightarrow b$ 是指一个定义域为 a 且陪域为 b 的函数类型，还是命题「 a 蕴含 b 」。

事实证明，这两个概念不仅在形式上「看起来」相同，它们「确实」是同一个东西。这被称为^{PAT Principle}**PAT 原理**，其中 PAT 是一个双重助记词：

$$\begin{array}{ll} \text{Propositions as Types} & \text{Proofs as Terms} \\ \text{PAT} = \text{命题即类型} & \text{PAT} = \text{证明即项} \end{array}$$

此外，由于类型也是项，这也意味着命题也是项。然而，PAT 并不是一个四重助记词（~~PAT = 证明即类型~~，^{Proofs as Types}**证明即类型**）。另请注意，并非所有类型都是命题，也并非所有项都是证明。

通过使用项和类型来表示证明和命题，依值类型论实现了概念上的极大精简。问题

「H 是 P 的证明吗？」

变得等同于

「项 H 的类型是 P 吗？」

因此，在 Lean 内部，并没有专门的证明检查器，只有类型检查器。证明由类型检查器进行检查。

让我们逐一回顾这些主要实体。我们使用数学变量 σ 和 τ 表示类型；使用 P 和 Q 表示命题；使用 t 、 u 和 x 表示项；使用 h 、 G 和 H 表示证明。

从「命题即类型」开始，对于类型，我们有：

- $\sigma \rightarrow \tau$ 是从 σ 到 τ 的函数类型。
- $(x : \sigma) \rightarrow \tau[x]$ 是从 $x : \sigma$ 到 $\tau[x]$ 的依值函数类型。

相比之下，对于命题，我们有：

- $P \rightarrow Q$ 可以读作「P 蕴含 Q」，或读作将 P 的证明映射到 Q 的证明的函数类型。
- $\forall x : \sigma, Q[x]$ 可以读作「对于所有的 x, Q[x]」，或读作类型为 $(x : \sigma) \rightarrow Q[x]$ 的函数类型，它将类型为 σ 的值 x 映射到 Q[x] 的证明。

继续看「证明即项」，对于项，我们有：

- 常量是一个项。
- 变量是一个项。
- $t\ u$ 是函数 t 对参数 u 的应用。
- $\text{fun } x \mapsto t[x]$ 是一个将 x 映射到 $t[x]$ 的函数。

相比之下，对于证明（即证明项），我们有：

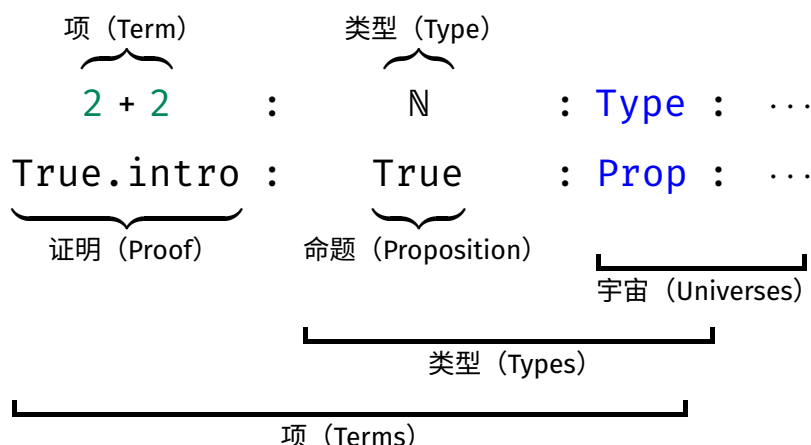
- 定理或假设的名称是一个证明。
- $H\ t$ 是一个证明，它通过项 t 对证明 H 陈述中的首个 $\forall$ 量词进行^{Instantiate}实例化。
- $H\ G$ 是一个证明，它通过证明 G ^{Discharge}解除了 H 陈述中的首个前提。这种操作被称为^{Modus Ponens}肯定前件。
- $\text{fun } h : P \mapsto H[h]$ 是 $P \rightarrow Q$ 的一个证明，前提是对于所有 $h : P$, $H[h]$ 都是 Q 的证明。
- $\text{fun } x : \sigma \mapsto H[x]$ 是 $\forall x : \sigma, Q[x]$ 的一个证明，前提是对于所有 $x : \sigma$, $H[x]$ 都是 Q[x] 的证明。

最后两种情况由 Fun' 规则支持。与原始证明项不同，在结构化证明中，我们会使用 assume 或 fix 来代替 fun，且为了可读性，我们通常还想使用 show 重复一遍结论，如下所示：

```
theorem case_4 :
  P → Q :=
  assume h : P
  show Q from
    H[h]

theorem case_5 :
  ∀x : σ, Q[x] :=
  fix x : σ
  show Q[x] from
    H[x]
```

依值类型论的术语可能会让人感到相当困惑，因为某些词具有狭义和广义之分。下面的图表展示了重要词汇的各种含义：



从广义上讲，任何表达式都是一个^{Term}项，任何可能出现在^{Typing Judgment}类型判断 $(:)$ 右侧的表达式都是一个^{Type}类型，而任何出现在类型判断右侧，且其左侧为一个类型的表达式，都是一个^{Universe}宇宙。这与将 $t : u$ 读作「 t 的类型为 u 」以及宇宙是类型的类型这一概念是一致的。

根据 PAT 原理，一些 Lean 命令虽然有两个名称，但本质上是相同的。`fix` 和 `assume` 就是这种情况；事实上，它们都是 LoVeLib 中 `fun` 的别名。

还有一些成对出现的命令，其行为略有不同，例如 `def/theorem` 和 `let/have`。根本区别在于：当我们定义某个函数或数据时，我们不仅关心其类型，还关心其函数体——也就是行为。另一方面，一旦我们证明了一个定理，证明本身就变得^{Irrelevant}无关紧要了。唯一重要的是^{Proof Irrelevance}存在一个证明。我们将在第 12 章回到证明无关性这一话题。

下表总结了策略证明、结构化证明和^{Raw Proof Terms}原始证明项之间的区别：

策略证明	结构化证明	原始证明项
<code>intro x</code>	<code>fix x : τ</code>	<code>fun x $\mapsto$</code>
<code>intro h</code>	<code>assume h : P</code>	<code>fun h $\mapsto$</code>
<code>have k := H</code>	<code>have k := H</code>	<code>(fun k $\mapsto \dots$) H</code>
<code>let x := t</code>	<code>let x := t</code>	<code>(fun x $\mapsto \dots$) t</code>
<code>exact (H : P)</code>	<code>show P from H</code>	<code>H : P</code>
<code>calc</code>	<code>calc</code>	<code>calc</code>

注意，后跟 u 的 `let x := t` 本质上是编写 `(fun x  $\mapsto$  u) t` 的一种方式，只不过^{Reduction} β -归约被禁用了。

4.8 通过模式匹配与递归进行归纳

在第 3.6 节中，我们回顾了使用 `induction` 策略来进行归纳证明的方法。另一种更灵活的风格依赖于模式匹配和 ^{PAT Principle}**PAT 原理**。

回想一下在第 2.2.3 节末尾给出的列表^{Reverse}反转的定义：

```
def reverse {α : Type} : List α → List α
| []      => []
| x :: xs => reverse xs ++ [x]
```

事实上，`reverse` 在 Lean 的标准库中以 `List.reverse` 的形式存在，其定义更高效，但对于推理来说不那么方便。一个值得证明的有用性质是 `reverse` 是它自身的逆：对于所有列表 `xs`，均有 `reverse (reverse xs) = xs`。然而，如果我们尝试通过归纳法证明它，很快就会遇到障碍。其^{Induction Step}归纳步骤为：

```
: ∀ xs, reverse (reverse xs) = xs ⊢ reverse (reverse xs ++ [x]) = x ::
xs ih
```

请注意，在这个双重 `reverse` 的「夹层」中出现了令人不悦的 `++ [x]`。我们需要一种方法将外层的 `reverse`「分配」到 `++` 上，以获得一个能与归纳假设左侧匹配的项。诀窍是证明并使用以下定理：

```
theorem reverse_append {α : Type} :
  ∀ xs ys : List α,
    reverse (xs ++ ys) = reverse ys ++ reverse xs
| [],      ys => by simp [reverse]
| x :: xs, ys => by simp [reverse, reverse_append xs]
```

该定理的证明与其说是证明，倒不如说更像是一个递归函数的定义。左侧的模式 `[]` 和 `x :: xs` 对应于 $\forall$ 量化变量 `xs` 的两个^{Constructor}构造子。在每个 `=>` 符号的右侧是对应情况的证明。我们可以进行模式匹配的变量是那些出现在 $\forall$ 量词中的变量，按出现顺序排列（此处为 `xs` 和 `ys`）。在归纳步的证明内部，归纳假设使用的名称与我们正在证明的定理名称相同（`reverse_append`）。

我们显式地将 `xs` 作为参数传递给归纳假设。这限制了该假设，使其仅适用于 `xs` 而不适用于其他列表。特别是，这确保了定理不会应用于 `x :: xs`，否则会导致循环证明：「为了证明 `reverse_append (x :: xs)`，请使用 `reverse_append (x :: xs)`」。Lean 的终止性检查器会发现该证明是非良基的并报错，但我们希望避免这种情况。此外，显式参数 `xs` 也可以作为文档，实际上是在说：「我们需要的该定理的唯一递归实例是在 `xs` 上的实例」。

作为参考，^{Tactical Proof}策略式证明如下：

```
theorem reverse_append_tactical {α : Type} (xs ys : List
α) :
```

```
reverse (xs ++ ys) = reverse ys ++ reverse xs :=
by
  induction xs with
  | nil          => simp [reverse]
  | cons x xs' ih => simp [reverse, ih]
```

如果我们用 `[y]` 代替 `ys`，该定理同样是可证明且有用的。但尽可能一般地陈述定理是一个好习惯，这会使库更具可重用性。此外，在归纳证明中，为了获得足够强的归纳假设，这通常是必需的。一般而言，寻找合适的归纳方式和定理需要思考和创造力。

Lean 支持对多个变量同时进行模式匹配（例如上面的 `xs` 和 `ys`）。此时模式之间用逗号分隔。其一般格式为：

```
theorem name (参数1 : 类型1) ... (参数m : 类型m) :
  语句
| 模式1 => 证明1
  :
| 模式n => 证明n
```

注意它与 `def` 的语法（第 2.2 节）有很强的相似性。事实上，这两个命令几乎完全相同，但 `theorem` 认为定义的项或证明是^{Opaque}不透明的，而 `def` 则保持其^{Transparent}透明。由于一旦定理被证明，实际的证明就不再重要了（第 12 章），因此稍后无需对其进行展开。`let` 和 `have` 之间也存在类似的区别。

根据 PAT 原理，通过模式匹配和递归进行的归纳证明等同于一个递归^{Proof Term}证明项。当我们调用归纳假设时，实际上只是在递归地调用一个递归函数。这解释了为什么归纳假设与我们证明的定理同名。Lean 的终止性检查器被用来建立归纳证明的良基性。

有了 `reverse_append` 定理，我们可以回到最初的目标：

通过模式匹配和递归进行归纳在 Lean 用户中很受欢迎。它的主要优点是语法方便，且支持^{Well-founded Induction}良基归纳，这比 `induction` 策略（第 3.7 节）提供的^{Structural Induction}结构归纳更强大。然而，在本指南中，我们不需要良基归纳的全部威力。此外，出于微妙的逻辑原因，通过模式匹配和递归进行的归纳不适用于^{Inductive Predicates}归纳谓词，这也是第 6 章的主题。基于这些原因，我们通常更喜欢使用 `induction` 策略。

4.9 新引入的 Lean 结构总结

证明命令

<code>assume</code>	声明假设
<code>calc</code>	通过传递性组合证明

<code>fix</code>	固定变量
<code>have</code>	声明中间定理
<code>let</code>	引入局部定义
<code>show</code>	声明目标

策略

<code>calc</code>	通过传递性组合证明
<code>have</code>	声明中间定理
<code>let</code>	引入局部定义

第二部分

函数式与逻辑编程

函数式编程

我们将深入探讨带类型函数式编程的本质：归纳类型、归纳证明、递归函数、模式匹配、结构体（记录）以及类型类。这里涵盖的概念在《Lean 4 定理证明》[2] 的第 7 到 10 章中有更详细的描述。

5.1 归纳类型

归纳类型模仿了带类型函数式编程语言（如 Haskell、ML、OCaml）中的数据类型。它们也让人联想到 Scala 中的密封类。在第 2 章中，我们看到了一些基本的归纳类型：自然数、算术表达式类型以及有限列表。在本章中，我们将重新审视列表并研究二叉树。我们还将简要了解长度为 n 的向量，这是一种依值类型。

回想一下作为归纳类型的自然数定义：

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

这定义了类型 `Nat` 以及两个常量 `Nat.zero` 和 `Nat.succ`，它们被称为构造子。该定义还断言了构造子的一些性质。此外，它还引入了额外的常量，用于在内部支持归纳和递归。

正如我们在第 2.1 节中看到的，归纳类型是一种其成员全部由（且仅由）有限次应用其构造子所构建的值组成的类型。格言：

- **无杂质**：类型中不包含除使用构造子可表达的值之外的任何值。
- **无混淆**：使用不同构造子组合构建的值是不同的。

对于自然数,「无杂质」意味着不存在无法使用 `Nat.zero` 和 `Nat.succ` 的有限组合来表达的特殊值 (如 -1 、 ε 、 ∞ 或 `NaN`) ; 「无混淆」则确保了对于所有 n , $\text{Nat.zero} \neq \text{Nat.succ } n$, 且 Nat.succ 是^{Injective}单射的。

此外, 归纳类型的值总是有限的。无限项

`Nat.succ (Nat.succ (Nat.succ (Nat.succ ...)))`

并不是一个值。同样地, 也不存在一个值 n 使得 $\text{Nat.succ } n = n$, 这一点我们将在下文中证明。

归纳类型使用起来非常方便, 因为它们支持归纳和递归, 且它们的构造子表现良好; 但并非所有类型都能定义为归纳类型。特别地, 诸如 $\mathbb{Q}$ (有理数) 和 $\mathbb{R}$ (实数) 等数学类型需要更复杂的构造, 这些构造基于^{Quotienting}商构造和^{Subtyping}子类型化。这将在第 12 和第 14 章中详细说明。

5.2 结构归纳

Structural Induction

结构归纳是数学归纳法向任意归纳类型的推广。为了通过对 n 的结构归纳来证明目标 $n : \mathbb{N} \vdash P[n]$, 只需证明两个子目标, 传统上称为^{Base Case}基本情况和^{Induction Step}归纳步骤:

$\vdash P[0]$

$k : \mathbb{N}, ih : P[k] \vdash P[k + 1]$

当然, 我们也可以写 `Nat.zero` 和 `Nat.succ k` 来代替 0 和 $k + 1$ 。

通常情况下, 情况会更复杂。目标可能包含一些不依赖于 n 的额外假设 (例如 Q), 以及一些依赖于 n 的假设 (例如 $R[n]$)。假如我们每种假设各有一个, 这就给出了初始目标:

$hQ : Q, n : \mathbb{N}, hR : R[n] \vdash S[n]$

对 n 进行结构归纳会产生两个子目标:

$hQ : Q, hR : R[0] \vdash S[0]$

$hQ : Q, k : \mathbb{N}, ih : R[k] \rightarrow S[k], hR : R[k + 1] \vdash S[k + 1]$

假设 Q 只是简单地从初始目标原封不动地继承下来, 而 $R[n] \vdash S[n]$ 处理起来几乎就像目标的目的变成了 $R[n] \rightarrow S[n]$ 。在上面的第一个例子中, 令 $P[n] := R[n] \rightarrow S[n]$ 即可轻松验证这一点。由于这种通用格式非常冗长且几乎不包含额外信息 (既然我们已经理解了其工作原理), 从现在起, 我们将以尽可能简单的形式呈现目标, 不带额外假设。

对于列表, 给定目标 $xs : \text{list } \alpha \vdash P[xs]$, 对 xs 进行结构归纳得到:

$\vdash P[[]]$

$y : \alpha, ys : \text{list } \alpha, ih : P[ys] \vdash P[y :: ys]$

当然，我们也可以写 `List.nil` 和 `List.cons y ys`。这里没有与 `y` 关联的归纳假设，因为 `y` 不是列表类型的。

对于算术表达式，基本情况是：

$$i : \mathbb{Z} \vdash P[\text{AExp.num } i] \qquad x : \text{String} \vdash P[\text{AExp.var } x]$$

归纳步骤是：

$$e_1 \ e_2 : \text{AExp}, ih_1 : P[e_1], ih_2 : P[e_2] \vdash P[\text{AExp.add } e_1 \ e_2]$$

$$e_1 \ e_2 : \text{AExp}, ih_1 : P[e_1], ih_2 : P[e_2] \vdash P[\text{AExp.sub } e_1 \ e_2]$$

$$e_1 \ e_2 : \text{AExp}, ih_1 : P[e_1], ih_2 : P[e_2] \vdash P[\text{AExp.mul } e_1 \ e_2]$$

$$e_1 \ e_2 : \text{AExp}, ih_1 : P[e_1], ih_2 : P[e_2] \vdash P[\text{AExp.div } e_1 \ e_2]$$

注意关于 e_1 和 e_2 的两个归纳假设。

通常情况下，结构归纳会为每个构造子产生一个子目标。在每个子目标中，对于我们正在进行归纳的类型的构造子参数，归纳假设都是可用的。

给定一个归纳类型 τ ，计算子目标的步骤总是相同的：

1. 将 $P[]$ 中的空缺替换为应用于新变量（例如 $y :: ys$ ）的每个可能的构造子，产生与构造子数量相同的子目标。
2. 将这些新变量（例如 y 、 ys ）添加到局部语境中。
3. 为所有类型为 τ 的新变量添加归纳假设。

例如，我们将证明对于所有 $n : \mathbb{N}$ ， $\text{Nat.succ } n \neq n$ 。我们从一个非形式证明开始：

证明采用对 n 的结构归纳。

情况 0: 我们必须证明 $\text{Nat.succ } 0 \neq 0$ 。这遵循归纳类型构造子的「无混淆」性质。

情况 $\text{Nat.succ } k$: 归纳假设是 $\text{Nat.succ } k \neq k$ 。我们必须证明 $\text{Nat.succ } (\text{Nat.succ } k) \neq \text{Nat.succ } k$ 。通过 Nat.succ 的单射性，我们得到 $\text{Nat.succ } (\text{Nat.succ } k) = \text{Nat.succ } k$ 等价于 $\text{Nat.succ } k = k$ 。因此，只需证明 $\text{Nat.succ } k \neq k$ ，这恰好对应于归纳假设。 $\square$

请注意此非形式证明的主要特征，你应该在自己的非形式论证中努力复现这些特征：

- 证明以明确声明我们正在进行的证明类型开始（例如，哪种归纳以及对哪个变量进行归纳）。
- 各个情况都被清晰地识别出来，并且对于每个情况，都陈述了目标的目的和假设。

- 明确调用了证明所依赖的关键定理（例如，`Nat.succ` 的单射性）。

现在让我们在 Lean 中完成这个证明：

```
theorem Nat.succ_neq_self (n : ℕ) :
  Nat.succ n ≠ n :=
by
  induction n with
  | zero      => simp
  | succ n' ih =>
    intro hsucc
    apply ih
    apply Nat.succ.inj hsucc
```

这个证明有些刻意，因为它可以用单个 `simp` 调用所取代，但它仍然拥有启发意义。

5.3 结构化递归

Structural Recursion

结构化递归是一种递归形式，它允许我们从进行递归的值中剥离出一个构造子。下面的阶乘函数就是结构化递归的：

```
def fact : ℕ → ℕ
| 0      => 0
| n + 1 => (n + 1) * fact n
```

我们在这里剥离的构造子是 `Nat.succ`（写作 `+ 1`），这样的函数保证在递归停止前只调用自身有限次。例如，`fact 12345` 将调用自身 12345 次。该函数被称为是 ^{Terminate}**终止**的，这一性质有助于确保逻辑一致性。

在结构化递归中，等式的数量与构造子的数量相同。初学者往往倾向于提供额外的冗余情况，如下例所示：

```
def factThreeCases : ℕ → ℕ
| 0      => 0
| 1      => 1
| n + 1 => (n + 1) * factThreeCases n
```

抵制这种诱惑对你最有好处。定义中的情况越多，对其进行推理的工作量就越大。请记住那句格言：「一个好的定义抵得上三个定理」。

对于结构化递归函数，Lean 可以自动证明其终止性。对于更通用的递归模式，终止性检查可能会失败。有时失败是有充分理由的，如下例所示：

```
-- fails
def illegal : ℕ → ℕ
```

```
| n => illegal n + 1
```

如果 Lean 接受这个定义,我们就可以利用它来证明 $0 = 1$,只需从等式 $\text{illegal } n = \text{illegal } n + 1$ 的两边减去 $\text{illegal } n$ 。从 $0 = 1$, 我们可以推导出 `False`, 再从 `False`, 我们可以推导出任何结论。显然, 我们不要这样的结果。

如果我们使用 `opaque` 和 `axiom`, 那就没有任何办法了:

```
opaque immoral : ℕ → ℕ

axiom immoral_eq (n : ℕ) :
  immoral n = immoral n + 1

theorem proof_of_False :
  False :=
  have hi : immoral 0 = immoral 0 + 1 :=
    immoral_eq 0
  have him :
    immoral 0 - immoral 0 = immoral 0 + 1 - immoral 0 :=
    by rw [←hi]
  have h0eq1 : 0 = 1 :=
    by simp at him
  show False from
    by simp at h0eq1
```

我们更倾向于使用 `def` 而非 `opaque` 和 `axiom` 的另一个原因是, 定义的等式可以用于计算。像 `rfl` 这样在计算意义下进行统一的策略, 在我们每次引入一个定义时都会变得更强大; 而诊断命令 `#eval` 和 `#reduce` 也可以用于已定义的常量。

细心的读者会注意到, 上面关于阶乘的定义在数学上是错误的: 无论参数是什么, `fact` 和 `factThreeCases` 竟然都返回 0。我们确实搞砸了。这些令人尴尬的错误提醒我们要「测试」我们的定义, 并「证明」它们的一些性质。虽然有缺陷的公理偶尔会出现, 但更常见的是定义未能捕获预想的概念。仅仅因为一个函数被命名为 `fact`, 并不意味着它实际上就在计算阶乘。

5.4 模式匹配表达式

模式匹配不仅可以在 `def` 命令的顶层进行, 还可以通过 `match` 表达式深入到项的内部。该构造拥有以下通用语法:

```

match 项1, ..., 项m with
  | 模式11, ..., 模式1m => 结果1
  ⋮
  | 模式n1, ..., 模式nm => 结果n

```

该构造让人联想到一些现代编程语言中的 match。上述 match 表达式的含义如下：

考虑项 项₁, ..., 项_m。

如果它们的形式分别为模式₁₁, ..., 模式_{1m},
则得到 结果₁。

⋮

如果它们的形式分别为模式_{n1}, ..., 模式_{nm},
则得到 结果_n。

模式可以包含变量、构造子和匿名占位符 (`_`)。表达式 结果_i 可以引用相应模式中引入的变量。

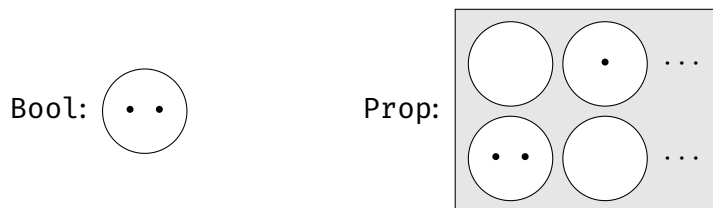
下面的函数定义演示了表达式内部模式匹配的语法：

bcount 函数计算列表中满足给定谓词 p 的元素数量。该谓词的 ^{Codomain}陪域是 Bool。
^{Boolean}布尔值类型 Bool，而在陈述程序性质时使用 ^{Proposition}命题类型 Prop。Bool 类型的两个值被称为 false 和 true（以小写字母形式）。

¹

^{Connective}逻辑联结词被称为 or（中缀：||）、and（中缀：&&）以及 not（前缀：!）。

下图展示了 Bool 和 Prop 的解释：



点代表元素，圆圈代表类型，矩形代表类型的类型，即宇宙。我们可以看到，Bool 被解释为拥有两个值的集合，而 Prop 由无限多个命题（即类型）组成，每个命题都有零个或多个证明（即元素）。我们将在第 6 章中完善这张图。

我们不能对命题（类型为 Prop）进行模式匹配，但我们可以使用 if-then-else 来代替。例如，自然数上的 min 运算符可以定义如下：

¹Lean 也允许我们使用 True 和 False，但它们会被隐式地从 Prop 转换为 Bool。我们通常建议避免这种隐式强制转换。遗憾的是，目前没有办法禁用它们。

```
def min (a b : ℕ) : ℕ :=
  if a ≤ b then a else b
```

这需要一个^{Decidable}**可判定**(即可执行)的命题。对于 $\leq$ 来说确实如此: 给定具体的参数(如 35 和 49), Lean 可以将 $35 \leq 49$ 约化为 True。Lean 使用一种称为^{Type Class}**类型类**的机制来跟踪可判定性, 下文将对此进行解释。

5.5 结构体

Lean 提供了定义^{Structure}**结构体**(也称为^{Record}**记录**)的便捷语法。它们本质上是拥有一些语法糖的非递归、单构造子的归纳类型。

下面的定义引入了一个名为 RGB 的结构体, 它有三个类型为 $\mathbb{N}$ 的字段, 分别名为 red、green 和 blue:

```
structure RGB where
  red    : ℕ
  green  : ℕ
  blue   : ℕ
```

此定义的效果大致相当于以下命令:

```
inductive RGB : Type where
  | mk : ℕ → ℕ → ℕ → RGB

def RGB.red : RGB → ℕ
  | RGB.mk r _ _ => r

def RGB.green : RGB → ℕ
  | RGB.mk _ g _ => g

def RGB.blue : RGB → ℕ
  | RGB.mk _ _ b => b
```

我们可以将一个新结构体定义为现有结构体的扩展。下面的定义用第四字段 alpha 扩展了 RGB:

```
structure RGBA extends RGB where
  alpha : ℕ
```

定义结构体的通用语法为:

```
structure 结构体名 (参数1 : 类型1) ... (参数k : 类型k)
  [extends 结构体1, ..., 结构体m] where
```

```

    字段名1 : 字段类型1
    ⋮
    字段名n : 字段类型n

```

参数 参数₁、...、参数_k 实际上也是额外的字段，但与 字段名₁、...、字段名_n 不同，它们存储在类型中，作为类型构造子 (结构体名) 的参数。

可以使用多种语法来指定值：

```

def pureRed : RGB :=
  RGB.mk 0xff 0x00 0x00

def pureGreen : RGB :=
  { red    := 0x00
    green  := 0xff
    blue   := 0x00 }

def semitransparentGreen : RGBA :=
  { pureGreen with
    alpha := 0x7f }

```

semitransparentGreen 的定义从 pureGreen 中复制了所有值，但显式设置了 alpha 字段。

接下来，我们定义一个名为 shuffle 的操作：

```

def shuffle (c : RGB) : RGB :=
  { red    := RGB.green c
    green  := RGB.blue c
    blue   := RGB.red c }

```

该定义依赖于生成的选择器 RGB.red、RGB.green 和 RGB.blue。除了 RGB.red c，我们也可以写成 c.red，其他字段同理。有时我们会在 Lean 的输出中看到这种记法，尽管我们自己不使用它。

连续三次应用 shuffle 就相当于完全没有应用它：

```

theorem shuffle_shuffle_shuffle (c : RGB) :
  shuffle (shuffle (shuffle c)) = c :=
  by rfl

```

5.6 类型类

Type Class

类型类是由 Haskell 推广的一种机制，目前存在于多个证明助手中。在 Lean 中，

类型类是一种结合了抽象常量及其性质的结构体类型。²

通过为常量提供具体定义并证明其性质成立，可以将一个类型声明为类型类的实例。^{Instance}根据类型，Lean 会检索相关的实例。

一个简单的例子是 `Inhabited` 类型类，它只需要一个常量 `Inhabited.default`，而不需要任何性质：

```
class Inhabited (α : Type) : Type where
  default : α
```

类的定义语法与结构体相同，只是使用关键字 `class` 代替了 `structure`。参数 α 代表可以作为该类成员的任意类型。这个特定的类型类拥有单个参数和单个字段，但通常情况下，一个类型类可以拥有多个参数和多个字段。

任何拥有至少一个元素的类型都可以被注册为 `Inhabited` 类型类的实例。例如，我们可以通过选择一个任意数字作为默认值来注册 `N`：

```
instance Nat.Inhabited : Inhabited N :=
  { default := 0 }
```

这指定了一个名字为 `Nat.Inhabited`、类型为 `Inhabited N` 的值。因为我们使用的是关键字 `instance` 而非 `def`，因此该结构体值被注册为「**典范实例**」^{Canonical Instance}，以便在需要类型为 `Inhabited N` 的结构体时使用。在存储类型类实例的全局表中，现在有一个条目：

$$\text{Inhabited } N \mapsto \text{Nat.Inhabited}$$

对于列表，即使 α 不包含任何居留元，空列表也是一个显而易见的默认值：

```
instance List.Inhabited {α : Type} : Inhabited (List α) :=
  { default := [] }
```

这在全局表中添加了以下条目：

$$\text{Inhabited (List ?}\alpha) \mapsto \text{List.Inhabited}$$

作为一个例子，观察到类型为 $\alpha \rightarrow \beta$ 的有限函数可以用它们的函数表来表示，函数表的类型为 $\beta \times \dots \times \beta$ （包含 $|\alpha|$ 个 β 的副本）。因此， $|\alpha \rightarrow \beta| = |\beta|^{|\alpha|}$ 。要使结果为 0，必须满足 $|\beta| = 0$ 且 $|\alpha| \neq 0$ 。换句话说，如果 (1) β 有居留元或 (2) α 「没有」居留元，则类型 $\alpha \rightarrow \beta$ 有居留元。我们关注情况 (1)：

```
instance Fun.Inhabited {α β : Type} [Inhabited β] :
  Inhabited (α → β) :=
  { default := fun a : α ↦ Inhabited.default }
```

²尽管名称如此，但相比 Haskell 的类型类而言，Lean 的类型类机制与 Scala 的^{implicit argument}隐式参数的关系更密切。

该实例本身依赖于类型类相同但类型不同的实例, 这种情况经常发生。

^{Pair}
偶对的类型 $\alpha \times \beta$, 也称为^{Product Type}**积类型**, 包含形式为 (a, b) 的值, 其中 $a : \alpha$ 且 $b : \beta$ 。给定一个偶对 $ab : \alpha \times \beta$, 可以通过编写 `Prod.fst ab` 和 `Prod.snd ab` 来提取其第一和第二分量。要提供 $\alpha \times \beta$ 的居留元, 我们需要 α 的居留元和 β 的居留元:

```
instance Prod.Inhabited {α β : Type}
  [Inhabited α] [Inhabited β] :
  Inhabited (α × β) :=
  { default := (Inhabited.default, Inhabited.default) }
```

使用 `Inhabited` 类型类, 我们可以定义列表上的「head」操作: 即返回列表第一个元素的函数。由于空列表不包含任何元素, 因此在这种情况下没有可以返回的有意义的值。给定一个属于 `Inhabited` 类型类的类型, 我们可以简单地返回其默认值:

```
def head {α : Type} [Inhabited α] : List α → α
  | []      => Inhabited.default
  | x :: _ => x
```

通过编写 `[Inhabited α]`, 我们要求 α 属于 `Inhabited`。这允许我们在定义中访问 `Inhabited.default`。

语法 `[Inhabited α]` 为 `head` 常量添加了一个隐式参数。但与其他隐式参数不同, Lean 会通过所有已声明的实例进行「类型类搜索」, 以确定该参数的值。因此, 在运行命令

```
#eval head ([] : List ℕ)
```

时, Lean 会寻找 `Inhabited ℕ` 的实例并找到我们上面声明的实例 `Nat.Inhabited`。在那次声明中, 我们将 `default` 设置为 0, 因此这就是 `#eval` 打印出的内容。如果有多个实例适用且 Lean 选择了错误的一个, 我们可以使用 `@` 语法将类型类参数转换为显式参数, 并提供所需的类型类实例。

让我们仔细看看 `Inhabited.default`:

```
Inhabited.default {α : Type} [Inhabited α] : α
```

注意方括号 `[ ]` 的使用。当我们使用 `Inhabited.default α` 来定义 `head` 时, Lean 会在注册实例的全局表和局部语境中寻找 `Inhabited α` 的实例。全局表包含 `Inhabited ℕ` 的条目, 但没有匹配 `Inhabited α` 的条目。另一方面, 局部语境中包含一个类型为 `Inhabited α` 的匿名参数, 它会被使用。

Lean 的核心库对 `List.head` 的定义与我们完全相同。在实践中, 几乎所有的类型都是非空的 (明显的例外是 `False`), 所以 `Inhabited` 限制几乎不是问题。

我们可以证明关于 `Inhabited` 类型类的抽象定理, 例如:

```
theorem head_head {α : Type} [Inhabited α] (xs : List α)
:
  head [head xs] = head xs
```

为了在类型为 $\text{List } \alpha$ 的列表上使用运算符 `head`，需要假设 $[\text{Inhabited } \alpha]$ 。如果我们省略这个假设，Lean 会抛出一个错误，告诉我们「类型类合成失败」。

还有更多只有常量但没有性质的类型类，包括：

```
class Zero (α : Type)
  where
    zero : α

class One (α : Type)
  where
    one : α

class Neg (α : Type)
  where
    neg : α → α

class Inv (α : Type)
  where
    inv : α → α

class Add (α : Type)
  where
    add : α → α → α

class Mul (α : Type)
  where
    mul : α → α → α
```

Syntactic Type Class

这些**语法类型类**引入了在不同语境下拥有不同语义的常量。例如，`one` 可以代表自然数 1、整数 1、实数 1、单位矩阵以及许多其他「一」的概念。这些类型类的主要目的是为丰富的代数类型类层次结构（群、Monoid幺半群、环、域等）奠定基础，并允许**重载**Overloading常见的数学符号，如 $+$ 、 $*$ 、 0 、 1 和 $^{-1}$ 。

Semantic Type Class

语法类型类不会对可以声明为实例的类型强加严格的限制。相比之下，**语义类型类**包含的性质则用于约束给定常量的行为。

在第 3.6 节中，我们遇到了以下语义类型类：

```
class Std.Commutative (α : Type) (f : α → α → α) where
  comm : ∀ a b, f a b = f b a

class Std.Associative (α : Type) (f : α → α → α) where
  assoc : ∀ a b c, f (f a b) c = f a (f b c)
```

这一次，关联不再是从类型到常量，而是从类型「以及一个函数」到性质。Lean 并不介意这种称谓上的偏差：尽管它们被称为类型类，但 Lean 的类型类非常灵活，可以用来表示各种约束。

从概念上讲，`Std.Commutative` 是一个三元组 (α, f, comm) 的依值类型，`Std.Associative` 也是如此。 f 的类型取决于 α ，而 comm 的类型取决于 α 和 f 。尽管它们是参数，但 α 和 f 也与 comm 一起存储。

在第 3.6 节中，我们将 $\mathbb{N}$ 上的 `add` 函数注册为一个可交换且可结合的操作：

```
instance Associative_add : Std.Associative add :=
  { assoc := add_assoc }

instance Commutative_add : Std.Commutative add :=
  { comm := add_comm }
```

每当我们尝试访问 `@Std.Commutative.comm  $\mathbb{N}$  add` 时，都会得到 `add_comm`，对于 `@Std.Associative.assoc  $\mathbb{N}$  add` 也是如此。策略 `ac_refl` 会尝试查找问题中所有二元运算符的 `comm` 和 `assoc` 性质，并在性质存在时利用它们。

定义类型类的通用语法如下：

```
class 类名 (参数1 : 类型1) ... (参数k : 类型k)
  [extends 结构体1, ..., 结构体m] where
  常量名1 : 常量类型1
  ⋮
  常量名n : 常量类型n
  性质名1 : 命题1
  ⋮
  性质名p : 命题p
```

实例化类型类的通用语法如下：

```
instance 实例名 : 类型类 参数 :=
  { 常量1 := 定义1,
    ⋮
    常量n := 定义n,
    性质1 := 证明1,
    ⋮
    性质p := 证明p }
```

5.7 列表

Lean 提供了丰富的有限列表函数库。在本节中，我们将回顾其中的一些，并定义一些我们自己的函数；这些练习有助于我们熟悉 Lean 中的函数式编程。

在第一个例子中，我们对一个列表进行分类讨论：

```
theorem head_head_cases {α : Type} [Inhabited α]
  (xs : List α) :
  head [head xs] = head xs :=
```

```

by
  cases xs with
  | nil      => rfl
  | cons x xs' => rfl

```

该证明依赖于 `cases`，它是 `induction` 的近亲。它对其参数进行分类讨论，但不生成归纳假设。调用 `cases xs` 将目标 $\vdash P[xs]$ 转换为两个子目标： $\vdash P[[]]$ 和 $\vdash P[x :: xs']$ 。我们其实也可以使用 `induction xs`。如果你在 `induction` 和 `cases` 之间犹豫不决，那么总是可以选择 `induction`，然后看看你是否需要归纳假设——如果你不需要它，那么将 `induction` 替换为 `cases` 来明确表示不需要是一种良好的风格。

在结构化证明中，我们可以使用 `match` 表达式来执行分类讨论：

```

theorem head_head_match {α : Type} [Inhabited α]
  (xs : List α) :
  head [head xs] = head xs :=
match xs with
| List.nil      => by rfl
| List.cons x xs' => by rfl

```

在下一个例子中，我们展示了如何利用构造子^{Injectivity}的**单射性**。当等式两边都应用了相同的构造子时，`cases` 策略会利用单射性来化简等式。在下面的证明中，化简前的等式是 $x :: xs = y :: ys$ ：

```

theorem injection_example {α : Type} (x y : α) (xs ys :
  List α)
  (h : x :: xs = y :: ys) :
  x = y ∧ xs = ys :=
by
  cases h
  simp

```

`cases` 策略在整个目标中将 `y` 替换为 `x`，将 `ys` 替换为 `xs`，产生子目标

$$\vdash x = x \wedge xs = xs$$

这可以由 `simp` 轻松证明。

当构造子不同时，`cases` 策略也可用于识别不可能的情况：

```

theorem distinctness_example {α : Type} (y : α) (ys :
  List α)
  (h : [] = y :: ys) :

```

```
False :=
by cases h
```

接下来，我们定义列表上的^{Map Function}**映射函数**：这种函数将其参数 f （其本身也是一个函数）应用到集合中存储的所有元素上。

```
def map {α β : Type} (f : α → β) : List α → List β
| []      => []
| x :: xs => f x :: map f xs
```

请注意，由于 f 在递归调用中不会改变，因此我们将其作为一个整体定义的参数。另一种选择（也是在递归调用中会改变的参数的唯一选择）如下：

```
def mapArgs {α β : Type} : (α → β) → List α → List β
| _, []      => []
| f, x :: xs => f x :: mapArgs f xs
```

映射函数的一个基本性质是，如果其参数是恒等函数 ($\text{fun } x \mapsto x$)，则它们没有效果：

```
theorem map_ident {α : Type} (xs : List α) :
  map (fun x ↦ x) xs = xs :=
by
  induction xs with
  | nil      => rfl
  | cons x xs' ih => simp [map, ih]
```

另一个基本性质是，连续的映射可以压缩为单个映射，其参数是所涉及函数的复合：

```
theorem map_comp {α β γ : Type} (f : α → β) (g : β → γ)
  (xs : List α) :
  map g (map f xs) = map (fun x ↦ g (f x)) xs :=
by
  induction xs with
  | nil      => rfl
  | cons x xs' ih => simp [map, ih]
```

在引入新操作时，将这些操作与其他操作组合使用时的行为展示出来是非常有用的。示例如下：

```
theorem map_append {α β : Type} (f : α → β)
  (xs ys : List α) :
```

```
map f (xs ++ ys) = map f xs ++ map f ys :=
by
  induction xs with
  | nil          => rfl
  | cons x xs' ih => simp [map, ih]
```

值得注意的是,最后三个证明在文本上是完全相同的。这些是典型的 `induction-rfl-simp` 证明。

下一个列表操作删除列表的第一个元素,返回其尾部:

```
def tail {α : Type} : List α → List α
| []      => []
| _ :: xs => xs
```

对于 `[]`,我们简单地返回 `[]` 作为其自身的尾部。

与 `tail` 对应的是提取列表第一个元素的函数。我们已经在第 5.6 节中回顾了一种使用 `Inhabited` 类型类的方案。另一种可行的定义是使用 `Option` 包装:

```
def headOpt {α : Type} : List α → Option α
| []      => Option.none
| x :: _ => Option.some x
```

类型 `Option α` 有两个构造子: `Option.none` 和 `Option.some a`, 其中 $a : \alpha$ 。当没有有意义的值可以返回时,我们使用 `Option.none`, 否则使用 `Option.some`。我们可以将 `Option.none` 视为函数式编程中的空指针, 但与空指针 (和空引用) 不同, 类型系统会防止不安全的解引用。要检索存储在 `Option` 中的值, 我们必须进行模式匹配。示意图如下:

```
match headOpt xs with
| Option.none    => handleTheError
| Option.some x => doSomethingWithValue x
```

我们不能简单地写 `doSomethingWithValue (headOpt xs)`, 因为这会产生类型错误。类型系统强迫我们考虑错误处理。

利用依值类型的力量, 我们有另一种实现偏函数的方法 —— 即, 我们可以指定一个前置条件。调用者必须随后传递一个前置条件得到满足的证明作为参数:

```
def headPre {α : Type} : (xs : List α) → xs ≠ [] → α
| [],      hxs => by simp at *
| x :: _, hxs => x
```

`headPre` 函数接受两个显式参数。第一个参数 `xs` 是一个列表。第二个参数 `hxs` 是 `xs ≠ []` 的证明。由于第二个参数的类型 `xs ≠ []` 取决于第一个参数, 我们必须

使用依值类型语法 $(xs : \text{List } \alpha) \rightarrow$ 而不是 $\text{List } \alpha \rightarrow$ ，以便我们可以命名第一个参数。函数的结果是类型为 α 的值；由于有了前置条件，不需要 `Option` 包装。

第二个参数用于排除 xs 为 `[]` 的情况。在这种情况下，该参数（称为 hxs ）是 $[] \neq []$ 的证明，而这是不可能的。证明由此导出一个^{Contradiction}矛盾，并利用它导出一个任意的 α 。从矛盾中，我们可以导出任何东西，甚至是 α 的一个居留元。

然后我们可以如下调用该函数：

```
#eval headPre [3, 1, 4] (by simp)
```

这会打印出 3。第二个参数 `by simp` 是 $[3, 1, 4]$ 不为 `[]` 的证明。

现在，给定两个长度相同的列表 $[x_1, \dots, x_n]$ 和 $[y_1, \dots, y_n]$ ，「zip」操作构造一个由偶对组成的列表 $[(x_1, y_1), \dots, (x_n, y_n)]$ ：

```
def zip {α β : Type} : List α → List β → List (α × β)
| x :: xs, y :: ys => (x, y) :: zip xs ys
| [], _           => []
| _ :: _, []      => []
```

如果一个列表比另一个短，该函数也是有定义的。例如， $\text{zip } [a, b, c] [x, y] = [(a, x), (b, y)]$ 。请注意，这种拥有三个情况的递归偏离了结构化递归架构。列表的长度是通过递归定义的：

```
def length {α : Type} : List α → ℕ
| []      => 0
| x :: xs => length xs + 1
```

我们可以关于 `zip` 结果的长度说一些有趣的事情——即，它是两个输入列表长度中的较小值：

```
theorem length_zip {α β : Type} (xs : List α) (ys : List β) :
  length (zip xs ys) = min (length xs) (length ys) :=
by
  induction xs generalizing ys with
  | nil          => simp [zip, min, length]
  | cons x xs' ih =>
    cases ys with
    | nil          => rfl
    | cons y ys' => simp [zip, length, ih ys',
      min_add_add]
```

上面的证明教会了我们又一个技巧。归纳假设是

```
ih : ∀ys : list β, length (zip xs ys) = min (length xs) (length ys)
```

为什么这里有一个 $\forall$ 量词? induction xs generalizing ys 策略^{Generalized}泛化了定理陈述, 使得归纳假设不再局限于证明目标中的某个固定 ys, 而是可以用于 ys 的任意值。这里需要这种灵活性, 因为我们想用 ys 的尾部 (称为 ys') 而不是 ys 本身来实例化该量词。

证明依赖于一个关于 min 函数的定理, 我们需要自己证明它:

```
theorem min_add_add (l m n : ℕ) :
  min (m + l) (n + l) = min m n + l :=
  by
    cases Classical.em (m ≤ n) with
    | inl h => simp [min, h]
    | inr h => simp [min, h]
```

回想定义 $\min a b = (\text{if } a \leq b \text{ then } a \text{ else } b)$ 。要对 min 进行推理, 我们经常需要对条件 $a \leq b$ 进行分类讨论。这可以通过使用 `cases Classical.em (a ≤ b)` 来实现。这会创建两个子目标: 一个以 $a \leq b$ 为假设, 另一个以 $\neg a \leq b$ 为假设。假设在每种情况下都可以作为 h 使用。

在结构化证明中, 有两种方法可以对命题进行分类讨论:

```
theorem min_add_add_match (l m n : ℕ) :
  min (m + l) (n + l) = min m n + l :=
  match Classical.em (m ≤ n) with
  | Or.inl h => by simp [min, h]
  | Or.inr h => by simp [min, h]

theorem min_add_add_if (l m n : ℕ) :
  min (m + l) (n + l) = min m n + l :=
  if h : m ≤ n then
    by simp [min, h]
  else
    by simp [min, h]
```

我们再次看到, 用于编写函数式程序的机制 (如 match 和 if-then-else) 也可用于编写结构化证明 (毕竟证明也是项)。我们现在可以为第 4.7 节末尾呈现的表格添加几行:

策略式证明	结构化证明	原始证明项
<code>cases t</code>	<code>match t with</code>	<code>match t with</code>
<code>cases Classical.em Q</code>	<code>if Q then ... else</code>	<code>if Q then ... else</code>

我们以一个关于 map 和 zip 的分配律结束:

```

theorem map_zip {α α' β β' : Type} (f : α → α') (g : β
  → β') :
  ∀xs ys, map (fun (a, b) ↦ (f a, g b)) (zip xs ys) =
    zip (map f xs) (map g ys)
| x :: xs, y :: ys => by simp [zip, map, map_zip f g
  xs ys]
| [], _ => by rfl
| _ :: _, [] => by rfl

```

左侧的模式与 zip 定义中使用的模式完全对应。这比分别对 xs 进行归纳和对 ys 进行分类讨论要简单得多，就像我们在证明 length_zip 时所做的那样。好的证明通常遵循它们所基于的定义的结构。

在 zip 的定义和 map_zip 的证明中，我们小心地指定了三个不重叠的模式。也可以编写拥有重叠模式的等式，例如：

```

def f {α : Type} : List α → ...
| [] => ...
| xs => ... xs ...

```

由于模式是按顺序应用的，上述命令定义的函数与以下函数相同：

```

def f {α : Type} : List α → ...
| [] => ...
| x :: xs => ... (x :: xs) ...

```

我们通常建议使用后者，即更显式的风格，因为这样产生的意外更少。

5.8 二叉树

树状对象通过带构造子的归纳类型来定义，其构造子接受多个递归参数。^{Binary Tree}二叉树的节点最多有两个子节点。Lean 对二叉树的定义如下：

```

inductive Tree (α : Type) : Type
| nil : Tree α
| node : α → Tree α → Tree α → Tree α

```

对于二叉树，结构归纳会产生两个归纳假设，内部节点的每个子树各一个。为了通过对 t 进行结构归纳来证明目标 $t : \text{Tree } \alpha \vdash P[t]$ ，我们需要展示以下子目标：

$$\vdash P[\text{Tree.nil}]$$

$$a : \alpha, l r : \text{Tree } \alpha, ih_l : P[l], ih_r : P[r] \vdash P[\text{Tree.node } a \ l \ r]$$

对树而言，对应列表翻转操作的是^{Mirror Operation}镜像操作：

```
def mirror {α : Type} : Tree α → Tree α
| Tree.nil           => Tree.nil
| Tree.node a l r => Tree.node a (mirror r) (mirror l)
```

镜像可以直接定义，无需借助于某些追加操作。因此，关于 `mirror` 的推理比关于 `reverse` 的推理更简单，如下所示：

```
theorem mirror_mirror {α : Type} (t : Tree α) :
  mirror (mirror t) = t :=
by
  induction t with
  | nil           => rfl
  | node a l r ih_l ih_r => simp [mirror, ih_l, ih_r]
```

更详细的非形式证明如下：

证明采用对 t 的结构归纳法。

情况 `Tree.nil`: 我们必须证明 `mirror (mirror Tree.nil) = Tree.nil`。这直接遵循 `mirror` 的定义。

情况 `Tree.node a l r`: 归纳假设为

$$(ih_l) \text{ mirror (mirror } l) = l \quad (ih_r) \text{ mirror (mirror } r) = r$$

我们必须证明 `mirror (mirror (Tree.node a l r)) = Tree.node a l r`。我们有

```
mirror (mirror (Tree.node a l r))
= mirror (Tree.node a (mirror r) (mirror l)) (通过 mirror
的定义)
= Tree.node a (mirror (mirror l)) (mirror (mirror r)) (同
上)
= Tree.node a l (mirror (mirror r)) (通过 ih_l)
= Tree.node a l r (通过 ih_r)
```

□

为了在 Lean 证明中达到同样的详细程度，我们可以使用计算块（第 4.4 节）而非 `simp`：

```
theorem mirror_mirror_calc {α : Type} :
  ∀ t : Tree α, mirror (mirror t) = t
| Tree.nil           => by rfl
| Tree.node a l r =>
```

```

calc
  mirror (mirror (Tree.node a l r))
  = mirror (Tree.node a (mirror r) (mirror l)) :=
    by rfl
  _ = Tree.node a (mirror (mirror l))
    (mirror (mirror r)) :=
      by rfl
  _ = Tree.node a l (mirror (mirror r)) :=
      by rw [mirror_mirror_calc l]
  _ = Tree.node a l r :=
      by rw [mirror_mirror_calc r]

```

我们以以下定理结束本节，该定理在第 6 章中会很有用：

```

theorem mirror_Eq_nil_Iff {α : Type} :
  ∀ t : Tree α, mirror t = Tree.nil ↔ t = Tree.nil
| Tree.nil           => by simp [mirror]
| Tree.node _ _ _ => by simp [mirror]

```

5.9 Cases 策略

cases

```

cases 项 [with
| 构造子1 名称列表1 => 策略1
  ⋮
| 构造子n 名称列表n => 策略n]

```

cases 策略对指定的项进行分类讨论。这会产生与该项类型定义中的构造子数量相同的子目标。该策略与 induction 类似，不同之处在于它不产生归纳假设，并且会自动排除不可能的情况。可选名称 名称列表₁, ..., 名称列表_n 用于任何出现的变量或假设。

cases 形如 $l = r$ 的假设

cases 策略也可以用于形如 $l = r$ 的假设 h 。它将 r 与 l 进行匹配，并在目标的各处将所有出现的 r 中的变量替换为 l 中相应的项。如果需要，可以使用 `clear h` 删除剩余的假设 $l = l$ 。

```

cases Classical.em (命题) with
| inl 为真时的名称 => 为真时的策略
| inr 为假时的名称 => 为假时的策略

```

cases 策略还可以用于对命题进行分类讨论。会出现两种情况：一种是命题为真，另一种是命题为假。可选名称 *为真时的名称* 和 *为假时的名称* 分别用于真和假情况下的假设。

5.10 依值归纳类型

归纳类型 `List α` 和 `Tree α` 属于 Lean 的简单类型部分。归纳类型也可以依赖于（非类型的）项。一个典型的例子是长度为 n 的列表，或者称为^{Vector}向量：

```
inductive Vec (α : Type) : ℕ → Type where
  | nil : Vec α 0
  | cons (a : α) {n : ℕ} (v : Vec α n) : Vec α (n + 1)
```

因此，项 `Vec.cons 3 (Vec.cons 1 Vec.nil)` 的类型为 `Vec ℕ 2`。通过在类型中编码向量长度，我们可以提供有关函数结果的更精确信息。例如 `Vec.reverse` 函数反转向量，它会将值 `Vec α n` 映射到拥有相同 n 的相同类型的另一个值。而 `Vec.zip` 可以要求其两个参数拥有相同的长度。固定长度的向量和矩阵在计算机科学和数学中都很有用。

不幸的是，这种更精确的信息是有代价的。当依值归纳类型所依赖的项是可证明相等但在计算上语法相等（例如 $m + n$ 与 $n + m$ ）时，它们会引起困难。在本指南中，我们通常会避免使用依值归纳类型。本节简要介绍它们是为了完整性。明确地说：这不属于考试内容。

下面的定义引入了列表和向量之间的转换：

```
def listOfVec {α : Type} : ∀ {n : ℕ}, Vec α n → List α
  | _, Vec.nil      => []
  | _, Vec.cons a v => a :: listOfVec v

def vecOfList {α : Type} :
  ∀ xs : List α, Vec α (List.length xs)
  | []      => Vec.nil
  | x :: xs => Vec.cons x (vecOfList xs)
```

`listOfVec` 转换接受类型 α 、长度 n 以及 α 上长度为 n 的向量作为参数，并返回 α 上的列表。虽然我们不在乎长度 n ，但它仍然需要作为一个参数，因为它出现在第三个参数的类型中。我们将前两个参数 α 和 n 设置为隐式的，因为它们可以从第三个参数的类型中推断出来。

`vecOfList` 转换接受类型 α 和 α 上的列表作为参数，并返回与列表长度相等的向量。Lean 的类型检查器足够强大，可以确定两个右侧拥有所需的类型。

根据 PAT 原理，证明类似于函数定义。让我们验证将向量转换为列表是否保持其长度：

```

theorem length_listOfVec {α : Type} :
  ∀(n : ℕ) (v : Vec α n), List.length (listOfVec v) =
    n
| _, Vec.nil      => by rfl
| _, Vec.cons a v =>
  by simp [listOfVec, length_listOfVec _ v]

```

为了通过对 v 进行结构归纳来证明目标 $v : \text{Vec } \alpha \ n \vdash P[v]$ ，我们可能会天真地认为展示以下两个子目标就足够了：

$$\vdash P[\text{Vec.nil}]$$

$$m : \mathbb{N}, a : \alpha, u : \text{Vec } \alpha \ m, \text{ih} : P[u] \vdash P[\text{Vec.cons } a \ u]$$

这是天真的，因为子目标甚至不是类型正确的： $P[\]$ 中的洞拥有类型 $\text{Vec } \alpha \ n$ (其原始^{Inhabitant}居留元 v 的类型)，所以我们不能简单地将类型为 $\text{Vec } \alpha \ 0$ 、 $\text{Vec } \alpha \ m$ 和 $\text{Vec } \alpha \ (m + 1)$ 的 Vec.nil 、 u 或 $\text{Vec.cons } a \ u$ 填入那个洞中。我们必须每次都调整 P ，将 n 替换为 0 、 m 或 $m + 1$ 。使用符号 $P_t[\]$ 表示 $P[\]$ 的变体，其中所有出现的 n 都被项 t 替换，我们得到：

$$\vdash P_0[\text{Vec.nil}]$$

$$m : \mathbb{N}, a : \alpha, u : \text{Vec } \alpha \ m, \text{ih} : P_m[u] \vdash P_{m+1}[\text{Vec.cons } a \ u]$$

使用 cases 策略的分类讨论证明与之类似，但不含归纳假设。通常，长度 n 不会是一个变量，而是一个复杂的项。那么将 $P[\]$ 中的 n 替换可能并不拥有直观意义。使用 cases，相应的子目标会被默默地消除。因此，对类型为 $\text{Vec } \alpha \ 0$ 的值进行分类讨论将只产生一个子目标，形式为 $\vdash P[\text{Vec.nil}]$ ，因为 0 永远不可能等于形式为 $m + 1$ 的项。

依值类型的模式匹配是微妙的，因为我们要匹配的值的类型可能会根据构造子的不同而改变。给定 $v : \text{Vec } \alpha \ n$ ，我们可能会尝试编写：

```

match v with
| Vec.nil      => ...
| Vec.cons a u => ...

```

但这与我们上面第一次尝试归纳证明时一样天真。由于类型 $\text{Vec } \alpha \ n$ 中的项 n 可能会根据构造子的不同而改变，我们必须对 n 也进行模式匹配：

```

match n, v with
| 0,      Vec.nil      => ...
| m + 1, Vec.cons a u => ...

```

从本质上讲，这等同于

```
match n, v with
| 0,      @Vec.nil α          => ...
| m + 1, @Vec.cons α a m u => ...
```

通常，在第一列放置占位符就足够了：

```
match n, v with
| _, Vec.nil      => ...
| _, Vec.cons a u => ...
```

对 n 进行模式匹配却忽略其结果可能看起来有些自相矛盾，但如果没有它，Lean 就无法推断 `Vec.cons` 的第二个隐式参数。在这方面，`cases` 比 `match` 更用户友好。

5.11 新引入的 Lean 结构总结

声明

<code>class</code>	将结构体类型声明为类型类
<code>instance</code>	将结构体值声明为类型类的实例
<code>structure</code>	引入结构体类型及其选择器

表达式

<code>if ... then ... else</code>	^{Decidable} 对可判定命题进行分类讨论
<code>match ... with</code>	进行模式匹配

策略

<code>cases</code>	执行分类讨论
--------------------	--------

归纳谓词

Inductive Predicate

归纳谓词，或归纳定义的命题，是一种指定类型为 $\cdots \rightarrow \text{Prop}$ 的函数的便捷方式。它能让人联想到形式系统（第 1.3 节）和 Prolog 风格的逻辑编程。但 Lean 比 Prolog 的表达能力强得多，为此我们需要做一些工作来建立定理，而不仅仅是运行 Prolog 解释器。看待 Lean 的一种视角是：

Lean = 函数式编程 + 逻辑编程 + 更多逻辑

6.1 入门示例

除非你接触过 Prolog 或逻辑编程，否则你可能会好奇归纳谓词是什么，以及它们为什么有用。我们首先回顾三个展示其多种用途的示例：偶数、网球比赛和自反传递闭包。

6.1.1 偶数

数学家经常将特定集合定义为满足某些标准的最小集合。考虑以下定义：

Even Natural Number

偶自然数的集合 E 被定义为在以下规则下封闭的最小集合 S ：

- (1) $0 \in S$;
- (2) 对于每个 $k \in \mathbb{N}$ ，如果 $k \in S$ ，那么 $k + 2 \in S$ 。

根据 Knaster-Tarski 定理，这样的集合是存在的。

（上一句话通常会省略。）我们很容易相信 E 包含了所有的偶数，且仅包含这些数。让我们带上数学家的帽子，证明 4 是偶数：

根据规则 (1)，我们有 $0 \in E$ 。

因此，根据规则 (2)（取 $k := 0$ ），我们有 $2 \in E$ 。

由此，根据规则 (2)（取 $k := 2$ ），我们有 $4 \in E$ ，证毕。 $\square$

相比之下，计算机科学家可能会使用一个由两个^{Derivation Rule}**推导规则**组成的形式系统来指定相同的集合：

$$\frac{}{0 \in E} \text{ZERO} \qquad \frac{k \in E}{k + 2 \in E} \text{ADDTWO}_k$$

那么证明就是一棵^{Derivation Tree}**推导树**：

$$\frac{\frac{\frac{}{0 \in E} \text{ZERO}}{2 \in E} \text{ADDTWO}_0}{4 \in E} \text{ADDTWO}_2$$

如果我们自上而下阅读，证明是正向的；如果我们自底向上阅读，证明就是逆向的。

归纳谓词是逻辑学家实现相同结果的方式。在 Lean 中，我们不会定义一个集合，而是归纳地定义一个^{Characteristic Predicate}**刻画谓词**：

```
inductive Even : ℕ → Prop where
  | zero      : Even 0
  | add_two   : ∀k : ℕ, Even k → Even (k + 2)
```

这看起来应该很熟悉。除了使用 Prop 代替 Type 之外，我们还使用了相同的语法来定义归纳类型。根据 PAT 原理，在 Lean 中，归纳类型和归纳谓词是由同一种机制提供的。

上面的命令定义了一个一元谓词 Even 以及两个^{Introduction Rule}**引入规则** Even.zero 和 Even.add_two，它们可用于证明形如 $\vdash \text{Even } \dots$ 的目标。回想一下，一个符号（例如 Even）的引入规则是一个结论包含该符号的定理（第 4.3 节）。根据 PAT 原理，Even n 可以被视为像 $\text{Vec } \alpha \ n$ （第 5.10 节）那样的依值归纳类型，而 Even.zero 和 Even.add_two 则是像 Vec.nil 和 Vec.cons 那样的构造子。

如果我们将 Lean 的定义翻译回中文，我们会得到类似上面的 Knaster-Tarski 风格的定义：

以下子句定义了偶数。

(1) 0 是偶数；

(2) 若 k 是偶数，则任何形如 $k + 2$ 的数都是偶数。

任何其他数都不是偶数。

值得注意的是，归纳定义的符号没有定义。因此，我们不能使用 `simp` ^{Unfold} 展开它们的定义。唯一可用的推理原则是引入和消去。

作为一个热身练习，这里是 `Even 4` 的证明：

```
theorem Even_4 :
  Even 4 :=
  have Even_0 : Even 0 :=
    Even.zero
  have Even_2 : Even 2 :=
    Even.add_two _ Even_0
  show Even 4 from
    Even.add_two _ Even_2
```

例如，证明项 `Even.add_two _ Even_0` 的类型为 `Even (0 + 2)`，它在计算意义下语法等价于 `Even 2`，因此在类型检查器看来是相等的。下划线代表 `0`。

得益于归纳定义的「无杂质」保证，`Even.zero` 和 `Even.add_two` 是构造 $\vdash \text{Even } \dots$ 的证明仅有的两种方式。通过检查结论 `Even 0` 和 `Even (k + 2)`，我们发现永远不存在证明 `1` 是偶数的风险。

看待上述归纳定义的另一种方式如下：第一行引入了一个谓词，而第二行和第三行引入了我们希望该谓词满足的公理。相应地，我们可以写成：

```
opaque Even :  $\mathbb{N} \rightarrow \text{Prop}$ 
axiom Even.zero : Even 0
axiom Even.add_two :  $\forall k : \mathbb{N}, \text{Even } k \rightarrow \text{Even } (k + 2)$ 
```

这里将 `inductive` 替换为 `opaque`，将 `|` 替换为 `axiom`。

但这个公理性版本没有告诉我们 `Even` 什么时候为假。我们无法用它来证明 $\neg \text{Even } 1$ 或 $\neg \text{Even } 17$ 。据我们所知，`Even` 对所有自然数都可能为真。相比之下，归纳定义保证了我们获得满足引入规则 `Even.zero` 和 `Even.add_two` 的最小谓词（即「最假」的谓词），并提供消去和归纳原则，允许我们证明 $\neg \text{Even } 1$ 、 $\neg \text{Even } 17$ 或 $\neg \text{Even } (2 * n + 1)$ 。

既然我们可以定义递归函数，为什么还要麻烦地使用归纳谓词呢？事实上，以下定义是完全合法的：

```
def evenRec :  $\mathbb{N} \rightarrow \text{Bool}$ 
| 0      => true
| 1      => false
| k + 2 => evenRec k
```

每种风格都有其优缺点。递归版本迫使我们指定一个假的情况（第二个等式），并且迫使我们考虑停机问题。另一方面，由于它具有等式性和计算性，因此它能很好地与 `rfl`、`simp`、`#eval` 和 `#reduce` 配合使用。归纳版本可以说更加地抽象和优雅，每个引入规则都是独立声明的。我们可以添加或删除规则，而无需考虑停机或可执行性。

定义 `Even` 的另一种方式是使用取模运算符 (%) 进行非递归定义：

```
def evenNonrec (k : ℕ) : Prop :=
  k % 2 = 0
```

数学家可能更喜欢这个版本。但归纳版本是一个方便的「Hello, World!」示例，类似于许多现实的归纳定义。它可能只是个玩具，但它是是个有用的玩具。

6.1.2 网球比赛

Transition System

迁移系统由连接「前」状态和「后」状态的迁移规则组成。作为迁移系统的范例，我们考虑网球比赛中从 0-0（^{Love all}「**双方零分**」）开始可能的迁移。网球比赛也是一个玩具示例，但在第 9 章中，我们将以类似的风格将命令式编程语言的语义定义为迁移系统。

以下摘录自国际网球联合会的《网球规则》，介绍了网球的计分规则。

标准局的计分方式如下，发球方的得分先报：

零分 - 「^{Love}零分」
 第一分 - 「15」
 第二分 - 「30」
 第三分 - 「40」
 第四分 - 「^{Game}局」

但如果每位球员/每对球员都赢得了三分，比分就是「^{Deuce}平分」。平分之后，赢得下一分的球员/球员对获得「^{Advantage}占先」。如果该球员/球员对也赢得了下一分，则该球员/球员对赢得该「^{Game}局」；如果对方球员/球员对赢得了下一分，比分再次变为「平分」。球员/球员对需要在「平分」之后立即连赢两分才能赢得该「局」。

我们首先定义一个归纳类型来表示比分：

```
inductive Score : Type where
  | vs      : ℕ → ℕ → Score
  | advServ : Score
  | advRecv : Score
  | gameServ : Score
```

```
| gameRecv : Score
```

诸如 30-15 之类的比分表示为 `Score.vs 30 15`，我们也将使用中缀记法将其记作 `30 - 15`。我们忽略了计分规则中一些最繁琐的方面，将「^{Love}零分」记作 0，将「^{Deuce}平分」记作 `40 - 40`。如果愿意，我们可以引入诸如 `def love : ℕ := Score.vs 0 0` 之类的别名。

下一阶段是引入一个二元谓词 `Step`，它确定迁移是否可能：

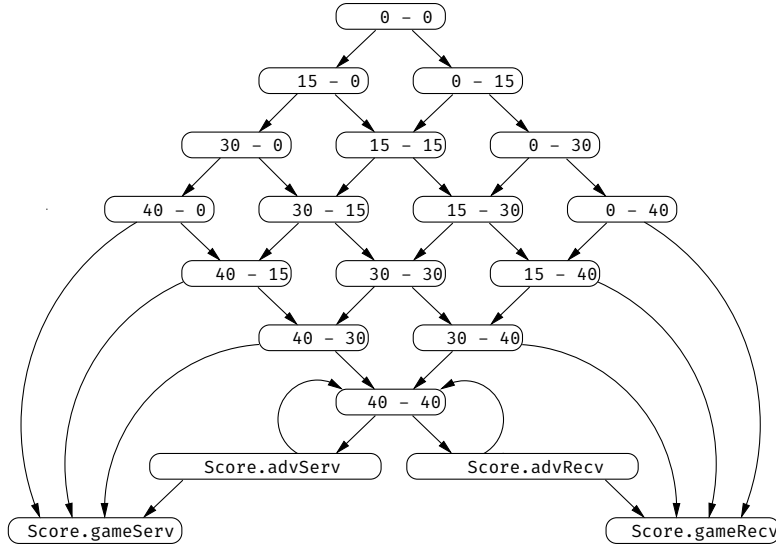
```
inductive Step : Score → Score → Prop where
  | serv_0_15      : ∀n, Step (0 - n) (15 - n)
  | serv_15_30     : ∀n, Step (15 - n) (30 - n)
  | serv_30_40     : ∀n, Step (30 - n) (40 - n)
  | serv_40_game   : ∀n, n < 40 → Step (40 - n)
    Score.gameServ
  | serv_40_adv    : Step (40 - 40) Score.advServ
  | serv_adv_40    : Step Score.advServ (40 - 40)
  | serv_adv_game  : Step Score.advServ Score.gameServ
  | recv_0_15      : ∀n, Step (n - 0) (n - 15)
  | recv_15_30     : ∀n, Step (n - 15) (n - 30)
  | recv_30_40     : ∀n, Step (n - 30) (n - 40)
  | recv_40_game   : ∀n, n < 40 → Step (n - 40)
    Score.gameRecv
  | recv_40_adv    : Step (40 - 40) Score.advRecv
  | recv_adv_40    : Step Score.advRecv (40 - 40)
  | recv_adv_game  : Step Score.advRecv Score.gameRecv
```

令 $s \rightsquigarrow t$ 作为 `Step s t` 的简写。一场比赛是一条链 $s_0 \rightsquigarrow s_1 \rightsquigarrow s_2 \rightsquigarrow \dots \rightsquigarrow s_n$ 其中 $s_0 = 0 - 0$ ，且从 s_n 无法进行任何迁移。该谓词允许诸如 $15 - 99 \rightsquigarrow 30 - 99$ 之类的无意义迁移，但由于比分 $15 - 99$ 无法从「双方零分」(0-0) 达到，因此这些迁移是无害的。

有了形式化定义，我们就可以提出并正式回答以下问题：比赛是否有最大长度？有多少种不同的最终比分？从「双方零分」开始是否能达到 15-99？比分是否可能回到 0-0？让我们使用 Lean 来反驳最后一项断言：

```
theorem no_Step_to_0_0 (s : Score) :
  ¬ s ⤵ 0 - 0 :=
by
  intro h
  cases h
```

下图总结了哪些比分可以从哪些比分到达：



6.1.3 自反传递闭包

归纳谓词一个特别有用的应用是 **二元关系** r 的 **自反传递闭包** r^* 。非形式地， r^* 是表示 r 的零次或多次步骤的关系。例如，如果 r 对应于在迁移系统中（如自动机）执行一次迁移，那么 r^* 则对应于执行任意步数（包括零步）。作为一个更具体的例子，请考虑以下情况：

$$r = \{(1, 2), (2, 4), (4, 8)\}$$

$$r^* = \{(n, n) \mid n \in \mathbb{N}\} \cup \{(1, 2), (1, 4), (1, 8), (2, 4), (2, 8), (4, 8)\}$$

星号 (*) 算子通常被严格地定义为一个形式系统：

$$\frac{(a, b) \in r}{(a, b) \in r^*} \text{BASE} \quad \frac{}{(a, a) \in r^*} \text{REFL} \quad \frac{(a, b) \in r^* \quad (b, c) \in r^*}{(a, c) \in r^*} \text{TRANS}$$

这些规则将 r^* 定义为包含 r （通过 BASE）且满足自反性（通过 REFL）和传递性（通过 TRANS）的 **最小关系**。如果我们想定义 **传递闭包** r^+ ，只需省略 REFL 规则即可。如果我们想要 **自反对称闭包**，我们会将 TRANS 规则替换为 SYMM 规则。在形式系统中，我们只需声明我们希望为真的属性，而无需考虑停机或可执行性。

将上述推导规则直接翻译为归纳谓词的引入规则也很简单：

```
inductive Star {α : Type} (R : α → α → Prop) : α → α
  → Prop
where
  | base (a b : α)      : R a b → Star R a b
  | refl (a : α)        : Star R a a
  | trans (a b c : α) : Star R a b → Star R b c → Star
    R a c
```

我们将^{Relation}关系表示为^{Binary Predicate}二元谓词而非^{Pair}偶对的集合，将 $(a, b) \in R$ 写作 $R\ a\ b$ 。^{Interchangeable}关系和谓词是^{Reflexive Transitive Closure}可互换的，但在证明助手中，谓词通常更方便使用。 R 的^{Introduction Rule}自反传递闭包记作 $\text{Star } R$ 。请注意， a 、 b 和 c 在冒号左侧被声明为^{Introduction Rule}引入规则的参数。我们也可以写成：

```
| base : ∀ a b : α, R a b → Star R a b
```

或者在另一个极端写成：

```
| base (a b : α) (hab : R a b) : Star R a b
```

所有这些形式都在^{Logically and Operationally Equivalent}逻辑和操作上等价。

归纳谓词的一般格式如下：

```
inductive 谓词名称 (参数1 : 类型1) ... (参数k : 类型k) :
  类型k+1 → ... → 类型k+p → Prop where
| 规则名称1 (参数11 : 类型11) ... (参数1m1 : 类型1m1) :
  命题1
  ⋮
| 规则名称n (参数n1 : 类型n1) ... (参数nmn : 类型nmn) :
  命题n
```

其中每个 命题_j 的结论必须是已定义的谓词 ^{谓词名称} 在某些^{Argument}实参^{Application}上的应用。这些参数可以是任意项；它们不需要是^{Constructor Pattern}构造子模式。如果我们想让对应于参数的实参变成^{Implicit}隐式的，也可以使用花括号 $\{ \}$ 而非圆括号 $()$ 。

上述 Star 的定义确实非常优雅。如果你仍然对此表示怀疑，请尝试将其实现为^{Recursive Function}递归函数：

```
def starRec {α : Type} (R : α → α → Bool) :
  α → α → Bool :=
```

总结一下，归纳谓词 P 的每个引入规则由以下部分组成（从左到右）：

- 一个^{Name}名称；
- 可能出现在规则中的^{Variable}变量；
- 必须满足的零个或多个^{Condition}条件，这些条件可能会^{Recursively}递归地应用 P ；
- 将 P 应用于某些参数，形成一个^{Pattern}模式。

因此，对于规则 Star.base ，模式是 $\text{Star } R\ a\ b$ ，条件是 $R\ a\ b$ ，变量是 a 和 b 。

6.1.4 反例

并非所有的归纳定义都允许一个^{Least Solution}最小解。最简单的反例是：

```
-- fails
inductive Illegal : Prop where
  | intro : ¬ Illegal → Illegal
```

如果 Lean 接受了这个定义，我们就可以用它来证明^{Equivalence} $\text{Illegal} \leftrightarrow \neg \text{Illegal}$ ，从而很容易推导出 False。幸运的是，Lean 拒绝了 this 定义：

```
arg #1 of 'Illegal.intro' has a non positive
occurrence of the datatypes being declared
```

翻译：

'Illegal.intro' 的第 1 个参数具有正在声明的数据类型的
^{non positive occurrence}
非正出现

它所抱怨的**非正出现**是指^{Nonpositive Occurrence}在^{Negation}**否定**下的 Illegal 出现。数学家会根据 Knaster-
^{Monotonicity Condition}**单调性条件**不满足而拒绝该定义。
Tarski 定理的

6.2 逻辑符号

虽然 Even 是本指南中第一个公开的归纳谓词，但前面的章节已经悄悄介绍了其他归纳谓词。其中第一个是^{Equality}**相等**(=)，在第 2 章中引入，随后是^{Logical Symbol}**逻辑符号** $\wedge$ 、 $\vee$ 、 $\leftrightarrow$ 、 $\exists$ 、True 和 False。它们的定义值得仔细研究：

```
inductive And (a b : Prop) : Prop where
  | intro : a → b → And a b
```

```
inductive Or (a b : Prop) : Prop where
  | inl : a → Or a b
  | inr : b → Or a b
```

```
inductive Iff (a b : Prop) : Prop where
  | intro : (a → b) → (b → a) → Iff a b
```

```
inductive Exists {α : Type} (P : α → Prop) : Prop where
  | intro : ∀ a : α, P a → Exists P
```

```
inductive True : Prop where
  | intro : True
```

```
inductive False : Prop where
```

```
inductive Eq {α : Type} : α → α → Prop where
| refl : ∀ a : α, Eq a a
```

(严格来说, 在 Lean 中, 上述一些定义实际上是^{Structure}结构体而非归纳谓词, 但区别只是表面的。正如我们在第 5.5 节中看到的, 结构体本质上是带有一些^{Syntactic Sugar}语法糖的单构造子归纳谓词。)

传统记法 $P \wedge Q$ 、 $\exists x : \alpha, P$ 和 $x = y$ 等都是 $\text{And } PQ$ 、 $\text{Exists}(\text{fun } x : \alpha \mapsto P)$ 和 $\text{Eq } x \ y$ 等的语法糖。请注意 fun 是如何充当全能^{Binder}绑定子的。

另请注意, False 没有构造子。不存在 False 的证明, 就像不存在 $\text{Even } 1$ 的证明一样。对于归纳谓词, 我们只陈述我们希望为真的规则。

符号 $\forall$ (包括其特例 $\rightarrow$) 直接内置于逻辑中, 并未定义为归纳谓词。它也没有^{Introduction Principle}引入或^{Anonymous Function}消去规则。^{Elimination Principle}引入原理是匿名函数 $\text{fun } x \mapsto _$, 而消去原理是函数应用 $_ \ u$ 。

对于任何归纳谓词, 只需指定引入规则即可。第 3.3 节和第 3.4 节中介绍的消去规则必须手动推导。

6.3 规则归纳

正如我们可以在归纳类型的项上进行归纳一样, 我们也可以在归纳谓词的证明上进行归纳。例如, 给定目标 $h : \text{Even } n \vdash P \ n$, 我们可以调用 $\text{induction } h$ 并得到两个子目标, 分别对应 Even.zero 和 Even.add_two 。这被称为「在 h 的推导结构上归纳」或简称为「^{Rule Induction}规则归纳」, 因为归纳是在谓词的引入规则 (即证明项的构造子) 上进行的。

看待规则归纳有两种方式: 「满足.....的最小谓词」视角和「PAT」视角。要理解「满足.....的最小谓词」视角, 请回想一下, 归纳定义将一个符号引入为满足引入规则的最小 (即最假) 谓词。相应地, Even 是满足性质 $Q \ 0$ 和 $\forall k, Q \ k \rightarrow Q \ (k + 2)$ 的最小谓词 Q 。因此, 如果我们能证明对于某个谓词 P , 性质 $P \ 0$ 和 $\forall k, P \ k \rightarrow P \ (k + 2)$ 成立, 那么 P 要么是 Even 本身, 要么比 Even 更大 (即更真)。结果是, $\text{Even } n$ 蕴含 $P \ n$, 这正是我们证明目标 $h : \text{Even } n \vdash P \ n$ 所需要的。

「满足.....的最小谓词」视角为规则归纳提供了一个很好的直观解释, 可用于^{Informal Argument}非形式论证中, 例如以下关于「对于所有 n , $\text{Even } n$ 蕴含 $n \% 2 = 0$ 」的证明:

证明对假设 $\text{Even } n$ 使用规则归纳。

情况 Even.zero : 我们必须证明 $0 \% 2 = 0$ 。这可以通过计算得出。

情况 $\text{Even.add_two } k$: 归纳假设是 $k \% 2 = 0$ 。我们必须证明 $(k + 2) \% 2 = 0$ 。这可以通过基本算术推理得出。 $\square$

Lean 证明具有相同的结构：

```
theorem mod_two_Eq_zero_of_Even (n : ℕ) (h : Even n) :
  n % 2 = 0 :=
by
  induction h with
  | zero          => rfl
  | add_two k hk ih => simp [ih]
```

PAT Principle

PAT 原理为我们提供了另一种看待规则归纳的有效方式。其核心思想是，在形如 $h : \text{Even } n \vdash P[h]$ 的目标中对 h 进行规则归纳，完全类似于在^{Dependent Inductive Type}**依值归纳类型**（例如 $\text{Vec } \alpha \ n$ ，见第 5.10 节）的值上进行结构归纳。将 $P[\]$ 中 n 被替换为项 u 的变体记作 $P_u[\]$ ，我们得到以下子目标：

$$\vdash P_0[\text{Even.zero} : \text{Even } 0]$$

$$k : \mathbb{N}, hk : \text{Even } k, ih : P_k[hk] \vdash P_{k+2}[\text{Even.add_two } k \ hk : \text{Even } (k + 2)]$$

这些实际上就是 `induction` 策略产生的子目标。

无论归纳谓词 Q 为何，计算子目标的步骤总是相同的：

1. 在 $P[h]$ 中，将 h 替换为应用在一些新变量上的每一个可能的引入规则（例如 $\text{Even.add_two } k \ hk$ ），并在 $P[\]$ 中对 n 进行实例化以使 $P[\]$ 类型正确。这会产生与引入规则数量相等的子目标。
2. 将这些新变量（例如 k 、 hk ）添加到本地^{Context}**语境**中。
3. 为所有断言 $Q \dots$ 的新假设添加归纳假设。

注意上述假设中出现了 $hk : \text{Even } k$ 。在「满足……的最小谓词」视角中，它是缺失的，而且它并不是必不可少的，因为总是可以通过将 P 强化为 $\text{Even } n \wedge \dots$ 的形式来恢复它。

在几乎所有的实际情况中， h 不会出现在 $P[h]$ 中。我们可以简单地写作：

$$\vdash P_0 \qquad k : \mathbb{N}, hk : \text{Even } k, ih : P_k \vdash P_{k+2}$$

在极少数情况下， h 会出现在 $P[h]$ 中。正如我们在第 5.7 节中尝试提取列表头部时看到的那样，证明可能作为子项出现在任意项中。

Reflexive Transitive Closure

接下来，我们考虑**自反传递闭包** $\text{Star } R$ 。给定目标 $h : \text{Star } R \ x \ y \vdash P$ ，对 h 进行规则归纳会产生以下子目标，其中 $P_{t,u}$ 表示将 P 中的 x 和 y 分别替换为 t 和 u 的变体：

$$a \ b : \alpha, hab : R \ a \ b \vdash P_{a,b}$$

$$a : \alpha \vdash P_{a,a}$$

$$a \ b \ c : \alpha, hab : \text{Star } R \ a \ b, hbc : \text{Star } R \ b \ c, ihab : P_{a,b}, ihbc : P_{b,c} \vdash P_{a,c}$$

这就是「证明助手」中「助手」这一方面发挥作用的地方。Star 的关键性质之一是^{Idempotence}**幂等性**——对 Star R 应用 Star 不会有任何效果。这可以在 Lean 中如下证明，在等价性的 $\rightarrow$ 方向上使用规则归纳：

```
theorem Star_Star_Iff_Star {α : Type} (R : α → α →
  Prop)
  (a b : α) :
  Star (Star R) a b ↔ Star R a b :=
by
  apply Iff.intro
  • intro h
    induction h with
    | base a b hab          => exact hab
    | refl a                => apply Star.refl
    | trans a b c hab hbc ihab ihbc =>
      apply Star.trans a b
      • exact ihab
      • exact ihbc
  • intro h
    apply Star.base
    exact h
```

我们小心地为新出现的变量赋予直观的名称。在包含长的、自动生成的名称的目标中很容易迷失。第 3.8 节中介绍的清理策略在面对大型目标时也很有帮助。

我们可以用相等性而非等价性来更常规地陈述幂等性质：

```
@[simp] theorem Star_Star_Eq_Star {α : Type}
  (R : α → α → Prop) :
  Star (Star R) = Star R :=
by
  apply funext
  intro a
  apply funext
  intro b
  apply propext
  apply Star_Star_Iff_Star
```

由于 Lean 的逻辑是^{Classical}**经典逻辑**，证明中使用了以下两个定理：

```
funext : (∀x, ?f x = ?g x) → ?f = ?g
propext : (?a ↔ ?b) → ?a = ?b
```

Functional Extensionality
函数外延性(funext) 指出, 如果两个函数对所有输入都产生相等的结果, 那么这两个函数必然相等。Propositional Extensionality
命题外延性(propext) 指出, 命题的等价性与相等性是一致的。在这些短语中, 外延性意味着类似于「所见即所得」的意思。虽然这些性质看起来显而易见, 但有些证明助手是建立在较弱的、Intuitionistic Logic
直觉主义逻辑之上的, 在这些逻辑中, 这些性质通常不成立。

我们将定理 `Star_Star_Eq_Star` 注册为 `simp` 规则, 因为以从左到右的重写规则来看, 它确实用一个更简单的项替换了一个复杂的项。很难想象会有我们不希望 `simp` 将 `Star (Star ...)` 重写为 `Star ...` 的情况。

对于规则归纳, 我们使用 `induction` 策略。Pattern Matching Recursion 由于一些微妙的逻辑原因 (这在第 12 章中会更加清晰), 这里不允许通过**模式匹配**和**递归**进行规则归纳。

在第 5.4 节中, 我们看到了一张并排的描述 `Bool` 和 `Prop` 解释的图表。该图表暗示存在无限数量的命题, 但我们目前知道的恰好有两个命题: `False` 和 `True`。以下是修订后的图表:



我们看到, 只有两个命题, 一个没有证明, 另一个有若干证明。图中显示了三个证明。我们将在第 12 章中再次完善这幅图。

6.4 线性算术策略

linarith

Linear Arithmetic
`linarith` 策略可用于证明涉及**线性算术**等式 ($=$)、不等式 ($<$ 、 $>$ 、 $\leq$ 和 $\geq$) 以及非等式 ($\neq$) 的目标。「线性」意味着不会出现乘法和除法, 或者如果出现, 则其中一个操作数必须是数字常量。例如, $2 * x < y$ 是一个线性约束 (可以改写为 $x + x < y$), 而 $x * y < y$ 是非线性的。

6.5 消去

给定一个归纳谓词 Q , 其引入规则通常具有 $\forall \dots, \dots \rightarrow Q \dots$ 的形式, 且可用于证明 $\vdash Q \dots$ 形式的目标。消去则以相反的方式工作: 它从定理或假设 $h : Q \dots$ 中提取信息。消去有多种形式: `cases` 和 `induction` 策略、模式匹配以及消去规则 (例如 `And.left`)。

在 $h : Q \dots$ 上调用的 `cases h` 策略执行的规则归纳大致与 `induction h` 相同, 但不产生任何归纳假设。我们在第 5 章中遇到了两个惯用法, 现在终于可以分析它们了。

第一个惯用法是当 h 的形式为 $l = r$ 时——即 $\text{Eq } l \ r$ (第 6.2 节)。假设目标是 $h : l = r \vdash P[h]$ 。第 6.3 节中介绍的步骤会产生子目标

$$a : \alpha \vdash P_{a,a}[\text{Eq.refl } a : a = a]$$

其中 $P_{t,u}[\]$ 代表 $P[\]$ 的变体, 其中 l 和 r 分别被 t 和 u 替换。(严格来说, 无用的假设 $h : a = a$ 也会出现在子目标中。)在实践中, $P[h]$ 可能并不依赖于 h 。此外, `cases` 会重用名称 l , 而不会使用像 a 这样不同的名称。因此, 我们会得到

$$l : \alpha \vdash P_{l,l}$$

换句话说, 原始目标中所有出现的 r 都被替换成了 l 。这对应于我们在第 5.7 节中观察到的行为。

第二个惯用法是策略 `cases Classical.em Q`, 其中 Q 是一个命题。`Classical.em Q` 部分是 $Q \vee \neg Q$ (即 $\text{Or } Q (\neg Q)$, 见第 6.2 节) 的一个证明项。然后应用 `cases` 来消去 $\vee$ 联结词。假设目标是 $\vdash P[\text{Classical.em } Q]$ 。根据 `Or` 谓词的定义, 新的子目标是

$$hQ : Q \vdash P[\text{Or.inl } hQ : Q \vee \neg Q]$$

$$hnQ : \neg Q \vdash P[\text{Or.inr } hnQ : Q \vee \neg Q]$$

不需要修改 P , 因为 `Or.inl hQ` 和 `Or.inr hnQ` 具有与 `Classical.em Q` 相同的类型——即 $Q \vee \neg Q$ 。在实践中, 目标通常不包含 `Classical.em Q`, 而仅仅是 $\vdash P$, 这样我们就得到子目标

$$hQ : Q \vdash P$$

$$hnQ : \neg Q \vdash P$$

同样, 这也是我们在第 5.7 节中观察到的行为。

在结构化证明中, 我们可以使用 `match` 表达式 (第 5.4 节) 来实现与 `cases` 相同的效果。这对于逻辑符号非常有效。然而, 对于像 `Even` 和 `Star` 这样参数会随归纳而变化的谓词, 我们最终会遇到**依值类型模式匹配**^{Dependently Typed Pattern Matching}, 这是很微妙的 (见第 5.10 节)。通常, 让 `cases` 来确定子目标的样式, 比我们自己进行模式匹配要容易得多。我们将在下面回顾这两种风格的示例。

展开形如 $Q(c \dots)$ 的假设通常很有用, 其中 c 是一个**构造子**^{Constructor}或某些其他常量。我们可以陈述并证明一个**反转规则**^{Inversion Rule}来支持这种消去推理。典型的反转规则具有以下形式:

$$\forall x_1 \dots x_n, Q(c \ x_1 \dots x_n) \rightarrow (\exists \dots, \dots \wedge \dots) \vee \dots \vee (\exists \dots, \dots \wedge \dots)$$

将引入和消去合并为一个定理也很有用, 这可以用于重写假设和子目标的目的。除了中间的联结词 $\leftrightarrow$ 之外, 格式是相同的:

$$\forall x_1 \dots x_n, Q(c \ x_1 \dots x_n) \leftrightarrow (\exists \dots, \dots \wedge \dots) \vee \dots \vee (\exists \dots, \dots \wedge \dots)$$

`Even` 的反转规则如下所示:

```

theorem Even_Iff (n : ℕ) :
  Even n ↔ n = 0 ∨ (∃m : ℕ, n = m + 2 ∧ Even m) :=
by
  apply Iff.intro
  • intro hn
    cases hn with
    | zero      => simp
    | add_two k hk =>
      apply Or.inr
      apply Exists.intro k
      simp [hk]
  • intro hor
    cases hor with
    | inl heq => simp [heq, Even.zero]
    | inr hex =>
      cases hex with
      | intro k hand =>
        cases hand with
        | intro heq hk =>
          simp [heq, Even.add_two _ hk]

```

一如既往，策略式证明并不是特别易读，但我们可以看到引入规则和消去式的 cases 策略在逻辑符号和 Even 谓词中都起着重要的作用。simp 策略则进行了最后的收尾工作。

如果你更喜欢结构化证明，这里有一个对 $hn : \text{Even } n$ 使用 ^{Dependently Typed Pattern Matching} 依值类型模式匹配的证明版本：

```

theorem Even_Iff_struct (n : ℕ) :
  Even n ↔ n = 0 ∨ (∃m : ℕ, n = m + 2 ∧ Even m) :=
Iff.intro
  (assume hn : Even n
   match hn with
   | Even.zero      =>
     show 0 = 0 ∨ _ from
       by simp
   | Even.add_two k hk =>
     show _ ∨ (∃m, k + 2 = m + 2 ∧ Even m) from
       Or.inr (Exists.intro k (by simp [*])))
  (assume hor : n = 0 ∨ (∃m, n = m + 2 ∧ Even m)

```

```

match hor with
| Or.inl heq =>
  show Even n from
    by simp [heq, Even.zero]
| Or.inr hex =>
  match hex with
  | Exists.intro m hand =>
    match hand with
    | And.intro heq hm =>
      show Even n from
        by simp [heq, Even.add_two _ hm])

```

6.6 更多示例

在对归纳谓词有了更深入的了解之后，我们现在可以依次回顾另外四个应用实例。

6.6.1 有序列表

我们的第一个例子是一个检查自然数列表是否按升序排列的谓词：

```

inductive Sorted : List ℕ → Prop where
| nil : Sorted []
| single (x : ℕ) : Sorted [x]
| two_or_more (x y : ℕ) {zs : List ℕ} (hle : x ≤ y)
  (hsorted : Sorted (y :: zs)) :
  Sorted (x :: y :: zs)

```

该定义捕捉了以下数学直觉：

有序列表的集合被定义为满足下列规则的最小闭集：

- (1) 列表 [] 是有序的；
- (2) 给定一个数字 x，列表 [x] 是有序的；
- (3) 给定两个数字 x、y 和一个列表 zs，如果 $x \leq y$ 且 $y :: zs$ 是有序的，那么 $x :: y :: zs$ 是有序的。

通过尝试一些小例子来测试我们的定义总是一个好主意。列表 [3, 5] 是有序的吗？看起来是的：

```

theorem Sorted_3_5 :
  Sorted [3, 5] :=
  by
    apply Sorted.two_or_more
    • simp
    • exact Sorted.single _

```

该示例使用了 `sorted` 的两个引入规则。下面是一个更紧凑的证明，使用了证明项：

```

theorem Sorted_3_5_raw :
  Sorted [3, 5] :=
  Sorted.two_or_more _ _ (by simp) (Sorted.single _)

```

同样的想法可以用来证明 `[7, 9, 9, 11]` 是有序的：

```

theorem sorted_7_9_9_11 :
  Sorted [7, 9, 9, 11] :=
  Sorted.two_or_more _ _ (by simp)
  (Sorted.two_or_more _ _ (by simp)
    (Sorted.two_or_more _ _ (by simp)
      (Sorted.single _)))

```

相反，我们可以证明某些列表不是有序的。为此，我们需要消去：

```

theorem Not_Sorted_17_13 :
  ¬ Sorted [17, 13] :=
  by
    intro h
    cases h with
    | two_or_more _ _ hlet hsorted => simp at hlet

```

从列表 `[17, 13]` 是有序的这一假设中，我们提取出不等式 $17 \leq 13$ 。cases 策略静默地排除了 `nil` 和 `single` 情况，因为它们无法与双元素列表匹配。然后我们调用 `simp` 来利用不可能成立的假设 $17 \leq 13$ 。

6.6.2 回文

回文是从左向右读和从右向左读都相同的列表。例如，`[a, b, b, a]` 和 `[a, h, a]` 都是回文。当且仅当作为参数传递的列表是回文时，以下归纳谓词为 `True`：

```

inductive Palindrome {α : Type} : List α → Prop where
  | nil : Palindrome []
  | single (x : α) : Palindrome [x]

```

```

| sandwich (x :  $\alpha$ ) (xs : List  $\alpha$ ) (hxs : Palindrome xs)
:
  Palindrome ([x] ++ xs ++ [x])

```

该定义区分了三种情况：(1) `[]` 是回文；(2) 对于任何元素 x ，单元素列表 `[x]` 是回文；(3) 对于任何元素 x 和任何回文 `[y1, ..., yn]`，列表 `[x, y1, ..., yn, x]` 是回文。

回文是归纳谓词大显身手的另一个例子。以下^{Naive Recursive Definition}朴素递归定义无法工作，因为 `[x] ++ xs ++ [x]` 不是一个^{Constructor Pattern}构造子模式，且变量 x 被重复了：

```

-- fails
def palindromeRec { $\alpha$  : Type} : List  $\alpha$  → Bool
| []           => true
| [_]          => true
| ([x] ++ xs ++ [x]) => palindromeRec xs
| _            => false

```

正确的递归定义是可能的，但超出了本指南的范围。

当然，回文的反转也是回文。这是一个很好的练习：

```

theorem Palindrome_reverse { $\alpha$  : Type} (xs : List  $\alpha$ )
  (hxs : Palindrome xs) :
  Palindrome (reverse xs) :=
by
  induction hxs with
  | nil           => exact Palindrome.nil
  | single x      => exact Palindrome.single x
  | sandwich x xs hxs ih =>
    simp [reverse, reverse_append]
    exact Palindrome.sandwich _ _ ih

```

非形式地：

证明通过对假设 hxs 进行^{Rule Induction}规则归纳。

CASE `Palindrome.nil`：我们必须证明 `Palindrome (reverse [])`。利用 `reverse [] = []`，这由 `Palindrome.nil` 得出。

CASE `Palindrome.single x`：我们必须证明 `Palindrome (reverse [x])`。利用 `reverse [x] = [x]`，这由 `Palindrome.single x` 得出。

CASE `Palindrome.sandwich x xs hxs`：在假设 (hxs) `Palindrome xs` 下，我们必须证明 `Palindrome (reverse ([x] ++ xs ++ [x]))`。归纳假设是 `Palindrome (reverse xs)`。通过简化，

只需证明 $\text{Palindrome}([x] ++ \text{reverse } xs ++ [x])$ 。根据 $\text{Palindrome.sandwich}$ ，只需证明 $\text{Palindrome}(\text{reverse } xs)$ ，这正是归纳假设。 $\square$

6.6.3 满二叉树

我们的第三个例子基于第 5.8 节中介绍的二叉树类型。如果二叉树的所有节点都有零个或两个子节点，则称该二叉树是^{Full}满的。这可以编码为一个归纳谓词：

```
inductive IsFull {α : Type} : Tree α → Prop where
| nil : IsFull Tree.nil
| node (a : α) (l r : Tree α)
  (hl : IsFull l) (hr : IsFull r)
  (hiff : l = Tree.nil ↔ r = Tree.nil) :
  IsFull (Tree.node a l r)
```

第一种情况指出空树是满树。第二种情况指出，如果一个非空树有两个本身也是满的子树，且这两个子树要么都是空的，要么都是非空的，那么该树就是满树。这两种情况整齐地遵循了归纳类型的结构，因此重用名称 `nil` 和 `node` 是很自然的。

子节点是空树的节点组成的树是满树。这是一个简单的证明：

```
theorem IsFull_singleton {α : Type} (a : α) :
  IsFull (Tree.node a Tree.nil Tree.nil) :=
by
  apply IsFull.node
  · exact IsFull.nil
  · exact IsFull.nil
  · rfl
```

满树的一个更有趣的性质是，^{Mirror Operation}满树性在镜像操作下是保持不变的。我们的第一个证明是通过对 $ht : \text{IsFull } t$ 进行^{Rule Induction}规则归纳：

```
theorem IsFull_mirror {α : Type} (t : Tree α)
  (ht : IsFull t) :
  IsFull (mirror t) :=
by
  induction ht with
  | nil                                     => exact
  IsFull.nil
  | node a l r hl hr hiff ih_l ih_r =>
    rw [mirror]
```

```

    apply IsFull.node
    • exact ih_r
    • exact ih_l
    • simp [mirror_Eq_nil_Iff, *]

```

由于 IsFull 的定义遵循了 Tree 的定义，因此对树 t 进行^{Structural Induction}**结构归纳**也是合理的：

```

theorem IsFull_mirror_struct_induct {α : Type} (t : Tree
α) :
  IsFull t → IsFull (mirror t) :=
by
  induction t with
  | nil                               =>
    intro ht
    exact ht
  | node a l r ih_l ih_r =>
    intro ht
    cases ht with
    | node _ _ _ hl hr hiff =>
      rw [mirror]
      apply IsFull.node
      • exact ih_r hr
      • apply ih_l hl
      • simp [mirror_Eq_nil_Iff, *]

```

这里的关键是对假设 $ht : \text{IsFull}(\text{Tree.node } a \ l \ r)$ 进行的^{Case Distinction}**分类讨论**。cases ^{Tactic}**策略**注意到 IsFull.nil 引入规则不可能被用来推导出 ht，因此它只能产生一种情况，对应于 IsFull.node。一如既往，如果你在 Visual Studio Code 中通过移动光标来检查策略式证明，它会更有意义。

6.6.4 一阶项

我们的最后一个例子基于^{First-Order Term}**一阶项**的归纳类型：

```

inductive Term (α β : Type) : Type where
  | var : β → Term α β
  | fn  : α → List (Term α β) → Term α β

```

一阶项要么是一个变量 x ，要么是一个应用于参数列表的函数符号 f : $f(t_1, \dots, t_n)$ ，其中数学变量 $t_1, \dots, t_n$ 代表参数，它们本身也是项。因此， $\sin(\max(x, y))$ 是一个一阶项。参数 α 和 β 分别是函数符号和变量的类型。

并非所有项都是合法的。例如，项 $\min(\cos(a), \cos(a, b))$ 被认为是^{Ill-formed}非良构的，因为函数 $\cos$ 调用时的参数数量不一致（1 个对比 2 个）。除了 α 和 β ，我们还考虑了^{Arity}元数，由函数 $\text{arity} : \alpha \rightarrow \mathbb{N}$ 表示，指示每个函数符号接受多少个参数。例如，二元符号的元数为 2。

`WellFormed` 谓词接着检查给定的项是否仅包含具有指定数量参数的函数符号应用：

```
inductive WellFormed {α β : Type} (arity : α → ℕ) :
  Term α β → Prop where
| var (x : β) : WellFormed arity (Term.var x)
| fn (f : α) (ts : List (Term α β))
  (hargs : ∀t ∈ ts, WellFormed arity t)
  (hlen : length ts = arity f) :
  WellFormed arity (Term.fn f ts)
```

`var` 情况指出变量总是良构的。`fn` 情况检查参数 `ts` 是否递归地良构，并且 `ts` 的长度是否等于所讨论函数符号 `f` 的指定元数。

一阶项的另一个有趣性质是它们是否包含变量。这可以使用归纳谓词轻松检查：

```
inductive VariableFree {α β : Type} : Term α β → Prop
where
| fn (f : α) (ts : List (Term α β))
  (hargs : ∀t ∈ ts, VariableFree t) :
  VariableFree (Term.fn f ts)
```

没有对应于 `Term.var` 的引入规则，因为变量永远不是^{variable-free}无变量的。

6.7 归纳中的陷阱

对归纳谓词调用 `induction` 时需要小心。归纳谓词的参数通常会在归纳过程中演变。这些细节在非形式证明中通常会被一带而过，但证明助手要求我们保持精确。

回想一下偶数的定义：

```
inductive Even : ℕ → Prop where
| zero      : Even 0
| add_two   : ∀k : ℕ, Even k → Even (k + 2)
```

如果目标的格式为 $h : \text{Even } n \vdash P n$ ，对 h 应用 `induction` 将产生以下子目标：

$$\vdash P 0 \qquad k : \mathbb{N}, hk : P k \vdash P (k + 2)$$

这符合预期。

问题出现在 `Even` 的参数不是变量时。在目标 $hev : \text{Even } (2 * n + 1) \vdash \text{False}$ 中对假设 `hev` 应用 `induction` 会失败并报错：

```
index in target's type is not a variable (consider
using the `cases` tactic instead)
```

译文：

目标类型中的索引不是变量（考虑改用 ``cases`` 策略）

为了解决这个问题，我们需要用变量 m 替换 $2 * n + 1$ ，并添加等式 $m = 2 * n + 1$ 作为假设：

$$m : 2 * n + 1, hev : \text{Even } m \vdash \text{False}$$

这个目标在逻辑上是等价的，但现在 `induction` 会产生两个子目标：

$$m \ n : \mathbb{N}, hm : 0 = 2 * n + 1 \vdash \text{False}$$

$$m : 2 * n + 1, ih : m = 2 * n + 1 \rightarrow \text{False}, hm : m + 2 = 2 * n + 1 \vdash \text{False}$$

遗憾的是，第二个子目标是不可证明的。问题在于我们想要用 $n - 1$ 来实例化归纳假设 `ih` 中的 n ，但 n 是不可实例化的。

解决方案是什么？我们需要对 n 进行全称量化，以便能够在归纳假设中实例化它。这可以通过在 `induction h` 之后指定 `generalizing n` 来实现。现在我们得到如下子目标：

$$m \ n : \mathbb{N}, hm : 0 = 2 * n + 1 \vdash \text{False}$$

$$m : 2 * n + 1, ih : \forall n, m = 2 * n + 1 \rightarrow \text{False}, hm : m + 2 = 2 * n + 1 \vdash \text{False}$$

现在归纳假设中的变量 n 与目标其余部分中的变量 n 断开了联系，我们可以将归纳假设中的 n 实例化为 $n - 1$ 。

综上所述，我们得到：

```
theorem Not_Even_two_mul_add_one (m n : ℕ)
  (hm : m = 2 * n + 1) :
  ¬ Even m :=
by
  intro h
```

```

induction h generalizing n with
| zero          => linarith
| add_two k hk ih =>
  apply ih (n - 1)
  cases n with
  | zero      => simp at *
  | succ n' =>
    simp at *
    linarith

```

我们使用定理 `Nat.succ_eq_add_one` 将 `Nat.succ n` 形式的项重写为 `n + 1`，并使用 `linarith` 策略进行简单的算术推理。当我们面对 `Nat.succ` 和加法的混合时，该定理非常有用。

6.8 新引入的 Lean 结构总结

定理

<code>funext</code>	函数外延性
<code>propext</code>	命题外延性

策略

<code>linarith</code>	为线性算术调用过程
-----------------------	-----------

带作用的编程

纯函数式编程有时会让人感到过于受限。^{Effectful Functional Programming}带作用的函数式编程^{Idiom}提供了一些习语来^{Side Effect}缓解这些限制，让我们能够以带有副作用^{Effect}、异常、非确定性和其他作用的方式进行编程。

其底层的抽象被称为^{Monad}单子。单子推广了带有作用的程序。它们在 Haskell 中非常流行，用于编写指令式程序。在 Lean 中，它们被用于表达指令式程序并对其进行推理。它们甚至对于 Lean 自身的编程也非常有用，我们在第 8 章中就会看到。

本笔记受到了^{Programming in Lean}《Lean 语言编程》[3] 第 7 章的启发。我们也参考了^{Real World Haskell}《实战 Haskell》[24] 的第 14 章，以获得关于带作用的函数式编程的一般性介绍。^{Effectful Functional Programming}

7.1 入门示例

考虑以下编程任务：

实现一个函数 `sum257 ns`，对自然数列表 `ns` 的第二个、第五个和第七个元素求和。结果使用 `Option N`，以便在列表元素太少时返回 `Option.none`。

一个直白的解决方案如下：

```
def nth {α : Type} : List α → Nat → Option α
| [], _      => Option.none
| x :: _, 0   => Option.some x
| _ :: xs, n + 1 => nth xs n

def sum257 (ns : List N) : Option N :=
  match nth ns 1 with
```

```

| Option.none      => Option.none
| Option.some n2 =>
  match nth ns 4 with
  | Option.none      => Option.none
  | Option.some n5 =>
    match nth ns 6 with
    | Option.none      => Option.none
    | Option.some n7 => Option.some (n2 + n5 + n7)

```

(令人困惑的是, nth 从 0 开始计数元素。)这段代码一点也不优雅,因为它充斥着对 Option 的^{Pattern Matching}模式匹配。虽然这个编程任务是人为构造的,但我们都能回想起写代码时遇上层层嵌套的错误处理和不断增加的缩进层级的经历。

我们可以做得更好,方法是将这些丑陋之处集中在一个函数中:

```

def connect {α : Type} {β : Type} :
  Option α → (α → Option β) → Option β
| Option.none, _ => Option.none
| Option.some a, f => f a

```

connect 函数作用于 Option。如果其值为 Option.none, 我们保持原样。这对应于一种错误情况, 而错误是带有「粘性」^{Sticky}的。否则, 其值的形式为 Option.some a, 我们对 a 应用操作 f —— 或者说将 f 的参数^{Bind}绑定到 a。现在我们可以使用 connect 来编写求和函数:

```

def sum257Connect (ns : List ℕ) : Option ℕ :=
  connect (nth ns 1)
  (fun n2 ↦ connect (nth ns 4)
    (fun n5 ↦ connect (nth ns 6)
      (fun n7 ↦ Option.some (n2 + n5 + n7))))

```

直观上, 该程序执行以下步骤:

1. 使用 nth 从列表中提取第二个元素。如果没有该元素, nth 返回 Option.none; 直接返回此值即可。否则, 将 n₂ 绑定到该元素, 并继续下一步。
2. 对第五个和第七个元素执行相同的操作, 同理推导。
3. 返回 n₂、n₅ 和 n₇ 的和, 并包装在 Option.some 中。

在数学上, 我们的新函数 sum257Connect 等于原来的函数 sum257。

我们不必自己定义 connect, 而是使用 Lean 预定义的通用^{Bind Operation}绑定操作。它接受相同的参数和顺序。以下是新版代码:

¹译注:「粘性」指的是一旦程序进入了错误状态, 这个错误就会像胶水一样「粘」在处理流中并一直传递下去, 后续的正常操作都会被自动跳过。

```
def sum257Bind (ns : List N) : Option N :=
  bind (nth ns 1)
    (fun n2 ↦ bind (nth ns 4)
      (fun n5 ↦ bind (nth ns 6)
        (fun n7 ↦ pure (n2 + n5 + n7))))
```

我们也使用了预定义的^{Pure Function}纯函数 `pure`，而非使用 `Option.some` 来将一个纯 α 值转换为 `Option  $\alpha$` 。

使用预定义的 `bind` 的优点之一是它提供了^{Syntactic Sugar}语法糖，即 `>>=` 运算符的形式：

```
def sum257Op (ns : List N) : Option N :=
  nth ns 1 >>=
    fun n2 ↦ nth ns 4 >>=
      fun n5 ↦ nth ns 6 >>=
        fun n7 ↦ pure (n2 + n5 + n7)
```

语法 `oa >>= f` 展开为 `bind oa f`，其中 `oa` 的类型为 `Option  $\alpha$` 。

求和程序的倒数第二个版本使用了更重的语法糖：

```
def sum257Dos (ns : List N) : Option N :=
  do
    let n2 ← nth ns 1
  do
    let n5 ← nth ns 4
  do
    let n7 ← nth ns 6
    pure (n2 + n5 + n7)
```

^{do Notation}**do-记法**为带作用的程序提供了一种方便的语法。程序 `do let a ← oa ...` 等价于 `oa >>= (fun a ↦ ...)`。如果我们对 `oa` 的计算结果不感兴趣，可以省略 `let a ←` 绑定并写作 `do oa ...`，它会展开为 `oa >>= (fun _ ↦ ...)`。

`do`-记法可以方便地在单个块中允许重复多次 `let` 绑定。这引出了程序的最终版本：

```
def sum257Do (ns : List N) : Option N :=
  do
    let n2 ← nth ns 1
    let n5 ← nth ns 4
    let n7 ← nth ns 6
    pure (n2 + n5 + n7)
```

带有箭头 $\leftarrow$ 的每一行都尝试读取一个值。在失败的情况下，整个程序求值为 `Option.none`。

上述函数可以被读作一个指令式程序，其中每个 `nth` 调用都可以抛出一个异常。但是，即使这种表示法具有指令式风格，该函数仍然是一个纯函数式程序。

7.2 两个操作与三个定律

`Option` 类型构造子是单子的一个例子，被称为^{Option Monad}**选项单子**。通常，单子是一个一元类型构造子 $m : \text{Type} \rightarrow \text{Type}$ ，并配有两个特殊的操作：

$$\begin{aligned} \text{pure } \{\alpha : \text{Type}\} &: \alpha \rightarrow m \alpha \\ \text{bind } \{\alpha \beta : \text{Type}\} &: m \alpha \rightarrow (\alpha \rightarrow m \beta) \rightarrow m \beta \end{aligned}$$

通常，大括号表示隐式参数。对于 `Option` 而言，`pure` 操作就是单纯的 `Option.some`，而 `bind` 则等同于我们的 `connect`。

类型 $m \alpha$ 的值是一个带作用的程序。`pure` 操作将类型为 α 的纯的、无作用的程序嵌入到 $m \alpha$ 中。`bind` 操作组合了两个带作用的程序，其类型分别为 $m \alpha$ 和 $m \beta$ 。第一个程序的输出（类型为 α ）被传递给第二个程序。第二个程序的输出也就是二者组合的程序的输出。

我们可以将单子想象成一个包含某些数据的盒子。这个盒子捕捉了一些特殊的作用（例如异常、可变状态）。`pure` 操作将数据放入盒子中，而 `bind` 允许我们访问盒子中的数据并修改它——甚至可以改变它的类型，因为结果类型是 $m \beta$ ，而不是 $m \alpha$ 。然而，没有通用的方法可以从盒子中提取数据——即从 $m \alpha$ 中获得一个 α 。盒子里可能没有任何 α 值，也可能有多个。

总而言之，`pure a` 是一个包含值 a 的盒子，没有作用，而 `bind ma f`（也可以写作 `ma >>= f` 或 `do a ← ma; f a`）执行 `ma`，然后使用 `ma` 的拆箱结果 a 来执行 f 。将类型为 $m \alpha$ 或 $m \beta$ 的「^{Boxed Value}装箱值」命名为 `ma` 或 `mb`，并将类型为 α 或 β 的数据命名为 `a` 或 `b` 是很方便的约定。

单子是一个具有许多应用场景的抽象概念。选项类型只是众多实例中的一个。下表概述了一些单子实例及其作用。

类型	作用
id	无作用
Option	简单异常
$\text{fun } \alpha \mapsto \sigma \rightarrow \alpha \times \sigma$	传递类型为 σ 的状态
Set	返回 α 值的非确定性计算
$\text{fun } \alpha \mapsto t \rightarrow \alpha$	读取类型为 t 的元素（例如配置）
$\text{fun } \alpha \mapsto \mathbb{N} \times \alpha$	附加运行时间（例如建模时间复杂度）
$\text{fun } \alpha \mapsto \text{String} \times \alpha$	附加文本输出（例如用于日志记录）
IO	与操作系统的交互
TacticM	与证明助手的交互

以上这些都是一元类型构造子 $m : \text{Type} \rightarrow \text{Type}$ 。一些作用可以组合（例如 $\text{fun } \alpha \mapsto \text{Option } (t \rightarrow \alpha)$ ）。有些作用是不可执行的（例如 Set）；尽管如此，它们对于抽象地建模程序非常有用。特定的类型构造子 m 可能会提供 pure 和 bind 之外的更多运算符。例如，它们可以提供提取装箱值的方法。

单子有几个好处。它们提供了高度可读的 do-记法。它们支持通用操作，例如 $\text{List.mmap } \{\alpha \beta : \text{Type}\} : (\alpha \rightarrow m \beta) \rightarrow \text{List } \alpha \rightarrow m (\text{List } \beta)$ ，这些操作对所有单子 m 都是统一的。引用 Programming in Lean《**Lean 语言编程**》[3] 中的话：

这种抽象的力量不仅在于它为所有这些不同的实例提供了通用的函数和记法，还在于它提供了一种思考它们共同点的有用的方式。

单子除了是一个有用的计算机科学概念外，还提供了一个 Axiomatic Reasoning**公理化推理**的好例子。

通常 bind 和 pure 操作需要遵守三条定律。bind 操作用于组合两个程序。如果其中任何一个程序是纯程序，我们可以将其内联并消除 bind。这引出了前两条定律：

```
do
  let a' ← pure a      =    f a
  f a'
```

以及

```
do
  let a ← ma           =    ma
  pure a
```

第三条定律是 bind 的结合律。它允许我们展平一个嵌套计算：

```

do
  let b ←
    do
      let a ← ma
      f a
    g b
=
do
  let a ← ma
  let b ← f a
  g b

```

之前我们将单子比作一个盒子。更具体地说，将盒子想象成一个瑞士银行账户可能会有所帮助，其中 α 是钱。第一条定律意味着如果你把一些钱存入账户，你就可以把它取出来。第二条定律意味着如果你取走一些钱然后又把它放回去，没有人会注意到。第三条定律意味着将两项银行操作结合在一起然后再进行第三项操作，与先单独执行第一项操作然后再执行另外两项操作是一样的。鉴于瑞士银行在保密方面的声誉，这三条定律似乎都是合理的。

7.3 一种类型类

单子是一种数学结构，因此我们在 Lean 中使用类型类来指定它们。回想一下，类型类是一个参数化的结构体类型，通常是以类型作为参数，但在这里是以类型构造子 $m : \text{Type} \rightarrow \text{Type}$ 作为参数。每当我们对具体的 m 使用类型类中的字段时，类型类推导机制就会检索相关的结构体值——即类型类的实例。

下面是一个可能的单子 Lean 定义，连同三条定律：

```

class LawfulMonad (m : Type → Type)
  extends Pure m, Bind m where
  pure_bind {α β : Type} (a : α) (f : α → m β) :
    (pure a >>= f) = f a
  bind_pure {α : Type} (ma : m α) :
    (ma >>= pure) = ma
  bind_assoc {α β γ : Type} (f : α → m β) (g : β → m γ)

    (ma : m α) :
    ((ma >>= f) >>= g) = (ma >>= (fun a ↦ f a >>= g))

```

让我们逐步研究这个定义：

- 我们正在创建一个由一元类型构造子 m 参数化的结构体——即一个类型为 $\text{Type} \rightarrow \text{Type}$ 的值。
- 该结构体从名为 `Pure` 和 `Bind` 的结构体中继承了字段和所有语法糖。这提供了 m 上的 `pure` 和 `bind` 操作，以及期望的类型和语法糖。

- 最后, 在 Pure 和 Bind 已经提供的字段基础上, 增加了三个字段 (pure_bind、bind_pure 和 bind_assoc)。每个字段都是三条定律之一的证明。

我们将我们的类型类称为 LawfulMonad, 因为要求这三条定律成立。要实例化此定义, 我们必须提供类型构造子 m、合适的 bind 和 pure 运算符, 以及定律的证明。

Lean 包含了它自己的单子概念, 也称为 LawfulMonad。它与我们的定义大致等价, 但分布在多个类型类中。

7.4 无作用

在本节以及第 7.5 节到第 7.7 节中, 我们将回顾由特定单子提供的各种作用。我们从^{Identity Monad}恒等单子开始, 它根本不提供任何特殊作用。

Lean 的常量 `id {α : Type} : α → α` 被定义为恒等函数 `fun x ↦ x`。恒等类型构造子是通过取 `α := Type` 获得的。我们可以将其注册为单子:

```
def id.pure {α : Type} : α → id α
| a => a

def id.bind {α β : Type} : id α → (α → id β) → id β
| a, f => f a

instance id.LawfulMonad : LawfulMonad id :=
{ pure      := id.pure
  bind      := id.bind
  pure_bind :=
    by
      intro α β a f
      rfl
  bind_pure :=
    by
      intro α ma
      rfl
  bind_assoc :=
    by
      intro α β γ f g ma
      rfl }
```

注册过程要求我们提供五个组件: pure 和 bind 操作以及三条定律的证明。

Identity Monad

恒等单子是可能的最简单的单子。它提供了一个简单的盒子，其中只有一个值，没有任何作用。它在加法律术中扮演着与 0 类似的角色。我们可以将其他单子看作是它的变体；例如，选项单子就是增加了代表错误状态的特殊 `Option.none` 值的恒等单子。

7.5 基本异常

正如我们在上面看到的，选项类型提供了一个基本的异常机制。下面的代码展示了如何将 `Option : Type → Type` 注册为一个合法单子：

```
def Option.pure {α : Type} : α → Option α :=
  Option.some

def Option.bind {α β : Type} :
  Option α → (α → Option β) → Option β
| Option.none, _ => Option.none
| Option.some a, f => f a

instance Option.LawfulMonad : LawfulMonad Option :=
{ pure      := Option.pure
  bind      := Option.bind
  pure_bind :=
    by
      intro α β a f
      rfl
  bind_pure :=
    by
      intro α ma
      cases ma with
      | none   => rfl
      | some _ => rfl
  bind_assoc :=
    by
      intro α β γ f g ma
      cases ma with
      | none   => rfl
      | some _ => rfl }
```

这三个证明都很简单直接。

除了标准操作之外，抛出和捕获异常也很有用。这可以实现如下：

```
def Option.throw {α : Type} : Option α :=
  Option.none

def Option.catch {α : Type} : Option α → Option α →
  Option α
| Option.none,    ma' => ma'
| Option.some a, _  => Option.some a
```

`Option.throw` 操作会抛出一个异常，使程序处于错误状态 (`Option.none`)。`Option.catch` 操作可用于从之前的异常中恢复。如果程序当前处于错误状态，`Option.catch` 会调用一些异常处理代码（其第二个参数）。这段代码可能会依次抛出一个新的异常。如果 `Option.catch` 应用于正常状态（形式为 `Option.some a`），则什么也不会发生。

作为 `Option.catch ma ma'` 的一种便捷替代方案，Lean 支持语法 `ma.catch ma'`。这里有一个示意性例子，演示了使用此语法进行抛出和捕获：

```
do
  ...
  if ... then
    Option.throw
  else
    ...
.catch do
  ...
```

相应的 Java 代码如下所示：

```
try {
  ...
  if (...) {
    throw new UnknownException();
  } else {
    ...
  }
} catch (UnknownException e) {
  ...
}
```

Option
选项只应对一种错误状态。一种更通用的抽象被称为^{Error Monad}**错误单子**，它支持不同的错误，就像 Java 和其他编程语言中的异常一样。

7.6 可变状态

State Monad

状态单子提供了一种对应于可变状态的抽象。对于某些编程语言，编译器可以检测到状态单子的使用，并将使用它们的程序翻译成更高效的指令式程序。

无可否认，「对应于可变状态的抽象」听起来可能有些抽象，所以让我们考虑一个半具体的例子。如果你有一些函数式编程的经验，那么你可能写过非常类似下面这个片段的代码：

```
def welcomeNewUser (userName : String) (ctxt : Context)
:
  (N × String) × Context :=
let
  (userId, ctxt') := createUser userName ctxt
  (password, ctxt'') := generateTemporaryPassword
  userId ctxt'
  (ok, ctxt''') := sendUnencryptedEmail userId
  password ctxt''
in
  ((userId, password), ctxt''')
```

（通过未加密的邮件发送密码，并忽略函数的 ok 状态是非常重要的。）该函数接受一些全局状态或^{Context}语境作为输入，并依次调用三个函数，其中每个函数都接受一个语境值，并返回一些数据和新的语境。语境实际上是在程序中「^{Threaded Through}穿针引线」。这使我们能够在没有副作用的编程语言中拥有可变状态。

上述方法容易出错——很容易忘记一个撇号（'）并将错误的语境传递给函数。代码也被所有的语境变量弄得杂乱无章。如果我们能像下面这样写呢？

```
def welcomeNewUserDo (userName : String) :
  Context → (N × String) × Context :=
do
  let userId ← createUser userName
  let password ← generateTemporaryPassword userId
  let ok ← sendUnencryptedEmail userId password
  pure (userId, password)
```

State Monad

这正是**状态单子**所提供的。

Binary Type Constructor

状态单子建立在^{Computation Action}二元类型构造子 Action 之上，它捕捉了在类型为 σ 的^{State}状态上进行**计算或操作**的概念，其返回值类型为 α 。在 Lean 中，Action $\sigma \alpha$ 被定义为等同于 $\sigma \rightarrow \alpha \times \sigma$ ：²

²这个类型构造子在传统上被称为 State，但这个名称非常容易引起混淆。

```
def Action (σ α : Type) : Type :=
  σ → α × σ
```

(由于类型也是项, 我们也可以使用 `def` 来定义类型缩写。) 对于给定的类型 σ , 我们有 $\text{Action } \sigma : \text{Type} \rightarrow \text{Type}$ 是一个单子。类型 σ 对确切的内存布局进行了抽象。例如, 我们可以使用偶对或列表来表示内存, 并相应地实例化抽象的状态 σ 。

一个^{Stateful Action}**带状态的操作**是一个函数, 它接收某个状态并返回一个值和一些新的状态。`Action` 的定义中的 $\sigma \rightarrow$ 部分给出了旧状态; 笛卡尔积的左分量 α 给出了计算的结果; 而积的右分量 σ 给出了新的状态。因此, 状态被隐式地贯穿于程序之中。与其他带作用的程序一样, `do` 记法只暴露数据 —— 类型为 α 的值 —— 并隐藏了作用 —— 旧的和新的 σ 状态。

当 $\sigma := \text{Unit}$ 时, 这是一个^{Cardinality}**基数为一**的类型 (类似于 C 或 Java 中的 `void`), 其唯一的值写作 `()`, 这对应于恒等单子: 类型 $\text{Unit} \rightarrow \alpha \times \text{Unit}$ 与 α 是^{Isomorphic}**同构**的。这种直觉可以指导我们定义 `pure` 和 `bind`。

我们首先定义基本操作: 两个操作 `read` 和 `write` 用于访问内存, 以及标准操作 `bind` 和 `pure`:

```
def Action.read {σ : Type} : Action σ σ
  | s => (s, s)

def Action.write {σ : Type} (s : σ) : Action σ Unit
  | _ => ((), s)

def Action.pure {σ α : Type} (a : α) : Action σ α
  | s => (a, s)

def Action.bind {σ : Type} {α β : Type} (ma : Action σ α
)
  (f : α → Action σ β) :
  Action σ β
  | s =>
    match ma s with
    | (a, s') => f a s'
```

`read` 操作简单地返回当前状态 s (在第一个^{Pair}**偶对**分量中), 并保持状态不变 (在第二个偶对分量中)。`write` 操作将当前状态替换为 s 并返回 `()`。`pure` 操作返回未改变的当前状态 s , 并与给定的值 a 组成^{Tuple}**元组**。`bind` 操作将初始状态传递给 `ma` 参数, 产生一个结果 a 和一个新的状态 s' 。这些被传递给 `f`, 它返回一个新的结果和新的状态。

为了将 `Action` 类型构造子注册为一个^{Lawful Monad}**合法单子**，我们需要像以前一样证明这三条定律：

```
instance Action.LawfulMonad {σ : Type} :
  LawfulMonad (Action σ) :=
  { pure      := Action.pure
    bind      := Action.bind
    pure_bind :=
      by
        intro α β a f
        rfl
    bind_pure :=
      by
        intro α ma
        rfl
    bind_assoc :=
      by
        intro α β γ f g ma
        rfl }
```

作为一个具体的例子，下面的程序会删除列表中所有小于前一个元素的元素，留下一个递增的元素列表。最大元素存储为状态 σ 。请注意状态是如何通过 `read` 和 `write` 访问的。

```
def increasingly : List N → Action N (List N)
| []      => pure []
| (n :: ns) =>
  do
    let prev ← Action.read
    if n < prev then
      increasingly ns
    else
      do
        Action.write n
        let ns' ← increasingly ns
        pure (n :: ns')
```

要执行该程序，我们必须提供一个初始状态。最后一个状态连同结果列表一起返回。它对应于列表中遇到的最大元素或起始状态。因此，命令

```
#eval increasingly [1, 2, 3, 2] 0
```

```
#eval increasingly [1, 2, 3, 2, 4, 5, 2] 0
```

产生如下输出：

```
([1, 2, 3], 3)
([1, 2, 3, 4, 5], 5)
```

7.7 非确定性

选项单子存储零个或一个 α 值，恒等单子和状态单子存储恰好一个值，而^{Set Monad}集合单子存储可能无限数量的值。这^{Nondeterminism}对于建模非确定性很有用，即作为一组可能的行为。

Lean 的类型 $\text{Set } \alpha$ 定义为 $\alpha \rightarrow \text{Prop}$ 。换句话说，集合是通过其^{Characteristic Predicate}特征谓词来标识的。它支持我们熟悉的运算符，例如空集 ($\emptyset$)、全集 (Set.univ)、并集 ($\cup$)、交集 ($\cap$) 和属于 ($\in$)，以及传统的花括号表示法，例如 $\{a\}$ 、 $\{a, b\}$ 和 $\{x \mid P x\}$ 。许多集合构造可以通过 `simp` 来简化。

`Set` 类型构造子可以按如下方式注册为合法单子：

```
def Set.pure {α : Type} : α → Set α
| a => {a}

def Set.bind {α β : Type} : Set α → (α → Set β) → Set
  β
| A, f => {b | ∃ a, a ∈ A ∧ b ∈ f a}

instance Set.LawfulMonad : LawfulMonad Set :=
{ pure      := Set.pure
  bind      := Set.bind
  pure_bind :=
    by
      intro α β a f
      simp [Set.pure, Set.bind]
  bind_pure :=
    by
      intro α ma
      simp [Set.pure, Set.bind]
  bind_assoc :=
    by
      intro α β γ f g ma
      simp [Set.bind]
```

```
aesop }
```

`pure` 操作简单地将给定的值 a 放入单元素集合 $\{a\}$ 中。`bind` 操作对集合 A 中的所有值调用 f ，并返回所有结果的并集。例如，如果 $A := \{3, 8\}$ 且 $f := (\text{fun } a \mapsto \{a + 1, a + 2\})$ ，那么 `Set.bind A f` 等于 $\{4, 5, 9, 10\}$ 。

请注意，在这三个证明中，我们都展开了通用 `pure` 和 `bind` 常量的定义，随后是 `Set.pure` 和 `Set.bind` 的定义。

最后一个证明依赖于 `aesop` 策略。目标的目的是：

$$\begin{aligned} & \forall x, \{b \mid \exists a, (\exists a_1 \in ma, a \in f a_1) \wedge b \in g a\} \\ & = \{b \mid \exists a \in ma, \exists a_1 \in f a, b \in g a_1\} \end{aligned}$$

等式的两侧除了存在量词的位置——以及令人困惑的绑定变量名称之外，都是相同的。这个丑陋的命题可以通过繁琐的引入和消去序列来证明，但我们想要更多的自动化。

Satisfiability Modulo Theory

上述 `aesop` 的一个替代方案是调用 `grind`，这是一个灵感来自所谓的**可满足性模理论**(SMT)求解器的策略。

7.8 通用证明策略

`aesop`

Automated Extensible Search For Obvious Proofs

`aesop` 策略 [19]，其名称代表「**用于明显证明的自动化可扩展搜索**」，是一个通用的证明搜索策略。此外，它还可以对假设中的逻辑符号 $\wedge$ 、 $\vee$ 、 $\leftrightarrow$ 和 $\exists$ 进行消去，并在目标中引入 $\wedge$ 、 $\leftrightarrow$ 和 $\exists$ ，并且它还会定期调用 `simp`。它可以成功证明目标、失败或部分成功，从而给我们留下一些未完成的子目标。

`grind`

`grind` 策略的灵感来自现代 SMT 求解器。它结合了多种推理引擎共同协作，以解决涉及等值推理、分类讨论和线性算术等目标。与 `simp` 不同，`grind` 以无向（即非从左到右）的方式对等式进行推理。它系统地从目标中存在的等式中推导出新的等式。例如，如果目标包含 $b = a$ 和 $f b \neq f a$ 作为假设，`grind` 将从 $b = a$ 推导出 $f b = f a$ ，并发现与 $f b \neq f a$ 的矛盾。

有些目标可以用 `grind` 解决而不能用 `aesop` 解决，反之亦然。很难预测哪种策略对给定的目标会取得成功。

7.9 一个通用算法：在列表上迭代

Effectful

假设我们使用 `map` 在列表的所有元素上应用一个**带作用的**函数 f 。然后我们得到一个普通的带作用值的列表。例如：

```
def nthFine {α : Type} (xss : List (List α)) (n : ℕ) :
  List (Option α) :=
  List.map (fun xs ↦ nth xs n) xss
```

函数 `nthFine xss n` 试图提取 `xss` 中每个列表的第 $n + 1$ 个元素。运行

```
#eval nthFine [[11, 12, 13, 14], [21, 22, 23]] 2
```

返回 `[Option.some 13, Option.some 23]`。

这些 `Option.some` 构造子可能会很不方便。通常，我们只关心是否出现了错误。这引出了我们的最后一个例子：一个通用的带作用程序 `mmap`，它在列表上迭代，并将带作用函数 `f` 应用于每个元素。该定义是递归的：

```
def mmap {m : Type → Type} [LawfulMonad m] {α β : Type}
  (f : α → m β) :
  List α → m (List β)
| []      => pure []
| a :: as =>
  do
    let b ← f a
    let bs ← mmap f as
    pure (b :: bs)
```

请注意，该函数返回一个包含列表的单个 `m` 值，而不是一个 `m` 值的列表。试着弄清楚为什么它是良类型的，并且具有预期的行为。

我们现在可以尝试使用 `mmap` 而不是 `List.map`：

```
def nthCoarse {α : Type} (xss : List (List α)) (n : ℕ)
  :
  Option (List α) :=
  mmap (fun xs ↦ nth xs n) xss
```

运行

```
#eval nthCoarse [[11, 12, 13, 14], [21, 22, 23]] 2
```

返回 `Option.some [13, 23]`，在纯列表周围只有一个 `Option.some`。

`mmap` 函数在追加运算符 `++` 上具有分配性。`do` 记法不仅对定义函数有用，而且对陈述它们的性质也很有用：

```
theorem mmap_append {m : Type → Type} [LawfulMonad m]
  {α β : Type} (f : α → m β) :
  ∀ as as' : List α, mmap f (as ++ as') =
  do
```

```

    let bs ← mmap f as
    let bs' ← mmap f as'
    pure (bs ++ bs')
| [], _ =>
  by simp [mmap, LawfulMonad.bind_pure,
    LawfulMonad.pure_bind]
| a :: as, as' =>
  by simp [mmap, mmap_append _ as as',
    LawfulMonad.pure_bind,
    LawfulMonad.bind_assoc]

```

7.10 新引入的 Lean 结构总结

记法

<code>do</code>	表示一个带作用程序的开始
<code>let ... ← ...</code>	在带作用程序中分配变量
<code>>>=</code>	组合带作用的计算

定理

<code>Set.ext</code>	集合外延性
----------------------	-------

策略

<code>aesop</code>	使用通用搜索过程证明命题
<code>grind</code>	使用 SMT 风格的推理证明命题

元编程

和大多数证明助手一样，Lean 可以通过自定义策略和其他功能进行扩展。对 Lean 自身进行编程，而不仅仅是使用它，被称为^{Metaprogramming}**元编程**。Lean 的元编程框架大多使用与 Lean 输入语言相同的概念和语法，因此我们不需要学习不同的语言来对 Lean 进行编程。单子被用来访问 Lean 的状态。

以归纳类型呈现的^{Abstract Syntax Tree}**抽象语法树**反映了内部数据结构。证明助手的内部结构通过 Lean 函数公开，我们可以使用这些函数来访问当前目标、统一项、查询和修改全局语境，以及设置属性（例如 `q[simp]`）。

下面是元编程的一些示例应用：

- 目标转换（例如：应用安全的引入规则，将目标置于^{Negation Normal Form}**否定范式**）；
- 启发式证明搜索（例如：结合^{Backtracking}**回溯**应用非安全的引入规则）；
- ^{Decision Procedure}**判定过程**（例如：针对线性算术、命题逻辑）；
- 定义生成器（例如：归纳类型的 Haskell 风格的 `deriving`）；
- 建议工具（例如：定理查找器、反例生成器）；
- 导出器（例如：文档生成器）；
- ^{ad-hoc}**特设**证明自动化（为了避免^{Boilerplate}**样板代码**或重复）。

正如数学家及 Lean 用户 Kevin Buzzard 所写的：¹

如果你发现自己正在^{Grinding}**「刷怪」**（借用电脑游戏的用语），一遍又一遍地做着同样的事情，因为你需要这样做才能取得进展，那么你可以试着说服一位计算机科学家为你编写一个策略（或者如果你足够勇敢，可以编写^{meta}**元** Lean 代码，甚至可以自己编写策略）。

¹<https://xenaproject.wordpress.com/2020/02/09/lean-is-better-for-proper-maths-than-all-the-other-theorem-provers/>

8.1 策略组合子

首先，有一些术语：如果应用一个策略产生了错误，则该策略^{Fail}**失败**；否则，它^{Succeed}**成功**。策略成功的一种方式是完全证明目标。另一种方式是产生替换当前目标的新子目标。有些策略通过什么都不做来取得成功。

在编写我们自己的策略时，我们经常需要在多个目标上重复某些操作，或者在策略失败时进行恢复。在这种情况下，^{Tactic Combinator}**策略组合子**会很有帮助。

最实用的策略组合子之一是 `repeat' tactic`。它在所有目标上重复调用 `tactic`，然后在产生的子目标上调用，再在产生的子子目标上调用，依此类推，直到 `tactic` 在所有可用目标上都失败为止。下面是一个涉及第 6 章中介绍的 `Even` 谓词的 `repeat'` 示例：

```
theorem repeat'_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  repeat' apply Even.add_two
```

在第一个 `repeat'` 行之后，证明状态包含四个目标：

`⊢ Even 4` `⊢ Even 7` `⊢ Even 3` `⊢ Even 0`

请注意，所有的合取项都消失了。第二个 `repeat'` 重复应用定理 `Even.add_two`： $\forall k, \text{Even } k \rightarrow \text{Even } (k + 2)$ ，留下了这些目标：

`⊢ Even 0` `⊢ Even 1` `⊢ Even 1` `⊢ Even 0`

第一个和最后一个目标很烦人，因为它们对应于定理 `Even.zero`。我们可以在应用 `Even.add_two` 失败时尝试应用 `Even.zero` 来证明它们。这可以通过以下方式实现：

```
theorem repeat'_first_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  repeat'
    first
    | apply Even.add_two
    | apply Even.zero
```

策略组合子 `first | tactic1 | ... | tacticn` 首先尝试执行其第一个参数 `tactic1`。如果失败，则尝试 `tactic2`，依此类推。如果所有指定的策略都失败，则整个组合子也会失败。在上面的示例中，我们还剩两个无法证明的目标：

$\vdash \text{Even } 1$ $\vdash \text{Even } 1$

下一个组合子 `all_goals tactic` 在每个目标上恰好调用一次策略。该组合子仅在 `tactic` 在「所有」目标上都成功时才成功。在下面的示例中它会失败，因为 `Even.add_two` 无法应用于目标 $\vdash \text{Even } 0$ ：

```
theorem all_goals_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  all_goals apply Even.add_two  -- fails
```

为了忽略 `tactic` 的失败，我们可以将其包装在 `try` 组合子中：

```
theorem all_goals_try_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  all_goals try apply Even.add_two
```

结果状态为

$\vdash \text{Even } 2$ $\vdash \text{Even } 5$ $\vdash \text{Even } 1$ $\vdash \text{Even } 0$

结构 `try tactic` 等价于 `first | tactic | skip`，其中 `skip` 是一个不做任何事情就成功的策略。因此 `try tactic` 总是成功。一个相关的策略是 `done`：如果没有剩余目标则成功，否则失败。

另一个变体是 `any_goals tactic` 组合子。它尝试在每个目标上调用一次 `tactic`，但与 `all_goals` 不同，只要 `tactic` 在**任何**目标上成功，它就会成功。示例

```
theorem any_goals_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  any_goals apply Even.add_two
```

结果状态为

$\vdash \text{Even } 2$ $\vdash \text{Even } 5$ $\vdash \text{Even } 1$ $\vdash \text{Even } 0$

这与前一个示例中的状态相同。通常，区别在于 `any_goals tactic` 可能会失败，而 `all_goals try tactic` 总是成功。

有时我们希望除非能完全证明一个目标，否则不去管它。组合子 `solve | tactic1 | ... | tacticn` 首先尝试执行其第一个参数 `tactic1`。如果这未能证明目标，则尝试 `tactic2`，依此类推。如果所有指定的策略都未能证明目标，则整个组合子失败。（将此行为与 `first | tactic1 | ... | tacticn` 的行为进行比较，后者只要求指定的策略之一成功，而不要求证明目标。）考虑这个示例：

```
theorem any_goals_solve_repeat_first_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  repeat' apply And.intro
  any_goals
    solve
    | repeat'
      first
      | apply Even.add_two
      | apply Even.zero
```

第一个和第四个目标已被证明，剩下的两个无法证明的目标与定理陈述中的完全一致：

$\vdash \text{Even } 7$ $\vdash \text{Even } 3$

请注意，`repeat'` 组合子可能会导致无限循环。考虑这个示例：

```
theorem repeat'_Not_example :
  ¬ Even 1 :=
by repeat' apply Not.intro
```

`Not.intro` 规则是 $(?a \rightarrow \text{False}) \rightarrow \neg ?a$ ，因此它应用一次将目标转换为 $\vdash \text{Even } 1 \rightarrow \text{False}$ 。由于 $\neg ?a$ 被定义为 $?a \rightarrow \text{False}$ ，该规则再次应用，再次产生相同的目标。该策略会发生死循环。

最后，^{And Then}「继而」运算符 `<;>` 可用于连接两个策略。左侧在第一个目标上执行。右侧在产生的每个子目标上执行（但不在原始的第二个、第三个等目标上执行）。因此，我们可以这样写

```
by
  induction n <;>
  aesop
```

而非更冗长的版本

```
by
  induction n with
  | zero      => aesop
  | succ n' ih => aesop
```

8.2 宏

让我们通过将自定义策略编码为^{Macro}宏，来进行一些实际的元编程。该策略体现了我们在上面的 solve 示例中硬编码的行为：

```
macro "intro_and_even" : tactic =>
  '(tactic|
    (repeat' apply And.intro
      any_goals
      solve
      | repeat'
        first
        | apply Even.add_two
        | apply Even.zero))
```

第一行将 intro_and_even 声明为属于 tactic ^{Syntactic Category}语法范畴的宏。在其余行中，'(tactic| tactic) 结构将指定的策略 tactic 嵌入到宏中。策略本身是使用标准语法指定的。

一旦我们定义了自定义策略，就可以在证明中调用它：

```
theorem intro_and_even_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  intro_and_even
```

这会产生子目标

⊢ Even 7

⊢ Even 3

8.3 元编程单子

宏是一种可用于编写简单证明自动化的机制。然而，对于大多数元编程任务，我们需要使用元编程单子：MetaM 和 TacticM。

MetaM 结合了几种单子的属性：

- 它是一个状态单子，提供对全局^{Context}语境（包括所有定义和归纳类型）、记法和属性（例如 `@[simp]` 定理列表）等的访问。
- 它的行为类似于选项单子。元程序 `failure` 表示策略已失败。
- 它支持^{tracing}追踪，因此我们可以使用程序 `logInfo` 来显示消息。
- 它支持指令式结构，例如 `for-in` 循环、`continue` 语句和 `return` 语句。

TacticM 通过目标管理扩展了 MetaM：它提供对目标列表的访问。它还允许我们运行 Lean 来填充表达式中的隐式 `{ }` 和类型类 `[ ]` 参数、展开宏等等。

在 Lean 内部，每个目标都表示为一个^{Metavariable}元变量 `?m`，代表一个缺失的项（通常是一个证明项）。每个元变量都有一个类型（通常是一个命题）和一个局部语境，指定了可以用来证明与该元变量相关联的目标的变量和假设。

让我们通过定义一个利用 TacticM 追踪功能的策略，来实际使用一下 TacticM。该策略将显示 Lean 版本号、目标列表以及第一个目标的目的（Lean 称之为^{Main Goal}主目标）：

```
def traceGoals : TacticM Unit :=
  do
    logInfo m!"Lean version {Lean.versionString}"
    logInfo "All goals:"
    let goals ← getUnsolvedGoals
    logInfo m!"{goals}"
    match goals with
    | [] => return
    | _ :: _ =>
      logInfo "First goal's target:"
      let target ← getMainTarget
      logInfo m!"{target}"

  elab "trace_goals" : tactic =>
    traceGoals
```

这段代码有许多有趣的特性：

- 第一行声明了一个类型为 `TacticM Unit` 的函数 `traceGoals` —— 这是返回 `Unit`（一个基数为 1 的平凡类型）的策略类型。注意，元编程函数的定义使用了与任何 Lean 函数相同的语法。

- 第二行进入了单子。其余行是访问 Lean 内部结构的带作用的操作。
- `m! "..."` 语法指定了一个字符串，其中每个出现的 `{term}`（其中 `term` 是一个 Lean 项）都会被求值并序列化为字符串。例如，如果 `Lean.versionString` 是「v4.24.0」，那么 `m!"Lean version {Lean.versionString}"` 会求值为字符串「Lean version v4.24.0」。
- 最后两行使用 `elab` 命令，告诉 Lean 的解析器将字符串「`trace_goals`」识别为一个策略。当 Lean 遇到这个策略时，它会运行 `elab` 命令主体中的元程序——在这里是 `traceGoals`。（当我们之前使用更高层级的 `macro` 命令时，`elab` 是在底层被调用的。）

下面是一个展示如何使用新策略的示例：

```
theorem Even_18_and_Even_20 (α : Type) (a : α) :
  Even 18 ∧ Even 20 :=
  by
    apply And.intro
    trace_goals
    intro_and_even
```

将鼠标悬停在 `trace_goals` 上，可见输出如下：

```
Lean version v4.24.0
All goals:
[case left
α : Type
a : α
⊢ Even 18
,
case right
α : Type
a : α
⊢ Even 20]
First goal's target:
Even 18
```

虽然 Lean 使用熟悉的 `C ⊢ P` 语法显示目标，但它们实际上是元变量。上述程序中使用的常量具有以下类型：

```
logInfo : MessageData → TacticM Unit
getUnsolvedGoals : TacticM (List MVarId)
getMainTarget : TacticM Expr
```

其中 `MessageData` 代表消息, `MVarId` 代表元变量标识符, 而 `Expr` 代表项。

8.4 第一个示例：一个 Assumption 策略

我们的第一个较大的示例实现了一个 `hypothesis` 策略, 它像预定义的 `assumption` 策略一样, 寻找正确类型 (即正确的命题) 的假设, 并应用它来证明目标:

```
def hypothesis : TacticM Unit :=
  withMainContext
    (do
      let target ← getMainTarget
      let lctx ← getLCtx
      for ldecl in lctx do
        if ! LocalDecl.isImplementationDetail ldecl
      then
        let eq ← isDefEq (LocalDecl.type ldecl)
        target
          if eq then
            let goal ← getMainGoal
            MVarId.assign goal (LocalDecl.toExpr ldecl)
            return
          failure)

elab "hypothesis" : tactic =>
  hypothesis
```

在 `hypothesis` 函数中, 我们首先提取第一个目标的目的和局部语境。为了确保 `getLCtx` 给出的是当前第一个目标的局部语境, 我们将整个 `do` 块传递给 `withMainContext` 函数。通常, 任何 `TacticM` 计算都是在环境局部语境中执行的, 该语境为表达式中出现的自由变量赋予了意义。`withMainContext` 函数将此局部语境设置为当前第一个目标的局部语境。

在 `do` 块内部, 我们使用方便的单子结构 `for-in` 遍历局部语境中的所有声明。对于每个不是所谓「实现细节」(即 Lean 插入的对用户不可见的假设) 的局部变量或假设 `h`, 我们检查其类型 (通常是其命题) 在计算和元变量实例化后是否等于目标。如果是, 我们获取与第一个目标关联的元变量, 并将 `h` 分配给它, 从而证明该目标。最后, 我们 `return`。

由于目标由元变量表示, 将一个项分配给元变量 `?m` 是 Lean 证明目标的底层方式。该项中出现的新元变量对应于必须证明的新子目标。

下面是 `hypothesis` 的一个简单调用:

```
theorem hypothesis_example {α : Type} {p : α → Prop} {a
  : α}
  (hpa : p a) :
  p a :=
  by hypothesis
```

如果我们添加追踪，那么可以看到在找到匹配的假设 hpa 并成功应用之前，会依次尝试 α 、p 和 a。

该示例使用了以下新常量：

```
getLCtx : TacticM LocalContext
LocalDecl.isImplementationDetail : LocalDecl → Bool
isDefEq : Expr → Expr → TacticM Bool
LocalDecl.type : LocalDecl → Expr
getMainGoal : TacticM MVarId
MVarId.assign : MVarId → Expr → TacticM Unit
LocalDecl.toExpr : LocalDecl → Expr
failure {α : Type} : TacticM α
```

8.5 表达式

元编程框架围绕着^{Expression}表达式(或^{Term}项)的类型 Expr 展开。表达式的一个重要组成部分是类型为 Name 的^{Name}名称。我们从它们开始。

名称可以使用单反引号指定。例如，'x 代表名称 x，它可以赋予变量或常量。在引用常量时，我们必须指定包括命名空间在内的完整名称；因此，要引用第 6 章中的 Even 谓词，我们必须编写 'LoVe.Even 而非 'Even。

如果我们想引用一个现有的常量，Lean 提供了双反引号语法，它使用 Lean 通常的名称^{Elaboration}阐释规则查找名称，并将其扩展为完整名称。因此，''Even 和 ''LoVe.Even 都指向名称 LoVe.Even，而如果我们编写一些未声明的名称（如 ''EvenIf），Lean 会报错。

类型 Expr 的定义如下：

```
inductive Expr : Type where
  | const    : Name → List Level → Expr
  | sort     : Level → Expr
  | fvar     : FVarId → Expr
  | mvar     : MVarId → Expr
  | app      : Expr → Expr → Expr
  | lam      : Name → Expr → Expr → BinderInfo → Expr
```

```

| bvar      : Nat → Expr
| forallE   : Name → Expr → Expr → BinderInfo → Expr
| letE      : Name → Expr → Expr → Expr → Bool →
  Expr
| lit       : Literal → Expr
| mdata     : MData → Expr → Expr
| proj      : Name → Nat → Expr → Expr

```

让我们来看看主要的构造子：

- `Expr.const name levels` 代表名为 `name` 的常量，例如 `Nat.add` 或 `N`。`levels` 参数代表^{Universe Levels}**宇宙层级**，这一概念将在第 12 章中进行解释。例如，`Expr.const 'Nat.add []` 代表 `Nat.add`，而 `Expr.const 'Nat []` 代表 `Nat`（即 `N`）。
- `Expr.sort level` 用于表示类型的类型。例如，`Expr.sort Level.zero` 代表 `Prop`，而 `Expr.sort (Level.succ Level.zero)` 代表 `Type`。
- `Expr.fvar id` 代表局部语境中的自由变量（例如 `a`、`h`）。`id` 参数是该变量的唯一标识符。
- `Expr.mvar id` 代表元变量，即带问号的变量 `?m`。`id` 参数是该元变量的唯一标识符。
- `Expr.app t u` 代表函数 `t` 应用于参数 `u`。例如，`Expr.app (Expr.const 'Nat.succ []) (Expr.const 'Nat.zero [])` 代表 `Nat.succ Nat.zero`。
- `Expr.lam name σ t bi` 代表匿名函数（或 λ -表达式）。`name` 参数是绑定变量的名称，`σ` 参数是绑定变量的类型，`t` 参数是函数体，而 `bi` 参数存储该变量是显式 `()`、隐式 `{ }` 还是类型类 `[]` 参数。
- `Expr.bvar i` 代表绑定变量，使用的是一种被称为^{De Bruijn Index}**De Bruijn 索引**的记法。`Expr.var 0` 指代由最近的^{Binder}**绑定子**绑定的变量，`Expr.var 1` 指代由第二近的绑定子绑定的变量，依此类推。

因此，

```

Expr.lam 'x (Expr.const 'Nat []) (Expr.bvar 0)
  BinderInfo.default

```

代表 `fun x : N ↦ x`，而

```

Expr.lam 'x (Expr.const 'Nat [])
  (Expr.lam 'y (Expr.const 'Nat []) (Expr.bvar 1)
    BinderInfo.default)
  BinderInfo.default

```

代表 $\text{fun } x \ y : \mathbb{N} \mapsto x$ 。

- `Expr.forallE name  $\sigma$   $\tau$  bi` 代表可能^{Dependent}依值的函数类型。`name` 参数是绑定变量的名称, σ 参数是定义域类型, τ 参数是结果类型, 而 `bi` 与上述 `Expr.lam` 相同。例如,

```
Expr.forallE 'n (Expr.const 'Nat [])
  (Expr.app (Expr.const 'Even []) (Expr.bvar 0))
  BinderInfo.default
```

代表 $(n : \mathbb{N}) \rightarrow \text{Even } n$ (也写成 $\forall n : \mathbb{N}, \text{Even } n$), 而

```
Expr.forallE 'dummy (Expr.const 'Nat [])
  (Expr.const 'Bool []) BinderInfo.default
```

代表 $\mathbb{N} \rightarrow \text{Bool}$ 。

8.6 第二个示例：一个合取式分解策略

在本节和下一节中, 我们定义了另外两个完成明确任务的策略。这两个策略中的第一个被称为 `destruct_and`, 它自动消除前提中的合取式。我们的目标是使如下证明自动化:

```
theorem abc_a (a b c : Prop) (h : a ∧ b ∧ c) :
  a :=
  And.left h
```

```
theorem abc_b (a b c : Prop) (h : a ∧ b ∧ c) :
  b :=
  And.left (And.right h)
```

```
theorem abc_bc (a b c : Prop) (h : a ∧ b ∧ c) :
  b ∧ c :=
  And.right h
```

```
theorem abc_c (a b c : Prop) (h : a ∧ b ∧ c) :
  c :=
  And.right (And.right h)
```

在每种情况下, 我们都希望只需编写 `by destruct_and h` 作为证明。

我们的策略依赖于一个辅助函数, 该函数以证明项 `hP` (最初是假设 `h`) 作为参数, 我们从中提取^{Conjunct}合取项:

```

partial def destructAndExpr (hP : Expr) : TacticM Bool
:=
withMainContext
  (do
    let target ← getMainTarget
    let P ← inferType hP
    let eq ← isDefEq P target
    if eq then
      let goal ← getMainGoal
      MVarId.assign goal hP
      return true
    else
      match Expr.and? P with
      | Option.none          => return false
      | Option.some (Q, R) =>
        let hQ ← mkAppM 'And.left #[hP]
        let success ← destructAndExpr hQ
        if success then
          return true
        else
          let hR ← mkAppM 'And.right #[hP]
          destructAndExpr hR)

```

与 hypothesis 一样，我们将整个 do 块传递给 withMainContext 函数。这确保了 inferType 和 isDefEq 在正确的局部语境中运行。

在 do 块内部，我们首先提取第一个目标的目的和 hP 的类型（通常是其命题）P。如果它们在计算和元变量实例化后相等，我们就通过分配其元变量来关闭目标，就像我们在 hypothesis 示例中所做的那样，并返回 true 表示成功。否则，我们检查 hP 的命题是否具有 $Q \wedge R$ 的形式。如果是，我们使用证明项 $hQ := \text{And.left } hP$ （它是 Q 的证明）递归调用辅助函数。如果成功，则完成；否则，我们尝试证明项 $hR := \text{And.right } hP$ （它是 R 的证明）。

注意函数定义开始处的关键字 partial。这里需要它是因为 Lean 无法证明该函数总是终止。由于该函数仅用作元程序，而不是在命题内部，因此终止是可选的，我们可以通过指定 partial 来禁用终止检查。

同样值得注意的是 mkAppM 函数，它用于构造常量对参数数组的^{Curried}柯里化应用。数组类似于列表，但书写时带有前缀符号 #（例如 $\#[1, 2, 3]$ ）。使用 mkAppM 比多次应用 Expr.app 构造子更方便。此外，mkAppM 允许我们省略隐式参数（例如命题 Q 和 R），否则我们必须将它们作为参数提供给 And.left 和 And.right。

主函数剩下的工作很少了：

```
def destructAnd (name : Name) : TacticM Unit :=
  withMainContext
    (do
      let h ← getFVarFromUserName name
      let success ← destructAndExpr h
      if ! success then
        failure)

elab "destruct_and" h:ident : tactic =>
  destructAnd (getId h)
```

该函数使用 `getFVarFromUserName` 获取假设 `h`，并调用辅助函数。如果辅助函数返回 `false`，则策略失败。

我们现在可以在这些动机性示例中使用我们的新工具了：

```
theorem abc_a_again (a b c : Prop) (h : a ∧ b ∧ c) :
  a :=
  by destruct_and h

theorem abc_b_again (a b c : Prop) (h : a ∧ b ∧ c) :
  b :=
  by destruct_and h

theorem abc_bc_again (a b c : Prop) (h : a ∧ b ∧ c) :
  b ∧ c :=
  by destruct_and h

theorem abc_c_again (a b c : Prop) (h : a ∧ b ∧ c) :
  c :=
  by destruct_and h
```

上述元程序中使用了以下新常量：

```
inferType : Expr → TacticM Expr
Expr.and? : Expr → Option (Expr × Expr)
mkAppM : Name → Array Expr → TacticM Expr
getFVarFromUserName : Name → TacticM Expr
```

8.7 第三个示例：一个直接证明查找器

有时我们陈述了一个定理，证明了它，后来才意识到该定理已经存在。这可以通过使用 `prove_direct` 来防止，该策略会遍历所有可用的定理，并检查其中是否有一个可以证明当前目标。我们将分步审阅其代码。

第一步是一个函数 `isTheorem`，如果声明是公理或定理，它返回 `true`，否则返回 `false`：

```
def isTheorem : ConstantInfo → Bool
  | ConstantInfo.axiomInfo _ => true
  | ConstantInfo.thmInfo _   => true
  | _                        => false
```

我们将使用此函数来过滤掉我们不感兴趣的声明。

下一个函数将名为 `name` 的定理应用于当前目标：

```
def applyConstant (name : Name) : TacticM Unit :=
do
  let cst ← mkConstWithFreshMVarLevels name
  liftMetaTactic (fun goal ↦ MVarId.apply goal cst)
```

给定一个名称，`mkConstWithFreshMVarLevels` 函数创建一个代表该常量的表达式 `cst`。函数名称中提到的「全新元变量层级」将在第 12 章中变得更加清晰。`MVarId.apply`（不要与 `MVarId.assign` 混淆）然后将该常量应用于当前目标，设置 $?m := cst \text{ ?}m_1 \dots ?m_n$ ，并返回代表 `cst` 前提的全新元变量 $?m_j$ 。

`liftMetaTactic` 函数检索第一个目标的标识符，在底层 `MetaM` 单子中对该目标运行给定的函数，并将该目标替换为函数返回的子目标。

下一个函数实现了一个其行为类似于 `<;>` 但可以从元程序中使用的组合子：

```
def andThenOnSubgoals (tac1 tac2 : TacticM Unit) :
  TacticM Unit :=
do
  let origGoals ← getGoals
  let mainGoal ← getMainGoal
  setGoals [mainGoal]
  tac1
  let subgoals1 ← getUnsolvedGoals
  let mut newGoals := []
  for subgoal in subgoals1 do
    let assigned ← MVarId.isAssigned subgoal
    if ! assigned then
```

```

    setGoals [subgoal]
  tac₂
  let subgoals₂ ← getUnsolvedGoals
  newGoals := newGoals ++ subgoals₂
  setGoals (newGoals ++ List.tail origGoals)

```

TacticM 单子跟踪当前要证明的目标。我们可以使用 `getGoals` 获取列表，并使用 `setGoals` 设置列表。如果我们希望策略暂时专注于特定的子目标，设置子目标列表是非常有用的。

在这里，我们首先专注于第一个目标 (`setGoals [mainGoal]`) 并调用第一个策略。对于产生的每个子目标，我们专注于它 (`setGoals [subgoal]`) 并调用第二个策略。从第二个策略产生的所有未解决的子子目标都收集在可变变量 `newGoals` 中。由于证明一个目标有时会实例化另一个元变量，我们在每次迭代中检查当前子目标的元变量是否已被分配，如果是，则跳过该子目标。最后，我们更新目标以包含所有待处理的目标：`newGoals` 中的目标，以及 `origGoals` 中除第一个目标外所有我们尚未考虑的目标。

通常，在策略结束时，我们应该确保目标列表由所有待证明的目标组成。否则，我们可能会遇到一些令人费解的错误，例如：

```
declaration has metavariables
```

我们还需要一个策略，它尝试使用由其名称指定的定理来证明目标，并调用 `hypothesis` 来证明任何产生的子目标：

```

def proveUsingTheorem (name : Name) : TacticM Unit :=
  andThenOnSubgoals (applyConstant name) hypothesis

```

这在程序上等同于证明 `apply name <;> hypothesis`。

最后，我们准备好审阅主函数了：

```

def proveDirect : TacticM Unit :=
  do
    let origGoals ← getUnsolvedGoals
    let goal ← getMainGoal
    setGoals [goal]
    let env ← getEnv
    for (name, info)
      in SMap.toList (Environment.constants env) do
      if isTheorem info && ! ConstantInfo.isUnsafe info
    then
      try

```

```

    proveUsingTheorem name
    logInfo m!"Proved directly by {name}"
    setGoals (List.tail origGoals)
    return
  catch _ =>
    continue
failure

elab "prove_direct" : tactic =>
  proveDirect

```

我们专注于第一个目标，然后遍历环境中声明的所有常量。如果该常量是一个定理，并且不是「unsafe」（一个与我们的不安全规则和策略概念无关的 Lean 概念），我们尝试使用我们的辅助函数 `proveUsingTheorem` 来应用它。如果成功，就打印「Proved directly by name」，其中 `name` 是该定理的名称，然后返回。如果失败，就继续遍历。如果整个遍历都结束了，就报告失败。

下面是该策略的实际应用：

```

theorem Nat.symm (x y : ℕ) (h : x = y) :
  y = x :=
  by prove_direct

```

这将打印「Proved directly by symm」。

该消息很有帮助，因为我们可以直接应用指定的定理，而无需依赖相对较慢的 `prove_direct` 策略。具体来说，我们可以结合 `hypothesis` 来应用定理 `symm`，如下所示：

```

theorem Nat.symm_manual (x y : ℕ) (h : x = y) :
  y = x :=
  by
    apply symm
    hypothesis

```

以下是本示例中出现的新常量列表：

```

mkConstWithFreshMVarLevels : Name → TacticM Expr
  liftMetaTactic : (MVarId → MetaM (List MVarId)) →
    TacticM Unit
  MVarId.apply : MVarId → Expr → MetaM (List MVarId)
  getGoals : TacticM (List MVarId)
  setGoals : List MVarId → TacticM Unit
  MVarId.isAssigned : MVarId → TacticM Bool
  getEnv : TacticM Environment
  SMap.toList : ConstMap → List (Name × ConstantInfo)
  Environment.constants : Environment → ConstMap
  ConstantInfo.isUnsafe : ConstantInfo → Bool

```

至此，我们对 `prove_direct` 的审阅就结束了。在 `mathlib` 中也有一个类似的策略，名为 `apply?`。

8.8 杂项策略

虽然本章的重点是开发新策略，但我们也遇到了三个预定义的策略。

skip

`skip` 策略在不执行任何操作的情况下成功。在我们开发自定义策略时，它有时可用作构建基块。

done

如果还有一些剩余目标，`done` 策略会产生失败；否则，它在不执行任何操作的情况下成功。与 `skip` 一样，它可以用作构建基块。

apply?

`apply?` 策略在加载的库中搜索能够精确证明目标的定理。成功时，它会建议一种形式为 `exact ...` 的策略调用，该调用可以插入到^{Formalization}形式化中。

8.9 新引入的 Lean 结构总结

声明

`partial`

作为可能不终止的元程序声明的前缀

引号

<code>'n</code>	引用一个字面名称
<code>''n</code>	引用一个经过阐释和检查的字面名称

策略

<code>apply?</code>	搜索能够证明当前目标的定理
<code>done</code>	如果还有目标剩余，则失败
<code>skip</code>	不执行任何操作

策略组合子

<code><;></code>	在第一策略产生的所有子目标上调用第二策略
<code>all_goals</code>	在每个目标上调用一次策略，并期望全部成功
<code>any_goals</code>	在每个目标上调用一次策略，并期望至少一个成功
<code>first </code>	依次尝试这些策略，直到其中一个成功为止
<code>repeat'</code>	在所有目标和子目标上重复调用策略，直到失败为止
<code>solve </code>	尝试通过依次尝试这些策略来完全证明当前目标
<code>try</code>	尝试调用一个策略；如果失败，则不执行任何操作

第三部分

程序语义

操作语义

在本章和接下来的两章中，我们将学习如何使用 Lean 来指定编程语言的^{Syntax}语法和^{Semantics}语义，证明语义的性质，以及对^{Concrete Program}具体程序进行推理。

本章深受^{Concrete Semantics: With Isabelle/HOL}《具体语义：Isabelle/HOL 描述》[23] 第 7 章的启发。

9.1 形式语义

^{Formal Semantics}形式语义允许我们对编程语言以及用该语言编写的单个程序进行^{Reduction Reasoning}归约和推理。它可以作为已验证的编译器、解释器、验证器、静态分析器、类型检查器等的基础。^{Formal Proof}如果没有形式证明，这些工具几乎总是错误的。

以 WebAssembly 为例，这是一种用于网页浏览器的类汇编语言，被设计为 C++ 和 Rust 等高级语言的可移植编译目标。研究员 Conrad Watt [27] 使用 Isabelle/HOL ^{Proof Assistant}证明助手形式化了其语义和^{Type System}类型系统。他发现了许多问题（粗体为本书作者添加）：

我们实现了 WebAssembly 语言核心执行语义和类型系统的完整 Isabelle 机械化。此外，我们针对工作组论文中阐述的^{Type Soundness}类型可靠性性质创建了机械化证明。为了完成这一证明，官方 WebAssembly 规范中由我们的证明和建模工作所揭示的若干缺陷，需要由规范作者进行修正。在某些情况下，这意味着类型系统最初是不可靠的。

我们与工作组成员保持着建设性的对话，在新特性被加入到规范中时对其进行机械化和验证。特别是，WebAssembly 实现与其宿主环境交互的机制在工作组的原始论文中并未进行形式化指定。扩展我们的机械化来对该特性进行建模，揭示了 WebAssembly 规范中的一个缺陷，该缺陷破坏了类型系统的可靠性。

Watt 的研究只是众多例子之一。证明助手被广泛用于编程语言研究。每年，在 Principles of Programming Languages, POPL Formalization 会议上发表的论文中，有很大一部分都进行了 **形式化**。这之所以成为可能，是因为入门起步所需的机制相对较少。这些证明往往包含许多分支情况，这非常适合由计算机来处理。此外，当我们为编程语言扩展更多特性时，证明助手在跟踪需要修改的内容方面极其便利。

9.2 一个极简的命令式语言

我们引入 *WHILE*¹，这是一种具有以下文法的极简命令式语言：

$S ::= \text{skip}$	(空操作)
$x := a$	(赋值)
$S; S$	(顺序组合)
$\text{if } b \text{ then } S \text{ else } S$	(条件语句)
$\text{while } b \text{ do } S$	(while 循环)

其中 S 代表 Statement **语句** (也称为 Command **命令** 或程序)， x 代表程序变量， a 代表 Arithmetic Expression **算术表达式**，而 b 代表 Boolean Expression **布尔表达式**。

在我们的文法中，我们刻意不对算术和布尔表达式的语法进行具体指定。在 Lean 中，我们有以下选择：

- 我们可以使用像第 2.1 节中那样名为 *AExp* 的类型，对于布尔表达式也是如此。
- 我们可以简单地决定，算术表达式是一个从状态到数字的函数（例如 $\text{State} \rightarrow \mathbb{N}$ ），而布尔表达式是关于状态的谓词（例如 $\text{State} \rightarrow \text{Bool}$ 或 $\text{State} \rightarrow \text{Prop}$ ）。**状态** 是从程序变量到值的映射。因此， $x + y + 1$ 将由函数 $\text{fun } s : \text{State} \mapsto s \text{ "x" } + s \text{ "y" } + 1$ 表示，而 $a \neq b$ 将由谓词 $\text{fun } s : \text{State} \mapsto s \text{ "a" } \neq s \text{ "b" }$ 表示。

这两个选项对应于 Deep Embedding **深嵌入** 和 Shallow Embedding **浅嵌入** 之间的区别。某种语法（表达式、公式、程序等）的深嵌入由在证明助手中指定的 Abstract Syntax Tree **抽象语法树**（例如 *AExp*）及其语义（例如 *eval*）组成。相比之下，浅嵌入只是重用了逻辑中相应的机制（例如函数和谓词）。

深嵌入允许我们对程序的语法进行推理。浅嵌入更加轻量，因为我们可以直接使用它，而不需要定义语义。浅嵌入本身就是它自己的语义。

在第 7 章中，我们使用了 Effectful Program **带作用的程序** 的浅嵌入。在这里，我们将使用程序的深嵌入（我们对此感兴趣并希望仔细研究）以及算术和布尔表达式的浅嵌入（我们对此不那么感兴趣）。下面是我们的 Lean 程序定义：

¹喜欢分析首字母缩略词的读者可能会觉得这个很有趣：弱假设命令式语言示例（Weak Hypothetical Imperative Language Example）。

```

inductive Stmt : Type where
  | skip      : Stmt
  | assign    : String → (State → ℕ) → Stmt
  | seq       : Stmt → Stmt → Stmt
  | ifThenElse : (State → Prop) → Stmt → Stmt → Stmt
  | whileDo   : (State → Prop) → Stmt → Stmt

```

中缀语法 $S; T$ 是 $\text{Stmt.seq } S \ T$ 的简写。

Inductive Type Constructor

归纳类型的构造子与 WHILE 文法规则之间的对应关系应该是显而易见的。变量由字符串表示。类型 State 被定义为 $\text{String} \rightarrow \mathbb{N}$ ，即从变量名到值的映射。为简单起见，我们的程序变量都属于自然数类型，并且所有可能的变量名在状态中均存在且被赋予了一个值。

以下这个小程序展示了深嵌入的 WHILE 语法（及其浅嵌入的方面）：

```

def sillyLoop : Stmt :=
  Stmt.whileDo (fun s ↦ s "x" > s "y")
    (Stmt.skip;
     Stmt.assign "x" (fun s ↦ s "x" - 1))

```

9.3 大步语义

Operational Semantics

Interpreter

Big-Step Semantics

Small-Step Semantics

操作语义对应于一个理想化的**解释器**。它有两种主要变体：**大步语义**和**小步语义**。

我们将首先为我们的 WHILE 语言给出大步语义。

Natural Semantics

Judgment

在大步操作语义（也称为**自然语义**）中，**判断**的形式为 $(S, s) \Longrightarrow t$ ，并且具有以下直观的解释：

从状态 s 开始，执行 S 可能会在状态 t 中终止。

Deterministic Language

对于像 WHILE 这样的**确定性语言**，由于程序总是只有唯一的结果，因此「可能终止」与「必定终止」（或「终止」）含义相同。

根据 WHILE 程序的定义，状态 s 是一个类型为 $\text{String} \rightarrow \mathbb{N}$ 的函数。下面是一个判断示例：

$$(x := x + y; y := 0, [x \mapsto 3, y \mapsto 5]) \Longrightarrow [x \mapsto 8, y \mapsto 0]$$

Informal Notation

我们使用**非形式记法** $[x \mapsto 3, y \mapsto 5]$ 来表示函数 $\text{fun } v \mapsto \text{if } v = "x" \text{ then } 3 \text{ else if } v = "y" \text{ then } 5 \text{ else } 0$ ，对于 $[x \mapsto 8, y \mapsto 0]$ 也是类似。直观上，该判断成立。

Derivation Rule Formal System

指定这种语义的传统方法是通过一个**推导规则的形式系统**，其风格类似于第 1.3 和 4.6 节中介绍的**类型规则**。下面给出了大步语义判断的推导规则。这些规则可以被看作是 WHILE 程序的理想化解释器。

$$\begin{array}{c}
\frac{}{(\text{skip}, s) \Rightarrow s} \text{SKIP} \\
\\
\frac{}{(x := a, s) \Rightarrow s[x \mapsto a \ s]} \text{ASSIGN} \\
\\
\frac{(S, s) \Rightarrow t \quad (T, t) \Rightarrow u}{(S; T, s) \Rightarrow u} \text{SEQ} \\
\\
\frac{(S, s) \Rightarrow t}{(\text{if } b \text{ then } S \text{ else } T, s) \Rightarrow t} \text{IF-TRUE} \quad \text{若 } b \ s \text{ 为 true} \\
\\
\frac{(T, s) \Rightarrow t}{(\text{if } b \text{ then } S \text{ else } T, s) \Rightarrow t} \text{IF-FALSE} \quad \text{若 } b \ s \text{ 为 false} \\
\\
\frac{(S, s) \Rightarrow t \quad (\text{while } b \text{ do } S, t) \Rightarrow u}{(\text{while } b \text{ do } S, s) \Rightarrow u} \text{WHILE-TRUE} \quad \text{若 } b \ s \text{ 为 true} \\
\\
\frac{}{(\text{while } b \text{ do } S, s) \Rightarrow s} \text{WHILE-FALSE} \quad \text{若 } b \ s \text{ 为 false}
\end{array}$$

在这些规则中, $a \ s$ 表示算术表达式 a 在状态 s 下的值, 对于 $b \ s$ 也是类似。此外, 语法 $s[x \mapsto n]$ 表示与 s 完全相同的状态, 除了它将变量 x 映射到 n 。其形式化表示为:

$$s[x \mapsto n] = (\text{fun } v \mapsto \text{if } v = x \text{ then } n \text{ else } s \ v)$$

此语法由 LoVeLib 提供。

最复杂的规则无疑是 WHILE-TRUE。直观上, 它可以理解为:

假设条件 b 在状态 s 下为真。如果 (1) 在状态 s 下执行 S 转为状态 t , 并且 (2) 从状态 t 开始执行 $\text{while } b \text{ do } S$ 转为状态 u , 那么在状态 s 下执行 $\text{while } b \text{ do } S$ 将转为状态 u 。

另一种思考 WHILE-TRUE 的方式是将其视为^{Loop Unrolling}**循环展开**。如果循环条件为真, $\text{while } b \text{ do } S$ 就等价于^{Compound Statement}**复合语句** $S; \text{while } b \text{ do } S$ 。WHILE-TRUE 的两个^{Premise}**前提**对应于 $S; \text{while } b \text{ do } S$ 的^{Instance}**SEQ 规则实例**的两个前提。

作为练习, 让我们推导上面的示例判断。令 $s := [x \mapsto 3, y \mapsto 5]$ 、 $t := [x \mapsto 8, y \mapsto 5]$ 以及 $u := [x \mapsto 8, y \mapsto 0]$ 。然后我们有:

$$\frac{\frac{}{(x := x + y, s) \Rightarrow t} \text{ASSIGN} \quad \frac{}{(y := 0, t) \Rightarrow u} \text{ASSIGN}}{(x := x + y; y := 0, s) \Rightarrow u} \text{SEQ}$$

推导规则可以直观地进行阅读。考虑 SEQ:

如果 (1) 在状态 s 下执行 S 转为状态 t , 且 (2) 在状态 t 下执行 T 转为状态 u , 那么在状态 s 下执行顺序组合 $S; T$ 将转为状态 u 。

条件 (1) 和 (2) 对应于 SEQ 的两个前提。

在 Lean 中, 大步语义判断由一个^{Inductive Predicate}归纳谓词表示, 其^{Introduction Rule}引入规则与上述推导规则紧密对应:

```
inductive BigStep : Stmt × State → State → Prop where
| skip (s) :
  BigStep (Stmt.skip, s) s
| assign (x a s) :
  BigStep (Stmt.assign x a, s) s[x ↦ a s]
| seq (S T s t u) (hS : BigStep (S, s) t)
  (hT : BigStep (T, t) u) :
  BigStep (S; T, s) u
| if_true (B S T s t) (hcond : B s)
  (hbody : BigStep (S, s) t) :
  BigStep (Stmt.ifThenElse B S T, s) t
| if_false (B S T s t) (hcond : ¬ B s)
  (hbody : BigStep (T, s) t) :
  BigStep (Stmt.ifThenElse B S T, s) t
| while_true (B S s t u) (hcond : B s)
  (hbody : BigStep (S, s) t)
  (hrest : BigStep (Stmt.whileDo B S, t) u) :
  BigStep (Stmt.whileDo B S, s) u
| while_false (B S s) (hcond : ¬ B s) :
  BigStep (Stmt.whileDo B S, s) s
```

使用归纳谓词而非^{Recursive Function}递归函数使我们能够应对^{Nontermination}不终止(发散的 while), 以及对于^{Nondeterminism}比 WHILE 更丰富的语言, 能够应对非确定性。它可以说还提供了更美观的语法, 更接近科学文献中传统使用的判断规则。如果我们转而尝试使用如下的递归定义:

```
def eval : Stmt → State → State
| Stmt.skip, s => s
| Stmt.assign x a, s => s[x ↦ a s]
| Stmt.ifThenElse b S T, s =>
  if b s then eval S s else eval T s
| S; T, s => eval T (eval S s)
| Stmt.whileDo b S, s =>
```

```

    if b s then eval (Stmt.whileDo b S) (eval S s) else
      S

```

我们将面临情况 `Stmt.whileDo` 的不终止问题。事实上,由于程序 `Stmt.whileDo (fun _ ↦ True) Stmt.skip` 会无限循环,尝试使用 `eval` 对其求值将永远不会返回。

有了大步语义,我们就可以对诸如第 9.2 节中定义的具体程序进行推理,并证明如下的定理:

```

theorem sillyLoop_from_1_BigStep :
  (sillyLoop, (fun _ ↦ 0)["x" ↦ 1]) ⇒ (fun _ ↦ 0)
:=
by
  rw [sillyLoop]
  apply BigStep.while_true
  · simp
  · apply BigStep.seq
    · apply BigStep.skip
    · apply BigStep.assign
  · simp
  apply BigStep.while_false
  simp

```

9.4 大步语义的性质

大步语义允许我们对具体程序进行推理,证明将最终状态与初始状态联系起来的定理。同样重要的是,它允许我们证明编程语言的性质,例如^{Determinism}**确定性**和不终止。

我们从确定性开始。这看起来像是一个平凡的性质,但很容易因为写错规则而引入非确定性。例如,在赋值规则中,如果我们将 `x` 误写为 `y` 得到 `BigStep (Stmt.assign x a, s) (s[y ↦ a s])`,我们就可以利用该规则来随意修改任何变量 `y`。换句话说,程序的执行可能会随机修改任何变量。因此,让我们验证一下我们的 `WHILE` 语言确实是确定性的:

```

theorem BigStep_deterministic {Ss l r} (hl : Ss ⇒ l)
  (hr : Ss ⇒ r) :
  l = r

```

Lean 证明位于与本章相应的演示文件中。由于技术原因,偶对 (S, s) 由单个变量 `Ss` 表示。我们在此仅提供一个非形式证明的简述:

证明通过对 $(S, s) \Rightarrow l$ 进行^{Rule Induction}**规则归纳**来完成。

情况 SKIP: 要使 $(\text{skip}, s) \Rightarrow l$ 成立, 我们需要 $l = s$ 。类似地, 通过对 $(\text{skip}, s) \Rightarrow r$ 进行分类讨论, 我们得到 $r = s$ 。因此, $l = r$ 。

情况 ASSIGN: 与 SKIP 类似。

情况 SEQ: 我们有假设 $(S, s) \Rightarrow t$ 、 $(T, t) \Rightarrow l$ 、 $(S, s) \Rightarrow t'$ 和 $(T, t') \Rightarrow r$, 以及归纳假设 $\forall r, (S, s) \Rightarrow r \rightarrow t = r$ 和 $\forall r, (T, t) \Rightarrow r \rightarrow l = r$ 。由第一个归纳假设以及 $(S, s) \Rightarrow t'$, 我们推导出 $t = t'$ 。由第二个归纳假设以及 $(T, t') \Rightarrow r$, 我们推导出 $l = r$ 。

情况 IF-TRUE: 由于 $b \ s$ 为真, $(\text{if } b \text{ then } S \text{ else } T, s) \Rightarrow r$ 只能是通过 IF-TRUE 推导出来的, 因此 $(S, s) \Rightarrow r$ 。归纳假设为 $\forall r, (S, s) \Rightarrow r \rightarrow l = r$ 。我们可以将 $(S, s) \Rightarrow r$ 应用于其上以得到 $l = r$ 。

情况 IF-FALSE: 与 IF-TRUE 类似。

情况 WHILE-TRUE: 与 SEQ 类似。

情况 WHILE-FALSE: 与 SKIP 类似。 □

鉴于 WHILE 语言是确定性的, 对于大步语义, 其^{Termination}终止性将等同于以下性质:

theorem BigStep_terminates $\{S \ s\}$:
 $\exists t, (S, s) \Rightarrow t$

该性质意味着, 对于每一个语句 S 和状态 s , 都存在一个状态 t , 使得从 s 开始执行 S 可能会在 t 中终止。因为 WHILE 是确定性的, 「可能终止」与「必定终止」含义相同。

然而, 这一性质并不成立。

在对归纳谓词进行推理时, 通常使用^{Inversion Rule}反转规则(第 6.5 节)会很方便。因此, 我们证明了以下规则:

@[simp] theorem BigStep_skip_Iff $\{s \ t\}$:
 $(\text{Stmt.skip}, s) \Rightarrow t \leftrightarrow t = s$

@[simp] theorem BigStep_assign_Iff $\{x \ a \ s \ t\}$:
 $(\text{Stmt.assign } x \ a, s) \Rightarrow t \leftrightarrow t = s[x \mapsto a \ s]$

@[simp] theorem BigStep_seq_Iff $\{S \ T \ s \ u\}$:
 $(S; T, s) \Rightarrow u \leftrightarrow (\exists t, (S, s) \Rightarrow t \wedge (T, t) \Rightarrow u)$

@[simp] theorem BigStep_if_Iff $\{B \ S \ T \ s \ t\}$:
 $(\text{Stmt.ifThenElse } B \ S \ T, s) \Rightarrow t \leftrightarrow$
 $(B \ s \wedge (S, s) \Rightarrow t) \vee (\neg B \ s \wedge (T, s) \Rightarrow t)$

```

theorem BigStep_while_Iff {B S s u} :
  (Stmt.withDo B S, s)  $\Rightarrow$  u  $\leftrightarrow$ 
  (B s  $\wedge$   $\exists$ t, (S, s)  $\Rightarrow$  t  $\wedge$  (Stmt.withDo B S, t)  $\Rightarrow$ 
  u)
   $\vee$  ( $\neg$  B s  $\wedge$  u = s)

```

```

@[simp] theorem BigStep_while_true_Iff {B S s u}
  (hcond : B s) :
  (Stmt.withDo B S, s)  $\Rightarrow$  u  $\leftrightarrow$ 
  ( $\exists$ t, (S, s)  $\Rightarrow$  t  $\wedge$  (Stmt.withDo B S, t)  $\Rightarrow$  u)

```

```

@[simp] theorem BigStep_while_false_Iff {B S s t}
  (hcond :  $\neg$  B s) :
  (Stmt.withDo B S, s)  $\Rightarrow$  t  $\leftrightarrow$  t = s

```

我们将这些规则添加到 simp 集合中，但 BigStep_while_Iff 除外，因为它会导致 simp 陷入死循环。

9.5 小步语义

大步语义的一个局限性在于，它不能让我们对^{Intermediate State}中间状态进行推理。从判断^{Multithreaded Program}(S, s) $\Rightarrow$ t 中，我们能看到的只有初始状态 s 和最终状态 t。这对于推理多线程程序来说过于粗粒度了，因为在多线程程序中，若干个进程可能会相互影响彼此的中间状态。此外，对于非确定性语言，大步语义没有提供表达终止性的通用方法：判断指示的是一种可能性（在状态 s 下执行 S **可能会**转为状态 t），而非必然性。

小步操作语义提供了更精细的视角。^{Transition Predicate}迁移谓词 $\Rightarrow$ 的类型为 Stmt $\times$ State $\rightarrow$ Stmt $\times$ State $\rightarrow$ Prop。直观上，(S, s) $\Rightarrow$ (T, t) 意味着在状态 s 下执行程序 S 的一步，留下待执行 of 的程序 T 在状态 t。如果已经没有任何可执行的内容，我们则放置 skip。

^{Execution}一次执行是一个由「小」^{Configuration} $\Rightarrow$ 步骤组成的有限或无限链 (S₀, s₀) $\Rightarrow$ (S₁, s₁) $\Rightarrow$...。偶对 (S, s) 被称为^{Final}配置；如果对于任何 (T, t) 都无法进行形如 (S, s) $\Rightarrow$ (T, t) 的迁移，则称该配置是^{Final}终态的。一个执行的示例如下：

```

(x := x + y; y := 0, [x  $\mapsto$  3, y  $\mapsto$  5])
 $\Rightarrow$  (skip; y := 0, [x  $\mapsto$  8, y  $\mapsto$  5])
 $\Rightarrow$  (y := 0, [x  $\mapsto$  8, y  $\mapsto$  5])
 $\Rightarrow$  (skip, [x  $\mapsto$  8, y  $\mapsto$  0])

```

如果类比计算机处理器，配置 (S, s) 中的 S 部分可以被看作是 ^{Program Counter} **程序计数器**，它指示下一步应该执行哪些指令。程序的逐步执行类似于在调试器中运行程序，并在每一步都设有一个断点。

有效的小步判断由推导规则给出：

$$\begin{array}{c}
 \frac{}{(x := a, s) \Rightarrow (\text{skip}, s[x \mapsto a \ s])} \text{ASSIGN} \\
 \frac{(S, s) \Rightarrow (S', s')}{(S; T, s) \Rightarrow (S'; T, s')} \text{SEQ-STEP} \\
 \frac{}{(\text{skip}; T, s) \Rightarrow (T, s)} \text{SEQ-SKIP} \\
 \frac{}{(\text{if } b \text{ then } S \text{ else } T, s) \Rightarrow (S, s)} \text{IF-TRUE} \quad \text{若 } b \text{ 为 true} \\
 \frac{}{(\text{if } b \text{ then } S \text{ else } T, s) \Rightarrow (T, s)} \text{IF-FALSE} \quad \text{若 } b \text{ 为 false} \\
 \frac{}{(\text{while } b \text{ do } S, s) \Rightarrow (\text{if } b \text{ then } (S; \text{while } b \text{ do } S) \text{ else skip}, s)} \text{WHILE}
 \end{array}$$

这些规则令人联想到第 6.1.2 节中的网球比赛转移系统。那个系统同样刻画了小步转移：从 0-0 到 15-0，再到 15-15，依此类推。

与大步语义不同，小步语义中没有针对 skip 的规则。这是因为形如 (skip, s) 的 ^{Configuration} **配置** 被视为最终配置；skip 被理解为执行平凡的语句。通过检视这些规则，我们可以确信：一个配置是最终配置，当且仅当其第一个分量为 skip。

关于顺序复合 $S; T$ ，有两条规则。第一条规则在 S 尚能继续执行时适用；若 S 已是 skip，则无法继续推进，此时第二条规则适用。

针对 if 的规则检验条件 b 的真值，并据此将 then 分支或 else 分支作为后续待执行的计算。

对于 while 循环，只有一条无条件规则，它展开循环的一次迭代，引入一条 if 语句。随后由 IF-TRUE 和 IF-FALSE 规则处理该 if。在 IF-TRUE 的情形下，最终会再次抵达 while 循环，从而可以无限持续下去。

在 Lean 中，小步语义的规定如下：

```

inductive SmallStep : Stmt × State → Stmt × State →
  Prop where
| assign (x a s) :
  SmallStep (Stmt.assign x a, s) (Stmt.skip, s[x ↦ a
s])
| seq_step (S S' T s s') (hS : SmallStep (S, s) (S',
s')) :

```

```

SmallStep (S; T, s) (S'; T, s')
| seq_skip (T s) :
  SmallStep (Stmt.skip; T, s) (T, s)
| if_true (B S T s) (hcond : B s) :
  SmallStep (Stmt.ifThenElse B S T, s) (S, s)
| if_false (B S T s) (hcond : ¬ B s) :
  SmallStep (Stmt.ifThenElse B S T, s) (T, s)
| whileDo (B S s) :
  SmallStep (Stmt.whileDo B S, s)
    (Stmt.ifThenElse B (S; Stmt.whileDo B S) Stmt.skip
, s)

```

基于小步语义，我们可以**定义**大步语义如下：

$$(S, s) \Longrightarrow t \quad \text{当且仅当} \quad (S, s) \Rightarrow^* (\text{skip}, t)$$

其中 p^* 表示二元谓词 p 的^{Reflexive Transitive Closure}**自反传递闭包**(RTC)。另一方面，若我们已经定义了大步语义，则可以**证明**上述等价定理，以**验证**我们的定义。

小步语义的主要缺点在于，我们现在需要同时使用两个谓词 $\Rightarrow$ 与 $\Rightarrow^*$ ，其推导规则与证明相较于大步语义更为繁琐。下面的例子清晰地说明了这一点：该例子需要对每个小步都应用定理

RTC.head : ?R ?a ?b → RTC ?R ?b ?c → RTC ?R ?a ?c

每一步小步均须如此：

```

theorem sillyLoop_from_1_SmallStep :
  (sillyLoop, (fun _ ↦ 0) ["x" ↦ 1]) ⇒*
  (Stmt.skip, (fun _ ↦ 0)) :=
by
  rw [sillyLoop]
  apply RTC.head
  • apply SmallStep.whileDo
  • apply RTC.head
  • apply SmallStep.if_true
    aesop
  • apply RTC.head
  • apply SmallStep.seq_step
    apply SmallStep.seq_skip
  • apply RTC.head
  • apply SmallStep.seq_step

```

```

    apply SmallStep.assign
  • apply RTC.head
  • apply SmallStep.seq_skip
  • apply RTC.head
  • apply SmallStep.whileDo
  • apply RTC.head
    • apply SmallStep.if_false
      simp
    • simp
    apply RTC.refl

```

9.6 小步语义的性质

我们可以证明，配置 (S, s) 是最终配置，当且仅当 $S = \text{skip}$ 。这保证了我们没有遗漏任何推导规则，从而小步语义不会卡死。定理表述如下：

```

theorem SmallStep_final (S s) :
  (¬ ∃ T t, (S, s) ⇒ (T, t)) ↔ S = Stmt.skip

```

证明对 S 进行结构归纳。

与大步语义类似，小步语义同样是确定性的：

```

theorem SmallStep_deterministic {Ss Ll Rr}
  (hl : Ss ⇒ Ll) (hr : Ss ⇒ Rr) :
  Ll = Rr

```

证明对 hl 或 hr 进行规则归纳。

对于小步语义，若从配置 (S_0, s_0) 出发的所有执行均有限，则称其^{Terminate}**终止**： $(S_0, s_0) ⇒ (S_1, s_1) ⇒ \dots ⇒ (S_n, s_n)$ 。若存在无限链 $(S_0, s_0) ⇒ (S_1, s_1) ⇒ \dots$ ，则称其^{Non-Terminate}**非终止**。整个编程语言在所有配置均终止时才是终止的。取 $S_0 := \text{Stmt.whileDo}(\text{fun } _ \mapsto \text{True}) \text{ Stmt.skip}$ 可以轻易说明 WHILE 语言是非终止的：对任意 s_0 ，有

$$(S_0, s_0) \Rightarrow (S_0, s_0) \Rightarrow (S_0, s_0) \Rightarrow \dots$$

我们可以为小步语义定义^{Inversion Rule}**反转规则**，例如：

```

theorem SmallStep_skip {S s t} :
  ¬ ((Stmt.skip, s) ⇒ (S, t))

```

```

@[simp] theorem SmallStep_seq_Iff {S T s Ut} :

```

$$\begin{aligned}
 (S; T, s) \Rightarrow Ut &\leftrightarrow \\
 (\exists S' \ t, (S, s) \Rightarrow (S', t) \wedge Ut = (S'; T, t)) \\
 \vee (S = \text{Stmt.skip} \wedge Ut = (T, s))
 \end{aligned}$$

```
@[simp] theorem SmallStep_if_Iff {B S T s Us} :
  (Stmt.ifThenElse B S T, s)  $\Rightarrow$  Us  $\leftrightarrow$ 
  (B s  $\wedge$  Us = (S, s))  $\vee$  ( $\neg$  B s  $\wedge$  Us = (T, s))
```

更根本的结果是大步语义与小步语义之间的等价性：

```
theorem BigStep_Iff_RTC_SmallStep {Ss t} :
  Ss  $\Rightarrow^*$  t  $\leftrightarrow$  Ss  $\Rightarrow^*$  (Stmt.skip, t)
```

回顾一下， $\Rightarrow^*$ 表示小步谓词 $\Rightarrow$ 的 ^{Reflexive Transitive Closure} **自反传递闭包**。该定理的证明超出了本课程的范围，详见 ^{Concrete Semantics: With Isabelle/HOL} 《具体语义：Isabelle/HOL 描述》[23] 第 7 章，或本章的演示文件。

霍尔逻辑

Operational Semantics Interpreter Hoare Logic Verifier
 若**操作语义**对应于一个理想化的**解释器**，则**霍尔逻辑**对应于一个理想化的**验证器**。
 霍尔逻辑可以用于指定编程语言的语义，但更适合对具体程序进行推理并证明其正确性。它以其发明者 Charles Antony Richard (Tony) Hoare 的名字命名。霍尔逻辑
 Axiomatic Semantics
 也被称为**公理语义**。

Concrete Semantics: With Isabelle/HOL
 本章深受《**具体语义：Isabelle/HOL 描述**》[23][23] 第 12 章的启发。

10.1 霍尔三元组

Derivation Rule
 霍尔逻辑是一套基于**推导规则**的推理框架，它以机械的方式来推导有效正确的公式。它允许我们直接对程序的语法进行推理，而无需关心其操作语义。该方法的「机械性」体现在推导规则的适用性可以被轻松验证。

首先，我们会抽象地介绍霍尔逻辑，不涉及 Lean 的具体实现。其次，我们将看到如何在 Lean 中嵌入霍尔逻辑的**判断**。霍尔逻辑的基本判断称为**霍尔三元组**，其形式为 $\{P\} S \{Q\}$ ，其中 S 是一条 WHILE 语句， P 与 Q 是关于程序变量的逻辑公式。目前，我们将这些公式视为由熟悉的联结词和量词构建的语法对象。霍尔三元组的含义如下：

Precondition Postcondition
 若**前置条件** P 在 S 执行前为真，且执行正常终止，则**后置条件** Q 在终止时为真。

Partial Correctness
 这是一个**部分正确性**陈述：程序在**正常终止时**是正确的；否则，程序的行为可以是任意的。对于 WHILE 程序而言，不能正常终止的唯一方式是进入无限循环。对于其他编程语言，无限递归和运行时错误（如除以零）也可能导致发散或异常终止。直觉上，下列所有霍尔三元组都应当是有效的：

$$\begin{aligned}
& \{\text{True}\} b := 4 \{b = 4\} \\
& \{a = 2\} b := 2 * a \{a = 2 \wedge b = 4\} \\
& \{b \geq 5\} b := b + 1 \{b \geq 6\} \\
& \{\text{False}\} \text{skip} \{b = 10\} \\
& \{\text{True}\} \text{while } i \neq 10 \text{ do } i := i + 1 \{i = 10\}
\end{aligned}$$

前三个霍尔三元组是相当自然的。第四个三元组^{vacuously}空洞地为真,因为前置条件 False 永远不可能被满足。在霍尔三元组的定义中,当前置条件为假时,整个三元组自动为真。这个三元组等价于 $\text{False} \rightarrow b = 10$, 对任何 b 的值都成立。对于第五个三元组,虽然无法保证程序一定会退出循环,但如果确实退出了,那么循环条件在退出时必然为假,因此我们能得到所需的后置条件 $i = 10$ 。

现在来看一些特别有趣的三元组:

$$\begin{aligned}
& \{\text{False}\} S \{\text{True}\} \\
& \{\text{False}\} S \{\text{False}\} \\
& \{\text{True}\} S \{\text{True}\} \\
& \{\text{True}\} S \{\text{False}\}
\end{aligned}$$

前两个三元组对任意语句 S 均为真 (因而没有实质意义): 前置条件永远不满足,故任何后置条件均空真成立。第三个三元组同样永远为真,与 S 无关。第四个三元组在 S 永不终止时为真 (例如 $S := \text{while True do skip}$); 否则为假。

10.2 霍尔规则

下面我们给出一套完整的推导规则,用于对 WHILE 程序进行推理:

$$\begin{array}{c}
\frac{}{\{P\} \text{skip} \{P\}} \text{SKIP} \\
\\
\frac{}{\{Q[a/x]\} x := a \{Q\}} \text{ASSIGN} \\
\\
\frac{\{P\} S \{R\} \quad \{R\} T \{Q\}}{\{P\} S; T \{Q\}} \text{SEQ} \\
\\
\frac{\{P \wedge B\} S \{Q\} \quad \{P \wedge \neg B\} T \{Q\}}{\{P\} \text{if } B \text{ then } S \text{ else } T \{Q\}} \text{IF} \\
\\
\frac{\{P \wedge B\} S \{P\}}{\{P\} \text{while } B \text{ do } S \{P \wedge \neg B\}} \text{WHILE}
\end{array}$$

$$\frac{P' \rightarrow P \quad \{P\} S \{Q\} \quad Q \rightarrow Q'}{\{P'\} S \{Q'\}} \text{CONSEQ}$$

在 ASSIGN 规则中, 表达式 $Q[a/x]$ 表示将条件 Q 中所有出现的 x 替换为 a 后所得的条件。该规则有时也以如下方式呈现:

$$\frac{}{\{Q[a]\} x := a \{Q[x]\}} \text{ASSIGN}$$

其中 $[x]$ 提取出 Q 中所有 x 出现的位置。

ASSIGN 规则也许看起来反直觉, 因为它是逆向工作的: 从后置条件出发, 计算前置条件。尽管如此, 它正确地捕捉了赋值语句的语义, 如下所示:

$$\begin{aligned} &\{0 = 0\} x := 0 \{x = 0\} \\ &\{0 = 0 \wedge y = 5\} x := 0 \{x = 0 \wedge y = 5\} \\ &\{x + 1 \geq 5\} x := x + 1 \{x \geq 5\} \end{aligned}$$

利用基本算术, 我们可以化简计算得到的前置条件; 例如, $0 = 0$ 等价于 True , $x + 1 \geq 5$ 等价于 $x \geq 4$ 。

SEQ 规则要求我们提出一个中间条件 R , 它在 S 执行后、 T 执行前成立。下面是 SEQ 的一个应用示例:

$$\frac{\frac{}{\{a = 2\} b := a \{b = 2\}} \text{ASSIGN} \quad \frac{}{\{b = 2\} c := b \{c = 2\}} \text{ASSIGN}}{\{a = 2\} b := a; c := b \{c = 2\}} \text{SEQ}$$

WHILE 规则是最复杂的。条件 P 被称为^{Invariant}**不变式**。它既是循环本身的前置和后置条件, 也是其循环体的前置和后置条件。由于在执行循环体之前 B 必须为真, 循环体的前置条件得以增强。类似地, 由于在循环退出时 B 必须为假, 循环的后置条件也得以增强。

考虑一个执行了^{Iteration} n 次迭代的循环。假设初始状态为 s_0 , 且在循环的第 i 次迭代之后的状态为 s_i 。那么以下条件将会成立:

$$P s_0 \quad P s_1 \wedge B s_1 \quad \cdots \quad P s_{n-1} \wedge B s_{n-1} \quad P s_n \wedge \neg B s_n$$

如果 $n = 0$, 我们立即可得 $P s_0 \wedge \neg B s_0$, 并且不会进入循环。

CONSEQ 是^{Premise}唯一一个在其前提中包含逻辑公式 (而非霍尔三元组) 的规则。这些^{Discharge}条件必须被解除, 无论是使用纸笔还是使用证明助手。CONSEQ 可以用于增强前置条件 (即使其限制更严格)、削弱后置条件 (即使其限制更宽松), 或者两者兼有。以下是一个推导示例:

$$\frac{x > 8 \rightarrow x > 4 \quad \frac{}{\{x > 4\} y := x \{y > 4\}} \text{ASSIGN} \quad y > 4 \rightarrow y > 0}{\{x > 8\} y := x \{y > 0\}} \text{CONSEQ}$$

从上到下阅读该树，我们把三元组的前置条件从 $x > 4$ 增强为 $x > 8$ ，并将后置条件从 $y > 4$ 削弱为 $y > 0$ 。我们也可以从下往上阅读该树：要证明三元组 $\{x > 8\} y := x \{y > 0\}$ ，只需证明 $\{x > 4\} y := x \{y > 4\}$ 即可，其中前置条件被削弱，而后置条件被增强。

除了 CONSEQ 之外，其他规则都是^{Syntax-driven}语法驱动的：我们只需检查当前的语句，就能知道每种情况下该应用哪个规则。对于赋值语句，我们应用 ASSIGN；对于 while 循环，我们应用 WHILE；以此类推。

SEQ、IF 和 CONSEQ 规则是^{Bidirectional}双向的：它们的结论形式均为 $\{P\} \dots \{Q\}$ ，其中 P 和 Q 是不同的数学变量。这使得它们在应用时非常方便。通过将其他规则与 CONSEQ 结合，我们可以推导出双向的变体：

$$\begin{array}{c}
 \frac{P \rightarrow Q}{\{P\} \text{ skip } \{Q\}} \text{SKIP}' \\
 \\
 \frac{P \rightarrow Q[a/x]}{\{P\} x := a \{Q\}} \text{ASSIGN}' \\
 \\
 \frac{\{P \wedge B\} S \{P\} \quad P \wedge \neg B \rightarrow Q}{\{P\} \text{ while } B \text{ do } S \{Q\}} \text{WHILE}'
 \end{array}$$

作为练习，您可以尝试使用 SKIP、ASSIGN 或 WHILE 结合 CONSEQ 推导出这些规则。

10.3 霍尔逻辑的语义方法

在 Lean 中编码霍尔逻辑的一种自然方式是沿用我们在大步和小步语义中的做法：定义霍尔三元组的句法概念以及一个归纳谓词，为每条核心霍尔规则提供一个引入规则；为了表示前置条件和后置条件，我们在状态上使用谓词 ($\text{State} \rightarrow \text{Prop}$)。随后，我们可以在大步语义上证明 *soundness*，其含义如下：只要 $\{P\} S \{Q\}$ 可导出，且若 $P s$ 与 $(S, s) \Rightarrow t$ 成立，则 $Q t$ 成立。这仅是对霍尔三元组直观含义的逻辑呈现：

如果前置条件 P 在 S 执行前为真（即 $P s$ ），且执行正常终止（即 $(S, s) \Rightarrow t$ ），则后置条件 Q 在终止时为真（即 $Q t$ ）。

我们不采用这种方法，而是建议直接在 Lean 中以语义方式定义霍尔三元组，基于大步语义，使其**按定义正确**。随后我们将把霍尔规则作为定理推导，而不是把它们写成引入规则。结合使用谓词来表示公式，这种方法彻底采用语义化。

霍尔三元组（部分正确版）的定义如下：

```
def PartialHoare (P : State → Prop) (S : Stmt)
```

$$(Q : \text{State} \rightarrow \text{Prop}) : \text{Prop} := \\ \forall s, t, P \ s \rightarrow (S, s) \Longrightarrow t \rightarrow Q \ t$$

我们引入了一些^{Syntactic Sugar}语法糖, 以 $\{ * P * \} S \{ * Q * \}$ 代替 $\text{PartialHoare } P \ S \ Q$, 从而更接近非形式语法 $\{P\} S \{Q\}$ 。

核心霍尔规则如下所示:

```
theorem skip_intro {P} :
  { * P * } Stmt.skip { * P * }

theorem assign_intro (P) {x a} :
  { * fun s ↦ P s[x ↦ a s] * } Stmt.assign x a { * P * }

theorem seq_intro {P Q R S T} (hS : { * P * } S { * Q * })
  (hT : { * Q * } T { * R * }) :
  { * P * } S; T { * R * }

theorem if_intro {B P Q S T}
  (hS : { * fun s ↦ P s ∧ B s * } S { * Q * })
  (hT : { * fun s ↦ P s ∧ ¬ B s * } T { * Q * }) :
  { * P * } Stmt.ifThenElse B S T { * Q * }

theorem while_intro (P) {B S}
  (h : { * fun s ↦ P s ∧ B s * } S { * P * }) :
  { * P * } Stmt.whileDo B S { * fun s ↦ P s ∧ ¬ B s * }

theorem consequence {P P' Q Q' S}
  (h : { * P * } S { * Q * }) (hp : ∀s, P' s → P s)
  (hq : ∀s, Q s → Q' s) :
  { * P' * } S { * Q' * }
```

以上所有定理都有基于大步语义的证明。有些三元组需要引用状态, 例如 `assign_intro` 中的前置条件。于是我们使用匿名函数来访问它。回想一下, P 与 $\text{fun } s \mapsto P \ s$ 是相等的 (通过 η -转换)。此外, 对于非形式写作 $P \rightarrow Q$ 的前提, 在 Lean 中我们必须写作 $\forall s, P \ s \rightarrow Q \ s$ 。如第 9 章所述, 赋值规则中的语法 $s[x \mapsto n]$ 表示与 s 相同的状态, 只是它将 x 映射到 n 。

可以从核心规则导出以下便利的规则:

```
theorem consequence_left (P') {P Q S}
  (h : { * P * } S { * Q * }) (hp : ∀s, P' s → P s) :
  { * P' * } S { * Q * }
```

```

theorem consequence_right (Q) {Q' P S}
  (h : {* P *} S {* Q *}) (hq :  $\forall s, Q\ s \rightarrow Q'\ s$ ) :
  {* P *} S {* Q' *}

theorem skip_intro' {P Q} (h :  $\forall s, P\ s \rightarrow Q\ s$ ) :
  {* P *} Stmt.skip {* Q *}

theorem assign_intro' {P Q x a}
  (h :  $\forall s, P\ s \rightarrow Q\ s[x \mapsto a\ s]$ ):
  {* P *} Stmt.assign x a {* Q *}

theorem seq_intro' {P Q R S T} (hT : {* Q *} T {* R *})
  (hS : {* P *} S {* Q *}) :
  {* P *} S; T {* R *}

theorem while_intro' {B P Q S} (I)
  (hS : {* fun s  $\mapsto$  I s  $\wedge$  B s *} S {* I *})
  (hP :  $\forall s, P\ s \rightarrow I\ s$ )
  (hQ :  $\forall s, \neg B\ s \rightarrow I\ s \rightarrow Q\ s$ ) :
  {* P *} Stmt.WhileDo B S {* Q *}

```

使用双向的 `assign_intro'`，我们可以导出赋值规则的正向版本：

```

theorem assign_intro_forward (P) {x a} :
  {* P *}
  (Stmt.assign x a)
  {* fun s  $\mapsto$   $\exists n_0, P\ s[x \mapsto n_0] \wedge s\ x = a\ s[x \mapsto n_0]$  *}
  :=
by
  apply assign_intro'
  intro s hP
  apply Exists.intro (s x)
  simp [*]

```

变量 n_0 代表赋值之前 x 的值。因此，在后置条件中， $s[x \mapsto n_0]$ 表示赋值之前的状态。由于 P 是前置条件，我们有 $P(s[x \mapsto n_0])$ 。此外，由 $s\ x$ 给出的 x 的新值，必须等于表达式 a 在旧状态 $s[x \mapsto n_0]$ 中求值所得的值。

正向规则不如逆向规则方便，因为后置条件包含一个存在量词。也可以用类似风格陈述逆向规则，从而揭示一个隐藏的对称性：

```

theorem assign_intro_backward (Q) {x a} :
  {* fun s  $\mapsto$   $\exists n', Q\ s[x \mapsto n'] \wedge n' = a\ s\ x$  *}

```

```
(Stmt.assign x a)
{* Q *}
```

注意，这个存在量词可以用单点规则消去（第 4.3 节）。于是我们得到熟悉的逆向规则 `assign_intro`，其前置条件为 $\text{fun } s \mapsto P(s[x \mapsto a \ s])$ 。

10.4 第一个程序：交换两个变量

让我们运用霍尔逻辑来验证第一个程序：一个三行程序，它交换变量 `a` 和 `b` 的值，并使用 `t` 作为临时存储。该程序定义如下：

```
def SWAP : Stmt :=
  Stmt.assign "t" (fun s ↦ s "a");
  Stmt.assign "a" (fun s ↦ s "b");
  Stmt.assign "b" (fun s ↦ s "t")
```

正确性陈述如下：

```
theorem SWAP_correct (a₀ b₀ : ℕ) :
  {* fun s ↦ s "a" = a₀ ∧ s "b" = b₀ *}
  (SWAP)
  {* fun s ↦ s "a" = b₀ ∧ s "b" = a₀ *}
```

逻辑变量 `a₀` 和 `b₀`「冻结」了程序变量 `a` 和 `b` 的初始值，使我们可以在后置条件中引用它们。毕竟，使用 $\text{fun } s \mapsto s \text{ "a" } = s \text{ "b" } \wedge s \text{ "b" } = s \text{ "a"}$ 作为后置条件是没有意义的。

正确性证明如下：

```
by
  apply PartialHoare.seq_intro'
  apply PartialHoare.seq_intro'
  apply PartialHoare.assign_intro
  apply PartialHoare.assign_intro
  apply PartialHoare.assign_intro'
  aesop
```

顺序组合规则和赋值规则的应用由程序的语法引导。程序中有两个顺序组合和三个赋值，因此对应规则也会被调用这么多次。最后我们得到一个非常不美观的子目标：

```

⊢ ∀s : State,
  s "a" = a₀ ∧ s "b" = b₀ →
  s["t" ↦ s "a"] ["a" ↦ s["t" ↦ s "a"] "b"]
    ["b" ↦ s["t" ↦ s "a"] ["a" ↦ s["t" ↦ s "a"] "b"] "t"] "a" = b₀ ∧
  s["t" ↦ s "a"] ["a" ↦ s["t" ↦ s "a"] "b"]
    ["b" ↦ s["t" ↦ s "a"] ["a" ↦ s["t" ↦ s "a"] "b"] "t"] "b" = a₀.

```

幸运的是, `simp [*] at *` 可以大幅化简该子目标, 而 `aesop` 甚至能完全自动地证明它。

10.5 第二个程序：两个数相加

我们的第二个例子计算 $m + n$, 并将结果留在 m 中, 只使用以下原始操作: $k + 1$ 、 $k - 1$ 和 $k \neq 0$ (对于任意 k):

```

def ADD : Stmt :=
  Stmt.withDo (fun s ↦ s "n" ≠ 0)
    (Stmt.assign "n" (fun s ↦ s "n" - 1);
     Stmt.assign "m" (fun s ↦ s "m" + 1))

```

由于存在 `while` 循环, 证明会更复杂:

```

theorem ADD_correct (n₀ m₀ : ℕ) :
  {* fun s ↦ s "n" = n₀ ∧ s "m" = m₀ *}
  (ADD)
  {* fun s ↦ s "n" = 0 ∧ s "m" = n₀ + m₀ *} :=
  PartialHoare.with_intro' (fun s ↦ s "n" + s "m" = n₀
    + m₀)
  (by
    apply PartialHoare.seq_intro'
      · apply PartialHoare.assign_intro
      · apply PartialHoare.assign_intro'
        aesop)
  (by aesop)
  (by aesop)

```

第一步是应用带循环不变式的派生 `while` 规则。我们的不变式是, 程序变量 n 和 m 的和必须等于所需的数学结果 $n_0 + m_0$, 其中 n_0 和 m_0 分别对应 n 和 m 的初始值, 正如前置条件所要求的那样。

我们是怎样想出这个不变式的? 即使对于简单循环, 寻找合适的不变式也可能很有挑战。困难在于, 不变式必须:

1. 在进入循环时为真；
2. 如果在迭代之前为真，则在循环的每次迭代之后仍然为真；
3. 足够强，能够蕴含所需的循环后置条件。

像 True 这样的不变式满足要求 1 和 2，但通常不满足要求 3。类似地，False 满足要求 2 和 3，但不满足要求 1。在实践中，不变式往往具有如下形式：

$$\text{已完成工作} + \text{剩余工作} = \text{期望结果}$$

其中 + 表示某个合适的运算符（不一定是加法）。当我们进入循环时，**已完成工作**通常是 0（或其他合适的「零」值），而不变式变为

$$\text{剩余工作} = \text{期望结果}$$

这个等式必须能在循环开始处被证明：要么由前一条语句的后置条件推出，要么在没有前一条语句时，由整个程序所需的前置条件推出。当我们退出循环时，**剩余工作**应该是 0（或它的某种变体），而不变式变为

$$\text{已完成工作} = \text{期望结果}$$

已完成工作通常采取某个变量的形式，我们在其中累积结果；而**剩余工作**则类似于**期望结果**，但依赖于在循环内改变值的程序变量，并且会计入**已完成工作**。

对于 ADD 程序的循环，**已完成工作**是 m ，**剩余工作**是 n ，**期望结果**是 $n_0 + m_0$ 。进入循环时，不变式 $m + n = n_0 + m_0$ 成立，因为此时 $m = m_0$ 且 $n = n_0$ 。（这个例子有些不同寻常，**已完成工作**不是 0，因为作为一种优化，我们复用了输入 m 作为累加器。）退出循环时，我们有 $n = 0$ ，因此不变式变为 $m = n_0 + m_0$ 。我们可以从 m 中取回结果。

while_intro' 定理被直接用作证明项。它产生三个子目标。不变式由所需的前置条件推出、以及它蕴含的所需后置条件这两个证明都是平凡的：它们只需要调用 aesop 即可。唯一非平凡的子目标是执行循环体会保持不变式这一条件。

对于这个例子，霍尔逻辑确实很有帮助。直接对操作语义进行推理会很不方便，因为我们需要用归纳来推理 while 循环。借助霍尔逻辑，这个归纳已经在 while_intro 规则的证明中一劳永逸地完成了。

10.6 验证条件生成器

Verification Condition Generator

验证条件生成器 (VCG) 是应用霍尔逻辑规则的程序，它们产生必须手动证明的**验证条件**。我们可以把它们看作机械的公务员，负责处理霍尔逻辑中的繁文缛节。作为用户，我们必须在程序中以标注的形式提供足够强的循环不变式。数以百计的程序验证工具都基于这些原理。

VCG 通常从后置条件逆向工作，使用^{Backward}逆向规则，即陈述以任意 Q 作为其后置条件的规则。这种做法的效果很好，因为霍尔逻辑的核心规则，也就是赋值规则，是逆向的。

我们可以使用 Lean 的元编程框架来定义一个简单的 VCG。首先，我们引入一个名为 `Stmt.invWhileDo` 的常量；除了循环条件 B 和循环体 S 之外，它还携带用户提供的不变式 I ：

```
def Stmt.invWhileDo (I B : State → Prop) (S : Stmt) :
  Stmt :=
  Stmt_whileDo B S
```

我们为该构造提供两条霍尔规则：一条逆向规则和一条双向规则。二者都由双向的 `while_intro'` 规则来论证其正确性：

```
theorem invWhile_intro {B I Q S}
  (hS : { * fun s ↦ I s ∧ B s * } S { * I * })
  (hQ : ∀ s, ¬ B s → I s → Q s) :
  { * I * } Stmt.invWhileDo I B S { * Q * } :=
  while_intro' I hS (by aesop) hQ

theorem invWhile_intro' {B I P Q S}
  (hS : { * fun s ↦ I s ∧ B s * } S { * I * })
  (hP : ∀ s, P s → I s) (hQ : ∀ s, ¬ B s → I s → Q s) :
  { * P * } Stmt.invWhileDo I B S { * Q * } :=
  while_intro' I hS hP hQ
```

上述规则只是把不变式标注用作它们的不变式。使用这个框架时，我们必须小心地为所有 `while` 循环标注合适的不变式。如果指定了错误的不变式，我们会遇到不可证明的子目标，这表明必须修正该不变式。

VCG 的代码相当简洁：

```
def matchPartialHoare : Expr → Option (Expr × Expr × Expr)
| (Expr.app (Expr.app (Expr.app
  (Expr.const 'PartialHoare _) P) S) Q) =>
  Option.some (P, S, Q)
| _ =>
  Option.none

partial def vcg : TacticM Unit :=
```

```

do
  let goals ← getUnsolvedGoals
  if goals.length != 0 then
    let target ← getMainTarget
    match matchPartialHoare target with
    | Option.none          => return
    | Option.some (P, S, Q) =>
      if Expr.isAppOfArity S 'Stmt.skip 0 then
        if Expr.isMVar P then
          applyConstant 'PartialHoare.skip_intro
        else
          applyConstant 'PartialHoare.skip_intro'
      else if Expr.isAppOfArity S 'Stmt.assign 2 then
        if Expr.isMVar P then
          applyConstant 'PartialHoare.assign_intro
        else
          applyConstant 'PartialHoare.assign_intro'
      else if Expr.isAppOfArity S 'Stmt.seq 2 then
        andThenOnSubgoals
          (applyConstant 'PartialHoare.seq_intro')
    vcg
      else if Expr.isAppOfArity S 'Stmt.ifThenElse 3
then
      andThenOnSubgoals
        (applyConstant 'PartialHoare.if_intro) vcg
      else if Expr.isAppOfArity S 'Stmt.invWhileDo 3
then
      if Expr.isMVar P then
        andThenOnSubgoals
          (applyConstant '
PartialHoare.invWhile_intro) vcg
      else
        andThenOnSubgoals
          (applyConstant '
PartialHoare.invWhile_intro')
        vcg
      else

```

failure

```
elab "vcg" : tactic =>
  vcg
```

VCG 提取第一个目标的目的并检查它。如果它是一个霍尔三元组，VCG 就检查其前置条件 P 和语句 S 。如果前置条件是一个元变量(例如 $?P$)，只要存在逆向规则，VCG 就会应用它(通过我们在第 8.7 节定义的 `applyConstant` 函数)，因为这会实例化该元变量。否则，会使用一条双向规则，以任意变量作为其前置条件；这个前置条件可以与目标的前置条件匹配。对于 `while` 循环，我们只考虑使用 `Stmt.invWhileDo` 的程序，因为一般而言，我们无法以编程方式猜出不变式。

对于复合语句新产生的所有子目标，VCG 会递归调用自身(通过我们在第 8.7 节定义的 `andThenOnSubgoals` 函数)。

10.7 重新审视第二个程序：两个数相加

使用验证条件生成器，我们可以重新审视上文给出的 `ADD` 程序的正确性证明：

```
theorem ADD_correct_vcg (n0 m0 : ℕ) :
  { * fun s ↦ s "n" = n0 ∧ s "m" = m0 * }
  (ADD)
  { * fun s ↦ s "n" = 0 ∧ s "m" = n0 + m0 * } :=
show { * fun s ↦ s "n" = n0 ∧ s "m" = m0 * }
  (Stmt.invWhileDo (fun s ↦ s "n" + s "m" = n0 + m0)
    (fun s ↦ s "n" ≠ 0)
    (Stmt.assign "n" (fun s ↦ s "n" - 1);
      Stmt.assign "m" (fun s ↦ s "m" + 1)))
  { * fun s ↦ s "n" = 0 ∧ s "m" = n0 + m0 * } from
by
  vcg <;>
  aesop
```

首先，我们使用 `show` 为 `while` 循环标注不变式。回忆一下，`show` 命令会以计算等价的方式重述目标。这里，我们利用这个机制，将 `Stmt.whileDo` 替换为 `Stmt.invWhileDo`，后者按定义等于 `Stmt.whileDo`。除此之外，该程序及其前置条件和后置条件都与定理陈述中的相同。

我们调用 `vcg` 作为第一个证明步骤。这会应用所有必要的霍尔规则，并留下一些子目标；这些子目标对 `aesop` 来说不在话下。

10.8 完全正确性的霍尔三元组

到目前为止，本章一直关注部分正确性。当我们陈述霍尔三元组 $\{P\} S \{Q\}$ 时，我们声称：如果程序 S 终止，那么最终状态会满足 Q ；但当 S 不终止时，我们什么也不说。特别地，对于发散程序 `while True do skip`，我们可以证明任意后置条件。诚然，这太宽松了：如果考试要求你编写一个排序算法，你当然不应该交出 `while True do skip` 作为答案。

Total Correctness
完全正确性是一个更强的概念；它除了部分正确性之外，还断言程序会正常终止。我们先关注部分正确性，是因为它更简单，而且无论如何它都是完全正确性的必要组成部分。

完全正确性的霍尔三元组具有 $[P] S [Q]$ 的形式，其预期含义如下：

如果在执行 S 之前前置条件 P 成立，则执行会正常终止，并且后置条件 Q 在最终状态中成立。

对于确定性程序，这可以等价地表述如下：

如果在执行 S 之前前置条件 P 成立，则存在一个状态，使得执行在其中正常终止，并且后置条件 Q 在该状态中成立。

下面是一个有效的霍尔三元组示例：

$$[i \leq 10] \text{ while } i < 10 \text{ do } i := i + 1 [i = 10]$$

对于 WHILE 语言，部分正确性与完全正确性的区别只涉及 while 循环（以及包含它们的程序）。现在，while 的霍尔规则必须用一个 Variant **变元** v 来标注；它是一个自然数，每次迭代都会减少一或更多：

$$\frac{[P \wedge B \wedge v = v_0] S [P \wedge v < v_0]}{[P] \text{ while } B \text{ do } S [P \wedge \neg B]} \text{WHILE-VAR}$$

这里， v_0 是一个逻辑变量，它「冻结」 v 的初始值，并且其作用域是整个前提；而 v 是一个数学变量（就像 P 、 B 和 S ）。对于上面的例子，我们可以取 $10 - i$ 作为变元（也可以取 $50 - i$ 或 $1024 - i * i$ ）。

考虑一次执行 $s_0, s_1, \dots, s_{n-1}, s_n$ ，它对应于 n 次循环迭代，如第 10.2 节所述。以下条件将会成立：

$$\begin{array}{ccccccc} P s_0 & & P s_1 \wedge B s_1 & & \cdots & & P s_{n-1} \wedge B s_{n-1} & & P s_n \wedge \neg B s_n \\ v s_0 & > & v s_1 & > & \cdots & > & v s_{n-1} & > & v s_n \end{array}$$

Hoare Triple
完全正确性的**霍尔三元组**理论将在本章的习题中展开。

指称语义

现在我们回顾第三种指定编程语言语义的方式：^{Denotational Semantics}**指称语义**。指称语义直接把程序的含义指定为一个数学对象。如果^{Operational Semantics}**操作语义**对应于一个理想化的解释器，而霍尔逻辑对应于一个理想化的验证器，那么指称语义就对应于一个理想化的编译器：这样的编译器并不把源程序翻译成汇编语言，而是直接翻译成数学。

^{Concrete Semantics: With Isabelle/HOL}本章的核心内容紧密仿照《**具体语义：Isabelle/HOL 描述**》[23] 第 11 章。

11.1 组合性

指称语义把每个程序的含义定义为一个数学对象。抽象地说，它可以被看作一个函数

$$\llbracket \cdot \rrbracket : \text{syntax} \rightarrow \text{semantics}$$

指称语义的一个关键性质是^{Compositionality}**组合性**：复合语句的含义应当根据其组成部分的含义来定义。考虑以下直接定义

$$\llbracket S \rrbracket = \{(s, t) \mid (S, s) \Longrightarrow t\}$$

它用大步语义 ($\Longrightarrow$) 来给出。

这个定义指定了我们想要的语义，但由于缺乏组合性，它算不上指称语义：复合语句（顺序组合、if-then-else 和 while）的含义是直接给出的，并没有使用其组成部分的^{Denotation}**指称**。

完全组合式的定义让我们能够对程序进行等式推理，这通常比使用 $\Longrightarrow$ 的引入、消去和反演原理更方便。本质上，我们想要如下形式的结构递归方程：

$$\begin{aligned}\llbracket S ; T \rrbracket &= \dots \llbracket S \rrbracket \dots \llbracket T \rrbracket \dots \\ \llbracket \text{if } B \text{ then } S \text{ else } T \rrbracket &= \dots \llbracket S \rrbracket \dots \llbracket T \rrbracket \dots \\ \llbracket \text{while } B \text{ do } S \rrbracket &= \dots \llbracket S \rrbracket \dots\end{aligned}$$

其中右边除了作为 $\llbracket \cdot \rrbracket$ 的参数以外, 不再出现 S 和 T 。

算术表达式的求值函数

$$\text{eval} : \underbrace{\text{AExp}}_{\text{语法}} \rightarrow \underbrace{(\text{String} \rightarrow \mathbb{Z}) \rightarrow \mathbb{Z}}_{\text{语义}}$$

满足如下方程:

$$\text{eval} (\text{AExp.add } e_1 \ e_2) \text{ env} = \text{eval } e_1 \text{ env} + \text{eval } e_2 \text{ env}$$

它就是一种指称语义, 因为 $\text{AExp.add } e_1 \ e_2$ 的语义是根据 e_1 和 e_2 的语义来定义的。

指称语义自然适用于算术表达式, 也适用于函数式程序。现在我们想为命令式程序给出一种方便的指称语义。由于 `while` 循环不保证终止, 我们需要发展一些额外的数学概念, 才能表述我们想要的语义。

11.2 关系式指称语义

确定性语言的指称语义通常以前置状态到后置状态的^{Function}函数给出, 但^{Relation}关系更加一般, 也更便于操作。我们将给出一种关系式指称语义。

一个程序的指称语义是类型为 $\text{Set} (\text{State} \times \text{State})$ 的数学对象。大步语义也使用过关系式方法, 它采取类型为 $\text{State} \rightarrow \text{State} \rightarrow \text{Prop}$ 的谓词形式。这两个类型是同构的, 但 $\text{Set } \alpha$ 被定义为 $\alpha \rightarrow \text{Prop}$, 支持许多有用的操作和记法, 例如 $\emptyset$ 、 $\cup$ 、 $\cap$ 、 $\in$ 以及 $\{\dots \mid \dots\}$ (第 7.7 节)。

我们的语义称为 `denote`, 并以 $\llbracket \cdot \rrbracket$ 作为语法糖。我们从定义的前四个方程开始, 把 `while` 留到后面:

```
def denote : Stmt → Set (State × State)
  | Stmt.skip           => Id
  | Stmt.assign x a     => {(s, t) | t = s[x ↦ a s]}
  | Stmt.seq S T        => denote S ∘ denote T
  | Stmt.ifThenElse B S T =>
    (denote S ↓ B) ∪ (denote T ↓ (fun s ↦ ¬ B s))
```

`skip` 语句被解释为状态上的恒等关系, 也就是所有形如 (s, s) 的元组构成的集合。这捕捉了 `skip` 想要的语义: 后状态总是与前状态相同。

赋值的语义是那些第二个分量反映赋值结果的元组所组成的集合。

顺序组合的语义可以优雅地表示为^{Relational Composition}关系组合 $\circ$, 它由下列方程定义:

$$r_1 \circ r_2 = \{(a, c) \mid \exists b, (a, b) \in r_1 \wedge (b, c) \in r_2\}$$

if-then-else 语句的语义由两个分支语义的并集给出，但会根据所处的分支，将其限制为只包含那些第一个分量使布尔条件为真或为假的元组。限制算子定义如下：

$$r \downarrow P = \{(a, b) \mid (a, b) \in r \wedge P a\}$$

注意两个退化情形：如果 $P := (\text{fun } _ \mapsto \text{True})$ ，则 $r \downarrow P = r$ ；如果 $P := (\text{fun } _ \mapsto \text{False})$ ，则 $r \downarrow P = \emptyset$ 。

当我们试图定义 while 循环的语义时，困难就出现了。我们希望写成

$$\begin{aligned} &| \text{Stmt}.\text{whileDo } B \ S \quad \Rightarrow \\ &\quad ((\text{denote } S \circ \text{denote } (\text{Stmt}.\text{whileDo } B \ S)) \downarrow B) \\ &\quad \cup (\text{Id} \downarrow (\text{fun } s \mapsto \neg B \ s)) \end{aligned}$$

但由于对 $\text{Stmt}.\text{whileDo } B \ S$ 的递归调用，这个定义不是良基的。我们需要别的东西。我们在右边寻找的是某个满足下列方程的项 X ：

$$X = ((\text{denote } S \circ X) \downarrow B) \cup (\text{Id} \downarrow (\text{fun } s \mapsto \neg B \ s))$$

我们寻找的是数学家所称的^{Fixpoint}**不动点**。接下来四节将构造一个算子 lfp ，用于计算给定方程的不动点。使用 lfp ，我们将能够如下定义 while 循环的语义：

$$\begin{aligned} &| \text{Stmt}.\text{whileDo } B \ S \quad \Rightarrow \\ &\quad \text{lfp } (\text{fun } X \mapsto ((\text{denote } S \circ X) \downarrow B) \\ &\quad \cup (\text{Id} \downarrow (\text{fun } s \mapsto \neg B \ s))) \end{aligned}$$

11.3 不动点

f 的不动点是方程中 X 的一个解：

$$X = f \ X$$

一般来说，对于某些 f ，不动点可能完全不存在。例如，如果 $f := \text{Nat}.\text{succ}$ ，那么不存在值 X 使得 $X = \text{Nat}.\text{succ } X$ 。不动点也可能有多个；例如，如果 $f := (\text{fun } x \mapsto x)$ ，那么任意 X 都是 $X = (\text{fun } x \mapsto x) \ X = X$ 的解。在关于 f 的某些条件下，可以保证存在唯一的^{Least Fixpoint}**最小不动点**和唯一的^{Greatest Fixpoint}**最大不动点**。

考虑以下不动点方程，其中 $X : \mathbb{N} \rightarrow \text{Prop}$ ：

$$X = (\text{fun } n : \mathbb{N} \mapsto n = 0 \vee (\exists m : \mathbb{N}, n = m + 2 \wedge X \ m))$$

这个方程是具有正确格式的方程经过 β -归约后的变体：

$$X = \overbrace{(\text{fun } (P : \mathbb{N} \rightarrow \text{Prop}) (n : \mathbb{N}) \mapsto n = 0 \vee (\exists m : \mathbb{N}, n = m + 2 \wedge P \ m))}^f X$$

一个解是 $X := \text{Even}$ ，即刻画偶自然数的谓词。回忆一下，我们证明过反演规则

$$\text{Even } n \leftrightarrow n = 0 \vee (\exists m : \mathbb{N}, n = m + 2 \wedge \text{Even } m)$$

见第 6.5 节。结果表明， Even 是唯一的不动点。一般而言，最小不动点和最大不动点可能不同。考虑 $X : \mathbb{N} \rightarrow \text{Prop}$ 上的方程

$$X = (\text{fun } P \mapsto P) X$$

最小不动点是 $\text{fun } _ \mapsto \text{False}$ ，最大不动点是 $\text{fun } _ \mapsto \text{True}$ 。按照约定，我们有 $\text{False} < \text{True}$ ，因此 $(\text{fun } _ \mapsto \text{False}) < (\text{fun } _ \mapsto \text{True})$ 。类似地，对于任意有居留元的类型 α ，有 $\emptyset < @\text{Set.univ } \alpha$ 。

对于 while 循环的语义， X 的类型将是状态之间关系的类型 $\text{Set}(\text{State} \times \text{State})$ ；而 f 对应于要么多执行一次循环迭代（如果条件 B 为真），要么取恒等关系（如果 B 为假）。

我们应该为 while 的语义使用哪个不动点？最大不动点还会允许循环的和发散的执行，而最小不动点只允许有限的（但可能无界的）执行。因此我们选择最小不动点。

11.4 单调函数

我们在上面声称，在关于 f 的某些条件下，最小和最大不动点保证存在。现在是时候把这点说得更精确了。令 α 和 β 为任意类型，二者各自配备一个偏序 $\leq$ 。如果对所有 a, b 都有 $a \leq b \rightarrow f a \leq f b$ ，则函数 $f : \alpha \rightarrow \beta$ 是单调的。如果 $f : \text{Set } \alpha \rightarrow \text{Set } \alpha$ 是单调的，那么它具有最小和最大不动点。

集合上的许多操作（例如并集 $\cup$ ）、关系上的许多操作（例如组合 $\circ$ ），以及函数上的许多操作（例如恒等函数 $\text{fun } x \mapsto x$ 、常量函数 $\text{fun } _ \mapsto k$ 、组合 $\cdot$ ）都是单调的，或会保持单调性。当然，并非所有函数都是单调的。下面是 $\text{Set } \alpha$ 上的一个非单调函数 f 的例子，其中 $\subseteq$ 是偏序：

$$f A = (\text{if } A = \emptyset \text{ then } \text{Set.univ} \text{ else } \emptyset)$$

如果 α 有居留元，则我们有 $\emptyset \subseteq \text{Set.univ}$ ，但 $f \emptyset = \text{Set.univ} \not\subseteq \emptyset = f \text{Set.univ}$ 。

11.5 完备格

为了给集合（包括关系）定义 lfp ，我们需要两个操作：子集 $\subseteq : \text{Set } \alpha \rightarrow \text{Set } \alpha \rightarrow \text{Prop}$ 和大交 $\bigcap : \text{Set}(\text{Set } \alpha) \rightarrow \text{Set } \alpha$ ，后者可定义为

$$\bigcap X = \{a \mid \forall A, A \in X \rightarrow a \in A\}$$

如果 X 是有限集合 $\{A_1, \dots, A_n\}$ ，则 $\bigcap X = A_1 \cap \dots \cap A_n$ 。

我们可以更一般地定义 lfp ，使它不仅适用于集合，也适用于称为**完备格**的代数结构的任意实例。完备格由以下内容组成：

1. 一个类型 α ;
2. 一个偏序 $\leq : \alpha \rightarrow \alpha \rightarrow \text{Prop}$ (即一个自反、反对称且传递的二元谓词);
3. 一个算子 $\sqcap : \text{Set } \alpha \rightarrow \alpha$, 称为**下确界**。

$\sqcap$ 算子满足以下两个条件：

1. $\sqcap A$ 是 A 的**下界**：对所有 $b \in A$ 都有 $\sqcap A \leq b$;
2. $\sqcap A$ 是**最大下界**：对所有满足 $\forall a, a \in A \rightarrow b \leq a$ 的 b ，都有 $b \leq \sqcap A$ 。

条件 1 和 2 合在一起保证 $\sqcap A$ 是唯一的最大下界。

格算子 $\leq$ 和 $\sqcap$ 分别推广了 $\subseteq : \text{Set } \alpha \rightarrow \text{Set } \alpha \rightarrow \text{Prop}$ 和 $\bigcap : \text{Set } (\text{Set } \alpha) \rightarrow \text{Set } \alpha$ 。注意， $\sqcap A$ 不一定属于 A 。例如， $\mathbb{R}$ 上的开区间 $]a, b[$ 的下确界是 $a \notin]a, b[$ 。集合的下确界是最小元素概念的一种推广。

下面是一些完备格的例子：

- 对所有类型 α ，关于 $\subseteq$ 和 $\bigcap$ 的 $\text{Set } \alpha$;
- 关于 $\rightarrow$ 和 $\text{fun } A \mapsto \forall a \in A, a$ 的 Prop ;
- 关于 $\leq$ 和某个合适下确界算子的 $\text{ENat} := \mathbb{N} \cup \{\infty\}$;
- 关于 $\leq$ 和某个合适下确界算子的 $\text{EReal} := \mathbb{R} \cup \{-\infty, \infty\}$ 。

如果 α 是完备格，那么 $\beta \rightarrow \alpha$ 也是完备格。如果 α 和 β 都是完备格，那么 $\alpha \times \beta$ 也是完备格。在这两种情形下， $\leq$ 和 $\sqcap$ 都逐分量定义。

下面是一些不是完备格的例子：关于 $\leq$ 的 $\mathbb{N}$ 、 $\mathbb{Z}$ 、 $\mathbb{Q}$ 和 $\mathbb{R}$ 。问题在于没有最大元素可以赋给 $\sqcap \emptyset$ 。另一个反例是 $\text{ERat} := \mathbb{Q} \cup \{-\infty, \infty\}$ ，因为 $\sqcap \{q \mid 2 < q * q\} = \text{sqrt } 2$ 不在 ERat 中。

在 Lean 中，用**类型类**来表示完备格是很自然的：

```
class CompleteLattice (α : Type)
  extends PartialOrder α : Type where
  Inf      : Set α → α
  Inf_le   : ∀A b, b ∈ A → Inf A ≤ b
  le_Inf   : ∀A b, (∀a, a ∈ A → b ≤ a) → b ≤ Inf A
```

类型 $\text{Set } \alpha$ 是该类型类的一个实例：

```
instance Set.CompleteLattice {α : Type} :
  CompleteLattice (Set α) :=
  { @Set.PartialOrder α with
    Inf      := fun X ↦ {a | ∀A, A ∈ X → a ∈ A}
    Inf_le   := by aesop
```

```
le_Inf      := by aesop }
```

11.6 最小不动点

利用完备格，我们可以定义最小不动点算子：

$$\text{lfp } f = \bigcap \{x \mid f x \leq x\}$$

在 Lean 中：

```
def lfp {α : Type} [CompleteLattice α] (f : α → α) : α
:=
  CompleteLattice.Inf {a | f a ≤ a}
```

我们在第 6.1 节简要提到过的^{Knaster-Tarski Theorem}**克纳斯特 – 塔斯基定理**¹ 为完备格上的任意单调函数 f 给出以下性质：

- $\text{lfp } f$ 是不动点： $\text{lfp } f = f (\text{lfp } f)$ ；
- $\text{lfp } f$ 小于任何其他不动点： $X = f X \rightarrow \text{lfp } f \leq X$ 。

11.7 关系式指称语义（续）

有了 lfp ，我们就可以兑现承诺，完成 WHILE 程序指称语义的定义：

```
| Stmt.whileDo B S      =>
  lfp (fun X ↦ ((denote S o X) ↓ B)
    ∪ (Id ↓ (fun s ↦ ¬ B s)))
```

为了验证我们的定义，可以证明指称语义和大步语义之间的如下联系：

```
theorem denote_Iff_BigStep (S : Stmt) (s t : State) :
  (s, t) ∈ ⟦S⟧ ↔ (S, s) ⇒ t
```

关于证明，我们请读者参阅^{Concrete Semantics: With Isabelle/HOL}《具体语义：Isabelle/HOL 描述》[23] 第 11 章，或本章配套的演示文件。

11.8 在程序等价上的应用

基于指称语义，我们引入^{Program Equivalence}**程序等价**的概念。如果两个程序具有相同的语义，它们就是等价的：

¹https://en.wikipedia.org/wiki/Knaster-Tarski_theorem

```
def DenoteEquiv (S1 S2 : Stmt) : Prop :=
  ⟦S1⟧ = ⟦S2⟧
```

我们把 $S_1 \sim S_2$ 写作 $\text{DenoteEquiv } S_1 \ S_2$ 的缩写。不难看出, $\sim$ 是一个等价关系。

如果两个子程序具有相同的语义, 程序等价可用于在较大的程序中用一个子程序替换另一个子程序。这通过以下同余规则实现:

```
theorem DenoteEquiv.seq_congr {S1 S2 T1 T2 : Stmt}
  (hS : S1 ~ S2) (hT : T1 ~ T2) :
  S1; T1 ~ S2; T2 :=
by
  simp [DenoteEquiv, denote] at *
  simp [*]

theorem DenoteEquiv.if_congr {B} {S1 S2 T1 T2 : Stmt}
  (hS : S1 ~ S2) (hT : T1 ~ T2) :
  Stmt.ifThenElse B S1 T1 ~ Stmt.ifThenElse B S2 T2 :=
by
  simp [DenoteEquiv, denote] at *
  simp [*]

theorem DenoteEquiv.while_congr {B} {S1 S2 : Stmt}
  (hS : S1 ~ S2) :
  Stmt.whileDo B S1 ~ Stmt.whileDo B S2 :=
by
  simp [DenoteEquiv, denote] at *
  simp [*]
```

Congruence Rule

同余规则是把某个等价关系提升到某些语境上的定理(这里是把 $\sim$ 提升到顺序组合、if-then-else 和 while 上)。

注意, 指称语义如何通过改写导向简短的证明。鉴于它本来就是为了等式化和组合性而设计的, 这并不令人意外。如果我们改用大步语义作为程序等价的基础, 这些证明会复杂得多。

现在, 我们可以通过等式推理证明简单程序等价:

```
theorem DenoteEquiv.skip_assign_id {x} :
  Stmt.assign x (fun s ↦ s x) ~ Stmt.skip :=
by simp [DenoteEquiv, denote, Id]
```

```

theorem DenoteEquiv.seq_skip_left {S} :
  Stmt.skip; S ~ S :=
  by simp [DenoteEquiv, denote, Id, comp]

theorem DenoteEquiv.seq_skip_right {S} :
  S; Stmt.skip ~ S :=
  by simp [DenoteEquiv, denote, Id, comp]

```

我们使用 `lfp` 算子定义了 `while` 的语义，但谁知道保证最小不动点存在的单调性是否满足呢？为了消除这种疑虑，我们证明以下定理：

```

theorem DenoteEquiv.if_seq_while {B S} :
  Stmt.ifThenElse B (S; Stmt.whileDo B S) Stmt.skip
  ~ Stmt.whileDo B S :=
  by
    simp [DenoteEquiv, denote]
    apply Eq.symm
    apply lfp_eq
    apply Monotone_while_lfp_arg

```

该定理给出了一种方便的方法，用来展开或收缩循环的一次迭代。第二个 `apply` 调用定理 `lfp_eq : lfp f = f (lfp f)`，它陈述 `lfp` 是一个不动点。

最后一步应用一个定理来说服 Lean： `lfp` 的参数是单调的。该定理的证明相当单调乏味：

```

theorem Monotone_while_lfp_arg (S B) :
  Monotone (fun X ↦ [[S]] ∘ X ↓ B ∪ Id ↓ (fun s ↦ ¬ B s)) :=
  by
    apply Monotone_union
    • apply SorryTheorems.Monotone_restrict
      apply SorryTheorems.Monotone_comp
      • exact Monotone_const _
      • exact Monotone_id
    • apply SorryTheorems.Monotone_restrict
      exact Monotone_const _

```

11.9 基于归纳谓词的更简单方法

Inductive Predicate

Calculus of Inductive Constructions

Lean 的归纳谓词对应于最小不动点，但它们内建于 Lean 的逻辑 (归纳构造演算) 中，并不使用像 `lfp` 这样的不动点算子。本节大部分内容都用于构造一个 `lfp` 算子。我们是否本可以改用归纳谓词，从而省去这些工作呢？

答案是肯定的。首先，回忆一下，类型 $\text{Set} (\text{State} \times \text{State})$ 和 $\text{State} \rightarrow \text{State} \rightarrow \text{Prop}$ 是 Lean 中二元关系的等价表示。因此我们可以构造如下 `Awhile` 谓词；它类似于自反传递闭包，但会在给定条件 `B` 变为假时停止：

```
inductive Awhile (B : State → Prop)
  (r : Set (State × State)) :
  State → State → Prop
where
  | true {s t u} (hcond : B s) (hbody : (s, t) ∈ r)
    (hrest : Awhile B r t u) :
    Awhile B r s u
  | false {s} (hcond : ¬ B s) :
    Awhile B r s s
```

借助这个算子，指称语义的定义变为

```
def denoteAwhile : Stmt → Set (State × State)
  :
  | Stmt.whiledo B S      =>
    {(s, t) | Awhile B (denoteAwhile S) s t}
```

`Awhile` 谓词的引入规则与大步操作语义中的 `while` 规则非常相似。在不同外观的背后，操作语义和指称语义毕竟并没有那么不同。

第四部分

数学

数学的逻辑基础

Logical Foundation

在本章中，我们将更深入地探讨 Lean 的**逻辑基础**。这里描述的大多数特性都特别适合于定义数学对象，并证明关于它们的定理。更多细节可参考 Carneiro 的硕士论文 [6]。

12.1 宇宙

Dependent Type Theory

Term

在**依值类型论**中，不仅所有**项**都有类型，所有类型也都有类型。我们已经多次见到这种情况。PAT 原理告诉我们把证明看作项，把命题看作类型。例如，定理

$$\text{@And.intro} : \forall a\ b, a \rightarrow b \rightarrow a \wedge b$$

实际上是一个类型为 $\forall a\ b, a \rightarrow b \rightarrow a \wedge b$ 的项 `@And.intro`，而这个类型本身又有一个类型：

$$\forall a\ b, a \rightarrow b \rightarrow a \wedge b : \text{Prop}$$

那么 `Prop` 的类型是什么？`Prop` 与我们目前构造出的几乎所有类型都拥有相同的类型：

$$\text{Prop} : \text{Type}$$

`Type` 的类型又是什么？最简单的方案是令 $\text{Type} : \text{Type}$ ，但这一选择会导致 Girard 悖论，即类型论中对应于 Russell 悖论的东西。为了避免不一致性，我们需要一个新的、更大的类型来容纳 `Type`，并称之为 `Type 1`。`Type 1` 本身的类型是 `Type 2`，依此类推：

$$\begin{aligned} \text{Type} & : \text{Type } 1 \\ \text{Type } 1 & : \text{Type } 2 \\ \text{Type } 2 & : \text{Type } 3 \\ & \vdots \end{aligned}$$

事实上，不带参数的 `Type` 是 `Type 0` 的缩写。如果我们想把 `Prop` 纳入这个层级，可以使用语法 `Sort u`，其中 `Sort 0` 是 `Prop` 的别名，而 `Sort (u + 1)` 是 `Type u` 的别名。这个层级由下面的类型判断刻画：

$$\frac{}{C \vdash \text{Sort } u : \text{Sort } (u + 1)} \text{ SORT}$$

所有这些包含其他类型的类型称为**宇宙**，表达式 `Sort u` 中的 `u` 称为**宇宙层级**。尽管宇宙层级看起来像是类型 $\mathbb{N}$ 的项，但实际上它们甚至不是项。

除了到处写 `Type` 外，你也可以写 `Type _`，让定理稍微更一般一些，而不必考虑宇宙层级。Lean 随后会创建一个新的宇宙变量。这有助于维持一种错觉：我们仿佛工作在一个方便的逻辑中，其中 `Type : Type` 成立，但又不会引入任何悖论。实践中，对大多数计算机科学和数学而言，`Type 0` 已经足够大。

12.2 Prop 的特殊性

虽然 `Prop` 看似很好地嵌入了宇宙层级，但它在若干方面不同于其他宇宙。

12.2.1 非直谓性

从其他类型构造新类型时（例如从 $\alpha : \text{Type } u$ 和 $\beta : \text{Type } v$ 构造 $\alpha \rightarrow \beta$ ），新构造的类型比它的每个组成部分都更复杂，因此自然应将它放入所涉及的最大宇宙中（例如 $\alpha \rightarrow \beta : \text{Type } (\max u v)$ ）。Lean 正是这样做的。下面的类型规则一般性地表达了依值类型中的这一思想：

$$\frac{C \vdash \sigma : \text{Type } u \quad C, x : \sigma \vdash \tau[x] : \text{Type } v}{C \vdash (x : \sigma) \rightarrow \tau[x] : \text{Type } (\max u v)} \text{ ARROW-TYPE}$$

`Type` 宇宙的这种行为称为**直谓性**。一般来说，直谓性意味着对象**不**可以通过一个范围涵盖该对象自身的量词来定义。

然而，让 `Prop` 表现得不同会很方便。我们希望表达式 $\forall n : \mathbb{N}, n = n$ 具有类型 `Prop`，毕竟它是一个命题。展开 $\forall$ 的语法糖后，这个表达式与 $(n : \mathbb{N}) \rightarrow n = n$ 相同。如果这里是 `Type` 而不是 `Prop`，上面的类型规则会给出

$$(n : \mathbb{N}) \rightarrow n = n : \text{Type}$$

为了仍然强制 $\forall n : \mathbb{N}, n = n$ 这样的表达式具有类型 `Prop`，我们需要一条特殊的类型规则，处理主体为 `Prop` 的 $\forall$ 表达式：

$$\frac{C \vdash \sigma : \text{Sort } u \quad C, x : \sigma \vdash \tau[x] : \text{Prop}}{C \vdash (\forall x : \sigma, \tau[x]) : \text{Prop}} \text{ ARROW-PROP}$$

该规则给出

$$\forall n : \mathbb{N}, n = n : \text{Prop}$$

正如我们所希望的那样。上面的两条类型规则可以概括为单条规则

$$\frac{C \vdash \sigma : \text{Sort } u \quad C, x : \sigma \vdash \tau[x] : \text{Sort } v}{C \vdash (x : \sigma) \rightarrow \tau[x] : \text{Sort } (\text{imax } u \ v)} \text{ARROW}$$

其中 $\text{imax } u \ 0 = 0$, 并且 $\text{imax } u \ (v + 1) = \max u \ (v + 1)$ 。这种行为称为 Prop 的 Impredicativity **非直谓性**。一般来说, 非直谓性意味着对象可以通过一个范围涵盖它自身的量词来定义。

12.2.2 证明无关性

Prop 和 Type 的第二个区别是 Proof Irrelevance **证明无关性**。它意味着同一命题 a 的任意两个证明都是相等的:

```
theorem proof_irrel {a : Prop} (h1 h2 : a) :
  h1 = h2 :=
  by rfl
```

在 Lean 中, 这一等式是在计算意义下的 Syntactically Equal **语法等价**, 因此允许我们使用 `rfl` 策略。将命题看作类型、将证明看作该类型的元素时, 证明无关性意味着一个命题要么是空类型, 要么恰好有一个 Inhabitant **居留元**。如果命题是空类型, 它就是假的; 如果命题恰好有一个居留元, 它就是真的。如本章稍后将看到的, 证明无关性在对依值类型进行推理时非常有用。

在第 6.3 节中, 我们看到过一幅并排描绘 Bool 和 Prop 解释的图。那幅图没有考虑证明无关性, 并为同一个命题显示了多个证明。现在我们知道这并不准确。下面是修订后的图:



如果我们在脑中把 Bool 的两个元素 (false 和 true) 与 Prop 的两个类型 (False 和 True) 对应起来, 就会看到 Prop 几乎与 Bool 相同, 只是类型为 Prop 的命题可以按照 PAT 原理存储证明。下表总结了这种情况:

	False	: Prop
True.intro	: True	: Prop
false	: Bool	: Type
true	: Bool	: Type

为了让证明无关性成立，Lean 必须放弃归纳谓词的「无混淆」性质。事实上，同一命题的各种证明当然应当被「混淆」。这^{Inductive Predicate}只涉及^{Inductive Type}归纳谓词（例如 Even），而不涉及一般的归纳类型（例如 List α ）。

其他系统和逻辑做出了不同选择。例如，Rocq 默认是证明相关的，但兼容证明无关性。同伦类型论以及其他构造性或直觉主义类型论建立在等式证明中的数据之上，因此与证明无关性不兼容。

12.2.3 没有大消去

Prop 和 Type 的另一个区别是，Prop ^{Large Elimination}不允许大消去：通常不可能从命题的证明中提取信息并在程序中使用（也就是在一个属于 Type 的类型的值中使用）。毕竟，由于证明无关性，给定命题的所有证明都相等，因此它们不能携带能将彼此区分开的特定信息。

假设我们能够在程序内部从证明中提取信息。例如，我们可以在如下函数定义中使用 match 构造：

```
-- fails
def unsquare (i :  $\mathbb{Z}$ ) (hsq :  $\exists j, i = j * j$ ) :  $\mathbb{Z} :=$ 
  match hsq with
  | Exists.intro j _ => j
```

unsquare 函数接受一个平方数 i ，以及一个证明 i 确实是平方数的证明 hsq，并返回从该证明中提取出的、平方前的数 j 。Lean 报告错误

```
tactic 'induction' failed, recursor 'Exists.casesOn'
can only eliminate into Prop
```

如果它接受了这个定义，我们就可以如下推出 False。令

```
hsq1 := Exists.intro 3 (by linarith)
hsq2 := Exists.intro (-3) (by linarith)
```

为 $\exists j, (9 : \mathbb{Z}) = j * j$ 的两个证明。注意，它们为 j 使用了不同的^{Witness}见证（3 对 -3）。于是我们有 $\text{unsquare } 9 \text{ hsq}_1 = 3$ 以及 $\text{unsquare } 9 \text{ hsq}_2 = -3$ 。然而，由证明无关性可知 $\text{hsq}_1 = \text{hsq}_2$ 。因此， $\text{unsquare } 9 \text{ hsq}_2$ 等于 3。但我们已经确定它等于 -3。这意味着 $3 = -3$ ，矛盾。

缺少大消去的一个不幸后果是，我们不能通过模式匹配和递归来执行^{Rule Induction}规则归纳（第 4.8 节）。这种归纳依赖于一个^{Measure}「度量」——一个到 $\mathbb{N}$ 的函数，用来给参数赋予大小。没有大消去，就无法有意义地定义该度量。这解释了为什么我们总是使用 induction 策略进行规则归纳。

Small Elimination

作为折中，Lean 允许**小消去**，它只消去到 Prop 中；而大消去可以消去到任意大的宇宙 Type u 中。这意味着，只要 match 表达式本身位于证明中，我们就可以使用 match 分析证明的结构、提取存在见证等等。

Syntactic Subsingleton

作为进一步的折中，Lean 允许对**语法子单例**进行大消去：这指 Prop 中至多只能以一种方式证明的类型。例如，False 没有证明，而 $a \wedge b$ 的所有证明都形如 `And.intro _ _`。（递归地说， a 和 b 可能有多种证明方式。）更精确地说，语法子单例是一个归纳定义，它至多有一个构造子，并且其参数要么是 Prop，要么作为结果类型的直接参数出现。当我们把 $h : a \wedge b$ 与 `And.intro ha hb` 匹配时，没有关于 h 的信息会泄漏出来。

12.3 选择公理

Axiom of Choice

Lean 的逻辑包含**选择公理**，它使得我们能够获得任意非空类型的某个任意元素。考虑下面这个预定义谓词：

```
inductive Nonempty (α : Sort u) : Prop
| intro (val : α) : Nonempty α
```

该谓词表示 α 至少有一个元素。为了证明 `Nonempty α`，我们必须向 `Nonempty.intro` 提供一个 α 值：

```
theorem Nat.Nonempty :
  Nonempty ℕ :=
  Nonempty.intro 0
```

由于 `Nonempty` 居于 Prop，大消去不可用，因此我们不能提取在 `Nonempty α` 的证明中使用的那个元素。

在 Lean 中，选择公理采取函数的形式：给定 `Nonempty α` 的证明，它返回一个任意的 α 值：

```
Classical.choice {α : Sort u} : Nonempty α → α
```

我们无法知道返回的元素是否与用于证明 `Nonempty α` 的元素相同。它只是 α 的一个任意元素。

Noncomputable

常量 `Classical.choice` 是**不可计算的**。如果我们用 `#reduce` 或 `#eval` 询问 Lean 它的值，Lean 会拒绝计算它。换言之，证明可以是项，但它们不一定是程序。这也是一些逻辑学家偏好在没有选择公理的情况下工作的原因之一。相比之下，绝大多数数学家并不反对该公理。

与证明无关性以及大、小消去不同，选择公理并未内建于 Lean 内核，它只是核心库中的一个公理，我们可以自由选择不使用它。如果定义使用 `Classical.choice` 来定义 Type 中的常量，Lean 要求我们用 `noncomputable` 关键字标记这些定义。

下列工具依赖于 `Classical.choice`:

- 函数 `Classical.choose` 称为^{Hilbert's Choice Operator}**希尔伯特选择算子**。如果我们不关心具体是哪一个,它能帮助我们找到 $\exists a : \alpha, p a$ 的一个见证。它的伴随定理 `Classical.choose_spec` 给出一个证明,说明该见证确实是一个见证。

```
Classical.choose : (∃ a : α, p a) → α
Classical.choose_spec : ∀ h : (∃ a : α, p a), p (Classical.choose h)
```

直观上,选择算子告诉我们:「说服我存在一个满足 p 的元素,我就给你这样一个元素。」

- 我们也可以推导出传统形式的选择公理:

```
Classical.axiomOfChoice (α β : Type) {R : α → β → Prop} :
  (∀ x : α, ∃ y : β, R x y) → (∃ f : α → β, ∀ x : α, R x (f x))
```

- 从选择公理以及^{Propositional Extensionality}**命题外延性** 和 ^{Functional Extensionality}**函数外延性**(`propext`、`funext`) 出发,我们可以推导出^{Law of Excluded Middle}**排中律**:

```
Classical.em : ∀ a : Prop, a ∨ ¬ a
```

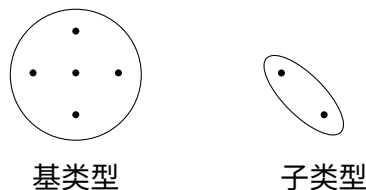
有了排中律,每个命题都是^{Decidable}**可判定的**。这意味着我们可以基于某个命题是否为真进行分类讨论,从而构造证明。

12.4 子类型

在适用时,归纳类型是一种非常方便的定义机制,但许多数学构造并不符合这种模式。Lean 提供了两种替代方案来处理这些情况:^{Subtype}**子类型**和^{Quotient}**商**。

^{Subtyping}**子类型化**是一种从既有类型创建新类型的机制。给定一个关于^{Base Type}**基类型**元素的谓词,子类型只包含基类型中满足该谓词的那些元素。更精确地说,子类型包含元素 - 证明偶对,它们把基类型的一个元素与该谓词对此元素为真的证明组合起来。

下图描绘了一个由保留基类型五个元素中的两个而创建的子类型:



子类型化适用于那些无法定义为归纳类型的类型。例如,任何试图按如下思路^{Finite Set}定义**有限集**类型的尝试

```
inductive Finset (α : Type) : Type where
  | empty : Finset α
  | insert : α → Finset α → Finset α
```

都会失败，因为给定集合可能有多个表示。例如， $\{1, 2\}$ 可以用以下任何一种方式表示，甚至还有更多方式：

```
Finset.insert 1 (Finset.insert 2 Finset.empty)
Finset.insert 2 (Finset.insert 1 Finset.empty)
Finset.insert 1 (Finset.insert 1 (Finset.insert 2 Finset.empty))
```

作为替代，我们可以把有限集定义为那些有限的（可能无限的）集合的子类型。一般地，子类型具有如下语法

{变量 : 基类型 // 变量的性质}

我们在第 4.6 节见过一个例子，即 $\{i : \mathbb{N} // i \leq n\}$ ，它由满足 $i \leq n$ 的自然数 i 组成，其中 n 固定在^{Context}语境中。基类型是 $\mathbb{N}$ ，性质是 $\text{fun } i \mapsto i \leq n$ 。同一类型还有一种不那么形象但也许更不易混淆的语法：`@Subtype  $\mathbb{N}$  (fun  $i \mapsto i \leq n$ )`。我们的介绍性例子，即某个类型 α 上的有限集类型，被指定为 $\{A : \text{Set } \alpha // \text{Set.Finite } A\}$ ，其中当且仅当参数有限时，`Set.Finite` 为 `True`。最后，注意 α 上的任何集合 A 都可以转换为类型 $\{a : \alpha // a \in A\}$ 。

12.4.1 第一个例子：满二叉树

为了说明子类型，我们将在第 5.8 节的 `Tree` 类型基础上，定义一种^{Full Binary Tree}满二叉树类型。在第 6.6.3 节中，我们引入了谓词 `IsFull`：当一棵树的每个节点都有零个或两个子节点时，它为真。基于这个类型和这个谓词，我们可以如下构造一个只包含满二叉树的子类型 `FullTree`：

```
def FullTree (α : Type) : Type :=
  {t : Tree α // IsFull t}
```

这是以下定义的语法糖

```
def FullTree (α : Type) : Type :=
  @Subtype (Tree α) IsFull
```

其中 `Subtype` 定义如下：

```
inductive Subtype {α : Type} (p : α → Prop) : Type
  | mk : (x : α) → p x → Subtype p
```

`FullTree` 的元素本质上是依值类型化的偶对，其中第一个分量是一棵树 `t`，第二个分量是 `t` 为满的证明。例如，下面定义了一棵空的满二叉树，以及一棵由单个标记为 6 的内部节点组成的满二叉树：

```
def nilFullTree : FullTree N :=
  Subtype.mk Tree.nil IsFull.nil

def fullTree6 : FullTree N :=
  Subtype.mk (Tree.node 6 Tree.nil Tree.nil)
    (by
      apply IsFull.node
      apply IsFull.nil
      apply IsFull.nil
      rfl)
```

给定一个类型为 `FullTree` 的值，我们可以通过 `Subtype.val` 和 `Subtype.property` 取回它的两个分量：

```
#reduce Subtype.val fullTree6
#check Subtype.property fullTree6
```

子类型最吸引人的方面是：只要操作保持子类型性质，我们就可以把基类型上的操作^{Lift}提升到子类型上，而不必从零开始构建一个库。我们只需要围绕基类型中的常量定义「包装器」。一般来说，围绕基类型上的操作 `f` 定义这样的包装器包含三个步骤：

1. 从包装器参数中提取基类型值；
2. 在这些基类型值上调用 `f`；
3. 使用 `Subtype.mk` 封装结果，同时给出一个证明，说明所得基类型值满足子类型性质。

利用这一过程，如果 `Tree` 函数保持性质 `IsFull`，我们就可以把它们提升为 `FullTree` 函数。例如，若要把 `mirror` 操作从类型 `Tree → Tree` 提升到类型 `FullTree → FullTree`，我们必须

1. 从包装器参数中提取 `Tree`；
2. 在该 `Tree` 上调用 `mirror`；
3. 使用 `Subtype.mk` 封装结果，同时给出一个证明，说明所得 `Tree` 满足 `IsFull`。

对于步骤 3，我们必须从参数中提取 `IsFull` 的证明，并使用第 6.6.3 节中的定理 `IsFull_mirror`。把所有内容放在一起，我们就得到

```
def FullTree.mirror {α : Type} (t : FullTree α) :
  FullTree α :=
```

```
Subtype.mk (LoVe.mirror (Subtype.val t))
  (by
    apply IsFull_mirror
    apply Subtype.property t)
```

输入是子类型 `FullTree` 的一个元素 `t`。我们把 `t` 分解为 `Subtype.val t : Tree` 和 `Subtype.property t : IsFull t`。我们使用前面的 `mirror` 函数反转 `t` 的树分量, 并使用定理 `IsFull_mirror` 以及 `t` 的性质分量, 来证明条件 `IsFull (mirror (Subtype.val t))`。

最后, 我们使用 `Subtype.mk` 构建一个偶对, 其中包含所得树以及该树为满的证明。`Subtype.mk` 构造子既可以看作类似偶对的构造子, 也可以看作从 `Tree` 到 `FullTree` 的强制转换, 其第二个参数保证该强制转换是安全的。

对于关于子类型的证明, 下面的定理很有用:

$$\text{Subtype.eq} : \text{Subtype.val } ?a = \text{Subtype.val } ?b \rightarrow ?a = ?b$$

该定理说明, 如果两个子类型值的 `Subtype.val` 分量相等, 那么这两个子类型值相等。Lean 中证明无关这一点至关重要, 因为我们不希望子类型中出现仅证明不同的虚假重复值。下面展示如何使用 `Subtype.eq` 证明 `FullTree` 的镜像的镜像就是该 `FullTree` 本身:

```
theorem FullTree.mirror_mirror {α : Type}
  (t : FullTree α) :
  (FullTree.mirror (FullTree.mirror t)) = t :=
by
  apply Subtype.eq
  simp [FullTree.mirror, LoVe.mirror_mirror]
```

应用定理 `Subtype.eq` 并展开 `full_Tree.mirror` 的定义, 会得到子目标

$$\text{mirror}(\text{mirror}(\text{Subtype.val } t)) = \text{Subtype.val } t$$

它与第 5.8 节中定理 `mirror_mirror` 的陈述相匹配。

12.4.2 第二个例子: 向量

作为第二个例子, 考虑下面的^{Vector}向量定义:

```
def Vector (α : Type) (n : ℕ) : Type :=
  {xs : List α // List.length xs = n}
```

向量被定义为具有给定长度的列表。对于列表而言, 所有长度的列表只有一个类型。对于向量而言, 每个向量长度都有一个专门的类型。这一方案的优点是, 向

^{Scalar Product}
量加法和**标量积**等操作要求所涉及的两个向量具有相同长度。我们在第 5.10 节见过一种不太实用的向量定义。

可以使用 Subtype.mk 从列表构建向量：

```
def vector123 : Vector ℤ 3 :=
  Subtype.mk [1, 2, 3] (by rfl)
```

向量上的基本操作可以通过以下方式定义：用 Subtype.val 和 Subtype.property 分解向量，操作底层列表，然后用 Subtype.mk 重新组合。例如，我们可以如下定

^{Componentwise Negation}
义整数向量的**逐分量取负**：

```
def Vector.neg {n : ℕ} (v : Vector ℤ n) : Vector ℤ n :=
  Subtype.mk (List.map Int.neg (Subtype.val v))
  (by
    rw [List.length_map]
    exact Subtype.property v)
```

我们使用函数 List.map 对底层列表的每个条目取负，并使用定理 List.length_map 说明这不会改变列表长度。

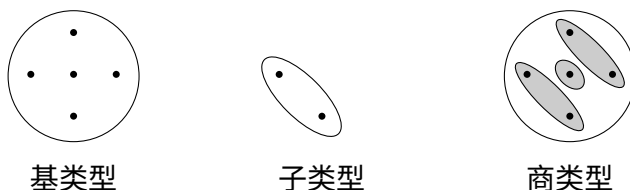
使用 Subtype.eq，我们可以证明下面关于 Vector.neg 的定理：

```
theorem Vector.neg_neg (n : ℕ) (v : Vector ℤ n) :
  Vector.neg (Vector.neg v) = v :=
  by
    apply Subtype.eq
    simp [Vector.neg]
```

应用 Subtype.eq 会把目标化归为证明底层列表上的相应性质。随后我们可以使用 simp 完成证明。

12.5 商类型

商是数学中的一种强大构造，可用于定义 $\mathbb{Z}$ 、 $\mathbb{R}$ 以及许多其他集合。Lean 支持 ^{Quotient Type}
商类型，也就是类型上的对应物。与子类型化一样，商化也从已有类型构造新类型。与子类型不同，商类型包含基类型的所有元素，只是基类型中某些不同的元素在商类型中可能被视为相等。用数学术语说，商类型同构于基类型的一个 ^{Partition}
划分。下图展示了一个由三路划分构建出的商类型：



图中的商类型只有三个元素，由灰色椭圆表示。这些元素中的每一个都对应于一个或多个基类型元素。

构造商类型的先决条件是一个基类型 τ ，以及一个等价关系 $R : \tau \rightarrow \tau \rightarrow \text{Prop}$ ，它指定基类型中的哪些元素将在商中被视为相等。为了构造商类型，我们首先需要证明 R 是 τ 上的**等价关系**。配备了等价关系的类型 τ 称为**集族**。在 Lean 中，Setoid 是一个**类型类**。我们可以使用命令声明一个实例

```
instance  $\tau$ .Setoid : Setoid  $\tau$  :=
{ r      := R
  iseqv :=
    { refl := ...
      symm := ...
      trans := ... } }
```

其中省略号代表相应性质缺失的证明。此外，这个实例声明为 $R\ a\ b$ 引入了记法 $a \approx b$ 。更重要的是，我们现在可以使用商类型 $\text{Quotient } \tau.\text{Setoid}$ 了。

每个元素 $a : \tau$ 都属于 $\text{Quotient } \tau.\text{Setoid}$ 中的某个元素，该元素由 $\text{Quotient.mk } \tau.\text{Setoid } a$ 给出，其中

$$\text{Quotient.mk } \{ \alpha : \text{Type} \} \rightarrow (s : \text{Setoid } \alpha) \rightarrow \alpha \rightarrow \text{Quotient } s$$

表达式 $\text{Quotient.mk } \tau.\text{Setoid } a$ 颇为冗长。幸运的是，Lean 允许我们把它缩写为 $\llbracket a \rrbracket$ 。

下面的公理保证，满足 R 的元素对在商类型中确实相等：

$$\text{Quotient.sound } \{ a\ b : \tau \} : a \approx b \rightarrow \llbracket a \rrbracket = \llbracket b \rrbracket$$

一个陈述了其逆命题的定理：

$$\text{Quotient.exact } \{ a\ b : \tau \} : \llbracket a \rrbracket = \llbracket b \rrbracket \rightarrow a \approx b$$

最后，我们可以使用 Quotient.lift ，把类型为 $\tau \rightarrow u$ 的函数提升到 $\text{Quotient } \tau.\text{Setoid} \rightarrow u$ ，其中 u 是某个任意类型；它满足下面这个在计算意义下的语法等式。给定某个 $f : \tau \rightarrow u$ ，以及 $h : \forall a\ b, a \approx b \rightarrow f\ a = f\ b$ ，我们有

$$\text{Quotient.lift } f\ h\ \llbracket a \rrbracket = f\ a$$

对所有 $a : \tau$ 成立。参数 h 是 f 与 $\approx$ ^{Compatible}兼容的证明；换言之，它不会区分 $\approx$ -等价的参数。

12.5.1 第一个例子：整数

作为商类型的第一个例子，我们将构造整数。一种方便的方法是在自然数偶对上构造商。其思想是，自然数偶对 (p, n) 表示整数 $p - n$ 。这样，我们可以用 $(p, 0)$ 表示所有非负整数 p ，用 $(0, n)$ 表示所有负整数 $-n$ 。我们还会得到同一整数的更多表示；例如， $(7, 0)$ 、 $(8, 1)$ 、 $(9, 2)$ 和 $(10, 3)$ 都表示整数 7。

首先，我们需要注册想要使用的等价关系。当 $p_1 - n_1$ 与 $p_2 - n_2$ 给出同一个整数时，我们希望两个偶对 (p_1, n_1) 和 (p_2, n_2) 相等。然而，条件 $p_1 - n_1 = p_2 - n_2$ 不可行，因为 $\mathbb{N}$ 上的减法行为不佳（例如 $0 - 1 = 0$ ）。因此，我们改用依赖加法的条件 $p_1 + n_2 = p_2 + n_1$ 。

我们给出等价关系的定义，随后证明它具有^{Reflexivity}自反性、^{Symmetry}对称性和^{Transitivity}传递性：

```
instance Int.Setoid : Setoid (ℕ × ℕ) :=
{ r :=
  fun (p₁, n₁) (p₂, n₂) ↦
    p₁ + n₂ = p₂ + n₁
  iseqv :=
    { refl :=
      by
        intro pn
        rfl
      symm :=
        by
          intro pn₁ pn₂ h
          simp [h]
      trans :=
        by
          intro pn₁ pn₂ pn₃ h₁₂ h₂₃
          linarith } }
```

现在我们可以用 $\approx$ 表示该等价关系：

```
theorem Int.Setoid_Iff (p₁ n₁ p₂ n₂ : ℕ) :
  (p₁, n₁) ≈ (p₂, n₂) ↔ p₁ + n₂ = p₂ + n₁ :=
  by rfl
```

随后我们可以把整数定义为

```
def Int : Type :=
  Quotient Int.Setoid
```

我们可以把整数零定义为

```
def Int.zero : Int :=
  [(0, 0)]
```

事实上，任何形如 $[(m, m)]$ 的项都表示零：

```
theorem Int.zero_Eq (m : ℕ) :
  Int.zero = [(m, m)] :=
  by
    rw [Int.zero]
    apply Quotient.sound
    rw [Int.Setoid_Iff]
    simp
```

接下来，我们在新的整数上定义加法。要在商类型上定义函数，我们不能像归纳类型那样简单地通过模式匹配来定义它们。相反，我们先在基类型上定义函数，然后把该定义提升到商上。为此，我们必须证明函数 f 的定义不依赖于所选择的 **等价类** 代表元（即 $a \approx b \rightarrow f a = f b$ ）。函数 Quotient.lift （用于一元函数）和 Quotient.lift_2 （用于二元函数）可用于以这种方式提升函数。

加法可以定义为对自然数偶对逐分量相加。随后我们需要证明它可以提升为商上的函数：只需说明 $pn_1 \approx pn_1'$ 和 $pn_2 \approx pn_2'$ 蕴含

$$[(\text{prod.fst } pn_1 + \text{prod.fst } pn_2, \text{prod.snd } pn_1 + \text{prod.snd } pn_2)] \\ = [(\text{prod.fst } pn_1' + \text{prod.fst } pn_2', \text{prod.snd } pn_1' + \text{prod.snd } pn_2')]$$

形式化地：

```
def Int.add : Int → Int → Int :=
  Quotient.lift₂
    (fun (p₁, n₁) (p₂, n₂) ↦ [(p₁ + p₂, n₁ + n₂)])
    (by
      intro pn₁ pn₂ pn₁' pn₂' h₁ h₂
      apply Quotient.sound
      rw [Int.Setoid_Iff] at *
      linarith)
```

所得函数 Int.add 具有预期行为：

```
theorem Int.add_Eq (p₁ n₁ p₂ n₂ : ℕ) :
  Int.add [(p₁, n₁)] [(p₂, n₂)] =
    [(p₁ + p₂, n₁ + n₂)] :=
  by rfl
```

如果 Lean 一开始就允许我们把这个定理输入为 Int.add 的定义，那会非常方便，大概使用如下语法：

这会是一种漂亮而直观的语法，但若没有定义与 $\approx$ 兼容的证明，我们就能定义荒谬的函数，并用它们推出 False。例如，我们可以定义

```
-- fails
def Int.fst : Int → ℕ
  | [(p, n)] => p
```

注意， $\text{Int.fst } [(0, 0)] = 0$ 且 $\text{Int.fst } [(1, 1)] = 1$ 。然而，由于 $[(0, 0)] = [(1, 1)]$ ，我们得到 $0 = 1$ ，矛盾。

我们可以使用特征定理 Int.add_Eq 证明关于 Int.add 的其他定理，例如

```
theorem Int.add_zero (i : Int) :
  Int.add Int.zero i = i :=
by
  induction i using Quotient.inductionOn with
  | h pn =>
    cases pn with
    | mk p n => simp [Int.zero, Int.add]
```

我们调用 `induction` 策略，并以 `Quotient.inductionOn` 作为归纳原理，对 i 进行分类讨论，把 i 替换为 $[(pn)]$ ，其中 pn 是基类型 $\mathbb{N} \times \mathbb{N}$ 的任意值。然后我们对 pn 进行分类讨论，得到一个偶对 (p, n) 。最后，我们使用 Int.zero 的定义和 Int.add 的特征等式化简目标。

12.5.2 第二个例子：无序偶对

Unordered Pair

无序偶对是不区分第一和第二分量的偶对。它们通常写作 $\{a, b\}$ 。我们将把 α 上的无序偶对的类型 $\text{UPair } \alpha$ 引入为偶对 (a, b) 关于「包含相同元素」这一关系的商：

```
instance UPair.Setoid (α : Type) : Setoid (α × α) :=
{ r :=
  fun (a₁, b₁) (a₂, b₂) ↦ ({a₁, b₁} : Set α) = ({a₂,
b₂})
iseqv :=
{ refl  := by simp
  symm  := by aesop
  trans := by aesop } }

theorem UPair.Setoid_Iff {α : Type} (a₁ b₁ a₂ b₂ : α) :
  (a₁, b₁) ≈ (a₂, b₂) ↔ ({a₁, b₁} : Set α) = ({a₂, b₂
}) :=
```

```
by rfl
```

```
def UPair (α : Type) : Type :=
  Quotient (UPair.Setoid α)
```

很容易证明我们的偶对确实是无序的：

```
theorem UPair.mk_symm {α : Type} (a b : α) :
  ([(a, b)] : UPair α) = [(b, a)] :=
by
  apply Quotient.sound
  rw [UPair.Setoid_Iff]
  aesop
```

无序偶对的另一种表示是作为基数为 1 或 2 的集合。下面的操作把 $\text{UPair } \alpha$ 值转换到这种表示：

```
def Set_of_UPair {α : Type} : UPair α → Set α :=
  Quotient.lift (fun (a, b) ↦ {a, b})
  (by
    intro ab1 ab2 h
    rw [UPair.Setoid_Iff] at *
    exact h)
```

12.5.3 通过正规化和子类型化给出的另一种定义

商类型的每个元素都对应于基类型中一类 $\approx$ -等价的元素。如果存在一种系统方法，可以为每个 $\approx$ -等价类获得一个^{Canonical Representative}典范代表元，我们就可以使用子类型而非商，只保留典范代表元并过滤掉其他元素。

考虑上面构造的整数商类型 Int 。我们观察到 $(7, 0)$ 、 $(8, 1)$ 、 $(9, 2)$ 和 $(10, 3)$ 都表示整数 7，但直观上 $(7, 0)$ 似乎优于其他表示。若 p 或 n 为 0，我们就称偶对 (p, n) 是^{Canonical}典范的：

```
inductive Int.IsCanonical : ℕ × ℕ → Prop where
  | nonpos {n : ℕ} : Int.IsCanonical (0, n)
  | nonneg {p : ℕ} : Int.IsCanonical (p, 0)
```

于是整数由自然数的典范偶对组成：

```
def Int : Type :=
  {pn : ℕ × ℕ // Int.IsCanonical pn}
```

显然，每个整数都能以唯一一种方式表示。因此， $+$ 和 $*$ 等整数运算必须给出典范^{Normalization}结果。幸运的是，**正规化**自然数偶对很容易：

```
def Int.normalize :  $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$ 
  | (p, n) => if p ≥ n then (p - n, 0) else (0, n - p)
```

```
theorem Int.IsCanonical_normalize (pn :  $\mathbb{N} \times \mathbb{N}$ ) :
  Int.IsCanonical (Int.normalize pn)
```

对于无序偶对，没有显然的^{Normal Form}正规形式，除非总是把较小元素放在前面（或后面）。
 这要求 α 上有一个^{Linear Order}线性序 $\leq$ ：

```
def UPair.IsCanonical { $\alpha$  : Type} [LinearOrder  $\alpha$ ] :
   $\alpha \times \alpha \rightarrow \text{Prop}$ 
  | (a, b) => a ≤ b
```

```
def UPair ( $\alpha$  : Type) [LinearOrder  $\alpha$ ] : Type :=
  {ab :  $\alpha \times \alpha$  // UPair.IsCanonical ab}
```

回到 Int.IsCanonical，我们注意到 $(0, 0)$ 的典范性有两个证明：可以使用 Int.IsCanonical.nonpos，也可以使用 Int.IsCanonical.nonneg。这不是问题，因为由证明无关性可知，这些证明必定相等。

12.6 新引入的 Lean 构造总结

声明

noncomputable 为不可计算声明添加前缀

常量

Classical.choice 返回非空类型中某个任意元素的函数

Classical.choose 给定一个 $\exists$ 的证明后返回见证的函数

Quotient	创建给定集族实例的商类型的函数
Quotient.lift	将一元函数提升到商类型上的函数
Quotient.lift₂	将二元函数提升到商类型上的函数
Setoid	带有其上的等价关系的类型的类型类
Sort u	层级为 u 的宇宙
Subtype.mk	用于构造子类型值的函数
Subtype.property	从子类型值中提取性质的函数
Subtype.val	从子类型值中提取基值的函数

记法

$\{x : \alpha \mid P[x]\}$	α 中所有满足 $P[x]$ 的 x 的子类型
$\approx$	集族上的等价关系（用于商化）
<code>Prop</code>	Sort 0 的缩写
<code>Type u</code>	Sort ($u + 1$) 的缩写

定理

<code>Classical.axiomOfChoice</code>	传统的选择公理
<code>Classical.choose_spec</code>	<code>Classical.choose</code> 的特征性质
<code>Quotient.exact</code>	商类型上的相等蕴含基类型上的 $\approx$
<code>Quotient.inductionOn</code>	商类型值上的归纳原理
<code>Quotient.sound</code>	基类型上的 $\approx$ 蕴含商类型上的相等
<code>Subtype.eq</code>	基类型上的相等蕴含子类型的相等

基本数学结构

在本章中，我们将介绍关于基本数学结构的定义和证明，例如群、域和线性序。

13.1 单个二元算子上的类型类

在数学中，^{Group}群是一个集合 G ，配备一个^{Binary Operator}二元算子 $\cdot : G \times G \rightarrow G$ ，并满足下列性质，这些性质称为群公理：

- 结合律：对所有 $a, b, c \in G$ ，都有 $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ ；
- 单位元：存在一个元素 $e \in G$ ，使得对所有 $a \in G$ ，都有 $e \cdot a = a$ ；
- 逆元：对每个 $a \in G$ ，都存在一个记作 a^{-1} 的逆元，使得 $a^{-1} \cdot a = e$ 。

在 Lean 中，群的^{Type Class}类型类可以定义如下：

```
class Group (α : Type) where
  mul      : α → α → α
  one      : α
  inv      : α → α
  mul_assoc : ∀ a b c, mul (mul a b) c = mul a (mul b c)
  one_mul  : ∀ a, mul one a = a
  mul_left_inv : ∀ a, mul (inv a) a = one
```

不过，这并不是官方定义。群是一个更大的^{Algebraic Structure}代数结构层级的一部分。

群运算可以写成乘法形式（使用算子 $*$ 、单位元 1 和逆元 a^{-1} ），也可以写成加法形式（使用算子 $+$ 、单位元 0 和逆元 $-a$ ）。这就是 Lean 为群提供两个类型类的原因：^{Multiplicative}乘法式的 Group 和^{Additive}加法式的 AddGroup。它们本质上相同，只是对常量和性质使用不同名称。

任何满足群公理的类型都可以注册为 `Group` 或 `AddGroup`。为说明这一点，我们将定义模 2 整数的类型 `Int2`，它也称为 $\mathbb{Z}/2\mathbb{Z}$ 或 $\mathbb{Z}_2$ ，并把它注册为 `AddGroup`。类型 `Int2` 有两个元素：

```
inductive Int2 : Type where
  | zero
  | one
```

加法定义如下：

```
def Int2.add : Int2 → Int2 → Int2
  | Int2.zero, a      => a
  | Int2.one,  Int2.zero => Int2.one
  | Int2.one,  Int2.one  => Int2.zero
```

为了实例化 `AddGroup`，我们需要提供下列常量和性质：

```
add : α → α → α
zero : α
neg : α → α
add_assoc : ∀ a b c, add (add a b) c = add a (add b c)
zero_add : ∀ a, add zero a = a
add_zero : ∀ a, add a zero = a
neg_add_cancel : ∀ a, add (neg a) a = zero
nsmul : ℕ → α → α
zsmul : ℤ → α → α
```

常量 `AddGroup.add`、`AddGroup.zero` 和 `AddGroup.neg` 分别对应二元算子、单位元和逆元。性质 `AddGroup.add_assoc`、`AddGroup.zero_add` 和 `AddGroup.neg_add_cancel` 对应三个群公理。出于技术原因，我们还必须证明冗余性质 `AddGroup.add_zero`，并提供 n 重加法的定义 `AddGroup.nsmul` 和 `AddGroup.zsmul`。

类型 `Int2` 可以如下注册为群：

```
instance Int2.AddGroup : AddGroup Int2 :=
{ add      := Int2.add
  zero     := Int2.zero
  neg      := fun a ↦ a
  add_assoc :=
    by
      intro a b c
      cases a <;>
```

```

        cases b <;>
        cases c <;>
      rfl
zero_add      :=
  by
    intro a
    cases a <;>
    rfl
add_zero      :=
  by
    intro a
    cases a <;>
    rfl
neg_add_cancel :=
  by
    intro a
    cases a <;>
    rfl
nsmul         :=
  @nsmulRec Int2 (Zero.mk Int2.zero) (Add.mk
Int2.add)
zsmul         :=
  @zsmulRec Int2 (Zero.mk Int2.zero) (Add.mk
Int2.add)
  (Neg.mk (fun a ↦ a))
  (@nsmulRec Int2 (Zero.mk Int2.zero) (Add.mk
Int2.add)) }

```

对于 `AddGroup.nsmul` 和 `AddGroup.zsmul`，我们使用 `mathlib` 提供的默认定义。

有了上面的类型类实例，我们现在可以写 `0`、`+`、`-` 等记法：

```
#reduce Int2.one + 0 - 0 - Int2.one
```

代数层级还包含更多带有一个二元算子的类型类。主要类型类列举如下：

类型类	性质	例子
Semigroup	$*$ 的结合律	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
Monoid	带单位元 1 的 Semigroup	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
LeftCancelSemigroup	带 $c * a = c * b \rightarrow a = b$ 的 Semigroup	
RightCancelSemigroup	带 $a * c = b * c \rightarrow a = b$ 的 Semigroup	
Group	带逆元 $^{-1}$ 的 Monoid	

这些结构大多都有^{Commutative}**交换版本** (即对所有元素 a, b , 都有 $a \cdot b = b \cdot a$): `CommSemigroup`、`CommMonoid`、`CommGroup`。这些结构也都有以前缀 `Add` 命名的加法对应物:

类型类	性质	例子
AddSemigroup	$+$ 的结合律	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
AddMonoid	带单位元 0 的 AddSemigroup	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
AddLeftCancelSemigroup	带有以下性质的 AddSemigroup $c + a = c + b \rightarrow a = b$	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
AddRightCancelSemigroup	带有以下性质的 AddSemigroup $a + c = b + c \rightarrow a = b$	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
AddGroup	带逆元 $-$ 的 AddMonoid	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}$

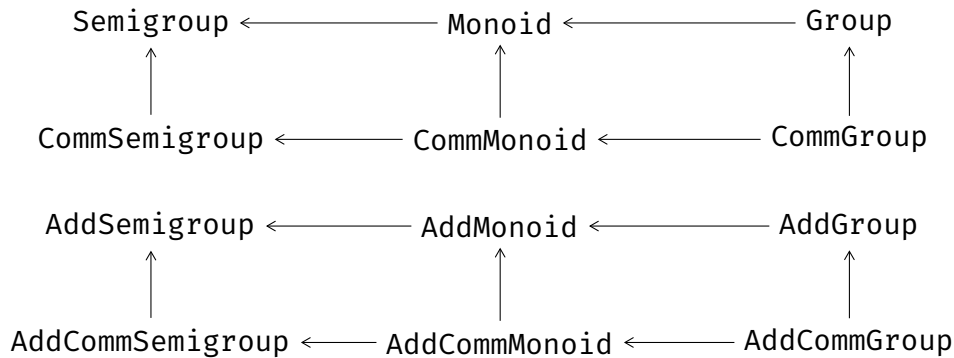
尽管加法类型类只是其乘法对应物的副本, 但在构造带有多个二元算子的代数结构时, 例如环和域, 它们至关重要。为了避免重复所有基于乘法类型类的定理和定义, 这一复制过程通过^{Metaprogram}**元程序**自动完成。

`AddMonoid` 的一个示例实例是类型 `List α`, 其中空列表 `[]` 作为零, 追加算子 `++` 作为加法:

```
instance List.AddMonoid {α : Type} : AddMonoid (List α)
:=
{ zero      := []
  add       := fun xs ys ↦ xs ++ ys
  add_assoc := List.append_assoc
  zero_add  := List.nil_append
  add_zero  := List.append_nil
  nsmul     :=
    @nsmulRec (List α) (Zero.mk [])
      (Add.mk (fun xs ys ↦ xs ++ ys)) }
```

我们还可以继续, 把带有 `[]` 和 `++` 的 `List α` 注册为 `AddLeftCancelSemigroup` 和 `AddRightCancelSemigroup`。

下图展示了其中一些类型类之间的关系。在本图以及后面的图中，从 X 指向 Y 的箭头表示「 X 从 Y 继承所有常量和性质」。



13.2 两个二元算子上的类型类

加法结构和乘法结构会合并起来，形成两个二元算子上的更复杂结构。其中之一是 ^{Field}域。域 F 由下列性质定义：

- F 在称为加法的算子 $+$ 下构成交换群，单位元为 0 。
- $F \setminus \{0\}$ 在称为乘法的算子 $*$ 下构成交换群。
- 乘法对加法满足 ^{Distributivity}分配律，即对所有 $a, b, c \in F$ ，都有 $a * (b + c) = a * b + a * c$ 。

运行 `#print Field` 可以显示 `Field` 所需的所有常量和性质。同样，由于其构造方式，这个类型类包含一些冗余的性质和定义。

现在我们将通过为 `Int2` 实例化 `Field` 类型类，来说明 `Int2` 是一个域。首先，我们必须在 `Int2` 上定义乘法：

```
def Int2.mul : Int2 → Int2 → Int2
| Int2.one, a => a
| Int2.zero, _ => Int2.zero
```

为了把 `Int2` 声明为域，我们可以使用语法 `Int2.AddGroup with`，复用上面定义的实例 `Int2.AddGroup`。余下性质可以如下证明：

```
instance Int2.Field : Field Int2 :=
{ Int2.AddGroup with
  one      := Int2.one
  mul      := Int2.mul
  inv      := fun a => a
  add_comm :=
    by
      intro a b
```

```

      cases a <;>
      cases b <;>
      rfl
exists_pair_ne :=
  by
    apply Exists.intro Int2.zero
    apply Exists.intro Int2.one
    simp
zero_mul      :=
  by
    intro a
    rfl
mul_zero      :=
  by
    intro a
    cases a <;>
    rfl
one_mul       :=
  by
    intro a
    rfl
mul_one       :=
  by
    intro a
    cases a <;>
    rfl
mul_inv_cancel :=
  by
    intro a h
    cases a
    • apply False.elim
      apply h
      rfl
    • rfl
inv_zero      := by rfl
mul_assoc     :=
  by

```

```

      intro a b c
      cases a <;>
      cases b <;>
      cases c <;>
      rfl
mul_comm      :=
  by
    intro a b
    cases a <;>
    cases b <;>
    rfl
left_distrib  :=
  by
    intro a b c
    cases a <;>
    cases b <;>
    rfl
right_distrib :=
  by
    intro a b c
    cases a <;>
    cases b <;>
    cases c <;>
    rfl
nnqsmul      := _
nnqsmul_def  :=
  by
    intro a b
    rfl
qsmul        := _
qsmul_def    :=
  by
    intro a b
    rfl
nnratCast_def :=
  by
    intro q

```

```
rfl }
```

有了这个类型类实例，我们现在可以使用 1、*、/ 等记法：

```
#reduce (1 : Int2) * 0 / (0 - 1)
```

该命令打印 Int2.zero。这里的类型标注 : Int2 是必要的，用来告诉 Lean 我们想在 Int2 中计算，而不是在默认的 $\mathbb{N}$ 中计算。我们甚至可以在 Int2 中使用任意数字。例如，数字 3 被解释为 $1 + 1 + 1$ ，这在 Int2 中等同于 1：

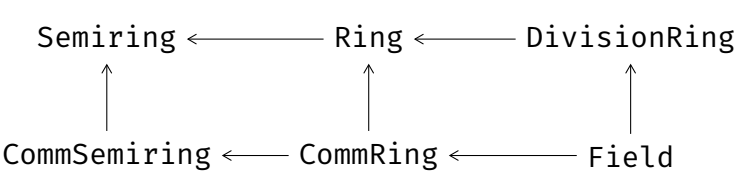
```
#reduce (3 : Int2)
```

该命令打印 Int2.one。

除了 Field，还有许多用于带两个二元算子的结构的类型类。主要类型类如下：

类型类	性质	例子
Semiring	带分配律的 Monoid 和 AddCommMonoid	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
CommSemiring	带 * 交换律的 Semiring	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
Ring	带分配律的 Monoid 和 AddCommGroup	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}$
CommRing	带 * 交换律的 Ring	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}$
DivisionRing	带乘法逆元的 Ring	$\mathbb{R}, \mathbb{Q}$
Field	带 * 交换律的 DivisionRing	$\mathbb{R}, \mathbb{Q}$

下图展示了这些类型类之间的关系。



Field 类型类要求性质 $\forall a, a / 0 = 0$ 。这只是为了让除法成为^{Total Function}全函数而采用的约定。数学家会把除法视为^{Partial Function}偏函数。以这种方式把偏函数全化并没有什么坏处。

一旦我们用某个具体类型实例化了这些类型类，就可以使用 ring 策略来^{Normalize}正规化包含该类型上算子的项。例如：

```
theorem ring_example (a b : Int2) :
  (a + b) ^ 3 = a ^ 3 + 3 * a ^ 2 * b + 3 * a * b ^ 2
  + b ^ 3
  :=
  by ring
```

该策略可用于任何被声明为域，或更一般地声明为交换半环的类型。

13.3 强制转换

在同一定理中组合来自 $\mathbb{N}$ 、 $\mathbb{Z}$ 、 $\mathbb{Q}$ 和 $\mathbb{R}$ 的数时，我们可能想从一个类型^{Coercion}强制转换到另一个类型。例如，给定一个自然数，我们可能需要把它转换为整数。考虑下面的定理，并注意乘法参数的类型：

```
theorem neg_mul_neg_Nat (n : ℕ) (z : ℤ) :
  (- z) * (- n) = z * n :=
  by simp
```

令人惊讶的是，尽管取负 $-n$ 并未定义在 $n : \mathbb{N}$ 上，且 $z : \mathbb{Z}$ 与 $n : \mathbb{N}$ 的乘法也未定义，但这个陈述并不会导致错误。

诊断命令 `#check neg_mul_neg_Nat` 会告诉我们发生了什么：

```
neg_mul_neg_Nat : ∀ (n : ℕ) (z : ℤ), -z * -↑n = z * ↑n
```

Lean 有一种在必要时引入强制转换的机制，记作 $\uparrow$ 或 `coe`。这个强制转换算子可以被设置为在任意类型之间提供隐式转换。许多强制转换已经就位，包括：

- `coe : ℕ → α` 将 $\mathbb{N}$ 转换到另一个半环 α ；
- `coe : ℤ → α` 将 $\mathbb{Z}$ 转换到另一个环 α ；
- `coe : ℚ → α` 将 $\mathbb{Q}$ 转换到另一个除环 α 。

我们可以提供类型标注来记录意图，或帮助 Lean 判断该在哪里放置强制转换，如下例：

```
theorem neg_Nat_mul_neg (n : ℕ) (z : ℤ) :
  (- n : ℤ) * (- z) = n * z :=
  by simp
```

在涉及强制转换的证明中，`norm_cast` 策略会很方便。它能帮助处理如下代码中的 $\vdash m n : \mathbb{N}, h : \uparrow m = \uparrow n \vdash m = n$ 这类目标：

```
theorem Eq_coe_int_imp_Eq_Nat (m n : ℕ)
  (h : (m : ℤ) = (n : ℤ)) :
  m = n :=
  by norm_cast at h
```

类似地，它也能帮助处理如下代码中的目标 $\vdash m n : \mathbb{N} \vdash \uparrow m + \uparrow n = \uparrow(m + n)$ ：

```
theorem Nat_coe_Int_add_eq_add_Nat_coe_Int (m n : ℕ) :
  (m : ℤ) + (n : ℤ) = ((m + n : ℕ) : ℤ) :=
  by norm_cast
```

`norm_cast` 策略依赖于如下定理：

```

Nat.cast_add : ∀ a b : ℕ, ↑(a + b) = ↑a + ↑b
Int.cast_add : ∀ a b : ℤ, ↑(a + b) = ↑a + ↑b
Rat.cast_add : ∀ a b : ℚ, ↑(a + b) = ↑a + ↑b

```

13.4 正规化策略

代数策略 `ring` 以及强制转换策略 `norm_cast` 通过正规化工作：它们重写表达式，希望所得结果^{Syntactically Equal}**语法等价**，此时等式便显然可证。与 `rw` 和 `simp` 一样，当它们取得了一些进展但未能完全证明目标时，会产生一个子目标。

可选参数 `position` 与重写策略中的相同（第 3.5 节）。

ring

```
ring [at position]
```

`ring` 策略通过正规化表达式并对结果进行语法比较，证明交换环和半环（例如 $\mathbb{N}$ 、 $\mathbb{Z}$ 、 $\mathbb{Q}$ 和 $\mathbb{R}$ ）上的等式。

norm_cast

```
norm_cast [at position]
```

`norm_cast` 策略把强制转换向表达式外侧移动，作为一种化简形式。

13.5 列表、多重集和有限集

在前几章中，我们已经见过许多使用列表的例子。但在给出新定义或陈述新定理时，我们也应该考虑其他选择，例如^{Multiset}**多重集**和^{Finite Set}**有限集**。

考虑下面的定义，它基于第 5.8 节中引入的二叉树：

```

def List.elems : Tree ℕ → List ℕ
| Tree.nil           => []
| Tree.node a l r => a :: List.elems l ++ List.elems r

```

该函数返回树中出现的所有元素组成的列表。它按从左到右的^{Depth First}顺序对树进行**深度优先**遍历。但对某些应用而言，我们可能并不关心元素顺序。

这正是多重集派上用场的地方。对于多重集，我们有 $\{3, 2, 1, 2\} = \{1, 2, 2, 3\}$ ，而两个列表 $[3, 2, 1, 2]$ 和 $[1, 2, 2, 3]$ 是不同的。多重集^{Quotient Type}被定义为列表在重排下的**商类型**。我们可以用多重集如下重做上面的定义：

```
def Multiset.elems : Tree N → Multiset N
| Tree.nil          => ∅
| Tree.node a l r =>
  {a} ∪ Multiset.elems l ∪ Multiset.elems r
```

使用这个定义，我们可以证明 $\text{Multiset.elems } t = \text{Multiset.elems } (\text{mirror } t)$ ，而 $\text{List.elems } t = \text{List.elems } (\text{mirror } t)$ 一般并不成立。

对某些应用而言，我们可能还想更进一步：不仅忽略顺序，也忽略每个元素在树中出现了多少次，只区分出现与未出现。这正是有限集，或者说 *finset* 派上用场的地方。在有限集上，我们有 $\{3, 2, 1, 2\} = \{1, 2, 3\}$ 。有限集被定义为不含任何重复元素的多重集的子类型。（另一种可能的定义是把它定义为有限集合的子类型。）我们可以用有限集如下重做上面的定义：

```
def Finset.elems : Tree N → Finset N
| Tree.nil          => ∅
| Tree.node a l r => {a} ∪ Finset.elems l ∪
  Finset.elems r
```

对于列表和多重集，Lean 提供了求和与求积算子，用来把所有元素相加或相乘。以下前两个命令打印 9，后两个命令打印 24：

```
#eval List.sum [2, 3, 4]
#eval Multiset.sum ({2, 3, 4} : Multiset N)

#eval List.prod [2, 3, 4]
#eval Multiset.prod ({2, 3, 4} : Multiset N)
```

这些算子要求元素类型被声明为 *AddMonoid* 的实例（用于求和）或 *Monoid* 的实例（用于求积）。*Multiset* 版本还要求声明 *AddCommMonoid* 或 *CommMonoid* 的实例，因为结果不能依赖于元素相加或相乘的顺序。

13.6 序类型类

上面引入的许多结构都可以带有序^{Order}。例如，自然数上我们熟悉的序可以定义为

```
inductive Nat.le : N → N → Prop where
| refl : ∀ a : N, Nat.le a a
| step : ∀ a b : N, Nat.le a b → Nat.le a (b + 1)
```

这是^{Linear Order}线性序的一个例子。^{Total Order}线性序（或全序）是一个二元关系 $\leq$ ，使得对所有 a 、 b 和 c ，下列性质成立：

- 自反性： $a \leq a$ ；

- 传递性：如果 $a \leq b$ 且 $b \leq c$ ，则 $a \leq c$ ；
- 反对称性：如果 $a \leq b$ 且 $b \leq a$ ，则 $a = b$ ；
- 完全性： $a \leq b$ 或 $b \leq a$ 。

如果一个关系具有前三个性质，它就是^{Partial Order}**偏序**。一个例子是集合、有限集或多重集上的子集关系 $\subseteq$ 。如果一个关系具有前两个性质，它就是^{Preorder}**预序**。一个例子是按长度比较列表。

在 Lean 中，不同类型的序对应不同的类型类：LinearOrder、PartialOrder 和 Preorder。Preorder 类有一个常量和两个性质：

```
le :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ 
le_refl :  $\forall a : \alpha, \text{le } a a$ 
le_trans :  $\forall a b c : \alpha, \text{le } a b \rightarrow \text{le } b c \rightarrow \text{le } a c$ 
```

PartialOrder 类还有额外性质

```
le_antisymm :  $\forall a b : \alpha, \text{le } a b \rightarrow \text{le } b a \rightarrow a = b$ 
```

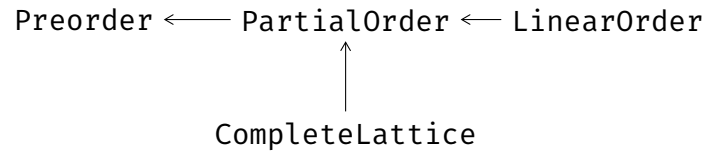
而 LinearOrder 还有额外性质

```
le_total :  $\forall a b : \alpha, \text{le } a b \vee \text{le } b a$ 
```

我们可以如下声明 List α 上按列表长度比较的预序：

```
instance List.length.Preorder { $\alpha : \text{Type}$ } : Preorder (
  List  $\alpha$ ) :=
{ le := fun xs ys  $\mapsto$  List.length xs  $\leq$  List.length ys
  lt := fun xs ys  $\mapsto$  List.length xs < List.length ys
  le_refl :=
    by
      intro xs
      apply Nat.le_refl
  le_trans :=
    by
      intro xs ys zs
      exact Nat.le_trans
  lt_iff_le_not_ge :=
    by
      intro a b
      exact Nat.lt_iff_le_not_le }
```

我们在第 11 章讨论过的^{Complete Lattice}完备格被形式化为另一个类型类 `CompleteLattice`，它扩展了 `PartialOrder`。



最后，Lean 提供了结合序与代数结构的类型类：`OrderedCancelCommMonoid`、`OrderedCommGroup`、`OrderedSemiring`、`LinearOrderedSemiring`、`LinearOrderedCommRing`、`LinearOrderedField`。所有这些数学结构都通过^{Monotonicity Rule}单调性规则（例如 $a \leq b \rightarrow c \leq d \rightarrow a + c \leq b + d$ ）和^{Cancellation Rule}消去规则（例如 $c + a \leq c + b \rightarrow a \leq b$ ），把 $\leq$ 和 $<$ 与常量 0 、 1 、 $+$ 和 $*$ 联系起来。

13.7 新引入的 Lean 构造总结

记法

$\uparrow$ 强制转换算子 `coe`

策略

`norm_cast` 正规化强制转换
`ring` 正规化环表达式

有理数与实数

在前几章中，我们已经看到如何把自然数 $\mathbb{N}$ 定义为^{Inductive Type}归纳类型，以及如何把整数 $\mathbb{Z}$ 定义为 $\mathbb{N} \times \mathbb{N}$ 上的^{Quotient}商。在本章中，我们回顾有理数 $\mathbb{Q}$ 和实数 $\mathbb{R}$ 的构造。这些构造用到的工具是归纳类型、^{Subtype}子类型和商。

下面的过程可用于构造具有特定性质的类型：

1. 创建一个足够大的新类型，使其能够表示所有元素，但表示方式不必唯一。
2. 对这种表示取商，按需把元素等同起来。
3. 通过从^{Base Type}基类型提升函数，在^{Quotient Type}商类型上定义算子，并证明它们与商关系兼容。

我们在第 12.5.1 节使用这种方法构造了类型 $\mathbb{Z}$ 。它也可用于 $\mathbb{Q}$ 和 $\mathbb{R}$ 。

14.1 有理数

^{Rational Number}

有理数是可以表示为整数 n 和 d 之分数 n/d 的数，其中 $d \neq 0$ ：

```
structure Fraction where
  num          :  $\mathbb{Z}$ 
  denom        :  $\mathbb{Z}$ 
  denom_Neq_zero : denom  $\neq$  0
```

数 n 称为^{Numerator}分子，数 d 称为^{Denominator}分母。有理数作为分数的表示并不唯一。例如，有理数 $1/2$ 、 $2/4$ 和 $-1/-2$ 都相等。这种分数表示将作为我们要取商的基类型。

若两个分数 n_1/d_1 和 n_2/d_2 的分子与分母之比相同，即 $n_1 * d_2 = n_2 * d_1$ ，它们就表示同一个有理数。为了按此关系构造类型 `Fraction` 上的商，我们要证明该关系是^{Equivalence Relation}等价关系。这可以通过把 `Fraction` 声明为 `Setoid` ^{Type Class}类型类的实例来实现：

```
namespace Fraction
```

```

instance Setoid : Setoid Fraction :=
{ r :=
  fun a b : Fraction ↦ num a * denom b = num b *
  denom a
  iseqv :=
    { refl  := by aesop
      symm  := by aesop
      trans :=
        by
          intro a b c heq_ab heq_bc
          apply Int.eq_of_mul_eq_mul_right (
            denom_Neq_zero b)
          calc
            num a * denom c * denom b
            = num a * denom b * denom c :=
              by ac_rfl
            _ = num b * denom a * denom c :=
              by rw [heq_ab]
            _ = num b * denom c * denom a :=
              by ac_rfl
            _ = num c * denom b * denom a :=
              by rw [heq_bc]
            _ = num c * denom a * denom b :=
              by ac_rfl } }

theorem Setoid_Iff (a b : Fraction) :
  a ≈ b ↔ num a * denom b = num b * denom a :=
  by rfl

end Fraction

```

于是我们可以把有理数类型定义为该集族上的商：

```

def Rat : Type :=
  Quotient Fraction.Setoid

```

为了定义零、一、加法、乘法以及其他运算，我们先在 Fraction 类型上定义它们。要将两个分数相加，我们把它们转换为同分母形式，并把分子相加。最容易使用的公分母就是两个分母的乘积：

```

namespace Fraction

```

```

instance Add : Add Fraction :=
{ add := fun a b : Fraction ↦
  { num      := num a * denom b + num b *
    denom a
    denom      := denom a * denom b
    denom_Neq_zero := by simp [denom_Neq_zero] } }

```

我们把这些运算注册为 Add 等语法型类型类的实例，以便能在 Fraction 上使用 + 等方便的记法。类似地，我们把零定义为 $0 := 0/1$ ，把一定义为 $1 := 1/1$ ，并把乘法定义为分子与分母分别相乘。

为了把这些运算提升到有理数类型 Rat 上，我们必须证明它们与 ^{Compatible} $\approx$ 兼容：

```

@[simp] theorem add_num (a b : Fraction) :
  num (a + b) = num a * denom b + num b * denom a :=
  by rfl

@[simp] theorem add_denom (a b : Fraction) :
  denom (a + b) = denom a * denom b :=
  by rfl

theorem Setoid_add {a a' b b' : Fraction} (ha : a ≈ a')
  (hb : b ≈ b') :
  a + b ≈ a' + b' :=
  by
    simp [Setoid_Iff, add_denom, add_num] at *
    calc
      (num a * denom b + num b * denom a)
        * (denom a' * denom b')
      = num a * denom a' * denom b * denom b'
        + num b * denom b' * denom a * denom a' :=
        by grind
      _ = num a' * denom a * denom b * denom b'
        + num b' * denom b * denom a * denom a' :=
        by simp [*]
      _ = (num a' * denom b' + num b' * denom a')
        * (denom a * denom b) :=
        by grind
end Fraction

```

然后我们可以使用 `Quotient.lift2` 定义 `Rat` 上的运算，并实例化相关的语法类型类型类，例如

```
namespace Rat

instance Add : Add Rat :=
{ add := Quotient.lift2 (fun a b : Fraction ↦ mk (a +
  b))
  (by
    intro a b a' b' ha hb
    apply Quotient.sound
    exact Fraction.Setoid_add ha hb) }

end Rat
```

从这里开始，我们可以继续证明让 `Rat` 成为 `Field` 实例所需的所有性质。

有理数的另一种定义 在 `mathlib` 中，有理数采用另一种方法定义。类型 `Rat` 被定义为 `Fraction` 的子类型，要求分母为正，并且分子与分母^{Coprime}**互质**（即除了 1 和 -1 外没有公因子）：

```
def Rat.IsCanonical (a : Fraction) : Prop :=
  Fraction.denom a > 0
  ∧ Nat.Coprime (Int.natAbs (Fraction.num a))
    (Int.natAbs (Fraction.denom a))

def Rat : Type :=
  {a : Fraction // Rat.IsCanonical a}
```

这是第 12.5.3 节所述一般策略的一个实例。采用这种方法时，不需要商，计算更高效，并且更多性质会成为在计算意义下的^{Syntactically Equal}**语法等价**。缺点是，由于需要^{Normalize}**正规化**分数，函数定义会变得更复杂。

14.2 实数

有些有理数序列看起来会收敛，因为序列中的数彼此越来越接近，但它们并不收敛到某个有理数。序列

$$\begin{aligned} a_0 &= 1 \\ a_1 &= 1.4 \\ a_2 &= 1.41 \\ a_3 &= 1.414 \\ a_4 &= 1.4142 \\ &\vdots \end{aligned}$$

就是这样的序列，其中 a_n 是满足 $a_n^2 < 2$ 且小数点后有 n 位的最大数。它看起来会收敛，因为每个 a_n 与后续任何数的距离至多为 10^{-n} ，但其极限是 $\sqrt{2} \notin \mathbb{Q}$ 。在这个意义上，有理数是不完备的，而实数是它们的^{Completion}**完备化**。为了构造实数，我们需要填补这些看起来会收敛但并不收敛的序列所揭示出的空隙。

^{Cauchy Sequence}**柯西序列**捕捉了「看起来会收敛」的序列这一概念。若对任意 $\varepsilon > 0$ ，都存在一个 $N \in \mathbb{N}$ ，使得对所有 $m \geq N$ ，都有 $|a_N - a_m| < \varepsilon$ ，则序列 $a_0, a_1, \dots$ 是柯西序列。换言之，无论我们把 ε 选得多小，总能在序列中找到一个点，从该点开始，所有后续数的偏离都小于 ε 。

我们把有理数序列形式化为函数 $f : \mathbb{N} \rightarrow \mathbb{Q}$ ，并用 abs 表示绝对值 $|\cdot|$ 。这给出了柯西序列的如下 Lean 定义：

```
def IsCauchySeq (f : ℕ → ℚ) : Prop :=
  ∀ε > 0, ∃N, ∀m ≥ N, abs (f N - f m) < ε
```

并非每个序列都是柯西序列：

```
theorem id_Not_CauchySeq :
  ¬ IsCauchySeq (fun n : ℕ => (n : ℚ)) :=
by
  rw [IsCauchySeq]
  intro h
  cases h 1 zero_lt_one with
  | intro i hi =>
    have hi_succ :=
      hi (i + 1) (by simp)
    simp at hi_succ
```

我们把柯西序列类型定义为一个子类型：

```
def CauchySeq : Type :=
  {f : ℕ → ℚ // IsCauchySeq f}
```

有一个从 CauchySeq 中提取实际序列的辅助函数会很方便：

```
def seqOf (f : CauchySeq) : ℕ → ℚ :=
  Subtype.val f
```

该构造的基本思想是用柯西序列表示实数。每个柯西序列都表示以它为极限的实数；例如，序列 $a_n = 1/n$ 表示实数 0，而序列 1, 1.4, 1.41, ... 表示实数 $\sqrt{2}$ 。

两个不同的柯西序列可以表示同一个实数；例如，序列 $a_n = 1/n$ 和常序列 $b_n = 0$ 都表示 0。因此，我们需要对表示同一实数的序列取商。当两个序列的差收敛到零时，它们表示同一个实数：

```
namespace CauchySeq

instance Setoid : Setoid CauchySeq :=
{ r :=
  fun f g : CauchySeq ↦
    ∀ε > 0, ∃N, ∀m ≥ N, abs (seqOf f m - seqOf g m) <
    ε
  iseqv :=
    { refl :=
      by
        intro f ε hε
        apply Exists.intro 0
        aesop
      symm :=
        by
          intro f g hfg ε hε
          cases hfg ε hε with
          | intro N hN =>
            apply Exists.intro N
            intro m hm
            rw [abs_sub_comm]
            apply hN m hm
      trans :=
        by
          intro f g h hfg hgh ε hε
          cases hfg (ε / 2) (by linarith) with
          | intro N₁ hN₁ =>
```

```

cases hgh ( $\epsilon / 2$ ) (by linarith) with
| intro N2 hN2 =>
  apply Exists.intro (max N1 N2)
  intro m hm
  calc
    abs (seqOf f m - seqOf h m)
    ≤ abs (seqOf f m - seqOf g m)
      + abs (seqOf g m - seqOf h m) :=
      by apply abs_sub_le
  _ <  $\epsilon / 2 + \epsilon / 2$  :=
    add_lt_add (hN1 m (le_of_max_le_left hm
))
      (hN2 m (le_of_max_le_right hm))
  _ =  $\epsilon$  :=
    by simp } }

theorem Setoid_iff (f g : CauchySeq) :
  f ≈ g ↔
  ∀ $\epsilon > 0$ , ∃N, ∀m ≥ N, abs (seqOf f m - seqOf g m) <  $\epsilon$ 
:=
  by rfl

end CauchySeq

```

使用这个 Setoid 实例，我们现在可以定义实数：

```

def Real : Type :=
  Quotient CauchySeq.Setoid

```

与有理数一样，我们需要定义零、一、加法、乘法以及其他算子。我们先在 CauchySeq 上定义它们，然后再提升到 Real。对于常量 0 和 1，我们可以简单地把它定义为常序列。任何常序列都是柯西序列：

```

namespace CauchySeq

def const (q :  $\mathbb{Q}$ ) : CauchySeq :=
  Subtype.mk (fun _ :  $\mathbb{N} \mapsto q$ )
    (by
      rw [IsCauchySeq]
      intro  $\epsilon$  h $\epsilon$ 
      aesop)

```

```
end CauchySeq
```

我们可以为 `Real` 声明^{Syntactic}语法型类型类 `Zero` 和 `One` 的实例：

```
namespace Real

instance Zero : Zero Real :=
  { zero := [[CauchySeq.const 0]] }

instance One : One Real :=
  { one := [[CauchySeq.const 1]] }

end Real
```

定义实数的加法 需要稍多一些工作。我们通过逐项相加序列元素来定义柯西序列上的加法：

```
namespace CauchySeq

instance Add : Add CauchySeq :=
  { add := fun f g : CauchySeq =>
    Subtype.mk (fun n : ℕ => seqOf f n + seqOf g n)
    sorry }

end CauchySeq
```

这个定义要求证明：在 f 和 g 是柯西序列的前提下，结果也是柯西序列。该证明很长，上面用 `sorry` 代替。

接下来，我们需要说明这个加法与 $\approx$ 兼容：

```
theorem Setoid_add {f f' g g' : CauchySeq} (hf : f ≈ f')
  (hg : g ≈ g') :
  f + g ≈ f' + g' :=
by
  intro ε₀ hε₀
  simp
  cases hf (ε₀ / 2) (by linarith) with
  | intro Nf hNf =>
    cases hg (ε₀ / 2) (by linarith) with
    | intro Ng hNg =>
      apply Exists.intro (max Nf Ng)
      intro m hm
      calc
        abs (seqOf (f + g) m - seqOf (f' + g') m)
        = abs ((seqOf f m + seqOf g m) - (seqOf f' m + seqOf g' m))
```

```

- (seqOf f' m + seqOf g' m)) :=
by rfl
_ = abs ((seqOf f m - seqOf f' m)
  + (seqOf g m - seqOf g' m)) :=
by
  have arg_eq :
    seqOf f m + seqOf g m
      - (seqOf f' m + seqOf g' m) =
    seqOf f m - seqOf f' m
      + (seqOf g m - seqOf g' m) :=
    by linarith
  rw [arg_eq]
_ ≤ abs (seqOf f m - seqOf f' m)
  + abs (seqOf g m - seqOf g' m) :=
by apply abs_add_le
_ < ε₀ / 2 + ε₀ / 2 :=
add_lt_add (hNf m (le_of_max_le_left hm))
  (hNg m (le_of_max_le_right hm))
_ = ε₀ :=
by simp

end CauchySeq

```

为了证明 $f + g \approx f' + g'$ ，给定 $\varepsilon_0 > 0$ 后，我们必须说明存在一个数 N ，使得

$$\forall m, m \geq N \rightarrow \text{abs} (\text{seqOf} (f + g) m - \text{seqOf} (f' + g') m) < \varepsilon_0.$$

为了得到 N ，我们使用 $f \approx f'$ 和 $g \approx g'$ 。等价式 $f \approx f'$ 告诉我们，对任意 $\varepsilon > 0$ ，都存在一个数 N_f ，使得对所有 $m \geq N_f$ ，都有 $\text{abs} (\text{seqOf} f m - \text{seqOf} f' m) < \varepsilon$ 。事实 $g \approx g'$ 给出了一个具有类似性质的数 N_g 。为了最终能完成计算，我们对 $\varepsilon := \varepsilon_0 / 2$ 取数 N_f 和 N_g 。然后选择 N 为 N_f 与 N_g 的最大值，这样对任意 $m \geq N$ 都能得到这些不等式。证明末尾的 `calc` 块建立了：对所有 $m \geq N$ ，

$$\text{abs} (\text{seqOf} (f + g) m - \text{seqOf} (f' + g') m) < \varepsilon_0.$$

证明了柯西序列上的加法与 $\approx$ 兼容之后，我们就可以在 `Real` 上定义加法：

```

namespace Real

instance Add : Add Real :=
{ add := Quotient.lift₂ (fun a b : CauchySeq ↦ [a + b])

```

```

      (by
        intro a b a' b' ha hb
        apply Quotient.sound
        exact CauchySeq.Setoid_add ha hb) }

    end Real

```

我们可以继续用这种方式定义乘法和其他算子。总之，实数被定义为柯西序列上的商，而柯西序列又被定义为 $\mathbb{N} \rightarrow \mathbb{Q}$ 的子类型。

实数的另一种定义 在 `mathlib` 中，实数的构造本质上如上所述。一些定义以更一般的方式陈述，以允许构造其他代数结构，例如 p -进数 [18]。

另一种方式是使用^{Dedekind Cut}**戴德金分割**定义实数。此时，数 $r : \mathbb{R}$ 被表示为满足 $x < r$ 的数 $x : \mathbb{Q}$ 的集合。另一个不依赖于 $\mathbb{Q}$ 的替代方案是使用二进制序列 $\mathbb{N} \rightarrow \text{Bool}$ 定义 $\mathbb{R}$ 。序列的元素表示该数的各位数字。如果我们只需要实区间 $[0, 1]$ ，这种方式特别好用。

14.3 最后的勉励

我们现在已经来到本指南的结尾。你现在已经了解了^{Interactive Theorem Proving}**交互式定理证明**中的基础理论和技术，以及一些应用领域。虽然我们使用的是 Lean，但你学到的内容应该可以迁移到其他系统，尤其是那些基于简单类型论或依值类型论的系统。你也应当能够阅读该领域的大多数科研论文。

即使你不选择以定理证明为职业，作者们也希望你能把^{Proof Assistant}**证明助手**带在身边，并在合适时使用它们：或是因为它们高度可信，或是因为它们便于跟踪复杂的证明目标。

如果你继续使用 Lean，合乎自然的下一步是熟悉 `mathlib` 及其文档。如果你在课堂环境之外使用 Lean，那么你经常会发现自己需要查找定义和定理。你一定会发现 `#find` 命令很有用。¹ 而且大多数 Lean 用户都会使用 Lean Zulip 聊天。²

14.4 新引入的 Lean 构造总结

诊断命令

`#find` 通过模式匹配查找定义或定理

¹https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Find.html

²<https://leanprover.zulipchat.com/>

参考文献

- [1] *The Lean 4 Manual*. 2021.
<https://leanprover.github.io/lean4/doc/>.
- [2] J. Avigad, L. de Moura, S. Kong, and S. Ullrich. *Theorem Proving in Lean 4*. 2021.
https://leanprover.github.io/theorem_proving_in_lean4/.
- [3] J. Avigad, L. de Moura, and J. Roesch. *Programming in Lean*. 2016.
- [4] H. P. Barendregt, W. Dekkers, and R. Statman. *Lambda Calculus with Types*. Perspectives in logic. Cambridge University Press, 2013.
- [5] K. Buzzard, J. Commelin, and P. Massot. Formalising perfectoid spaces. In J. Blanchette and C. Hritcu, editors, *CPP 2020*, pages 299–312. ACM, 2020.
- [6] M. Carneiro. The type theory of Lean. MSc thesis, Carnegie Mellon University, 2019. <https://github.com/digama0/lean-type-theory/releases/download/v1.0/main.pdf>.
- [7] G. Gonthier. Formal proof—the Four-Color Theorem. *Notices AMS*, 55(11):1382–1393, 2008.
<https://www.ams.org/notices/200811/tx081101382p.pdf>.
- [8] G. Gonthier, A. Asperti, J. Avigad, Y. Bertot, C. Cohen, F. Garillot, S. L. Roux, A. Mahboubi, R. O’Connor, S. O. Biha, I. Pasca, L. Rideau, A. Solovyev, E. Tassi, and L. Théry. A machine-checked proof of the Odd Order Theorem. In S. Blazy, C. Paulin-Mohring, and D. Pichardie, editors, *ITP 2013*, volume 7998 of *Lecture Notes in Computer Science*, pages 163–179. Springer, 2013.
<https://hal.inria.fr/hal-00816699/document>.
- [9] K. Gopinathan and I. Sergey. Certifying certainty and uncertainty in approximate membership query structures. In S. K. Lahiri and C. Wang, editors, *CAV 2020, Part II*, volume 12225 of *Lecture Notes in Computer Science*, pages 279–303. Springer, 2020. <https://arxiv.org/abs/2004.13312>.

- [10] R. Gu, Z. Shao, H. Chen, X. N. Wu, J. Kim, V. Sjöberg, and D. Costanzo. CertiKOS: An extensible architecture for building certified concurrent OS kernels. In K. Keeton and T. Roscoe, editors, *OSDI 2016*, pages 653–669. USENIX Association, 2016. <https://www.usenix.org/system/files/conference/osdi16/osdi16-gu.pdf>.
- [11] T. C. Hales, M. Adams, G. Bauer, D. T. Dang, J. Harrison, T. L. Hoang, C. Kaliszyk, V. Magron, S. McLaughlin, T. T. Nguyen, T. Q. Nguyen, T. Nipkow, S. Obua, J. Pleso, J. Rute, A. Solovyev, A. H. T. Ta, T. N. Tran, D. T. Trieu, J. Urban, K. K. Vu, and R. Zumkeller. A formal proof of the Kepler conjecture. *CoRR*, abs/1501.02155, 2015. <http://arxiv.org/abs/1501.02155>.
- [12] J. Harrison. Formal verification at Intel. In *LICS 2003*, pages 45–54. IEEE Computer Society, 2003. <https://ieeexplore.ieee.org/document/1210044>.
- [13] J. Harrison, J. Urban, and F. Wiedijk. History of interactive theorem proving. In J. H. Siekmann, editor, *Computational Logic*, volume 9 of *Handbook of the History of Logic*, pages 135–214. Elsevier, 2014. <https://www.cl.cam.ac.uk/~jrh13/papers/joerg.pdf>.
- [14] G. Klein, J. Andronick, K. Elphinstone, G. Heiser, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. seL4: Formal verification of an operating-system kernel. *Commun. ACM*, 53(6):107–115, 2010. https://www.researchgate.net/profile/Gernot_Heiser/publication/220910193_SeL4_Formal_verification_of_an_OS_kernel/links/09e4150f00292a0329000000/SeL4-Formal-verification-of-an-OS-kernel.pdf.
- [15] D. E. Knuth. *The TeXbook*. Addison-Wesley, 1986.
- [16] R. Kumar, M. O. Myreen, M. Norrish, and S. Owens. CakeML: A verified implementation of ML. In S. Jagannathan and P. Sewell, editors, *POPL 2014*, pages 179–192. ACM, 2014. <https://cakeml.org/popl14.pdf>.
- [17] X. Leroy. A formally verified compiler back-end. *J. Automated Reasoning*, 43(4):363–446, 2009. <https://arxiv.org/pdf/0902.2137.pdf>.
- [18] R. Y. Lewis. A formal proof of Hensel’s lemma over the p -adic integers. In A. Mahboubi and M. O. Myreen, editors, *CPP 2019*, pages 15–26. ACM, 2019. <https://arxiv.org/pdf/1909.11342.pdf>.

- [19] J. Limperg and A. H. From. Aesop: White-box best-first proof search for Lean. In B. Pientka and S. Zdancewic, editors, *CPP '23*. ACM, 2023. <https://people.compute.dtu.dk/ahfrom/aesop-camera-ready.pdf>.
- [20] A. Lochbihler. Verifying a compiler for Java threads. In A. D. Gordon, editor, *ESOP 2010*, volume 6012 of *Lecture Notes in Computer Science*, pages 427–447. Springer, 2010. https://link.springer.com/content/pdf/10.1007/978-3-642-11957-6_23.pdf.
- [21] The mathlib Community. The Lean mathematical library. In J. Blanchette and C. Hrițcu, editors, *CPP 2020*, pages 367–381. ACM, 2020. <https://arxiv.org/pdf/1910.09336.pdf>.
- [22] R. Nederpelt and H. Geuvers. *Type Theory and Formal Proof: An Introduction*. Cambridge University Press, 2014.
- [23] T. Nipkow and G. Klein. *Concrete Semantics: With Isabelle/HOL*. Springer, 2014. <http://www.concrete-semantics.org/concrete-semantics.pdf>.
- [24] B. O’Sullivan, D. Stewart, and J. Goerzen. *Real World Haskell*. O’Reilly, 2008. <http://book.realworldhaskell.org/read/>.
- [25] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [26] D. M. Russinoff. A mechanically checked proof of correctness of the AMD K5 floating point square root microcode. *Formal Methods in System Design*, 14(1):75–125, 1999. <https://link.springer.com/content/pdf/10.1023/A:1008669628911.pdf>.
- [27] C. Watt. Mechanising and verifying the WebAssembly specification. In J. Andronick and A. P. Felty, editors, *CPP 2018*, pages 53–65. ACM, 2018. <https://www.cl.cam.ac.uk/~caw77/papers/mechanising-and-verifying-the-webassembly-specification.pdf>.